DEVOPS ENGINEERING MASTERY

DOCKER & CONTAINER ENGINEERING

Container Fundamentals, Docker Architecture, Image Engineering & Production Container Operations

VOLUME 06 • FIRST EDITION

```
nahid@technova:~$ docker build -t technova/app:v1.4.0 .
✓ 12 layers  •  ✓ multi-stage  •  ✓ non-root user
nahid@technova:~$ docker compose -f compose.yml -f compose.prod.yml up -d
✓ api healthy  •  ✓ mongo healthy  •  ✓ web healthy
nahid@technova:~$ docker stats --no-stream
```

Docker • containerd • Linux Namespaces • Compose • MERN

First Edition • Published 9 November 2026 • Dhaka, Bangladesh

24 Chapters • 24 Guided Labs • 3 Major Projects • 120 MCQs

Docker & Container Engineering

DEVOPS ENGINEERING MASTERY — Volume 06

Container Fundamentals, Docker Architecture, Image Engineering ও Production Container Operations

First Edition — Published 9 November 2026.

Hands-on environment: Docker Engine and Docker Compose v2, with Node.js 20, MongoDB and PostgreSQL, and a MERN (MongoDB, Express, React, Node.js) example application called TechNova. All domains, IP addresses and credentials in examples are fictional (technova.example, 203.0.113.x, CHANGE-ME-...). Sample terminal outputs are illustrative. Cloud-provider comparisons (AWS, Microsoft Azure, Google Cloud, IBM Cloud) are high-level and operational; provider details vary by region and plan, and official provider documentation is the authority — no pricing or service limits are stated in this book.

How to use this book. Read the parts in order — each part builds on the previous one. Keep a Docker-ready machine beside you and do every hands-on exercise and every lab in the companion Lab Workbook before moving on. Do not copy a Dockerfile or Compose file blindly; read the explanation under each instruction first. Destructive operations are always flagged with a WARNING box and a safe alternative. Never practise on a production host: use a disposable virtual machine or an isolated lab environment, and never paste a real secret into an example — use the fictional placeholders shown. Never store a secret inside an image layer.

A note on honesty. Reading a book alone does not make anyone an expert. This book teaches the foundations and the practical judgment to progress toward professional engineering — but the judgment itself comes from doing the labs, breaking a test container, fixing it, and repeating. There is no shortcut for that practice.

Conventions. Bangla is the explanatory language; technical terms, command names, APIs and configuration keys stay in English. `monospace` marks commands, file paths and code. **CONCEPT** explains an idea, **REAL WORLD** shows an industry scenario, **WARNING** marks risk, **PRO TIP** gives professional advice, and **LAB** marks hands-on work.

© 2026 DevOps Engineering Mastery series. All rights reserved. This book is for personal learning. Container changes should first be practised in an isolated lab, never directly on a production system. The author and publisher accept no liability for data loss or service disruption caused by running commands without a backup or an approval gate.

Who this book is for

A learner who knows JavaScript, React, Next.js, Node.js, Express and MongoDB, understands basic Git and networking, has studied Linux server administration and CI/CD fundamentals, and wants to design, build, operate and troubleshoot real container systems — from a first `docker run` to a production multi-container MERN deployment with health checks, security controls, monitoring, backup and rollback.

Learning outcomes

By the end of this book you will be able to: explain how containers differ from virtual machines and how Linux namespaces and cgroups make isolation work; explain Docker Engine, containerd and OCI runtimes; write, optimise and secure Dockerfiles with multi-stage builds; tag, scan and distribute images through registries; design container networks and persistent storage; build multi-container MERN environments with Docker Compose; manage environment variables and secrets safely; implement health checks and service recovery; apply least- privilege container security; set resource limits and operate logging and monitoring; and deploy, update, roll back and troubleshoot production containers with a runbook.

সূচিপত্র — Contents

PART I — Container Fundamentals and Architecture

Chapter 01 — Virtual Machines vs Containers: Core Concepts 7

Chapter 02 — Containerization and Linux Isolation Fundamentals 17

Chapter 03 — Docker Architecture: Engine, Daemon, CLI and Runtime 27

Chapter 04 — Images, Containers, Layers and Union Filesystems 35

Chapter 05 — Container Lifecycle and Resource Management 44

PART II — Docker Image Engineering

Chapter 06 — Docker Installation and Development Environment 54

Chapter 07 — Docker CLI: Managing Images and Containers 63

Chapter 08 — Dockerfile Fundamentals and Build Instructions 72

Chapter 09 — Build Context, Layers, Cache and Build Optimization 80

Chapter 10 — Multi-Stage Builds and Production Image Design 87

Chapter 11 — Image Tagging, Registries and Image Distribution 94

PART III — Container Networking and Storage

Chapter 12 — Docker Networking Architecture 101

Chapter 13 — Bridge, Host, None and Custom Networks 110

Chapter 14 — Container-to-Container Communication and DNS 119

Chapter 15 — Volumes, Bind Mounts and Persistent Storage 129

Chapter 16 — Storage Backup, Restore and Data Management 140

PART IV — Multi-Container Application Engineering

Chapter 17 — Docker Compose Architecture and Configuration 151

Chapter 18 — Building Multi-Container MERN Applications 164

Chapter 19 — Environment Variables, Secrets and Configuration 175

Chapter 20 — Health Checks, Dependencies and Service Recovery 183

PART V — Production Container Operations

Chapter 21 — Container Security and Least-Privilege Design 193

Chapter 22 — Resource Limits, Logging and Container Monitoring 202

Chapter 23 — Production Deployment, Updates and Troubleshooting 209

Chapter 24 — Container Operations Architecture and Best Practices 216

Companion Volumes

Lab Workbook — 24 Guided Labs • 15 Assignments • 3 Major Projects • Capstone

Assessment and Answer Guide — 120 MCQs • Short Answers • Scenarios • Case Studies • Final Exam

Quick Reference Handbook — Glossary (180+ Terms) • CLI / Dockerfile / Compose References • Checklists • Decision Trees

Guided labs (24), major projects (3) and the capstone are in the separate *Docker Lab Workbook*. MCQs, assignments, examinations and the answer key are in the separate *Assessment and Answer Guide*. Glossary, CLI, Dockerfile and Compose references, checklists and decision trees are in the separate *Quick Reference Handbook*.

PART I • DEVOPS ENGINEERING MASTERY

Container Fundamentals and Architecture

ভার্চুয়াল মেশিন বনাম কনটেইনার, Linux isolation — namespace ও cgroup — Docker আর্কিটেকচার, ইমেজ ও লেয়ার, কনটেইনার লাইফসাইকেল।

অধ্যায় 1 • PART I — Container Foundations

Chapter 1 — Virtual Machines vs Containers: Core Concepts

১.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: (১) Virtual Machine (VM) ও Container-এর সংজ্ঞা এবং তাদের মৌলিক পার্থক্য ব্যাখ্যা করতে; (২) Hypervisor কী, Type 1 ও Type 2 Hypervisor কীভাবে কাজ করে তা বলতে; (৩) Host Operating System ও Guest Operating System-এর ভূমিকা আলাদা করতে; (৪) কেন Container "হালকা VM" নয় — Shared Kernel Architecture-এর অর্থ বোঝাতে; (৫) TechNova-র MERN অ্যাপের জন্য কখন VM আর কখন Container উপযুক্ত তা যুক্তিসহ সিদ্ধান্ত নিতে; (৬) Isolation, Portability ও Overhead — এই তিনটি মানদণ্ডে দুটি প্রযুক্তির তুলনা করতে।

১.২ কেন এই বিষয়টি DevOps-এ গুরুত্বপূর্ণ

DevOps-এর প্রতিদিনের কাজ — Deploy করা, Scale করা, Rollback করা, পরিবেশের পার্থক্য দূর করা — সবকিছুর নিচে একটি প্রশ্ন থাকে: অ্যাপ্লিকেশনটি কোথায়, কীসের ভেতরে চলবে? ভুল উত্তর দিলে খরচ বাড়ে, Deploy ধীর হয়, আর নিরাপত্তা-ঝুঁকি তৈরি হয়। অনেক শিক্ষার্থী প্রথমেই Docker কমান্ড মুখস্থ করতে যান, কিন্তু VM আর Container-এর পার্থক্য না বুঝলে তারা Container-কে VM-এর মতো ব্যবহার করেন — ভেতরে SSH Server চালান, একটিমাত্র Container-এ ডেটাবেস, API ও ফ্রন্টএন্ড একসঙ্গে ঢোকান, আবার Container বন্ধ হলেই ডেটা হারিয়ে অবাক হন। এই অধ্যায় সেই ভুলের মূল কেটে দেয়। আপনি যদি বোঝেন যে Container আসলে কী — Linux Kernel-এর Isolation সুবিধার ওপরে গড়া একটি প্রক্রিয়া-স্তরের (Process-level) আবরণ — তাহলে পরের অধ্যায়গুলোর Namespace, Cgroup, Image Layer, Lifecycle — সবকিছু নিজে নিজেই জায়গায় বসে যাবে। DevOps Engineer হিসেবে Architecture সিদ্ধান্ত নেওয়ার সময় এই বোঝাপড়াই আপনার সবচেয়ে বড় সম্পদ।

১.৩ পূর্বশর্ত জ্ঞান

Linux টার্মিনালে ফাইল দেখা, কমান্ড চালানো ও প্রসেস দেখার অভিজ্ঞতা (`ps`, `top`, `ls`), Operating System-এর প্রাথমিক ধারণা — Kernel, Process, Memory, File System — এবং React/Next.js, Node.js/Express ও MongoDB দিয়ে একটি MERN অ্যাপ চালানোর অভিজ্ঞতা থাকলেই চলবে। VirtualBox বা VMware-এ কখনো VM চালিয়ে থাকলে ভালো, না চালালেও এই অধ্যায়ে প্রতিটি ধারণা প্রথম নীতি থেকে পরিচয় করিয়ে দেওয়া হবে। Docker-এর পূর্ব অভিজ্ঞতা প্রয়োজন নেই।

১.৪ প্রথম নীতি থেকে ব্যাখ্যা

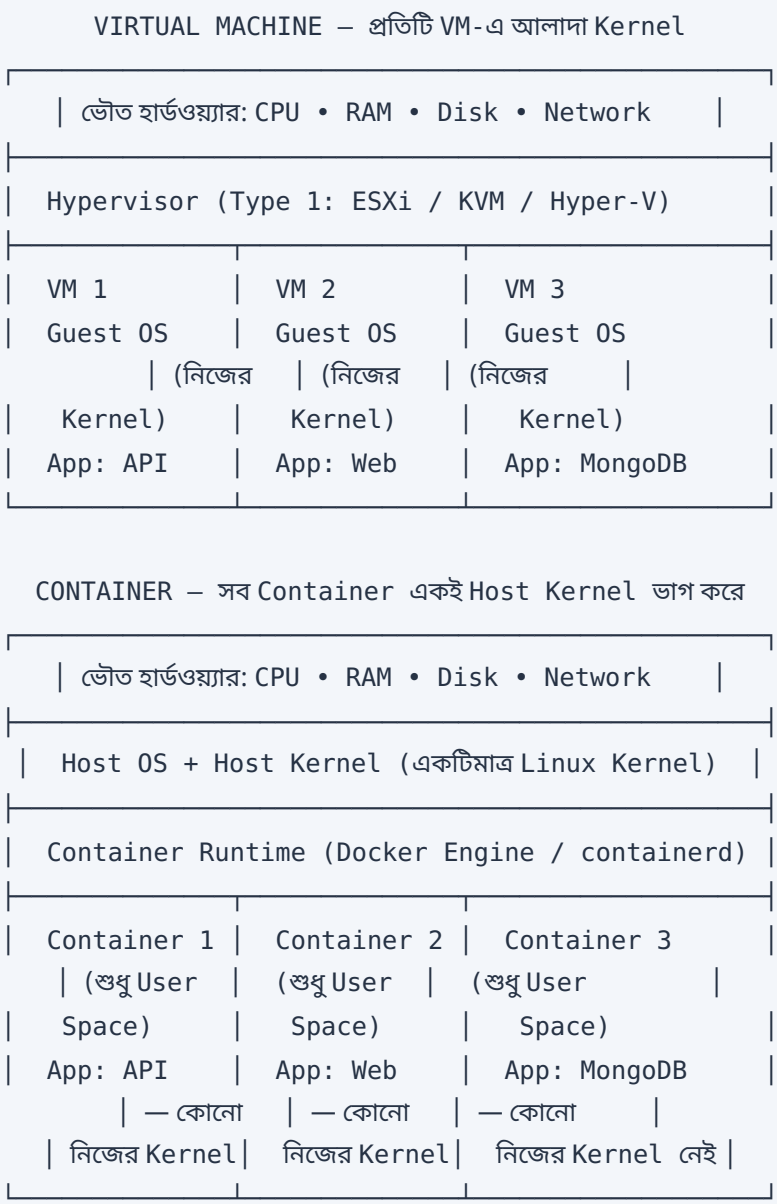
একটি কম্পিউটারে সফটওয়্যার চালাতে তিনটি জিনিস লাগে: হার্ডওয়্যার (CPU, RAM, Disk, Network), সেই হার্ডওয়্যার পরিচালনার জন্য একটি **Kernel** — Operating System-এর কেন্দ্রীয় অংশ, যা CPU-সময় ভাগ করে দেয়, মেমরি বরাদ্দ করে, ফাইল ও নেটওয়ার্ক পরিচালনা করে — এবং Kernel-এর ওপরে চলা **Process** — যেমন আপনার Node.js সার্ভার বা MongoDB। প্রশ্ন হলো: একই হার্ডওয়্যারে একাধিক আলাদা পরিবেশ চালাতে চাইলে কী করবেন? দুটি উত্তর আছে। প্রথম উত্তর — **Virtual Machine**: হার্ডওয়্যারটিকেই নকল করুন। Hypervisor নামের একটি সফটওয়্যার প্রতিটি VM-কে মনে করায় যে তার নিজের CPU, নিজের RAM, নিজের Disk আছে; প্রতিটি VM-এর ভেতরে আলাদা একটি পূর্ণ Operating System (Guest OS) চলে, যার নিজের Kernel আছে। দ্বিতীয় উত্তর — **Container**: হার্ডওয়্যার নকল না করে, Kernel-

এর Isolation ক্ষমতা ব্যবহার করুন। একটিমাত্র Host Kernel-এর ওপরে প্রতিটি Container একটি আলাদা Process-এর দল হিসেবে চলে — প্রতিটি Container মনে করে তার নিজের ফাইলসিস্টেম, নিজের নেটওয়ার্ক, নিজের Process-তালিকা আছে, কিন্তু আসলে তারা সবাই একই Kernel ভাগ করে নেয়। এই এক লাইনের পার্থক্য — "আলাদা Kernel বনাম ভাগ করা Kernel" — থেকেই VM ও Container-এর গতি, আকার, নিরাপত্তা ও ব্যবহারের সব পার্থক্য তৈরি হয়।

> **CONCEPT:** Container "হালকা VM" নয়। VM হলো হার্ডওয়্যার-স্তরের ভার্চুয়ালাইজেশন (Hardware Virtualization) — প্রতিটি VM-এর নিজের Kernel থাকে। Container হলো Operating System-স্তরের ভার্চুয়ালাইজেশন (OS-level Virtualization) — সব Container একই Host Kernel ভাগ করে। Container-কে VM ভাবলে আপনি এর নিরাপত্তা-সীমা ভুল বুঝবেন: Kernel-এর কোনো দুর্বলতা থাকলে তা সব Container-কে প্রভাবিত করতে পারে, যেখানে VM-এ Guest Kernel আলাদা থাকায় সেই ঝুঁকি কমে। এই বইয়ে আমরা এই ভুল ধারণাটি আর কখনো করব না।

Hypervisor বলতে সেই সফটওয়্যারকে বোঝায় যা ভৌত হার্ডওয়্যার ও VM-এর মাঝখানে বসে হার্ডওয়্যার ভাগ করে দেয়। **Type 1 Hypervisor** (Bare-Metal Hypervisor) — যেমন VMware ESXi, Microsoft Hyper-V, KVM — সরাসরি হার্ডওয়্যারের ওপরে চলে, কোনো Host OS লাগে না; এটি প্রোডাকশন ডেটা সেন্টারের মান। **Type 2 Hypervisor** (Hosted Hypervisor) — যেমন VirtualBox, VMware Workstation — একটি সাধারণ OS (Windows, macOS, Linux)-এর ওপরে অ্যাপ্লিকেশন হিসেবে চলে; এটি শেখা ও পরীক্ষার জন্য সুবিধাজনক। **Host OS** হলো ভৌত মেশিনে সরাসরি ইনস্টল করা OS, আর **Guest OS** হলো VM-এর ভেতরে চলা OS। Container-এর জগতে "Guest OS" বলে কিছু নেই — Container Image-এ থাকে শুধু অ্যাপ্লিকেশন ও তার লাইব্রেরি (User Space), Kernel থাকে না।

১.৫ অভ্যন্তরীণ আর্কিটেকচার ও চিত্র



উপরের চিত্রটিই এই অধ্যায়ের মূল কথা: VM-এ প্রতিটি বাক্সে একটি করে Kernel, Container-এ সব বাক্সে একটি ভাগ করা Kernel।

আরও একটি গুরুত্বপূর্ণ চিত্র — Resource Overhead: একটি সাধারণ Ubuntu VM চালু করতে 1–2 GB RAM ও কয়েক GB Disk লাগে শুধু Guest OS-এর জন্য, আর Boot করতে 30–60 সেকেন্ড লাগে। একটি Node.js Container চালু করতে মেমরি লাগে শুধু অ্যাপের যতটুকু দরকার (কয়েক দশ MB থেকে কয়েক শ MB), Disk-এ Image কয়েক শ MB, আর Start হতে লাগে এক সেকেন্ডেরও কম — কারণ নতুন Kernel Boot করতে হয় না, শুধু একটি Process শুরু হয়। এই গতির পার্থক্যই Container-কে CI/CD, Microservices ও দ্রুত Scale-এর জন্য উপযুক্ত করে তুলেছে।

১.৬ বিস্তারিত প্রযুক্তিগত ব্যাখ্যা

Hypervisor কীভাবে হার্ডওয়্যার ভাগ করে

CPU-তে একটি বিশেষ সুবিধা আছে — Hardware Virtualization Extension (Intel VT-x, AMD-V) — যা Hypervisor-কে Guest OS-কে সরাসরি CPU-তে চালাতে দেয়, কিন্তু প্রতিটি বিপজ্জনক নির্দেশ (যেমন হার্ডওয়্যার সরাসরি ছোঁয়া) আটকে দেয়। Hypervisor প্রতিটি VM-কে একটি **Virtual CPU (vCPU)**, ভার্চুয়াল RAM ও ভার্চুয়াল Disk দেয়; VM মনে করে এগুলোই আসল হার্ডওয়্যার। এই অনুকরণের (Emulation) খরচ আছে — CPU-তে 2–10%, I/O-তে আরও বেশি — তবে আধুনিক হার্ডওয়্যার-সহায়তায় তা অনেক কমে এসেছে। মেমরির ক্ষেত্রে Hypervisor একটি কৌশল ব্যবহার করে যাকে বলে Memory Overcommit — সব VM-এর বরাদ্দ মেমরির যোগফল ভৌত RAM-এর চেয়ে বেশি হতে পারে, কারণ সব VM একসঙ্গে সর্বোচ্চ মেমরি ব্যবহার করে না; কিন্তু সবাই একসঙ্গে চাইলে সিস্টেম ধীর হয়ে যায় বা VM বন্ধ হয়ে যায়।

Container কীভাবে Isolation পায়

Container কোনো হার্ডওয়্যার নকল করে না; বরং Linux Kernel-এর দুটি সুবিধা ব্যবহার করে — **Namespace** (কী দেখা যাবে তা আলাদা করা) ও **Control Group / Cgroup** (কতটুকু রিসোর্স ব্যবহার করা যাবে তা সীমা দেওয়া)। এগুলোর বিস্তারিত ব্যাখ্যা অধ্যায় ২-এ আসবে; এখানে শুধু মূল ধারণা: Namespace প্রতিটি Container-কে নিজের Process ID, নিজের Network Interface, নিজের Mount করা ফাইলসিস্টেম দেখায়, আর Cgroup বলে দেয় এই Container সর্বোচ্চ কত CPU ও RAM ব্যবহার করতে পারবে। এই দুটি মিলে একটি Process-কে মনে করায় যে সে একা একটি মেশিনে চলছে — যদিও আসলে সে Host-এর একটি সাধারণ Process মাত্র, যা `ps aux` চালালে Host-এ দেখা যায়।

বিষয়	Virtual Machine	Container
Isolation-এর স্তর	হার্ডওয়্যার স্তর; আলাদা Kernel	OS স্তর; ভাগ করা Kernel, Namespace + Cgroup
Boot সময়	30–60 সেকেন্ড (পূর্ণ OS Boot)	এক সেকেন্ডের কম (শুধু Process Start)
আকার	GB-এ মাপা হয় (Guest OS-সহ)	MB-এ মাপা হয় (শুধু অ্যাপ + লাইব্রেরি)
এক হোস্টে সংখ্যা	দশের ঘরে	শত শত, ক্ষেত্রবিশেষে হাজারের ঘরে
OS-এর স্বাধীনতা	প্রতিটি VM-এ আলাদা OS ও Kernel সংস্করণ সম্ভব; Linux হোস্টে Windows VM সম্ভব	সব Container একই Host Kernel ব্যবহার করে; Linux Container-এ Windows Kernel চলে না
নিরাপত্তা-সীমানা	শক্তিশালী — Kernel আলাদা	দুর্বলতর — Kernel ভাগ করা; Kernel দুর্বলতা সব Container-কে ছোঁয়
Portability	VM Image ভারী, সরাতে সময় লাগে	Image হালকা, Registry থেকে যেকোনো Docker Host-এ টানা যায়
সাধারণ ব্যবহার	আলাদা OS চাইলে, শক্ত Isolation চাইলে, পুরনো অ্যাপ চালাতে	Microservices, CI/CD, দ্রুত Scale, পরিবেশের সামঞ্জস্য

কেন Container VM-এর বিকল্প নয়, পরিপূরক

বাস্তব ডেটা সেন্টারে দুটিই একসঙ্গে থাকে: ভৌত সার্ভারে Hypervisor দিয়ে কয়েকটি VM, আর প্রতিটি VM-এর ভেতরে Docker দিয়ে অনেকগুলো Container। এই স্তরবিন্যাসকে বলে Defense in Depth — VM দেয় Kernel-স্তরের সীমানা,

Container দেয় দ্রুত Deploy ও ঘনত্ব (Density)। যেমন TechNova তাদের Production-এ ক্লাউড VM-এর ভেতরে Container চালায়: VM ভাড়া করা হয় ক্লাউড থেকে, Container-এ চলে API, Web ও Worker। কোনো গ্রাহকের ডেটা বিশেষভাবে সংবেদনশীল হলে সেই কাজটি আলাদা VM-এ রাখা হয়, যাতে Kernel ভাগ করার ঝুঁকি না থাকে।

Portability ও "It works on my machine" সমস্যা

VM-ও Portable — একটি VM Image অন্য হোস্টে নেওয়া যায় — কিন্তু তার আকার GB-এ মাপা হয় বলে তা ধীর। Container Image-এ শুধু অ্যাপ ও নির্ভরতা থাকে, তাই তা Registry থেকে সেকেন্ডে-মিনিটে টানা যায়, আর Development, Staging ও Production — তিন জায়গায় হুবহু একই Image চলে। এটাই Container-এর আসল জয়: পরিবেশের পার্থক্য দূর করা, VM-এর চেয়ে হালকা হওয়া নয়। "হালকা" বললে মানুষ নিরাপত্তা-সীমাটি ভুলে যায়; "একই Kernel ভাগ করে, কিন্তু আলাদা দেখায়" বললে সত্যটি মনে থাকে।

১.৭ Docker কমান্ড — লাইন ধরে ধরে ব্যাখ্যা

এই অধ্যায়ে আমরা এখনো Docker ইনস্টল করিনি, তবু দুটি প্রযুক্তির পার্থক্য নিজের চোখে দেখার মতো কমান্ড আছে। প্রথমে আপনার মেশিনটি VM না Container, তা বোঝার উপায় — এই কমান্ডগুলো Linux Host-এ চালানোর জন্য:

```
# 1. এই মেশিনে কোন Hypervisor আছে কি না দেখুন (VM হলে দেখা যাবে)
systemd-detect-virt
# আউটপুটের অর্থ: 'none' মানে ভৌত মেশিন বা Container; 'kvm'/'vmware' মানে VM-এর ভেতরে আছেন
# 'docker' বা 'lxc' মানে Container-এর ভেতরে আছেন — চিত্রণমূলক আউটপুট: docker

# 2. Kernel সংস্করণ দেখুন — Container ও Host একই Kernel দেখায়, VM আলাদা দেখায়
uname -r
# চিত্রণমূলক আউটপুট: 6.8.0-31-generic
# ব্যাখ্যা: Container-এর ভেতরে uname -r চালালে Host Kernel-এর সংস্করণই দেখাবে,
# কারণ Container-এর নিজের Kernel নেই। VM-এ Guest Kernel-এর সংস্করণ দেখাবে।

# 3. চলমান প্রসেস দেখুন — Host-এ Container-এর প্রসেস সাধারণ প্রসেস হিসেবে দেখা যায়
ps aux | head -n 5
# ব্যাখ্যা: ps = প্রসেস তালিকা; aux = সব ব্যবহারকারীর সব প্রসেস বিস্তারিত দেখাও;
# head -n 5 = প্রথম ৫ লাইন দেখাও। Docker Host-এ node সার্ভারটি এখানে দেখা যাবে।

# 4. Docker ইনস্টল থাকলে সংস্করণ ও তথ্য দেখুন (পরের অধ্যায়ে ইনস্টল করব)
docker version
# ব্যাখ্যা: Client ও Server (Daemon) — দুটি অংশের সংস্করণ আলাদাভাবে দেখায়;
# Server অংশ না চললে শুধু Client দেখাবে ও ত্রুটি জানাবে।

# 5. Docker থাকলে চলমান Container দেখুন — এটি অধ্যায় ৫-এর আগাম ঝলক
docker ps
# ব্যাখ্যা: ps = চলমান Container-এর তালিকা; প্রতিটি লাইনে Container ID, Image,
# চালু হওয়ার সময় ও Port দেখায়। কোনো Container না চললে তালিকা খালি থাকে।
```

মনে রাখবেন: `systemd-detect-virt` দিয়ে VM শনাক্ত করা যায়, কিন্তু Container শনাক্ত করার নির্ভরযোগ্য উপায় হলো `/.dockerenv` ফাইলের উপস্থিতি বা `/proc/1/cgroup` দেখা — অধ্যায় ২-এ আমরা এটি হাতে-কলমে করব।

১.৮ সম্পূর্ণ কনফিগারেশন উদাহরণ

একই MERN অ্যাপের দুটি স্থাপনার রূপরেখা — VM পদ্ধতি বনাম Container পদ্ধতি — পাশাপাশি রাখলে সিদ্ধান্তটি পরিষ্কার হয়। নিচের উদাহরণটি ধারণাগত; আসল Deploy অধ্যায় ৫ ও পরের Volume-এ আসবে:

```
# উদাহরণ A — VM পদ্ধতি: প্রতিটি স্তরে আলাদা VM (ধারণাগত রূপরেখা)
# VM 1: web-vm (203.0.113.10) — Ubuntu 24.04, Nginx + Next.js, 2 vCPU, 4 GB RAM
# VM 2: api-vm (203.0.113.11) — Ubuntu 24.04, Node.js + Express, 2 vCPU, 4 GB RAM
# VM 3: db-vm  (203.0.113.12) — Ubuntu 24.04, MongoDB, 4 vCPU, 8 GB RAM
# মোট: 3টি পূর্ণ OS, 3টি Kernel, 16 GB RAM শুধু OS-সহ বরাদ্দ
# সুবিধা: প্রতিটি স্তর Kernel-স্তরে আলাদা; অসুবিধা: ভারী, ধীর, ব্যয়বহুল

# উদাহরণ B — Container পদ্ধতি: একটি VM-এর ভেতরে তিনটি Container
# Host VM: docker-host (203.0.113.20) — Ubuntu 24.04, Docker Engine, 4 vCPU, 8 GB RAM
#   Container: technova-web  — Next.js (Image: technova-web:1.0.0), Port 3000
#   Container: technova-api  — Express (Image: technova-api:1.0.0), Port 4000
#   Container: technova-db   — MongoDB (Image: mongo:7), Port 27017, Volume-এ ডেটা
# মোট: 1টি OS, 1টি Kernel, 3টি Container — একই RAM-এ অনেক বেশি ঘনত্ব
# সুবিধা: দ্রুত Start, একই Image সব পরিবেশে; ঝুঁকি: Kernel ভাগ করা
# শিক্ষা: প্রোডাকশনে প্রায়ই A ও B একসঙ্গে থাকে — VM দেয় সীমানা, Container দেয় গতি
```

লক্ষ্য করুন উদাহরণ B-তে MongoDB Container-এর ডেটা Volume-এ রাখা হয়েছে — কারণ Container মুছে গেলে তার ভেতরের ফাইল মুছে যায়। Volume ও ডেটার স্থায়িত্ব (Persistence) নিয়ে বিস্তারিত আলোচনা অধ্যায় ৪-এ আসবে।

১.৯ বাস্তব প্রোডাকশন পরিস্থিতি

> **REAL WORLD — TechNova:** TechNova-র ই-কমার্স অ্যাপ app.technova.example ঈদের বিক্রির সময় হঠাৎ ট্রাফিক তিন গুণ হয়ে গেল। VM পদ্ধতিতে নতুন VM তৈরি করে OS ইনস্টল, অ্যাপ বসানো ও কনফিগার করতে ২০–৩০ মিনিট লাগত — ততক্ষণে গ্রাহক চলে যেতেন। Container পদ্ধতিতে একই API Image থেকে আরও পাঁচটি Container চালু করতে লাগল ৩০ সেকেন্ডেরও কম। তবে দল একটি ভুলও করেছিল: তারা ভেবেছিল Container মানেই VM-এর মতো নিরাপত্তা, তাই একটি Container-কে `--privileged` দিয়ে চালিয়েছিল — সেই Container Host-এর সব ডিভাইস দেখতে পেত। নিরাপত্তা-পর্যালোচনায় ধরা পড়ার পর তারা Privileged Mode তুলে দিয়ে শুধু প্রয়োজনীয় Capability দিতে শুরু করে (অধ্যায় ২-এ বিস্তারিত)। শিক্ষা: গতি পেতে গিয়ে নিরাপত্তা-সীমা ভুলে গেলে চলবে না।

আরেকটি বাস্তব দৃশ্য — Legacy অ্যাপ: TechNova-র একটি পুরনো অ্যাপ শুধু Windows Server 2012-এ চলে। Linux Container-এ তা চালানো অসম্ভব, কারণ Kernel আলাদা। সেখানে VM-ই একমাত্র পথ: Windows Guest OS-সহ একটি VM। বিপরীতে, নতুন MERN অ্যাপটি Linux Container-এ চলে চমৎকারভাবে। সঠিক সিদ্ধান্ত হলো "কোনটি ভালো" নয় — "কোন কাজে কোনটি" তা বোঝা।

১.১০ ধাপে ধাপে ব্যবহারিক বাস্তবায়ন

উদাহরণ: আপনার ল্যাপটপে VM ও Container-এর পার্থক্য নিজে মাপা

1. আপনার ল্যাপটপে `systemd-detect-virt` ও `uname -r` চালিয়ে দেখুন — আপনি ভৌত মেশিনে, VM-এ, নাকি Container-এ আছেন।
2. VirtualBox থাকলে একটি Ubuntu VM তৈরি করুন ও তার Boot সময় ঘড়ি দিয়ে মাপুন; পাশাপাশি Docker থাকলে একটি `hello-world` Container-এর Start সময় মাপুন (অধ্যায় ৩-এ শিখবেন)।
3. দুটির মেমরি ব্যবহার তুলনা করুন: VM-এর জন্য বরাদ্দ RAM লিখুন, আর Container-এর জন্য `docker stats`-এর মেমরি লিখুন (অধ্যায় ৫-এ শিখবেন)।

4. একটি কাগজে TechNova-র MERN অ্যাপের তিনটি স্তর (Web, API, MongoDB) আঁকুন — একবার তিনটি VM হিসেবে, একবার একটি Host-এ তিনটি Container হিসেবে — এবং প্রতিটি ক্ষেত্রে মোট RAM ও Boot সময়ের হিসাব লিখুন।
5. সিদ্ধান্ত লিখুন: কোন স্তরটি আলাদা VM চায় (যেমন সংবেদনশীল ডেটাবেস) আর কোনটি Container-এ যথেষ্ট — এবং কেন।

১.১১ সাধারণ ভুল ও ভুল ধারণা

TROUBLESHOOTING: "Container চালু করলাম, কিন্তু ভেতরে `systemctl` চলে না কেন?" — কারণ Container-এ কোনো Init System বা পূর্ণ OS নেই; Container-এর ভেতরে চলে শুধু আপনার অ্যাপের Process। `systemctl` চালাতে চাওয়া মানেই Container-কে VM ভাবা। সমাধান: Container-এর ভেতরে সার্ভিস পরিচালনা করবেন না — Container-কেই একটি সার্ভিস হিসেবে পরিচালনা করুন (Start/Stop/Restart), যা অধ্যায় ৫-এ শিখবেন।

- **ভুল ধারণা:** "Container হলো হালকা VM।" — না; Container-এর নিজের Kernel নেই, সে Host Kernel ভাগ করে। এর নিরাপত্তা-অর্থ সম্পূর্ণ আলাদা।
- **ভুল ধারণা:** "Container মানেই নিরাপদ, কারণ তা আলাদা।" — না; Namespace ও Cgroup Isolation শক্তিশালী, কিন্তু Kernel-স্তরের নয়। Kernel দুর্বলতা (Kernel Vulnerability) সব Container-কে ছুঁতে পারে।
- **ভুল:** একটি Container-এ অনেকগুলো সার্ভিস চালানো (Web + API + DB একসঙ্গে) — এতে লগ, Restart ও Scale — সবকিছু জটিল হয়। নীতি: একটি Container, একটি মূল Process।
- **ভুল:** Container-এর ভেতরে গুরুত্বপূর্ণ ডেটা রাখা — Container মুছলে ডেটা মুছে যায়। ডেটার জন্য Volume ব্যবহার করুন।
- **ভুল ধারণা:** "VM ধীর মানেই পুরনো প্রযুক্তি, সবসময় Container ব্যবহার করব।" — আলাদা Kernel বা আলাদা OS দরকার হলে VM-ই সঠিক পছন্দ; দুটি প্রযুক্তি পরিপূরক।
- **ভুল:** Container-এ SSH Server চালিয়ে ভেতরে ঢুকে কাজ করা — এটি VM-এর অভ্যাস। Container-এ ঢুকতে `docker exec` ব্যবহার করুন, আর সমস্যা হলে Container নতুন করে তৈরি করুন।

১.১২ নিরাপত্তা বিবেচনা

WARNING: VM ও Container-এর নিরাপত্তা-সীমানা এক নয় — এই পার্থক্য না বুঝলে সবচেয়ে বড় ঝুঁকি তৈরি হয়। VM থেকে বেরিয়ে Host-এ যাওয়া (VM Escape) অত্যন্ত কঠিন, কারণ Hypervisor ও Guest Kernel — দুটি স্তর ভাঙতে হয়। Container থেকে বেরিয়ে Host-এ যাওয়া (Container Escape) তুলনামূলকভাবে সহজ, কারণ একটিমাত্র Kernel ভাগ করা হয় — Kernel-এর একটি দুর্বলতাই যথেষ্ট। তাই কখনোই অবিশ্বস্ত (Untrusted) কোডকে Privileged Container-এ চালাবেন না, Host-এর `/` ডিরেক্টরি Container-এ Mount করবেন না, আর Docker Socket (`/var/run/docker.sock`) Container-এর ভেতরে দেওয়া মানে Container-কে Host-এর Docker-এর চাবি দিয়ে দেওয়া — এটি প্রোডাকশনে প্রায় সবসময় নিষিদ্ধ।

নিরাপত্তার আরেকটি দিক — আক্রমণের পৃষ্ঠ (Attack Surface): VM-এ প্রতিটি Guest OS-এর নিজের প্যাচ, নিজের সার্ভিস থাকে; Container-এ Host Kernel-এর প্যাচই সব Container-কে রক্ষা করে — Host Kernel আপডেট না করলে

সব Container ঝুঁকিতে থাকে। আবার Container Image-এ পুরনো লাইব্রেরি থাকলে সেই দুর্বলতা Image-এর সঙ্গে সব পরিবেশে ছড়িয়ে পড়ে। তাই Image Scan করা ও Host Kernel নিয়মিত আপডেট করা — দুটিই বাধ্যতামূলক, যার বিস্তারিত অধ্যায় ২ ও ৪-এ আসবে।

১.১৩ কর্মক্ষমতা ও রিসোর্স বিবেচনা

VM-এর Overhead আসে তিন জায়গা থেকে: Guest OS-এর মেমরি ও CPU, Hypervisor-এর অনুকরণের খরচ, আর Disk I/O-এর অতিরিক্ত স্তর। Container-এর Overhead প্রায় শূন্যের কাছাকাছি — অ্যাপটি Host-এ সরাসরি চলে, শুধু Namespace ও Cgroup-এর সামান্য খরচ যোগ হয়। তবে একটি সতর্কতা: Container-এ সীমা (Limit) না দিলে একটি Container পুরো Host-এর CPU ও RAM খেয়ে ফেলতে পারে — VM-এ যেমন বরাদ্দ নির্দিষ্ট থাকে, Container-এ তা ডিফল্টভাবে থাকে না। তাই প্রোডাকশনে প্রতিটি Container-এ CPU ও Memory Limit দেওয়া বাধ্যতামূলক (অধ্যায় ৫-এ `--memory` ও `--cpus` শিখবেন)। ঘনত্বের (Density) দিক থেকে: একই হার্ডওয়্যারে VM চলে দশের ঘরে, Container চলে শত শত — এ কারণেই ক্লাউড বিল কমাতে Container জনপ্রিয়, তবে ঘনত্ব বাড়লে "Noisy Neighbour" সমস্যা বাড়ে — একটি Container বেশি I/O করলে পাশের Container ধীর হয়।

১.১৪ সমস্যা সমাধান (Troubleshooting)

> **TROUBLESHOOTING:** "আমার অ্যাপ VM-এ চলে, Container-এ চলে না" — প্রথমে দেখুন Kernel-নির্ভর কিছু ব্যবহার করছেন কি না (যেমন বিশেষ Kernel Module, `iptables`-এর গভীর ব্যবহার, বা নির্দিষ্ট ফাইলসিস্টেম)। Container Host Kernel ভাগ করে, তাই Host Kernel-এ যা নেই, Container-এও তা নেই। দ্বিতীয়ত, দেখুন অ্যাপটি কি Host-এর কোনো ফাইলের ওপর নির্ভরশীল যা Container Image-এ নেই। তৃতীয়ত, নেটওয়ার্ক: VM-এর নিজের IP থাকলেও Container-এর Port Host-এ Publish করতে হয় — অধ্যায় ৩-এ `-p` ফ্ল্যাগ শিখবেন।

1. **লক্ষণ:** VM ধীর — **পরীক্ষা:** Host-এ CPU Overcommit আছে কি না দেখুন (`top`, Hypervisor-এর পরিসংখ্যান)। **সমাধান:** vCPU কমান বা Host-এ VM কমান।
2. **লক্ষণ:** Container থেকে Host-এর ফাইল দেখা যাচ্ছে না — **পরীক্ষা:** Volume Mount করেছেন কি না। **সমাধান:** Container আলাদা ফাইলসিস্টেম দেখে — এটাই স্বাভাবিক; ফাইল ভাগ করতে Volume লাগে।
3. **লক্ষণ:** "এই Container-এ `ping` চলে না" — **পরীক্ষা:** Image-এ `ping` টুল আদৌ আছে কি না; অনেক ছোট Image-এ (যেমন Alpine) অনেক টুল থাকে না। **সমাধান:** টুলের অনুপস্থিতিকে Container-এর দোষ ভাববেন না — Image-এ কী আছে তা `docker exec` দিয়ে দেখুন।
4. **লক্ষণ:** একই অ্যাপ দুই হোস্টে ভিন্ন আচরণ করে — **পরীক্ষা:** Kernel সংস্করণ ও Docker সংস্করণ দুই হোস্টে এক কি না। **সমাধান:** Container Image এক হলেও Host Kernel ভিন্ন হলে আচরণ ভিন্ন হতে পারে — এটি VM-এর ক্ষেত্রে ঘটে না।

১.১৫ সেরা অনুশীলন ও অ্যান্টি-প্যাটার্ন

সেরা অনুশীলন: প্রতিটি কাজের জন্য সঠিক স্তর বেছে নিন — আলাদা Kernel বা OS দরকার হলে VM, দ্রুত Deploy ও ঘনত্ব দরকার হলে Container; প্রোডাকশনে VM ও Container একসঙ্গে স্তরে স্তরে সাজান; Container-এ সবসময় রিসোর্স সীমা দিন; Host Kernel নিয়মিত আপডেট করুন। **অ্যান্টি-প্যাটার্ন:** Container-কে VM-এর মতো ব্যবহার করা — ভেতরে SSH, একাধিক সার্ভিস, হাতে কনফিগারেশন পরিবর্তন; "সবকিছু Container-এ চালাব" — ডেটাবেসের মতো State-যুক্ত কাজেও না ভেবে Container বেছে নেওয়া; নিরাপত্তা-সীমা না বুঝে Privileged Container চালানো; VM-এর মতো

Container-কে "পোষা প্রাণী" (Pet) ভাবা — Container হলো "গবাদি পশু" (Cattle): যেকোনো একটিকে মুছে নতুনটি তৈরি করা যায়, কোনোটিরই বিশেষ যত্ন নেওয়া হয় না।

১.১৬ অধ্যায়ের সারাংশ

VM হলো হার্ডওয়্যার-স্তরের ভার্চুয়ালাইজেশন — প্রতিটি VM-এর নিজের Guest OS ও Kernel থাকে, Hypervisor হার্ডওয়্যার ভাগ করে দেয়। Container হলো OS-স্তরের ভার্চুয়ালাইজেশন — সব Container একই Host Kernel ভাগ করে, Namespace ও Cgroup দিয়ে আলাদা দেখায়। VM শক্ত Isolation দেয় কিন্তু ভারী ও ধীর; Container দ্রুত, হালকা ও Portable, কিন্তু Kernel ভাগ করার কারণে নিরাপত্তা-সীমানা দুর্বলতর। Container "হালকা VM" নয় — এই সত্যটি বুঝলেই Docker-এর বাকি সব ধারণা সহজ হয়। বাস্তব প্রোডাকশনে দুটিই একসঙ্গে ব্যবহৃত হয়: VM দেয় সীমানা, Container দেয় গতি।

১.১৭ মূল প্রযুক্তিগত পরিভাষা

Virtual Machine (VM): হার্ডওয়্যারের ভার্চুয়াল কপি — যার ভেতরে আলাদা Guest OS ও Kernel চলে।

Hypervisor: ভৌত হার্ডওয়্যার ভাগ করে একাধিক VM চালানোর সফটওয়্যার; Type 1 সরাসরি হার্ডওয়্যারে চলে, Type 2 একটি Host OS-এর ওপরে চলে।

Host OS / Guest OS: ভৌত মেশিনের OS হলো Host OS; VM-এর ভেতরের OS হলো Guest OS।

Kernel: Operating System-এর কেন্দ্র — CPU, মেমরি, ফাইল ও নেটওয়ার্ক পরিচালনা করে।

Container: একই Host Kernel ভাগ করে চলা, Namespace ও Cgroup দিয়ে আলাদা করা Process-এর দল; নিজের Kernel থাকে না।

Namespace: Linux Kernel-এর সুবিধা — প্রতিটি Process-কে আলাদা দৃশ্য (Process, Network, File) দেখায়।

Cgroup (Control Group): Linux Kernel-এর সুবিধা — Process-দলের CPU, মেমরি ও I/O ব্যবহার সীমা দেয়।

User Space: Kernel-এর বাইরে অ্যাপ্লিকেশন ও লাইব্রেরি চলার জায়গা — Container Image-এ শুধু এটিই থাকে।

Density: একই হার্ডওয়্যারে কতগুলো VM বা Container চালানো যায় তার পরিমাপ।

Container Escape / VM Escape: আলাদা পরিবেশ থেকে বেরিয়ে Host-এ যাওয়ার আক্রমণ; Container Escape তুলনামূলকভাবে সহজ, কারণ Kernel ভাগ করা হয়।

১.১৮ জ্ঞান যাচাই — প্রশ্ন

1. VM ও Container-এর মূল পার্থক্যটি এক বাক্যে লিখুন — Kernel-এর ভূমিকা উল্লেখ করে।
2. Type 1 ও Type 2 Hypervisor-এর পার্থক্য কী? প্রতিটির একটি উদাহরণ দিন।
3. কেন Container-এর ভেতরে `uname -r` Host Kernel-এর সংস্করণ দেখায়?
4. একটি Linux Container-এ Windows অ্যাপ চালানো যায় না কেন?
5. Container দ্রুত Start হয় কেন, আর VM ধীর কেন — ভেতরের কারণ ব্যাখ্যা করুন।
6. "Container হালকা VM" — এই বাক্যটি কেন ভুল ও বিপজ্জনক, নিরাপত্তার দিক থেকে ব্যাখ্যা করুন।
7. কখন VM বেছে নেবেন আর কখন Container — দুটি করে বাস্তব পরিস্থিতি দিন।

১.১৯ ব্যবহারিক অনুশীলন

1. **শনাক্তকরণ:** আপনার মেশিনে `systemd-detect-virt`, `uname -r` ও `ps aux | head` চালিয়ে লিখুন — আপনি কোথায় আছেন এবং কীভাবে বুঝলেন।
2. **তুলনা সারণি:** VM বনাম Container — Boot সময়, আকার, Isolation, নিরাপত্তা, খরচ — এই পাঁচটি বিষয়ে নিজের ভাষায় একটি সারণি বানান।
3. **স্থাপনা নকশা:** TechNova-র MERN অ্যাপের জন্য একটি স্থাপনা-নকশা আঁকুন যেখানে VM ও Container দুটিই আছে — কোন স্তরে কোনটি, এবং কেন, তা লিখুন।

অধ্যায় 2 • PART I — Container Foundations

Chapter 2 — Containerization and Linux Isolation Fundamentals

২.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: (১) Containerization বলতে ঠিক কী বোঝায় তা প্রথম নীতি থেকে ব্যাখ্যা করতে; (২) সাতটি Linux Namespace — PID, Network, Mount, UTS, IPC, User ও Cgroup — প্রতিটির কাজ আলাদাভাবে বলতে; (৩) Cgroup v1 ও v2-এর পার্থক্য ও CPU/Memory/IO সীমা ব্যাখ্যা করতে; (৪) Linux Capabilities, Seccomp, AppArmor ও SELinux কীভাবে Container-কে বেঁধে রাখে তা বোঝাতে; (৫) OCI Specification (Runtime, Image, Distribution) কী এবং কেন এটি Docker-কে Portable করে তা বলতে; (৬) একটি Container-এর ভেতরে ঢুকে Namespace ও Cgroup নিজে পরীক্ষা করতে।

২.২ কেন এই বিষয়টি DevOps-এ গুরুত্বপূর্ণ

Docker কমান্ড শেখা সহজ, কিন্তু Container কেন আলাদা থাকে, কেন একটি Container Host-এর ফাইল দেখতে পায় না, কেন একটি Container পুরো CPU খেয়ে ফেলতে পারে না — এই "কেন"-এর উত্তর না জানলে আপনি প্রোডাকশনে অন্ধের মতো কাজ করবেন। Namespace না বুঝলে আপনি বুঝবেন না কেন দুটি Container-এর একই Port (যেমন 3000) নিয়ে সংঘাত হয় না; Cgroup না বুঝলে বুঝবেন না কেন `--memory` সীমা দেওয়া সত্ত্বেও অ্যাপ OOM-Killed হয়; Capability না বুঝলে বুঝবেন না কেন Root হয়েও Container-এর ভেতরে কিছু কাজ করা যায় না। সবচেয়ে বড় কথা — এই অধ্যায়ই সেই জায়গা যেখানে "Container হালকা VM নয়" কথাটি প্রমাণিত হয়: Container-এর সব Isolation আসে Host Kernel থেকে ধার করা সুবিধা দিয়ে, নিজের কোনো Kernel দিয়ে নয়। এই অধ্যায়টি এই বইয়ের সবচেয়ে গভীর ভিত্তি — এখানে সময় দিলে পরের প্রতিটি অধ্যায় অর্ধেক পরিশ্রমে বোঝা যাবে।

২.৩ পূর্বশর্ত জ্ঞান

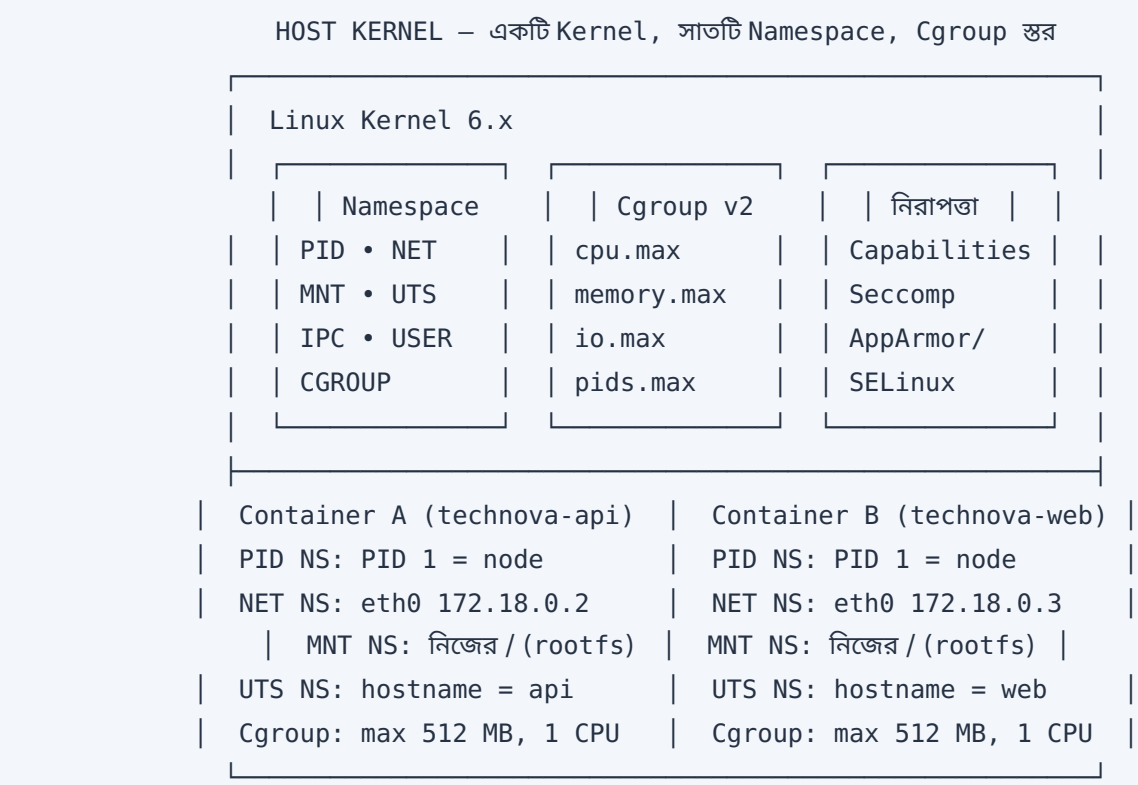
অধ্যায় ১-এর VM বনাম Container পার্থক্য, Linux Process-এর ধারণা (PID, Parent-Child), ফাইলসিস্টেম Mount-এর প্রাথমিক ধারণা, নেটওয়ার্ক Interface ও Port-এর ধারণা, এবং `ps`, `ls /proc`, `cat` কমান্ড চালানোর অভিজ্ঞতা প্রয়োজন। Kernel Module বা System Call-এর গভীর জ্ঞান প্রয়োজন নেই — প্রতিটি শব্দ প্রথম ব্যবহারে ব্যাখ্যা করা হবে।

২.৪ প্রথম নীতি থেকে ব্যাখ্যা

একটি Linux Process আসলে কী দেখে? সে দেখে একটি Process-তালিকা (কারা চলছে), একটি ফাইলসিস্টেম (কোন ফাইল কোথায়), একটি নেটওয়ার্ক (কোন IP, কোন Port), একটি Hostname, আর অন্য Process-এর সঙ্গে কথা বলার উপায়। স্বাভাবিকভাবে সব Process একই জিনিস দেখে — এটাই "একটি কম্পিউটার" অনুভূতির উৎস। **Containerization** মানে এই দৃশ্যটিকে ভেঙে দেওয়া: Kernel-কে বলা, "এই Process-দলকে আলাদা দৃশ্য দেখাও।" যে কৌশলে দৃশ্য আলাদা করা হয় তার নাম **Namespace** — প্রতিটি Namespace একটি নির্দিষ্ট জিনিসের আলাদা সংস্করণ দেয়। আর যে কৌশলে রিসোর্সের ব্যবহার মাপা ও সীমা দেওয়া হয় তার নাম **Cgroup**। একটি Container আসলে এই দুটি কৌশলের যোগফল, সঙ্গে আরও কয়েকটি নিরাপত্তা-স্তর: একটি Process-দল + তাদের জন্য আলাদা Namespace-সমূহ + তাদের জন্য Cgroup সীমা + একটি Root Filesystem (Image থেকে আসা) + কিছু নিরাপত্তা-নিয়ম (Capabilities, Seccomp, AppArmor/SELinux)। কোনো নতুন Kernel নেই, কোনো হার্ডওয়্যার অনুকরণ নেই — শুধু Kernel-এরই সুবিধা দিয়ে একটি Process-কে বিশ্বাস করানো যে সে একা একটি মেশিনে আছে।

CONCEPT: Namespace বলে "তুমি কী দেখতে পাবে", Cgroup বলে "তুমি কতটুকু ব্যবহার করতে পারবে"। Namespace ছাড়া Container সবকিছু দেখতে পেত — Host-এর সব Process, সব নেটওয়ার্ক। Cgroup ছাড়া Container সব রিসোর্স খেয়ে ফেলতে পারত। দুটিকে একসঙ্গে না বুঝলে Container-কে বোঝা হয় না। একটি উদাহরণ: PID Namespace-এ Container-এর ভেতরে আপনার Node.js অ্যাপের PID হয় 1 — সে মনে করে সে প্রথম Process — কিন্তু Host-এ তার আসল PID হয়তো 45231। দুটি সংখ্যাই সত্য, দুটি আলাদা দৃশ্যে।

২.৫ অভ্যন্তরীণ আর্কিটেকচার ও চিত্র



Host-এর ps-এ: node প্রসেসের PID 45231 ও 45287 — Namespace-এর বাইরে সব দেখা যায়।

মনে রাখবেন: Namespace একটি দৃশ্য (View), Cgroup একটি সীমা (Limit), আর Seccomp/Capabilities হলো অনুমতি (Permission)।

২.৬ বিস্তারিত প্রযুক্তিগত ব্যাখ্যা

সাতটি Namespace — এক এক করে

Namespace	যা আলাদা করে	Container-এর ভেতরে যা দেখা যায়	না থাকলে কী হতো
PID	Process ID ও Process-তালিকা	নিজের অ্যাপ PID 1; শুধু নিজের Process দেখা যায়	Host-এর সব Process দেখা যেত, অন্যদের Kill করা যেত

Network (NET)	Network Interface, IP, Port, Routing	নিজের eth0, নিজের IP (যেমন 172.18.0.2), নিজের Port 3000	দুটি Container একই Port নিয়ে সংঘাত করত
Mount (MNT)	Mount করা ফাইলসিস্টেমের দৃশ্য	নিজের Root Filesystem `/`; Host-এর `/` দেখা যায় না	Host-এর সব ফাইল দেখা ও বদলানো যেত
UTS	Hostname ও Domain Name	নিজের Hostname (যেমন `technova-api`)	সব Container একই Hostname দেখত
IPC	Inter-Process Communication — Shared Memory, Semaphore, Message Queue	শুধু নিজের Container-এর Process-এর সঙ্গে Shared Memory	এক Container অন্য Container-এর Shared Memory পড়তে পারত
User	User ID ও Group ID-র মানচিত্র	ভেতরে UID 0 (Root) হলেও Host-এ UID 100000-এর মতো সাধারণ ব্যবহারকারী	ভেতরের Root মানেই Host-এর Root হতো — ভয়ংকর ঝুঁকি
Cgroup	Cgroup-এর দৃশ্য (`/sys/fs/cgroup`)	নিজের সীমা ও ব্যবহার দেখা যায়	অন্য Container-এর সীমা দেখা যেত

PID Namespace গভীরে: Container-এর প্রথম Process (PID 1) একটি বিশেষ দায়িত্ব পায় — Orphan Process-কে দত্তক নেওয়া (Reaping) ও Signal গ্রহণ করা। আপনার Node.js অ্যাপ PID 1 হিসেবে চললে সে যদি Zombie Process পরিষ্কার না করে, Container-এর ভেতরে Zombie জমতে থাকে। এ কারণে প্রোডাকশনে অনেক সময় `--init` ফ্ল্যাগ দিয়ে একটি ছোট Init Process (tini) PID 1 হিসেবে বসানো হয়, যা Signal ও Zombie সামলায় — অধ্যায় ৫-এ বিস্তারিত। **Network Namespace গভীরে:** প্রতিটি Container পায় একটি Virtual Ethernet জোড়া (veth pair) — এক প্রান্ত Container-এর ভেতরে `eth0`, অন্য প্রান্ত Host-এর Bridge-এ যুক্ত। প্যাকেট Container থেকে বেরিয়ে Bridge হয়ে Host-এর নেটওয়ার্কে যায়; বাইরে থেকে আসা প্যাকেট Port Mapping (`-p 8080:3000`) দিয়ে ভেতরে ঢোকে। **Mount Namespace গভীরে:** Container-এর Root Filesystem আসে Image-এর Layer থেকে (অধ্যায় ৪), আর Host-এর ডিরেক্টরি শুধু তখনই দেখা যায় যখন আপনি ইচ্ছাকৃতভাবে Volume Mount করেন (`-v /host/path:/container/path`)।

Cgroup — রিসোর্সের সীমা ও হিসাব

Cgroup (Control Group) Kernel-এর হিসাবরক্ষক ও প্রহরী: প্রতিটি Process-দল কোন Cgroup-এ আছে তা দেখে Kernel ঠিক করে তারা কত CPU, কত RAM, কত Disk I/O ব্যবহার করতে পারবে। **Cgroup v1**-এ প্রতিটি রিসোর্সের জন্য আলাদা Hierarchy ছিল (cpu, memory, blkio আলাদা আলাদা গাছ); **Cgroup v2** (আধুনিক মান, Ubuntu 24.04-এ ডিফল্ট) — একটিমাত্র Unified Hierarchy, সহজ ও নির্ভরযোগ্য নিয়ম। মূল নিয়ন্ত্রণগুলো: `cpu.max` (যেমন `100000 100000` মানে 1 CPU), `memory.max` (যেমন 512 MB), `memory.swap.max`, `io.max` (Disk Read/Write সীমা), `pids.max` (সর্বোচ্চ Process সংখ্যা — Fork Bomb আটকায়)। মেমরি সীমা ছাড়ালে Kernel OOM Killer (Out-Of-Memory Killer) সেই Cgroup-এর Process-কে Kill করে — এটাই Container হঠাৎ বন্ধ হয়ে যাওয়ার সবচেয়ে সাধারণ কারণ, যার লক্ষণ অধ্যায় ৫-এ শিখবেন। CPU সীমা ছাড়ালে Process Kill হয় না — শুধু Throttle হয় (ধীর হয়ে যায়), তাই CPU সীমা "নরম" আর মেমরি সীমা "কঠোর"।

Capabilities — Root-কে ভেঙে ছোট করা

ঐতিহ্যগত Linux-এ Root (UID 0) মানে সব ক্ষমতা — যেকোনো ফাইল পড়া, যেকোনো Port খোলা, Kernel-এর সেটিং বদলানো। **Linux Capabilities** এই "সব বা কিছুই না" ক্ষমতাকে ছোট ছোট টুকরোয় ভেঙে দেয়:

`CAP_NET_BIND_SERVICE` (1024-এর নিচের Port খোলা), `CAP_SYS_ADMIN` (অনেক প্রশাসনিক কাজ — সবচেয়ে বিপজ্জনক), `CAP_CHOWN` (ফাইলের মালিক বদলানো), `CAP_NET_RAW` (Raw Packet পাঠানো) ইত্যাদি — মোট 40-এর বেশি। Docker ডিফল্টভাবে Container-কে শুধু প্রয়োজনীয় কয়েকটি Capability দেয়, `CAP_SYS_ADMIN`-সহ বিপজ্জনকগুলো কেড়ে নেয় — তাই Container-এর ভেতরে Root হয়েও আপনি `mount` করতে পারেন না বা Host-এর Kernel সেটিং বদলাতে পারেন না। `--cap-add` দিয়ে Capability যোগ করা যায়, `--cap-drop` দিয়ে আরও কাড়া যায়, আর `--privileged` দিলে সব Capability ফিরে আসে — প্রোডাকশনে এটি প্রায় সবসময় ভুল।

Seccomp, AppArmor ও SELinux — তিনটি নিরাপত্তা-স্তর

Seccomp (Secure Computing Mode) হলো System Call ফিল্টার: Process Kernel-কে কোন System Call (যেমন `open`, `mount`, `reboot`) করতে পারবে তা একটি Allowlist দিয়ে বেঁধে দেয়। Docker-এর ডিফল্ট Seccomp Profile 300-এর বেশি নিরাপদ System Call অনুমোদন করে, কিন্তু `mount`, `reboot`, `ptrace`-এর মতো বিপজ্জনকগুলো আটকায়। কোনো অ্যাপ অস্বাভাবিক System Call চাইলে Seccomp তা আটকে দেয় — তখন অ্যাপ ক্র্যাশ করতে পারে, আর লগে `Operation not permitted` দেখা যায়। **AppArmor** (Ubuntu-এর মান) ও **SELinux** (RHEL-এর মান) হলো Mandatory Access Control (MAC) — তারা ফাইল, নেটওয়ার্ক ও Capability-এর ওপরে নিয়মের একটি স্তর বসায়: কোন Process কোন ফাইল পড়তে পারবে, কোন Port খুলতে পারবে। Docker ডিফল্টভাবে Container-এ `docker-default` AppArmor Profile লাগায়। এই তিনটি স্তর একসঙ্গে কাজ করে: Capability বলে "কী ক্ষমতা আছে", Seccomp বলে "Kernel-কে কী বলতে পারবে", AppArmor/SELinux বলে "কোন ফাইলে হাত দিতে পারবে"।

OCI Specification — কেন Docker Portable

OCI (Open Container Initiative) হলো Container-এর মান-সংস্থা — Docker, Kubernetes ও অন্যরা মিলে ঠিক করেছে Container কেমন হবে। তিনটি মূল Spec: **Runtime Specification** — Container কীভাবে চালাতে হয় (Config ফাইল `config.json`-এর গঠন, Namespace ও Cgroup কীভাবে বসাতে হয়); **Image Specification** — Image কীভাবে সাজানো হয় (Layer, Manifest, Config); **Distribution Specification** — Registry থেকে Image কীভাবে টানা ও পাঠানো হয়। এই মানের কারণেই Docker দিয়ে তৈরি Image containerd, Podman বা Kubernetes-এও চলে — Runtime বদলালেও Image একই থাকে। Docker Engine-এর ভেতরে আসলে containerd (High-level Runtime) ও runc (Low-level Runtime, OCI Runtime) কাজ করে — অধ্যায় ৩-এ এই স্তরগুলো বিস্তারিত আসবে।

২.৭ Docker কমান্ড — লাইন ধরে ধরে ব্যাখ্যা

Namespace ও Cgroup নিজের চোখে দেখার জন্য একটি Container চালিয়ে ভেতরে ও বাইরে থেকে পরীক্ষা করুন। নিচের কমান্ডগুলো Docker ইনস্টল করা Linux Host-এ চালান (ইনস্টল-পদ্ধতি অধ্যায় ৩-এ):

```
# 1. একটি পরীক্ষামূলক Container চালু করুন — নাম দিয়ে, Background-এ
docker run -d --name iso-test --hostname technova-api nginx:1.27-alpine
# ব্যাখ্যা: run = নতুন Container তৈরি ও চালু; -d = Detached (Background-এ চলবে);
# --name iso-test = Container-এর নাম; --hostname = UTS Namespace-এর Hostname;
# nginx:1.27-alpine = Image (Nginx 1.27, Alpine Linux Base); শেষ আর্গুমেন্ট Image-এর নাম।

# 2. Container-এর ভেতরে PID 1 কে দেখুন — ভেতরে সে 1 নম্বর
docker exec iso-test ps aux
# চিত্রণমূলক আউটপুট:
#   PID   USER     COMMAND
#     1   root     nginx: master process nginx -g daemon off;
# ব্যাখ্যা: exec = চলমান Container-এর ভেতরে কমান্ড চালাও; ভেতরে Nginx-এর PID 1।

# 3. Host থেকে একই প্রসেসের আসল PID দেখুন — Host-এ সে অন্য নম্বর
```

```
ps aux | grep nginx
# চিত্রণমূলক আউটপুট: root  45231  ...  nginx: master process
# ব্যাখ্যা: Host-এ একই প্রসেসের PID 45231 — Namespace-এর দুটি দৃশ্যের প্রমাণ।

# 4. Container-এর Namespace তালিকা দেখুন — /proc/PID/ns-এ Symlink থাকে
ls -l /proc/45231/ns/
# ব্যাখ্যা: প্রতিটি Namespace একটি Symlink — pid, net, mnt, uts, ipc, user, cgroup;
# দুটি Container-এর একই ধরনের Namespace-এর নম্বর আলাদা হলে তারা আলাদা।

# 5. Container-এর Cgroup পথ ও সীমা দেখুন
docker inspect iso-test --format '{{ .HostConfig.Memory }} {{ .HostConfig.CpuShares }}'
# চিত্রণমূলক আউটপুট: 0 0
# ব্যাখ্যা: inspect = Container-এর বিস্তারিত তথ্য; Memory 0 মানে কোনো সীমা দেওয়া হয়নি —
# এটি ডিফল্ট, এবং প্রোডাকশনে বিপজ্জনক (অধ্যায় ৫)।

# 6. সীমাসহ Container চালান — Cgroup-এর সীমা বসানো
docker run -d --name limited --memory 256m --cpus 0.5 --pids-limit 100 nginx:1.27-alpine
# ব্যাখ্যা: --memory 256m = সর্বোচ্চ 256 MB RAM; --cpus 0.5 = অর্ধেক CPU;
# --pids-limit 100 = সর্বোচ্চ 100 Process — Fork Bomb থেকে সুরক্ষা।

# 7. সীমার প্রমাণ দেখুন — Host-এর Cgroup ফাইলসিস্টেমে
cat /sys/fs/cgroup/system.slice/docker-*/memory.max
# ব্যাখ্যা: Cgroup v2-তে memory.max ফাইলে সীমা লেখা থাকে; 268435456 মানে 256 MB।
# পথটি Docker সংস্করণভেদে বদলাতে পারে — docker inspect দিয়ে Container ID মিলিয়ে নিন।

# 8. User Namespace পরীক্ষা — ভেতরে Root, Host-এ সাধারণ ব্যবহারকারী
docker run --rm --user 1000:1000 alpine:3.20 id
# চিত্রণমূলক আউটপুট: uid=1000 gid=1000
# ব্যাখ্যা: --rm = কাজ শেষে Container মুছে ফেলো; --user = ভেতরে কোন UID-তে চলবে;
# id = বর্তমান ব্যবহারকারীর পরিচয় দেখাও। Root নয় এমন UID-তে চালানোই সেরা অভ্যাস।
```

২.৮ সম্পূর্ণ কনফিগারেশন উদাহরণ

TechNova-র API Container-এর জন্য একটি নিরাপদ, সীমাবদ্ধ চালুর সংজ্ঞা — `docker run` ও Docker Compose দুই রূপেই। Compose-এর পূর্ণ পরিচয় পরের Volume-এ আসবে; এখানে শুধু Isolation-সংক্রান্ত অংশটুকু দেখুন:

```
# নিরাপদ চালুর উদাহরণ — Shell আকারে (এক লাইনে লেখা হলেও প্রতিটি ফ্ল্যাগের কারণ আছে)
docker run -d --name technova-api \
  --memory 512m --memory-swap 512m --cpus 1.0 --pids-limit 200 \
  --user 1000:1000 --read-only --cap-drop ALL --cap-add NET_BIND_SERVICE \
  --security-opt no-new-privileges:true \
  --hostname technova-api --network technova-net \
  technova-api:1.0.0
# লাইন ধরে ধরে: --memory/--cpus = Cgroup সীমা; --pids-limit = Fork সীমা;
# --user 1000:1000 = Root নয় এমন ব্যবহারকারী; --read-only = Root ফাইলসিস্টেম লেখা যাবে না;
# --cap-drop ALL = সব Capability কেড়ে নেওয়া; --cap-add NET_BIND_SERVICE = শুধু
#   1024-এর নিচের Port খোলার ক্ষমতা ফেরত; --security-opt no-new-privileges = ভেতরে
#   Privilege বাড়ানো যাবে না; --network = আলাদা Network Namespace-যুক্ত Bridge।

# একই সংজ্ঞা Docker Compose আকারে (docker-compose.yml — অংশবিশেষ)
services:
  api:
    image: technova-api:1.0.0
    hostname: technova-api
    user: "1000:1000"
```

```
    read_only: true
    cap_drop:
      - ALL
    cap_add:
      - NET_BIND_SERVICE
    security_opt:
      - no-new-privileges:true
    deploy:
      resources:
        limits:
          cpus: "1.0"
          memory: 512M
        reservations:
          cpus: "0.25"
          memory: 128M
    networks:
      - technova-net
networks:
  technova-net:
    driver: bridge
```

লক্ষ্য করুন: `limits` হলো সর্বোচ্চ সীমা, `reservations` হলো নিশ্চিত বরাদ্দ — Scheduler (যেমন Swarm বা Kubernetes) Reservation দেখে ঠিক করে কোন Host-এ Container বসবে। দুটিকে গুলিয়ে ফেলবেন না।

২.৯ বাস্তব প্রোডাকশন পরিস্থিতি

> **REAL WORLD — TechNova:** TechNova-র একটি Worker Container হঠাৎ পুরো Host-এর RAM খেয়ে ফেলল — কারণ তাতে কোনো `--memory` সীমা ছিল না। Host-এর OOM Killer তখন API Container-কে Kill করল (যেটি বেশি মেমরি ধরে রেখেছিল), আর গ্রাহকদের Checkout বন্ধ হয়ে গেল। তদন্তে দেখা গেল Worker-এ একটি মেমরি লিক (Memory Leak) ছিল — প্রতিটি কাজের পর কিছু মেমরি ছাড়া হচ্ছিল না। সমাধান দুটি স্তরে: (১) প্রতিটি Container-এ Memory Limit বসানো, যাতে একটি Container-এর লিক পুরো Host-কে না মারে; (২) Worker-এর লিক ঠিক করা ও `docker stats` দিয়ে মেমরি-বৃদ্ধি পর্যবেক্ষণ করা। শিক্ষা: Cgroup সীমা কোনো "অপ্টিমাইজেশন" নয় — এটি Blast Radius (ক্ষতির পরিধি) সীমিত করার নিরাপত্তা-ব্যবস্থা।

আরেকটি দৃশ্য — Shared Memory: দুটি Container-কে একই IPC Namespace ভাগ করতে হয়েছিল একটি Legacy অ্যাপের জন্য (`--ipc shareable`)। দলটি প্রথমে বুঝতে পারেনি যে এর মানে একটি Container অন্য Container-এর Shared Memory পড়তে পারে — নিরাপত্তা-পর্যালোচনায় ধরা পড়ার পর তারা Legacy অ্যাপটি আলাদা করে ফেলে IPC ভাগাভাগি বন্ধ করে দেয়। শিক্ষা: Namespace ভাগ করা মানে Isolation ইচ্ছাকৃতভাবে কমানো — প্রতিবার এর কারণ লিখে রাখুন।

২.১০ ধাপে ধাপে ব্যবহারিক বাস্তবায়ন

উদাহরণ: একটি Container-এর Isolation নিজে পরীক্ষা করা

1. `docker run -d --name iso-test nginx:1.27-alpine` দিয়ে একটি Container চালু করুন।
2. `docker exec iso-test ps aux` চালিয়ে ভেতরের PID 1 দেখুন; তারপর Host-এ `ps aux | grep nginx` চালিয়ে আসল PID মেলান।
3. `docker exec iso-test hostname` ও `hostname` (Host-এ) তুলনা করুন — UTS Namespace-এর প্রমাণ।

4. `docker exec iso-test ip addr` চালিয়ে Container-এর IP দেখুন; Host-এ `ip addr` চালিয়ে Bridge ও veth Interface খুঁজুন।
5. `docker exec iso-test cat /proc/1/cgroup` চালিয়ে Cgroup পথ দেখুন; Host-এ `/sys/fs/cgroup/`-এ সেই পথে গিয়ে `memory.max` ও `cpu.max` পড়ুন।
6. এবার `--memory 256m --cpus 0.5` দিয়ে আরেকটি Container চালিয়ে `memory.max` আবার পড়ুন — সংখ্যাটি বদলেছে কি না দেখুন।

২.১১ সাধারণ ভুল ও ভুল ধারণা

- **ভুল ধারণা:** "Container-এর ভেতরে Root মানে Host-এর Root।" — না; ডিফল্ট User Namespace ছাড়াও Root-এর Capability কাটা থাকে, তবে User Namespace ছাড়া ভেতরের Root Host-এর Root-এর কাছাকাছি থাকে — তাই Root-এ চালানো ঝুঁকিপূর্ণ।
- **ভুল:** `--privileged` দিয়ে সমস্যা সমাধান করা — এটি সব Capability, সব Device ও সব Seccomp সুরক্ষা তুলে দেয়; Container প্রায় Host-এর সমান হয়ে যায়। পরিবর্তে শুধু প্রয়োজনীয় Capability যোগ করুন (`--cap-add`)।
- **ভুল ধারণা:** "Namespace থাকলেই সম্পূর্ণ Isolation।" — না; Kernel ভাগ করা হয়, আর কিছু জিনিস Namespace-এর আওতায় পড়ে না — যেমন Kernel-এর তথ্য (`/proc/sys`-এর কিছু অংশ), সময় (সব Container একই ঘড়ি দেখে), ও CPU-এর দুর্বলতা (যেমন Spectre/Meltdown ধরনের আক্রমণ)।
- **ভুল:** Cgroup সীমা না দেওয়া — "আমার অ্যাপ ছোট, সীমা লাগবে না" ভাবা। একটি লিক বা আক্রমণ পুরো Host-কে ফেলে দিতে পারে।
- **ভুল:** `--pid=host` বা `--net=host` অকারণে ব্যবহার করা — এতে PID বা Network Namespace ভাগ হয়ে যায়, Isolation নষ্ট হয়। শুধু বিশেষ ক্ষেত্রে (যেমন Monitoring Agent) ব্যবহার করুন, কারণ লিখে রাখুন।
- **ভুল ধারণা:** "Seccomp/AppArmor শুধু বড় কোম্পানির জন্য।" — না; Docker-এর ডিফল্ট Profile-গুলো এমনিতেই চলে — আপনি শুধু সেগুলো বন্ধ করবেন না।

২.১২ নিরাপত্তা বিবেচনা

> **WARNING:** এই অধ্যায়ের সবচেয়ে গুরুত্বপূর্ণ নিরাপত্তা-সত্য: Container-এর Isolation Kernel-এর ওপর নির্ভর করে, তাই Host Kernel-এর দুর্বলতা (যেমন Dirty Pipe, CVE-2022-0847) সব Container-কে ঝুঁকিতে ফেলে। Host Kernel নিয়মিত আপডেট করুন, আর অপরিচিত Image চালাবেন না — Image-এর ভেতরের কোড Container-এর ভেতরে চললেও Kernel-এর দুর্বলতা কাজে লাগিয়ে বেরিয়ে আসতে পারে। সবচেয়ে বিপজ্জনক তিনটি জিনিস কখনো করবেন না: (১) `--privileged` Container চালানো; (২) Docker Socket (`/var/run/docker.sock`) Container-এর ভেতরে Mount করা; (৩) Host-এর Root ডিরেক্টরি (`/`) Volume হিসেবে দেওয়া। এই তিনটির যেকোনো একটিই Container Escape-এর পথ খুলে দেয়।

আরও নিরাপত্তা-নিয়ম: Root-এ নয়, নির্দিষ্ট UID-তে Container চালান (`--user`); `--cap-drop ALL` দিয়ে শুরু করে শুধু প্রয়োজনীয় Capability যোগ করুন; `--read-only` দিয়ে Root ফাইলসিস্টেম লেখা বন্ধ করুন (লেখার দরকার হলে `tmpfs` Mount দিন); `no-new-privileges` চালু রাখুন; User Namespace Remapping (`userns-remap`) চালু করলে ভেতরের Root Host-এ সাধারণ ব্যবহারকারীতে ম্যাপ হয় — এটি প্রোডাকশন Host-এর জন্য শক্তিশালী সুরক্ষা।

২.১৩ কর্মক্ষমতা ও রিসোর্স বিবেচনা

Namespace-এর খরচ প্রায় শূন্য — একটি নতুন Namespace তৈরি করতে মাইক্রোসেকেন্ড লাগে। Cgroup-এর খরচও সামান্য — Kernel প্রতিটি Allocation গুনে, তবে তা নগণ্য। আসল খরচ আসে ভুল সীমা থেকে: CPU সীমা খুব কম দিলে অ্যাপ Throttle হয়ে ধীর হয় — `docker stats`-এ CPU 100% দেখালেও আসলে সীমার কারণে আটকে আছে কি না তা যাচাই করুন। মেমরি সীমা খুব কম দিলে OOM-Killed হয়; খুব বেশি দিলে Host-এর মেমরি নষ্ট হয়। সঠিক সীমা বের করার নিয়ম: প্রথমে সীমা ছাড়া Staging-এ চালিয়ে আসল ব্যবহার মাপুন (`docker stats`, ২৪ ঘণ্টা), তারপর ব্যবহারের ১.৫ গুণ সীমা হিসেবে বসান, আর Alert বসান ৮০% ব্যবহারে। `--memory-swap` সতর্কভাবে ব্যবহার করুন — Swap-এ গেলে অ্যাপ ভয়ংকর ধীর হয়; অনেক প্রোডাকশন দল Swap সম্পূর্ণ বন্ধ রাখে (`--memory-swap` = `--memory`-এর সমান)।

২.১৪ সমস্যা সমাধান (Troubleshooting)

TROUBLESHOOTING: "Container-এর ভেতরে `mount: permission denied`" — এটি সাধারণত Capability-এর অভাব: `CAP_SYS_ADMIN` ছাড়া Mount করা যায় না, আর ডিফল্টভাবে Docker তা দেয় না। সমাধান: Mount করার দরকার আছে কি না ভাবুন — বেশিরভাগ ক্ষেত্রে Host থেকে Volume Mount করাই সঠিক পথ (`-v`), ভেতরে Mount করা নয়। যদি সত্যিই দরকার হয়, `--cap-add SYS_ADMIN` যোগ করুন — কিন্তু কারণটি লিখে রাখুন, কারণ এটি নিরাপত্তা-সীমানা কমায়। একইভাবে `Operation not permitted` ত্রুটি দেখলে Seccomp বা AppArmor আটকাচ্ছে কি না দেখুন — Host-এ `dmesg` বা `journalctl -k`-এ Denied বার্তা খুঁজুন।

1. **লক্ষণ:** Container হঠাৎ বন্ধ, Exit Code 137 — **কারণ:** OOM-Killed (128 + 9 = 137, SIGKILL)। **পরীক্ষা:** `docker inspect`-এ `OOMKilled: true` দেখুন; Host-এ `dmesg | grep -i oom`। **সমাধান:** Memory Limit বাড়ান বা মেমরি লিক ঠিক করুন।
2. **লক্ষণ:** অ্যাপ ধীর, কিন্তু CPU ব্যবহার কম দেখায় — **কারণ:** CPU Throttling। **পরীক্ষা:** Cgroup-এ `cpu.stat`-এর `nr_throttled` দেখুন। **সমাধান:** `--cpus` বাড়ান।
3. **লক্ষণ:** দুটি Container একই Port-এ শুনতে চায় — **কারণ:** Host Port-এর সংঘাত, Container Port-এর নয়। **সমাধান:** Host-এ আলাদা Port Publish করুন (`-p 8081:3000` ও `-p 8082:3000`)।
4. **লক্ষণ:** Container-এর ভেতরে Hostname বদলানো যায় না — **কারণ:** UTS Namespace-এ Hostname বদলাতে Capability লাগে। **সমাধান:** `--hostname` দিয়ে চালুর সময় ঠিক করুন।

২.১৫ সেরা অনুশীলন ও অ্যান্টি-প্যাটার্ন

সেরা অনুশীলন: প্রতিটি Container-এ CPU, Memory ও PIDs সীমা দিন; Root নয় এমন ব্যবহারকারীতে চালান; `--cap-drop ALL` থেকে শুরু করুন; `--read-only` Root ফাইলসিস্টেম ব্যবহার করুন; User Namespace Remapping চালু করুন; Host Kernel আপডেট রাখুন; Namespace ভাগ করার (`--pid=host`, `--net=host`, `--ipc=host`) প্রতিটি সিদ্ধান্তের কারণ লিখে রাখুন। **অ্যান্টি-প্যাটার্ন:** `--privileged` দিয়ে সব সমস্যা "ঠিক" করা; সীমা ছাড়া প্রোডাকশনে চালানো; Docker Socket Container-এ দেওয়া; "Container আলাদা, তাই নিরাপদ" ভেবে Image Scan না করা; Seccomp/AppArmor বন্ধ করে দেওয়া কারণ একটি অ্যাপ তাতে অভিযোগ করেছিল — আগে Profile-টি ঠিক করুন, সুরক্ষা বন্ধ করবেন না।

২.১৬ অধ্যায়ের সারাংশ

Containerization মানে Linux Kernel-এর Namespace ও Cgroup দিয়ে একটি Process-দলকে আলাদা দৃশ্য ও সীমা দেওয়া — কোনো নতুন Kernel ছাড়াই। PID, Network, Mount, UTS, IPC, User ও Cgroup — সাতটি Namespace মিলে Container-কে নিজের Process, নেটওয়ার্ক, ফাইলসিস্টেম ও Hostname দেখায়। Cgroup v2 CPU, মেমরি, I/O ও Process সংখ্যার সীমা দেয়; মেমরি ছাড়ালে OOM Killer Kill করে, CPU ছাড়ালে Throttle হয়। Capabilities Root-এর ক্ষমতা ভেঙে ছোট করে, Seccomp System Call ফিল্টার করে, আর AppArmor/SELinux ফাইল-স্তরে নিয়ম বসায়। OCI Specification-এর কারণে Image ও Runtime Portable। এই সবকিছু মিলেই Container — আর এগুলো না বুঝে Docker চালানো মানে গাড়ির ইঞ্জিন না জেনে গাড়ি চালানো।

২.১৭ মূল প্রযুক্তিগত পরিভাষা

Containerization: Namespace, Cgroup ও নিরাপত্তা-নিয়ম দিয়ে একটি Process-দলকে আলাদা পরিবেশে চালানোর কৌশল।

Namespace: Kernel-এর দৃশ্য-আলাদাকরণ — প্রতিটি Namespace একটি জিনিসের (Process, Network, File) আলাদা সংস্করণ দেখায়।

Cgroup (Control Group): Kernel-এর রিসোর্স-নিয়ন্ত্রণ — CPU, মেমরি, I/O ও Process সংখ্যার সীমা ও হিসাব।

Cgroup v2: Cgroup-এর আধুনিক Unified Hierarchy সংস্করণ; Ubuntu 24.04-এ ডিফল্ট।

OOM Killer: মেমরি ফুরিয়ে গেলে Kernel যে Process-কে Kill করে; Container-এ Exit Code 137 হয়।

Linux Capability: Root-এর ক্ষমতার ছোট টুকরো (যেমন CAP_NET_BIND_SERVICE); Docker ডিফল্টভাবে বিপজ্জনকগুলো কেড়ে নেয়।

Seccomp: System Call ফিল্টার — Process Kernel-কে কোন কাজের অনুরোধ করতে পারবে তা বেঁধে দেয়।

AppArmor / SELinux: Mandatory Access Control — ফাইল ও নেটওয়ার্কে Process-এর অনুমতির নিয়ম বসায়।

OCI (Open Container Initiative): Container-এর খোলা মান — Runtime, Image ও Distribution Specification।

veth Pair: Container-এর Network Namespace ও Host Bridge-এর সংযোগকারী ভার্চুয়াল কেবল-জোড়া।

System Call: Process-এর Kernel-এর কাছে অনুরোধ — যেমন ফাইল খোলা (`open`) বা নেটওয়ার্ক শোনা।

Fork Bomb: দ্রুত Process তৈরি করে সিস্টেম অচল করার আক্রমণ; `pids.max` দিয়ে আটকানো যায়।

২.১৮ জ্ঞান যাচাই — প্রশ্ন

1. Namespace কী বলে আর Cgroup কী বলে — এক বাক্যে প্রতিটির কাজ লিখুন।
2. PID Namespace-এ Container-এর অ্যাপের PID 1 হয় কেন, আর Host-এ একই প্রসেসের PID আলাদা হয় কেন?
3. দুটি Container-এর Port 3000-এ সংঘাত হয় না কেন? Network Namespace ও Port Mapping-এর ভূমিকা ব্যাখ্যা করুন।
4. Cgroup v1 ও v2-এর মূল পার্থক্য কী?
5. মেমরি সীমা ছাড়ালে কী হয়, আর CPU সীমা ছাড়ালে কী হয় — পার্থক্যটি ব্যাখ্যা করুন।
6. Container-এর ভেতরে Root হয়েও `mount` করা যায় না কেন? Capabilities-এর ভূমিকা বলুন।
7. Seccomp, AppArmor ও Capabilities — তিনটির কাজ এক বাক্যে আলাদা করুন।
8. OCI Specification না থাকলে কী সমস্যা হতো?

২.১৯ ব্যবহারিক অনুশীলন

1. **Namespace পরীক্ষা:** এই অধ্যায়ের ২.১০ ধাপগুলো অনুসরণ করে PID, Hostname, IP ও Cgroup-এর প্রমাণ সংগ্রহ করে একটি ছোট রিপোর্ট লিখুন।
2. **সীমা পরীক্ষা:** `--memory 64m` দিয়ে একটি Container চালিয়ে ভেতরে মেমরি খাওয়া একটি ছোট Node.js স্ক্রিপ্ট চালান — OOM-Killed (Exit 137) দেখুন ও `docker inspect` -এ প্রমাণ খুঁজুন।
3. **নিরাপদ চালু:** ২.৮-এর নিরাপদ `docker run` কমান্ডটি নিজের MERN API Image-এর জন্য মানিয়ে লিখুন এবং প্রতিটি ফ্ল্যাগের কারণ এক লাইনে লিখুন।

অধ্যায় 3 • PART I — Container Foundations

Chapter 3 — Docker Architecture: Engine, Daemon, CLI and Runtime

৩.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: (১) Docker Engine-এর উপাদান — CLI, Daemon (dockerd), containerd, runc ও shim — প্রতিটির ভূমিকা ব্যাখ্যা করতে; (২) `docker` কমান্ড চালালে ভেতরে কী ঘটে তা ধাপে ধাপে বলতে; (৩) Docker Daemon ইনস্টল, চালু ও লগ দেখার পদ্ধতি জানতে; (৪) Docker Socket, REST API ও Remote Daemon-এর ধারণা বোঝাতে; (৫) Storage Driver ও Network Driver-এর ভূমিকা বলতে; (৬) Daemon কনফিগারেশন (`daemon.json`) লিখতে।

৩.২ কেন এই বিষয়টি DevOps-এ গুরুত্বপূর্ণ

`docker run` চালিয়ে Container চললেই কাজ শেষ নয় — প্রোডাকশনে Daemon বন্ধ হয়ে গেলে, Disk ভরে গেলে, বা CLI "Cannot connect to the Docker daemon" বললে আপনাকে ভেতরের স্তরগুলো জানতে হবে। অনেক শিক্ষার্থী মনে করেন Docker একটি একক প্রোগ্রাম; আসলে এটি একটি স্তরবিন্যাস (Layered Architecture) — CLI শুধু অনুরোধ পাঠায়, Daemon সিদ্ধান্ত নেয়, containerd Container-এর জীবনচক্র সামলায়, আর runc আসলে Kernel-এর সঙ্গে কথা বলে Container তৈরি করে। কোন স্তরে সমস্যা তা না জানলে Troubleshooting হয় আন্দাজে। CI/CD Pipeline-এ Docker ব্যবহার করতে হলেও এই আর্কিটেকচার জানা চাই — যেমন Docker-in-Docker কেন ঝুঁকিপূর্ণ, বা কেন Remote Daemon-এ TLS লাগে।

৩.৩ পূর্বশর্ত জ্ঞান

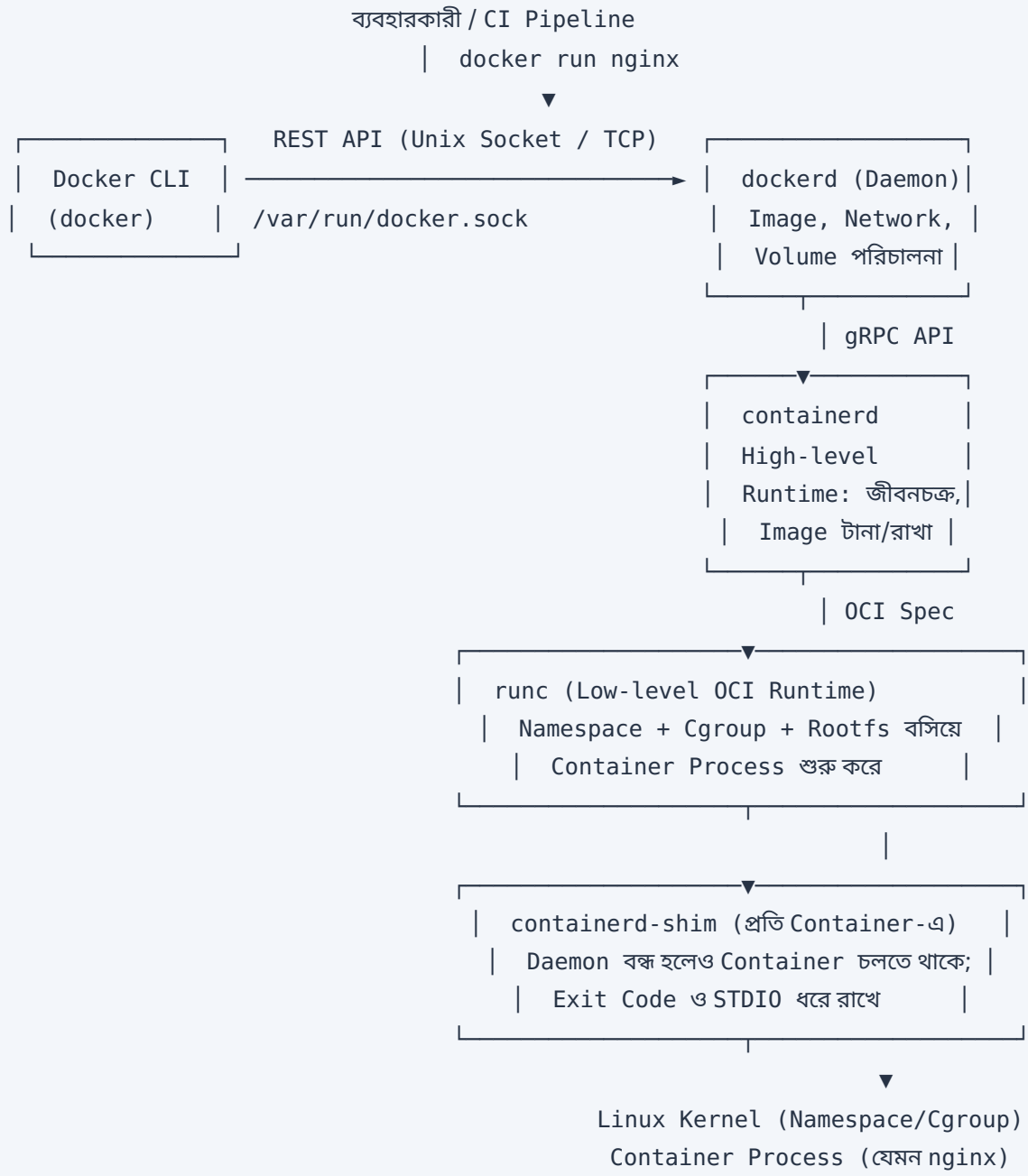
অধ্যায় ১–২-এর VM/Container পার্থক্য ও Namespace/Cgroup ধারণা, Linux-এ Service পরিচালনার প্রাথমিক ধারণা (`systemctl`), JSON ফাইলের গঠন, এবং Client-Server ধারণা (একটি প্রোগ্রাম অনুরোধ পাঠায়, আরেকটি উত্তর দেয়) জানা থাকলেই চলবে। REST API-এর গভীর জ্ঞান প্রয়োজন নেই।

৩.৪ প্রথম নীতি থেকে ব্যাখ্যা

Docker-কে একটি রেস্টুরেন্ট ভাবুন। আপনি (ব্যবহারকারী) ওয়েটারকে (CLI) অর্ডার দেন — "একটি API Container চালাও।" ওয়েটার অর্ডারটি রান্নাঘরের ম্যানেজারের (Daemon) কাছে নিয়ে যায়। ম্যানেজার নিজে রান্না করেন না — তিনি রেসিপি (Image) খুঁজে আনেন, রান্নার দলকে (containerd) নির্দেশ দেন, রান্নার দল আগুন জ্বালায় (runc — আসলে Kernel-এর Namespace ও Cgroup বসায়)। প্রতিটি স্তরের কাজ আলাদা, তাই একটি স্তর বদলানো যায় অন্যটি না ছুঁয়ে — যেমন runc-এর বদলে অন্য OCI Runtime ব্যবহার করা যায়, Image একই থাকে। এই বিভাজনই Docker-কে মজবুত ও Portable করেছে। **Docker Engine** বলতে এই পুরো ব্যবস্থাকে বোঝায় — CLI + Daemon + containerd + runc + নেটওয়ার্ক ও স্টোরেজ Driver। **Daemon (dockerd)** হলো Background-এ চলা মূল সার্ভিস — সে API শোনে, Image পরিচালনা করে, Container তৈরি করে। **CLI (`docker`)** হলো আপনার হাতের কমান্ড-লাইন টুল — সে নিজে কিছু চালায় না, শুধু Daemon-কে REST API অনুরোধ পাঠায়।

CONCEPT: `docker` কমান্ডটি Docker নয় — এটি শুধু একটি Client। আসল কাজ করে Daemon। প্রমাণ: Daemon বন্ধ থাকলে `docker ps` চালালেও "Cannot connect to the Docker daemon" ত্রুটি আসে — CLI ঠিক আছে, Server নেই। একইভাবে CLI আপনার ল্যাপটপে রেখে Daemon অন্য সার্ভারে (203.0.113.20) চালানো যায় — CLI শুধু নেটওয়ার্কে অনুরোধ পাঠায়।

৩.৫ অভ্যন্তরীণ আর্কিটেকচার ও চিত্র



লক্ষ্য করুন: runc কাজ শেষে বেরিয়ে যায়; Container চলতে থাকে Shim-এর তত্ত্বাবধানে। তাই Daemon Restart করলেও চলমান Container বন্ধ হয় না — এটাই Live Restore-এর ভিত্তি।

৩.৬ বিস্তারিত প্রযুক্তিগত ব্যাখ্যা

একটি `docker run`-এর ভেতরের যাত্রা

(১) CLI আপনার কমান্ডটি পড়ে একটি HTTP অনুরোধ বানায় — `POST /containers/create` ও `POST /containers/{id}/start` — এবং Unix Socket (`/var/run/docker.sock`) দিয়ে Daemon-কে পাঠায়। (২) Daemon অনুরোধ যাচাই করে: Image লোকাল আছে কি না দেখে; না থাকলে Registry থেকে টানে (Pull)। (৩) Daemon Container-এর জন্য একটি ডিরেক্টরি তৈরি করে — `/var/lib/docker/containers/<id>/` — যেখানে Config, লগ ও Hostname রাখা হয়। (৪) Daemon containerd-কে বলে Container তৈরি করতে; containerd Image-এর Layer থেকে Root Filesystem সাজায়, নেটওয়ার্ক Namespace তৈরি করে, আর runc-কে ডাকে। (৫) runc OCI Spec অনুযায়ী Namespace (PID, NET, MNT, UTS, IPC, User) ও Cgroup বসায়, Root Filesystem Mount করে, আর Container-এর মূল Process (যেমন `nginx`) শুরু করে। (৬) containerd-shim Process-টির Parent হয়ে থাকে, আর Daemon CLI-কে Container ID ফেরত দেয়। পুরো প্রক্রিয়াটি এক সেকেন্ডেরও কম সময়ে ঘটে — কারণ কোনো Kernel Boot হয় না।

Storage Driver ও Network Driver

Storage Driver ঠিক করে Image Layer কীভাবে Disk-এ সাজানো হয় ও Container-এর লেখার স্তর (Writable Layer) কীভাবে কাজ করে — Ubuntu-তে ডিফল্ট `overlay2` (অধ্যায় ৪-এ বিস্তারিত)। **Network Driver** ঠিক করে Container কীভাবে নেটওয়ার্ক পায় — `bridge` (ডিফল্ট: নিজের নেটওয়ার্ক, Port Mapping), `host` (Host-এর নেটওয়ার্ক সরাসরি), `none` (কোনো নেটওয়ার্ক নেই)। Daemon চালুর সময় `--storage-driver` বা `--data-root` দিয়ে Disk-এর অবস্থান বদলানো যায় — Disk ভরে গেলে এটি জরুরি হয়।

Docker Socket ও Remote Access

Daemon ডিফল্টভাবে শুধু লোকাল Unix Socket-এ শোনে — তাই শুধু ওই মেশিনের ব্যবহারকারী (বা `docker` Group-এর সদস্য) Daemon-কে নির্দেশ দিতে পারে। TCP-তে শুনতে হলে (`tcp://203.0.113.20:2376`) TLS বাধ্যতামূলক — TLS ছাড়া TCP খুলে দিলে যে কেউ ইন্টারনেট থেকে আপনার Host-এ Root-সমতুল্য Container চালাতে পারে, এটি প্রোডাকশনে সবচেয়ে বিপজ্জনক ভুলগুলোর একটি। **Docker Context** (`docker context`) দিয়ে একাধিক Daemon-এর মধ্যে বদলানো যায় — লোকাল ল্যাপটপ থেকে Staging Host-এর Daemon পরিচালনা করা যায়।

Rootless Docker ও Daemon-এর বিকল্প

সাধারণ Docker Daemon Root হিসেবে চলে — এটিই এর নিরাপত্তা-দুর্বলতা। **Rootless Docker** Daemon-কে সাধারণ ব্যবহারকারী হিসেবে চালায়, User Namespace ব্যবহার করে — নিরাপদ, তবে কিছু সীমা আছে (যেমন 1024-এর নিচের Port)। Podman হলো Daemon-বিহীন বিকল্প — প্রতিটি কমান্ড সরাসরি Container চালায়, কোনো স্থায়ী Daemon নেই। প্রোডাকশন Kubernetes-এ Docker Engine-এর বদলে সরাসরি containerd ব্যবহার করা হয় — Docker-এর Image ও Runtime মান (OCI) একই থাকায় তা সম্ভব।

৩.৭ Docker কমান্ড — লাইন ধরে ধরে ব্যাখ্যা

```
# 1. Docker ইনস্টল ও Daemon চালু (Ubuntu 24.04 — অফিসিয়াল পদ্ধতির সংক্ষেপ)
sudo apt update
# ব্যাখ্যা: প্যাকেজ তালিকা হালনাগাদ করো।
sudo apt install -y docker.io
# ব্যাখ্যা: Ubuntu-এর প্যাকেজ থেকে Docker ইনস্টল; প্রোডাকশনে Docker-এর অফিসিয়াল
# Repository থেকে ইনস্টল করা ভালো, যাতে সর্বশেষ সংস্করণ পাওয়া যায়।
sudo systemctl enable --now docker
```

```
# ব্যাখ্যা: enable = Boot-এ চালু হবে; --now = এখনই চালু করো; docker = সার্ভিসের নাম।

# 2. Daemon চলছে কি না ও সংস্করণ দেখুন
sudo systemctl status docker
# ব্যাখ্যা: Daemon-এর অবস্থা (active/running), লগের শেষ কয়েক লাইন দেখায়।
docker version
# ব্যাখ্যা: Client ও Server — দুই অংশের সংস্করণ; Server অংশ না চললে ত্রুটি দেখাবে।
docker info
# ব্যাখ্যা: Daemon-এর বিস্তারিত: Storage Driver (overlay2), Cgroup সংস্করণ,
# চলমান Container সংখ্যা, Disk ব্যবহার — Troubleshooting-এর প্রথম কমান্ড।

# 3. Daemon লগ দেখুন — সমস্যার সময় এটাই প্রথম দেখার জায়গা
sudo journalctl -u docker --since "1 hour ago" --no-pager | tail -n 50
# ব্যাখ্যা: journalctl = systemd লগ; -u docker = শুধু Docker সার্ভিসের লগ;
# --since = শেষ ১ ঘণ্টা; tail -n 50 = শেষ ৫০ লাইন।

# 4. Daemon কনফিগারেশন বদলানো — /etc/docker/daemon.json (উদাহরণ, ৩.৮ দেখুন)
sudo systemctl reload docker
# ব্যাখ্যা: কিছু সেটিং Reload-এ বদলায়; বেশিরভাগ ক্ষেত্রে Restart লাগে:
sudo systemctl restart docker
# সতর্কতা: Restart-এ Daemon বন্ধ হয়, তবে containerd-shim-এর কারণে চলমান
# Container সাধারণত চলতে থাকে (Live Restore চালু থাকলে)।

# 5. Docker Context — অন্য Host-এর Daemon পরিচালনা (TLS-সহ, সতর্কভাবে)
docker context create staging --docker "host=tcp://
203.0.113.20:2376,ca=ca.pem,cert=cert.pem,key=key.pem"
# ব্যাখ্যা: staging নামের Context — Staging Host-এর Daemon, TLS সার্টিফিকেট-সহ।
docker context use staging
# ব্যাখ্যা: এখন থেকে docker কমান্ড Staging Daemon-এ যাবে; ফিরতে: docker context use default
```

৩.৮ সম্পূর্ণ কনফিগারেশন উদাহরণ

```
# /etc/docker/daemon.json — TechNova Staging Host-এর উদাহরণ (JSON, মন্তব্য নয় — আসল ফাইলে মন্তব্য লিখবেন
না)
{
  "log-driver": "json-file",
  "log-opts": {
    "max-size": "10m",
    "max-file": "3"
  },
  "storage-driver": "overlay2",
  "data-root": "/var/lib/docker",
  "live-restore": true,
  "userns-remap": "default",
  "default-ulimits": {
    "nofile": {
      "Name": "nofile",
      "Hard": 65536,
      "Soft": 65536
    }
  },
  "insecure-registries": [],
  "registry-mirrors": []
}
# ব্যাখ্যা: log-driver json-file = Container লগ ফাইলে রাখা; max-size/max-file = লগ ঘোরানো
# (Log Rotation) — নইলে Disk ভরে যায়; live-restore = Daemon Restart-এ Container চলতে থাকে;
```

```
# userns-remap = User Namespace Remapping — ভেতরের Root Host-এ সাধারণ ব্যবহারকারী;
# default-ulimits nofile = প্রতিটি Container-এ সর্বোচ্চ খোলা ফাইল 65536;
# insecure-registries খালি = শুধু HTTPS Registry অনুমোদিত।
# সতর্কতা: ফাইলটি বদলে sudo systemctl restart docker চালাতে হয়; ভুল JSON লিখলে
# Daemon চালু হবে না — আগে python3 -m json.tool দিয়ে যাচাই করুন।
```

৩.৯ বাস্তব প্রোডাকশন পরিস্থিতি

REAL WORLD — TechNova: TechNova-র Staging Host-এ একদিন সকালে সব `docker` কমান্ড "Cannot connect to the Docker daemon" বলতে শুরু করল। দল প্রথমে ভাবল Docker ভেঙে গেছে, কিন্তু `systemctl status docker` দেখাল Daemon চলছে না — কারণ রাতে `/var/lib/docker` ডিস্ক ভরে গিয়েছিল (পুরনো Image ও লগে)। Daemon চালু হতে পারেনি। সমাধান: অন্য টার্মিনাল থেকে Disk খালি করা, Daemon Restart, আর ভবিষ্যতের জন্য `daemon.json`-এ লগ Rotation ও `docker system prune`-এর নিয়মিত Cron। শিক্ষা: Daemon-এর সমস্যার প্রথম পরীক্ষা `systemctl status` ও `journalctl` — CLI-এর ত্রুটি বার্তা শুধু লক্ষণ, কারণ নয়।

৩.১০ ধাপে ধাপে ব্যবহারিক বাস্তবায়ন

1. Ubuntu VM-এ Docker ইনস্টল করে `sudo systemctl status docker` ও `docker info` চালান — Storage Driver, Cgroup সংস্করণ ও Data Root লিখে রাখুন।
2. `docker run --rm hello-world` চালিয়ে দেখুন — Image টানা, Container তৈরি, বার্তা ছাপা ও Container মোছা — পুরো যাত্রাটি ঘটে।
3. `daemon.json`-এ লগ Rotation বসিয়ে Daemon Restart করুন; `docker info` দিয়ে সেটিং কার্যকর হয়েছে কি না যাচাই করুন।
4. ইচ্ছাকৃতভাবে Daemon বন্ধ করে (`sudo systemctl stop docker`) `docker ps` চালান — ত্রুটি বার্তাটি পড়ুন ও লিখুন; তারপর Daemon আবার চালু করুন।
5. `docker context ls` চালিয়ে Context তালিকা দেখুন — Default Context-এর Endpoint কী তা লিখুন।

৩.১১ সাধারণ ভুল ও ভুল ধারণা

- **ভুল ধারণা:** "`docker` কমান্ডই Docker।" — না; CLI শুধু Client, Daemon-ই আসল Engine।
- **ভুল:** Daemon-কে TLS ছাড়া TCP-তে খুলে দেওয়া (`-H tcp://0.0.0.0:2375`) — ইন্টারনেট থেকে যে কেউ Root-সমতুল্য Container চালাতে পারে।
- **ভুল:** প্রতিটি সমস্যায় Daemon Restart করা — চলমান Container-এর লগ ও অবস্থা হারাতে পারে; আগে `docker info` ও `journalctl` দেখুন।
- **ভুল ধারণা:** "Daemon বন্ধ হলে Container বন্ধ হয়।" — না; Shim-এর কারণে Container চলতে থাকে (Live Restore চালু থাকলে)।
- **ভুল:** `sudo` ছাড়া Docker চালাতে না পেরে সবাইকে Root দেওয়া — সঠিক পথ: ব্যবহারকারীকে `docker` Group-এ যোগ করা (`sudo usermod -aG docker nahid`) — তবে জানবেন Group-এর সদস্য কার্যত Root-সমতুল্য, তাই শুধু বিশ্বস্ত ব্যবহারকারীকে দিন।
- **ভুল:** Docker-in-Docker (DinD) অকারণে ব্যবহার করা — CI-তে Privileged DinD-এর বদলে Docker Socket Mount বা Rootless Runner বিবেচনা করুন।

৩.১২ নিরাপত্তা বিবেচনা

WARNING: Docker Socket (`/var/run/docker.sock`) যার হাতে, তার হাতে কার্যত Host-এর Root — কারণ Daemon-কে বললেই Host-এর `/` Mount করা Privileged Container চালানো যায়। তাই Socket কখনো Container-এর ভেতরে দেবেন না, কখনো TCP-তে TLS ছাড়া খুলবেন না, আর `docker` Group-এ শুধু বিশ্বস্ত ব্যবহারকারী রাখবেন। Rootless Docker ব্যবহার করলে এই ঝুঁকি অনেক কমে। Daemon-এর API-তে Authentication নেই — সুরক্ষা নির্ভর করে Socket-এর File Permission ও TLS-এর ওপর।

৩.১৩ কর্মক্ষমতা ও রিসোর্স বিবেচনা

Daemon নিজে সামান্য RAM (50–100 MB) ব্যবহার করে; আসল খরচ Image ও Container-এর Disk। `/var/lib/docker`-এর Disk পর্যবেক্ষণ করুন — `docker system df` দিয়ে Image, Container ও Volume-এর ব্যবহার দেখা যায়। লগ Rotation না থাকলে একটি শব্দ-ভরা Container (Verbose Logging) কয়েক ঘণ্টায় Disk ভরিয়ে ফেলতে পারে — তাই `daemon.json`-এ `max-size` বাধ্যতামূলক। Image Pull-এর সময় নেটওয়ার্ক ব্যান্ডউইথ খরচ হয় — Registry Mirror ব্যবহার করলে Pull দ্রুত হয়। Daemon-এর CPU খরচ সাধারণত নগণ্য, তবে অনেক Container একসঙ্গে Start করলে Daemon ব্যস্ত হয়ে পড়ে — Start-এ Stagger (একে একে) করুন।

৩.১৪ সমস্যা সমাধান (Troubleshooting)

TROUBLESHOOTING: "Cannot connect to the Docker daemon" — ধাপে ধাপে: (১) `sudo systemctl status docker` — Daemon চলছে কি না; (২) না চললে `sudo journalctl -u docker --no-pager | tail -n 50` — কেন থামল; (৩) Disk ভরা কি না দেখুন (`df -h /var/lib/docker`); (৪) `daemon.json`-এর JSON বৈধ কি না যাচাই করুন (`python3 -m json.tool /etc/docker/daemon.json`); (৫) Socket ফাইল আছে কি না দেখুন (`ls -l /var/run/docker.sock`)। "permission denied" হলে ব্যবহারকারী `docker` Group-এ আছে কি না দেখুন (`groups`) — Group যোগ করার পর Logout/Login লাগে।

1. **লক্ষণ:** Daemon চালু হয় না, লগে "failed to start daemon: error initializing graphdriver" — **কারণ:** Storage Driver বা Data Root-এর সমস্যা। **সমাধান:** `daemon.json` যাচাই করুন, Disk খালি করুন, প্রয়োজনে `/var/lib/docker` সরিয়ে নতুন করে শুরু করুন (সব Image মুছে যাবে — সতর্কতা)।
2. **লক্ষণ:** Image Pull ধীর — **পরীক্ষা:** Registry Mirror আছে কি না, নেটওয়ার্ক গতি কেমন। **সমাধান:** Mirror বসান, বা Layer Cache ব্যবহার করুন।
3. **লক্ষণ:** Container চালু হয় কিন্তু Daemon-এর লগে "no space left on device" — **সমাধান:** `docker system prune -a --volumes` সতর্কভাবে চালান (অব্যবহৃত সবকিছু মুছে যাবে — আগে যাচাই করুন)।

৩.১৫ সেরা অনুশীলন ও অ্যান্টি-প্যাটার্ন

সেরা অনুশীলন: Daemon কনফিগারেশন `daemon.json`-এ লিখে সংস্করণ-নিয়ন্ত্রণে রাখুন; লগ Rotation বাধ্যতামূলক করুন; Live Restore চালু রাখুন; User Namespace Remapping চালু করুন; Daemon-এর Disk পর্যবেক্ষণ করুন; Remote Daemon-এ শুধু TLS-সহ TCP ব্যবহার করুন। **অ্যান্টি-প্যাটার্ন:** TLS ছাড়া TCP খোলা; Socket Container-এ

দেওয়া; `daemon.json` হাতে সম্পাদনা করে ব্যাকআপ না রাখা; Disk ভরা পর্যন্ত অপেক্ষা করা; প্রতিটি সমস্যায় Daemon Restart করা।

৩.১৬ অধ্যায়ের সারাংশ

Docker Engine একটি স্তরবিন্যাস: CLI অনুরোধ পাঠায়, Daemon (dockerd) সিদ্ধান্ত নেয়, containerd জীবনচক্র সামলায়, runc Kernel-এর Namespace ও Cgroup বসিয়ে Container Process শুরু করে, আর Shim Daemon-এর অনুপস্থিতিতেও Container চালিয়ে রাখে। Daemon Unix Socket-এ শোনে, REST API-তে কথা বলে, Storage ও Network Driver পরিচালনা করে। Daemon কনফিগারেশন `daemon.json`-এ থাকে। এই স্তরগুলো জানলে Troubleshooting হয় সুনির্দিষ্ট, আন্দাজে নয়।

৩.১৭ মূল প্রযুক্তিগত পরিভাষা

Docker Engine: CLI, Daemon, containerd, runc ও Driver-এর সমন্বয়ে গড়া সম্পূর্ণ Docker ব্যবস্থা।

Daemon (dockerd): Background-এ চলা Docker সার্ভিস — API শোনে, Image ও Container পরিচালনা করে।

CLI: Command Line Interface — `docker` কমান্ড; নিজে কিছু চালায় না, Daemon-কে অনুরোধ পাঠায়।

containerd: High-level Runtime — Container-এর জীবনচক্র ও Image পরিচালনা করে।

runc: Low-level OCI Runtime — Namespace ও Cgroup বসিয়ে Container Process শুরু করে।

containerd-shim: প্রতি Container-এর তত্ত্বাবধায়ক Process — Daemon বন্ধ হলেও Container চালিয়ে রাখে।

Docker Socket: `/var/run/docker.sock` — CLI ও Daemon-এর যোগাযোগের Unix Socket।

Storage Driver: Image Layer Disk-এ সাজানোর কৌশল; ডিফল্ট `overlay2`।

Live Restore: Daemon Restart-এ চলমান Container চালিয়ে রাখার সুবিধা।

Rootless Docker: Root ছাড়া, সাধারণ ব্যবহারকারী হিসেবে Daemon চালানোর পদ্ধতি।

৩.১৮ জ্ঞান যাচাই — প্রশ্ন

1. CLI, Daemon, containerd, runc ও Shim — প্রতিটির কাজ এক বাক্যে লিখুন।
2. `docker run nginx` চালালে ভেতরে ধাপে ধাপে কী ঘটে?
3. Daemon বন্ধ হলে চলমান Container কেন বন্ধ হয় না?
4. Docker Socket-এর নিরাপত্তা-ঝুঁকি কী? TLS ছাড়া TCP খোলা কেন বিপজ্জনক?
5. `docker info` দিয়ে কী কী জানা যায়, আর কখন এটি চালাবেন?
6. Rootless Docker কী, আর কখন এটি ব্যবহার করবেন?

৩.১৯ ব্যবহারিক অনুশীলন

1. **আর্কিটেকচার মানচিত্র:** CLI থেকে Kernel পর্যন্ত স্তরগুলোর ডায়াগ্রাম নিজে আঁকুন ও প্রতিটি স্তরের কাজ লিখুন।
2. **Daemon পরীক্ষা:** ৩.১০-এর ধাপগুলো করে Daemon বন্ধ/চালু, লগ দেখা ও `docker info` পড়ার অনুশীলন করুন।
3. **daemon.json:** ৩.৮-এর উদাহরণটি আপনার ল্যাব VM-এ বসিয়ে Daemon Restart করুন ও `docker info` দিয়ে যাচাই করুন।

অধ্যায় 4 • PART I — Container Foundations

Chapter 4 — Images, Containers, Layers and Union Filesystems

৪.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: (১) Image, Container ও Layer-এর পার্থক্য ব্যাখ্যা করতে; (২) Dockerfile-এর প্রতিটি নির্দেশ কীভাবে একটি Layer তৈরি করে তা বলতে; (৩) Union Filesystem ও Copy-on-Write (CoW) কীভাবে কাজ করে তা গভীরে বোঝাতে; (৪) OCI Image Specification — Manifest, Config ও Layer Digest — ব্যাখ্যা করতে; (৫) `docker create`, `start`, `stop`, `remove`-এর ভেতরে ধাপে ধাপে কী ঘটে তা বলতে; (৬) Multi-Stage Build দিয়ে ছোট ও নিরাপদ Image বানাতে; (৭) Image-এর আকার, Cache ও Tag-সংক্রান্ত সিদ্ধান্ত নিতে।

৪.২ কেন এই বিষয়টি DevOps-এ গুরুত্বপূর্ণ

Image হলো Container জগতের মুদ্রা — CI/CD Pipeline-এ যা তৈরি হয়, Registry-তে যা জমা থাকে, আর Production-এ যা চলে, সবই Image। Image না বুঝলে আপনি বুঝবেন না কেন একটি ছোট পরিবর্তনে পুরো Image আবার তৈরি হয় না (Layer Cache), কেন দুটি Image একই Base Layer ভাগ করে Disk বাঁচায়, কেন Container-এর ভেতরে ফাইল বদলালে Image বদলায় না (Writable Layer), আর কেন Container মুছলে ডেটা মুছে যায়। প্রোডাকশনে সবচেয়ে সাধারণ বিপর্যয়গুলো — Image-এর আকার GB ছাড়িয়ে যাওয়া, Secret Image-এ ঢুকে যাওয়া, `latest` Tag-এর কারণে কোন সংস্করণ চলছে তা না জানা — সবগুলোর মূল এই অধ্যায়ে। Image-কে গভীরে বুঝলে Dockerfile লেখা হয় ইচ্ছাকৃত, আন্দাজে নয়।

৪.৩ পূর্বশর্ত জ্ঞান

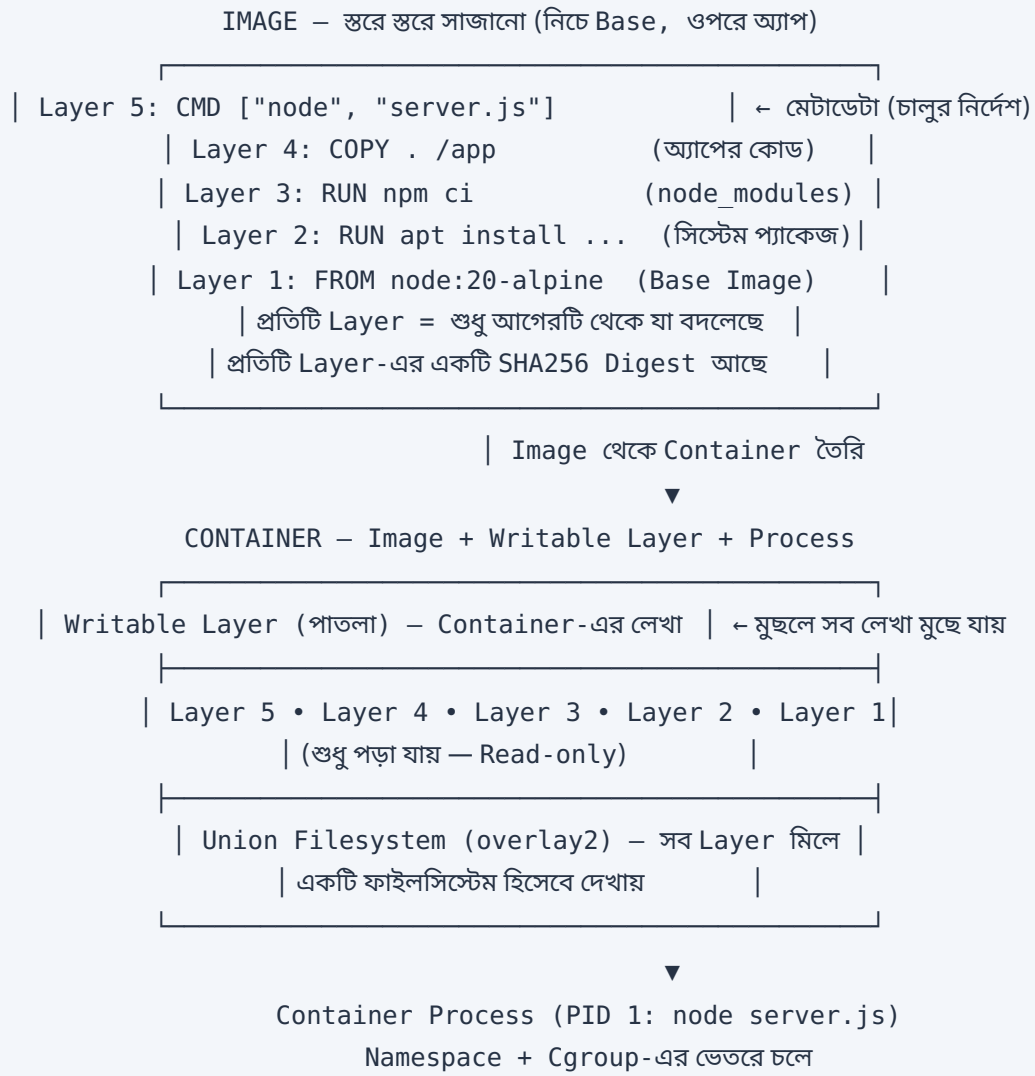
অধ্যায় ১–৩-এর VM/Container পার্থক্য, Namespace/Cgroup, Docker Engine-এর স্তরবিন্যাস, Linux ফাইলসিস্টেম ও Mount-এর ধারণা, এবং একটি MERN অ্যাপের গঠন (package.json, node_modules, Build আউটপুট) জানা থাকতে হবে। Dockerfile আগে লিখে না থাকলেও চলবে — এই অধ্যায়ে প্রতিটি নির্দেশ ব্যাখ্যা করা হবে।

৪.৪ প্রথম নীতি থেকে ব্যাখ্যা

একটি **Image** হলো একটি অ্যাপ চালানোর জন্য প্রয়োজনীয় সবকিছুর — Base OS ফাইল, Runtime (Node.js), লাইব্রেরি, অ্যাপের কোড, Environment সেটিং, আর চালু করার নির্দেশ (Entrypoint/Command) — একটি অপরিবর্তনীয় (Immutable), স্তরে স্তরে সাজানো প্যাকেজ। **Container** হলো সেই Image-এর একটি চলমান উদাহরণ (Running Instance) — Image-এর ওপরে একটি পাতলা লেখার স্তর (Writable Layer) বসিয়ে, তার ভেতরে Process শুরু করে দেওয়া হয়। তুলনা: Image হলো একটি ক্লাস (Class), Container হলো তার Object; Image হলো একটি CD, Container হলো সেই CD বাজানো; Image হলো রেসিপি, Container হলো রান্না করা খাবার। **Layer** হলো Image-এর একটি ধাপ — Dockerfile-এর প্রতিটি নির্দেশ (যেমন `RUN`, `COPY`) একটি নতুন Layer তৈরি করে, যাতে শুধু আগের Layer থেকে যা বদলেছে তা থাকে। Layer-গুলো একে অপরের ওপরে স্তূপীকৃত হয়, আর **Union Filesystem** সেগুলোকে একত্রে দেখায় — যেন একটি একক ফাইলসিস্টেম। **Copy-on-Write (CoW)** মানে: Container যখন কোনো ফাইল পড়ে, সে Layer থেকে সরাসরি পড়ে; যখন লিখতে যায়, তখন প্রথমে ফাইলটির একটি কপি তার নিজের Writable Layer-এ তোলা হয়, তারপর সেখানে লেখা হয় — নিচের Layer কখনো বদলায় না। এই অপরিবর্তনীয়তাই Image-কে নির্ভরযোগ্য করে: একই Image থেকে তৈরি প্রতিটি Container একইভাবে শুরু হয়।

CONCEPT: Image অপরিবর্তনীয়, Container পরিবর্তনশীল — এই পার্থক্যটিই সবকিছু। Container-এর ভেতরে ফাইল বদলালে Image বদলায় না; বদলায় শুধু সেই Container-এর Writable Layer। Container মুছলে Writable Layer মুছে যায় — তাই ডেটা রাখতে Volume লাগে। আর Image-এর কোনো Layer বদলাতে চাইলে নতুন Image তৈরি করতে হয় — পুরনো Layer কখনো সম্পাদনা করা হয় না, শুধু নতুন Layer যোগ করা হয়।

৪.৫ অভ্যন্তরীণ আর্কিটেকচার ও চিত্র



Overlay2-এর ভাষায়: `lowerdir` = নিচের Layer-গুলো (শুধু পড়া), `upperdir` = Writable Layer, `merged` = একত্রিত দৃশ্য যা Container দেখে, `workdir` = Overlay-এর অভ্যন্তরীণ কাজের জায়গা।

৪.৬ বিস্তারিত প্রযুক্তিগত ব্যাখ্যা

OCI Image Specification — Manifest, Config, Digest

OCI Image Spec অনুযায়ী একটি Image আসলে কয়েকটি ফাইলের সমষ্টি। **Manifest** হলো Image-এর সূচিপত্র — এতে লেখা থাকে Config-এর Digest ও প্রতিটি Layer-এর Digest ও আকার। **Config** হলো Image-এর মেটাডেটা — কোন Architecture (amd64/arm64), কোন OS, Environment Variable, Entrypoint, CMD, Exposed Port, কাজের ডিরেক্টরি। **Layer** হলো একটি Tar ফাইল — ফাইল ও ডিরেক্টরির সংকুচিত সংগ্রহ — যার **Digest** (SHA256 Hash) তার পরিচয়। Digest-এর কারণে Layer Content-Addressable: একই বিষয়বস্তুর Layer সবসময় একই Digest পায়, তাই Docker একই Layer দুবার ডাউনলোড করে না — দুটি Image একই Base Layer ভাগ করলে Disk-এ তা একবারই থাকে। **Tag** (`technova-api:1.0.0`) শুধু একটি নাম — একটি চলমান লেবেল; **Digest** (`sha256:9f86...`) হলো অপরিবর্তনীয় পরিচয়। প্রোডাকশনে Digest দিয়ে Image নির্দিষ্ট করা সবচেয়ে নিরাপদ, কারণ Tag বদলে যেতে পারে — `latest` Tag আজ যা দেখায়, কাল তা অন্য Image দেখাতে পারে।

Union Filesystem — Overlay2 গভীরে

Linux-এর **OverlayFS** দুটি ডিরেক্টরিকে একত্রে দেখায়: নিচের স্তর (`lowerdir` — একাধিক Layer, কোলন দিয়ে যুক্ত) শুধু পড়া যায়, ওপরের স্তর (`upperdir`) পড়া-লেখা যায়, আর `merged` ডিরেক্টরিতে দুটি মিলে একটি ফাইলসিস্টেম দেখায়। কোনো ফাইল দুই স্তরেই থাকলে ওপরেরটি দেখা যায় (Whiteout নিয়ম: নিচের ফাইল মুছতে চাইলে ওপরে একটি Whiteout চিহ্ন বসে, নিচের ফাইলটি মুছে না)। Docker-এর `overlay2` Storage Driver এই OverlayFS-কে ব্যবহার করে — প্রতিটি Layer Disk-এ একটি ডিরেক্টরি, প্রতিটি Container-এর Writable Layer একটি `upperdir`। এই কাঠামোর কারণে Image তৈরি দ্রুত (শুধু বদলানো Layer তৈরি হয়), Container তৈরি দ্রুত (শুধু একটি খালি Writable Layer বসে), আর Disk বাঁচে (Layer ভাগাভাগি হয়)। তবে সীমাও আছে: অনেক Layer (100-এর বেশি) হলে Mount ধীর হয়; Writable Layer-এ অনেক লেখা হলে CoW-এর খরচ বাড়ে — ডেটাবেসের মতো বেশি লেখা কাজের জন্য Volume ব্যবহার করা উচিত, Writable Layer নয়।

Dockerfile — প্রতিটি নির্দেশ একটি Layer

`FROM` Base Image ঠিক করে; `RUN` একটি কমান্ড চালিয়ে তার ফলকে Layer হিসেবে জমা করে; `COPY` / `ADD` ফাইলকে Layer হিসেবে যোগ করে; `ENV`, `WORKDIR`, `EXPOSE`, `CMD`, `ENTRYPOINT` — এগুলো নতুন Layer তৈরি করে না, শুধু Config-এ মেটাডেটা যোগ করে (তবে `ENV` পরের Layer-এর Cache-কে প্রভাবিত করে)। **Layer Cache** হলো Build-এর গতি: Dockerfile-এর কোনো নির্দেশ বদলালে শুধু সেই নির্দেশ থেকে নিচের Layer-গুলো আবার তৈরি হয়, ওপরের Layer-গুলো Cache থেকে নেওয়া হয়। তাই অপরিবর্তনশীল ধাপ (যেমন নির্ভরতা ইনস্টল) ওপরে, আর ঘন ঘন বদলানো ধাপ (যেমন অ্যাপের কোড কপি) নিচে রাখতে হয় — এটাই Dockerfile লেখার সবচেয়ে গুরুত্বপূর্ণ নিয়ম।

ভেতরে কী ঘটে: create / start / stop / remove

`docker create`: Daemon Image-এর Layer থেকে Root Filesystem সাজায় (Overlay Mount), একটি খালি Writable Layer তৈরি করে, Container-এর Config লেখে (`/var/lib/docker/containers/<id>/config.v2.json`), Network Namespace ও IP বরাদ্দ করে (কিন্তু Process শুরু করে না), আর একটি Container ID ফেরত দেয়। অবস্থা: `created`। **`docker start`:** Daemon containerd-কে ডাকে, containerd runc-কে ডাকে, runc Namespace ও Cgroup বসায়, Root Filesystem Mount করে, আর Entrypoint/CMD Process (PID 1) শুরু করে। অবস্থা: `running`। **`docker stop`:** প্রথমে `SIGTERM` (15) পাঠানো হয় — অ্যাপকে ভদ্রভাবে বন্ধ হওয়ার সুযোগ দেওয়া হয় (ডিফল্ট 10 সেকেন্ড); সময় পেরিয়ে গেলে `SIGKILL` (9) পাঠিয়ে জোর করে বন্ধ করা হয়। অবস্থা: `exited`। এই Graceful Shutdown-এর সময়টি `--time` দিয়ে বদলানো যায় — Node.js অ্যাপের জন্য 10 সেকেন্ড অনেক সময় যথেষ্ট

নয়, তাই প্রোডাকশনে 30 সেকেন্ড দেওয়া সাধারণ। **`docker rm`:** Writable Layer, Container-এর Config ও লগ মুছে ফেলা হয়; Image অপরিবর্তিত থাকে; Volume মুছে না — যদি না `-v` ফ্ল্যাগ দেওয়া হয়। **`docker run`** হলো `create` + `start` + (Attach থাকলে) লগ দেখানো — একসঙ্গে করা।

Multi-Stage Build — ছোট ও নিরাপদ Image

একটি MERN অ্যাপের Build-এ Node.js, npm, TypeScript Compiler, পরীক্ষার টুল — সব লাগে; কিন্তু চালাতে লাগে শুধু Build-এর ফল (যেমন `.next/` বা `dist/`) ও Runtime নির্ভরতা। **Multi-Stage Build**-এ প্রথম Stage (Builder) সব টুল দিয়ে Build করে, দ্বিতীয় Stage (Runner) শুধু Build-এর ফল কপি করে নেয় — ফলে চূড়ান্ত Image-এ Compiler, Source Code বা `node_modules`-এর Dev নির্ভরতা থাকে না। Image ছোট হয় (GB থেকে MB), আক্রমণের পৃষ্ঠ কমে, আর Secret (যেমন Build-সময়ের Token) চূড়ান্ত Image-এ পৌঁছায় না।

৪.৭ Docker কমান্ড — লাইন ধরে ধরে ব্যাখ্যা

```
# 1. Image তৈরি করুন — Build প্রসঙ্গ (Context) থেকে
docker build -t technova-api:1.0.0 -t technova-api:latest .
# ব্যাখ্যা: build = Image তৈরি; -t = Tag দাও (দুটি Tag দেওয়া হয়েছে);
# . = Build Context — বর্তমান ডিরেক্টরির সব ফাইল Daemon-কে পাঠানো হয়;
# Dockerfile না থাকলে -f দিয়ে পথ দিতে হয়।

# 2. Image-এর Layer দেখুন — প্রতিটি Layer-এর আকার ও নির্দেশ
docker history technova-api:1.0.0
# চিত্রণমূলক আউটপুট:
#   IMAGE          CREATED        SIZE      COMMENT
#   abc123...      2 hours ago    12 MB     CMD ["node" "server.js"]
#   def456...      2 hours ago    45 MB     COPY . /app
# ব্যাখ্যা: history = প্রতিটি Layer — কোন নির্দেশে তৈরি, কত বড়;
# কোন Layer বড় তা দেখে Image ছোট করার জায়গা খুঁজুন।

# 3. Image-এর বিস্তারিত দেখুন — Config, Entrypoint, Env
docker inspect technova-api:1.0.0
# ব্যাখ্যা: Manifest ও Config — Architecture, Env, Cmd, ExposedPorts দেখায়।

# 4. Image ট্যাগ করা ও Registry-তে পাঠানো
docker tag technova-api:1.0.0 registry.technova.example/technova-api:1.0.0
# ব্যাখ্যা: tag = একই Image-কে আরেক নাম দাও (Registry-এর নাম-সহ)।
docker push registry.technova.example/technova-api:1.0.0
# ব্যাখ্যা: push = Layer-গুলো Registry-তে পাঠাও; শুধু নতুন Layer যায়, পুরনো যায় না।

# 5. Image টানা — Digest দিয়ে, সবচেয়ে নিরাপদ
docker pull registry.technova.example/technova-
api@sha256:9f86d081884c7d659a2feaa0c55ad015a3bf4f1b2b0b822cd15d6c15b0f00a08
# ব্যাখ্যা: pull = Registry থেকে Image টানো; @sha256:... মানে Digest দিয়ে নির্দিষ্ট —
# Tag বদলে গেলেও এই Digest সবসময় একই Image দেখায়।

# 6. Lifecycle — create, start, stop, rm আলাদাভাবে
docker create --name api-test technova-api:1.0.0
# ব্যাখ্যা: শুধু তৈরি করো, চালু কোরো না — অবস্থা created।
docker start api-test
# ব্যাখ্যা: Process শুরু করো — অবস্থা running।
docker stop --time 30 api-test
# ব্যাখ্যা: SIGTERM দাও, 30 সেকেন্ড অপেক্ষা করো, তারপর SIGKILL; অবস্থা exited।
docker rm api-test
```

```
# ব্যাখ্যা: Container মুছে ফেলো — Writable Layer ও Config মুছে যাবে, Image থাকবে।
docker rm -v api-test
# ব্যাখ্যা: -v = সঙ্গে Anonymous Volume-ও মুছে ফেলো; Named Volume মুছে না।

# 7. Image-এর আকার ও Disk ব্যবহার দেখুন
docker images
# ব্যাখ্যা: লোকাল Image-এর তালিকা — Repository, Tag, Image ID, আকার, বয়স।
docker system df
# ব্যাখ্যা: Image, Container, Volume ও Build Cache — প্রতিটি কত Disk খাচ্ছে।
```

৪.৮ সম্পূর্ণ কনফিগারেশন উদাহরণ

TechNova-র API-এর জন্য Multi-Stage Dockerfile — Layer Cache, ছোট Image ও নিরাপত্তা — তিনটি একসঙ্গে:

```
# Dockerfile — technova-api (Node.js + Express, Multi-Stage)
# Stage 1: Builder — Build-এর জন্য সব টুল
FROM node:20-alpine AS builder
WORKDIR /app
# প্রথমে শুধু package ফাইল কপি — নির্ভরতা Layer Cache হবে, কোড বদলালে আবার ইনস্টল হবে না
COPY package.json package-lock.json ./
RUN npm ci --omit=dev
# তারপর কোড কপি ও Build (এই উদাহরণে TypeScript আছে ধরে নেওয়া হলো)
COPY . .
RUN npm run build
# Stage 2: Runner — শুধু চালানোর জন্য যা দরকার
FROM node:20-alpine AS runner
WORKDIR /app
# নিরাপত্তা: Root নয় এমন ব্যবহারকারী তৈরি
RUN addgroup -S appgroup && adduser -S appuser -G appgroup
# Builder থেকে শুধু ফল কপি — Source Code ও Dev নির্ভরতা আসে না
COPY --from=builder /app/node_modules ./node_modules
COPY --from=builder /app/dist ./dist
COPY --from=builder /app/package.json ./package.json
USER appuser
EXPOSE 4000
ENV NODE_ENV=production
CMD ["node", "dist/server.js"]
# ব্যাখ্যা: দুটি FROM মানে দুটি Stage; --from=builder মানে আগের Stage থেকে কপি;
# USER appuser = Root-এ চালানো নয়; EXPOSE = ডকুমেন্টেশন (Port খোলে না, শুধু জানায়);
# CMD = চালুর নির্দেশ — Dockerfile-এ শেষ CMD-টিই কার্যকর হয়।
```

আরও একটি গুরুত্বপূর্ণ ফাইল — `.dockerignore` — Build Context ছোট রাখে, যাতে Secret ও অপ্রয়োজনীয় ফাইল Image-এ না ঢোকে:

```
# .dockerignore — Build Context থেকে বাদ
node_modules
.git
.env
*.log
README.md
Dockerfile
.dockerignore
```

৪.৯ বাস্তব প্রোডাকশন পরিস্থিতি

REAL WORLD — TechNova: TechNova-র প্রথম Dockerfile-এ `COPY . .` ছিল প্রথম লাইনে, আর `RUN npm ci` ছিল তার পরে — ফলে কোডের একটি লাইন বদলালেই পুরো নির্ভরতা আবার ইনস্টল হতো, Build-এ লাগত ৮ মিনিট। Layer Cache-এর নিয়ম বুঝে তারা `package.json` আগে কপি করে `npm ci` আগে চালাল — Build নেমে এলো ৪০ সেকেন্ডে (কোড বদলে শুধু শেষ Layer তৈরি হয়)। আরেকটি ঘটনা: একটি Image-এ ভুল করে `.env` ফাইল কপি হয়ে গিয়েছিল — যাতে আসল Database Password ছিল — আর সেই Image Registry-তে Push করা হয়েছিল। Image-এর প্রতিটি Layer Tar ফাইলে থাকে, তাই Password মুছে ফেললেও পুরনো Layer-এ তা থেকে যায় — সমাধান ছিল Password বদলানো ও Image নতুন করে তৈরি করা। শিক্ষা: Secret কখনো Dockerfile-এর `COPY` বা `ENV`-এ লিখবেন না — Build Secret (`--secret`) বা Runtime Environment ব্যবহার করুন।

৪.১০ ধাপে ধাপে ব্যবহারিক বাস্তবায়ন

1. ৪.৮-এর Dockerfile দিয়ে আপনার MERN API-এর Image তৈরি করুন (`docker build -t technova-api:1.0.0 .`)।
2. `docker history technova-api:1.0.0` চালিয়ে প্রতিটি Layer-এর আকার লিখুন — সবচেয়ে বড় Layer কোনটি?
3. কোডের একটি লাইন বদলে আবার Build করুন — কোন Layer Cache থেকে এলো, কোনটি আবার তৈরি হলো, Build লগে `CACHED` চিহ্ন দেখে লিখুন।
4. `docker create`, `docker start`, `docker stop --time 30`, `docker rm` — আলাদাভাবে চালিয়ে প্রতিটি ধাপে `docker ps -a`-এ অবস্থা (created/running/exited) লিখুন।
5. Container-এর ভেতরে একটি ফাইল তৈরি করে (`docker exec api-test touch /app/test.txt`) Container মুছে আবার একই Image থেকে নতুন Container চালান — ফাইলটি আছে কি না দেখুন (থাকার কথা নয় — CoW ও Writable Layer-এর প্রমাণ)।

৪.১১ সাধারণ ভুল ও ভুল ধারণা

- **ভুল ধারণা:** "Container-এর ভেতরে ফাইল বদলালে Image বদলে যায়।" — না; বদলায় শুধু Writable Layer, Image অপরিবর্তিত থাকে।
- **ভুল:** `latest` Tag দিয়ে প্রোডাকশনে Deploy করা — কোন সংস্করণ চলছে তা জানা যায় না; সংস্করণ Tag (`1.0.0`) বা Digest ব্যবহার করুন।
- **ভুল:** `COPY . .` প্রথমে লেখা — Layer Cache নষ্ট হয়, প্রতিটি Build ধীর হয়। `package.json` আগে কপি করুন।
- **ভুল:** Secret (`.env`, API Key) Image-এ কপি করা — Layer-এ চিরকাল থেকে যায়; Build Secret বা Runtime Secret ব্যবহার করুন।
- **ভুল:** একটিমাত্র বড় Layer-এ সবকিছু করা (`RUN`-এ অনেক কাজ একসঙ্গে) — Cache-এর সুবিধা হারায়; আবার অপ্রয়োজনীয় অনেক Layer করাও ধীর করে — ভারসাম্য রাখুন।
- **ভুল ধারণা:** "`EXPOSE 4000` মানে Port খোলা।" — না; EXPOSE শুধু ডকুমেন্টেশন; Port খুলতে `-p` লাগে।
- **ভুল:** ডেটাবেসের ডেটা Writable Layer-এ রাখা — Container মুছলে ডেটা মুছে যায়; Volume ব্যবহার করুন।

৪.১২ নিরাপত্তা বিবেচনা

WARNING: Image-এর Layer-এ Secret লুকিয়ে থাকা সবচেয়ে সাধারণ ও বিপজ্জনক ভুল। `ENV MONGO_PASSWORD=CHANGE-ME-SECRET` বা `COPY .env .` লিখলে Secret Image-এর Layer-এ চিরকাল থেকে যায় — Image যার হাতে যায়, Secret তার হাতে যায়। Registry-তে Push করা Image যে কেউ টেনে Layer খুলে Secret পড়তে পারে। নিয়ম: Secret কখনো Image-এ নয় — Runtime-এ Environment Variable বা Secret Manager থেকে দিন; Build-সময়ে দরকার হলে Docker BuildKit-এর `--secret` ব্যবহার করুন, যা চূড়ান্ত Image-এ থাকে না। আর Base Image নিয়মিত আপডেট করুন — পুরনো Base Image-এ পরিচিত দুর্বলতা (CVE) থাকে; `docker scout` বা Trivy দিয়ে Image Scan করুন।

আরও নিরাপত্তা: ছোট Base Image ব্যবহার করুন (Alpine বা Distroless — কম প্যাকেজ মানে কম আক্রমণের পৃষ্ঠ); Root-এ নয়, নির্দিষ্ট ব্যবহারকারীতে চালান (`USER appuser`); `docker build`-এ BuildKit চালু রাখুন (`DOCKER_BUILDKIT=1`) — এটি Secret ও Cache নিরাপদে সামলায়।

৪.১৩ কর্মক্ষমতা ও রিসোর্স বিবেচনা

Image-এর আকার সরাসরি Pull সময়, Disk ব্যবহার ও Deploy গতিকে প্রভাবিত করে — 1 GB Image Pull করতে মিনিট লাগে, 100 MB Image-এ সেকেন্ড লাগে। আকার কমাতে: Multi-Stage Build ব্যবহার করুন; `RUN`-এর পর Package Cache মুছুন (`rm -rf /var/lib/apt/lists/*`); একাধিক `RUN` একত্রে লিখে অস্থায়ী ফাইল একই Layer-এ মুছুন (আলাদা Layer-এ মুছলে আকার কমে না — পুরনো Layer-এ ফাইল থেকেই যায়); `.dockerignore` দিয়ে Context ছোট রাখুন। Layer সংখ্যা 20–30-এর মধ্যে রাখা ভালো — বেশি Layer Mount ধীর করে। Writable Layer-এ বেশি লেখা CoW-এর খরচ বাড়ায় — ডেটাবেস ও লগের জন্য Volume ব্যবহার করুন।

৪.১৪ সমস্যা সমাধান (Troubleshooting)

TROUBLESHOOTING: "Build-এ Cache ব্যবহার হচ্ছে না, প্রতিবার সব আবার তৈরি হয়" — প্রথমে দেখুন কোন নির্দেশের পরে Cache ভাঙছে: `docker build`-এর লগে প্রথম `CACHED` নয় এমন ধাপটিই কারণ। সাধারণ কারণ: `COPY . .` খুব ওপরে থাকা (যেকোনো ফাইল বদলালে Cache ভাঙে), `.dockerignore` না থাকা (অপ্রয়োজনীয় ফাইল Context-এ ঢুকে Cache ভাঙে), বা Build Argument (`--build-arg`) বদলে যাওয়া। আরেকটি সাধারণ সমস্যা: "Container চালু হয়েই বন্ধ হয়ে যায় (Exited 0)" — কারণ মূল Process শেষ হয়ে গেছে; `docker logs` দেখুন — অ্যাপটি কি ভুল করে শেষ হয়ে গেছে, নাকি CMD ভুল লেখা হয়েছে।

1. **লক্ষণ:** `docker build`-এ "no space left on device" — **সমাধান:** `docker system df` দেখুন, `docker system prune` চালান, Build Cache সীমা দেখুন।
2. **লক্ষণ:** Image Pull-এ "manifest unknown" — **কারণ:** Tag বা Digest ভুল, বা Registry-তে সেই Image নেই। **সমাধান:** Tag যাচাই করুন, Registry-তে Image আছে কি না দেখুন।
3. **লক্ষণ:** Container-এর ভেতরে ফাইল বদলালেও Host-এ দেখা যায় না — **কারণ:** এটাই স্বাভাবিক — Writable Layer Container-এর নিজের। **সমাধান:** ফাইল ভাগ করতে Volume Mount করুন।
4. **লক্ষণ:** `docker stop`-এ Container 10 সেকেন্ড আটকে থাকে — **কারণ:** অ্যাপ SIGTERM-এ সাড়া দেয় না। **সমাধান:** Node.js অ্যাপে SIGTERM Handler লিখুন (Graceful Shutdown), বা `--time 30` দিন।

৪.১৫ সেরা অনুশীলন ও অ্যান্টি-প্যাটার্ন

সেরা অনুশীলন: Multi-Stage Build ব্যবহার করুন; `package.json` আগে কপি করে Layer Cache বাঁচান; সংস্করণ Tag ও Digest ব্যবহার করুন; `.dockerignore` রাখুন; ছোট Base Image (Alpine/Distroless) বেছে নিন; Root-এ চালাবেন না; Image Scan করুন; প্রতিটি Image-এ Label দিন (সংস্করণ, Commit Hash)। **অ্যান্টি-প্যাটার্ন:** `latest` দিয়ে প্রোডাকশন চালানো; Secret Image-এ রাখা; একটিমাত্র Stage-এ Build ও Runtime একসঙ্গে রাখা; `COPY . .` প্রথমে লেখা; `RUN apt-get update` ও `RUN apt-get install` আলাদা Layer-এ লেখা (Cache পুরনো হয়ে Package সংস্করণের অসামঞ্জস্য হয়)।

৪.১৬ অধ্যায়ের সারাংশ

Image হলো অপরিবর্তনীয়, স্তরে স্তরে সাজানো প্যাকেজ — প্রতিটি Layer-এর SHA256 Digest আছে, আর OCI Manifest সেগুলোর সূচিপত্র। Container হলো Image + Writable Layer + Process। Union Filesystem (overlay2) Layer-গুলোকে একটি ফাইলসিস্টেম হিসেবে দেখায়; Copy-on-Write নিয়মে লেখার সময় ফাইলটি Writable Layer-এ কপি হয়, নিচের Layer বদলায় না। `create` Root Filesystem সাজায়, `start` Process শুরু করে, `stop` প্রথমে SIGTERM তারপর SIGKILL দেয়, `rm` Writable Layer মুছে ফেলে। Multi-Stage Build Image ছোট ও নিরাপদ করে; Layer Cache Build দ্রুত করে; Secret কখনো Image-এ রাখা যায় না।

৪.১৭ মূল প্রযুক্তিগত পরিভাষা

Image: অপরিবর্তনীয়, স্তরে সাজানো অ্যাপ প্যাকেজ — Layer, Config ও Manifest-এর সমষ্টি।

Layer: Image-এর একটি ধাপ — Dockerfile-এর একটি নির্দেশের ফল; শুধু বদলানো অংশ থাকে।

Union Filesystem (OverlayFS / overlay2): একাধিক Layer-কে একটি ফাইলসিস্টেম হিসেবে দেখানোর কৌশল।

Copy-on-Write (CoW): পড়ার সময় সরাসরি পড়া, লেখার সময় প্রথমে Writable Layer-এ কপি করা।

Writable Layer: Container-এর নিজের লেখার স্তর — Container মুছলে মুছে যায়।

Digest: Layer বা Image-এর SHA256 Hash — অপরিবর্তনীয় পরিচয়।

Manifest: Image-এর সূচিপত্র — Config ও Layer-এর Digest-এর তালিকা।

Tag: Image-এর চলমান নাম (`1.0.0` , `latest`); Digest-এর মতো অপরিবর্তনীয় নয়।

Layer Cache: Build-এ অপরিবর্তিত Layer আবার ব্যবহার করার সুবিধা।

Multi-Stage Build: এক Dockerfile-এ একাধিক Stage — Builder Stage Build করে, Runner Stage শুধু ফল নেয়।

Build Context: `docker build`-এ Daemon-কে পাঠানো ফাইলের সমষ্টি; `.dockerignore` দিয়ে ছোট রাখা হয়।

Whiteout: ওপরের Layer-এ নিচের ফাইল মুছে ফেলার চিহ্ন — নিচের ফাইলটি মুছে না, শুধু লুকিয়ে যায়।

৪.১৮ জ্ঞান যাচাই — প্রশ্ন

1. Image ও Container-এর পার্থক্য নিজের ভাষায় ব্যাখ্যা করুন — Class/Object তুলনাটি ব্যবহার করে।
2. Copy-on-Write কীভাবে কাজ করে? একটি ফাইল পড়া ও লেখার সময় ধাপে ধাপে বলুন।
3. Layer Cache কেন ভাঙে? `COPY . .`-এর অবস্থান কেন গুরুত্বপূর্ণ?
4. Tag ও Digest-এর পার্থক্য কী? প্রোডাকশনে Digest কেন নিরাপদ?
5. `docker create`, `start`, `stop`, `rm` — প্রতিটির ভেতরে কী ঘটে?
6. Multi-Stage Build Image ছোট ও নিরাপদ করে কেন?
7. Container মুছলে ডেটা মুছে যায় কেন? Volume কীভাবে সমাধান দেয়?

৪.১৯ ব্যবহারিক অনুশীলন

1. **Layer মানচিত্র:** ৪.১০-এর ধাপগুলো করে আপনার Image-এর Layer তালিকা ও আকারের রিপোর্ট লিখুন।
2. **Multi-Stage:** ৪.৮-এর Dockerfile আপনার MERN API-এর জন্য মানিয়ে লিখুন ও এক-Stage Image-এর সঙ্গে আকার তুলনা করুন (`docker images`)।
3. **CoW প্রমাণ:** একটি Container-এর ভেতরে ফাইল তৈরি করে Container মুছে নতুন Container-এ ফাইলটি নেই — এই পরীক্ষাটি করে ফল লিখুন।

অধ্যায় 5 • PART I — Container Foundations

Chapter 5 — Container Lifecycle and Resource Management

৫.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: (১) Container-এর Lifecycle — created, running, paused, stopped, removed — প্রতিটি অবস্থা ও পরিবর্তন ব্যাখ্যা করতে; (২) Lifecycle Event (`docker events`) পড়ে কী ঘটছে তা বুঝতে; (৩) `docker stats` দিয়ে CPU, মেমরি, নেটওয়ার্ক ও Disk I/O পর্যবেক্ষণ করতে; (৪) CPU, মেমরি, PIDs ও I/O সীমা বসাতে ও যাচাই করতে; (৫) Restart Policy ও Healthcheck লিখতে; (৬) Log পরিচালনা ও Graceful Shutdown-এর নিয়ম বলতে।

৫.২ কেন এই বিষয়টি DevOps-এ গুরুত্বপূর্ণ

Container চালু করা সহজ, চালিয়ে রাখা কঠিন। প্রোডাকশনে Container হঠাৎ বন্ধ হয়, Restart লুপে পড়ে, মেমরি খেয়ে Host-কে ফেলে দেয়, বা লগে Disk ভরিয়ে ফেলে — আর প্রতিটি ক্ষেত্রেই প্রশ্ন একটাই: "কী ঘটেছে, কেন ঘটেছে, আর কীভাবে ঠিক করব?" Lifecycle না জানলে আপনি `docker ps`-এ একটি Container-কে "Exited" দেখে আন্দাজে Restart করবেন; Lifecycle জানলে Exit Code, Event ও লগ পড়ে কারণ বের করবেন। Resource Management না জানলে একটি Container-এর লিক পুরো Host-কে মারবে (অধ্যায় ২-এর TechNova ঘটনা)। এই অধ্যায়টি Container-কে "চালানো" থেকে "পরিচালনা করা"-তে নিয়ে যায় — এটাই DevOps Engineer-এর দৈনন্দিন কাজ।

৫.৩ পূর্বশর্ত জ্ঞান

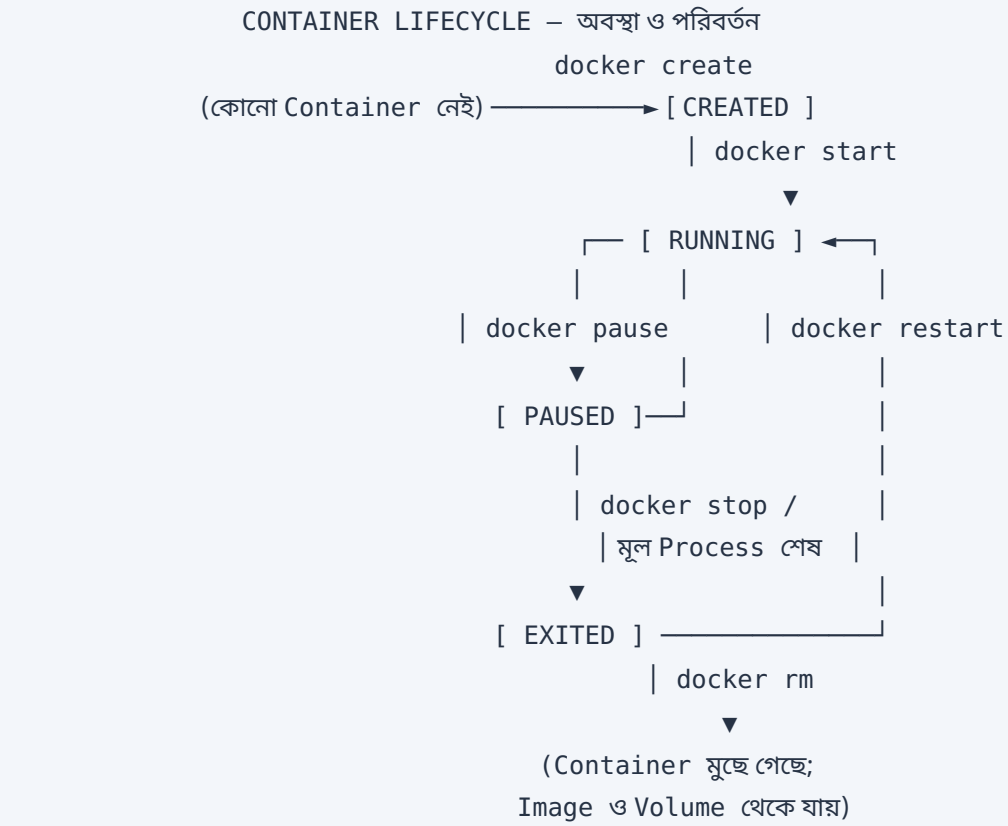
অধ্যায় ১–৪-এর VM/Container পার্থক্য, Namespace/Cgroup, Docker Engine-এর স্তর, Image/Layer ও create/start/stop/remove-এর ভেতরের কাজ জানা থাকতে হবে। Linux Signal-এর প্রাথমিক ধারণা (SIGTERM, SIGKILL) ও `ps` / `top`-এর অভিজ্ঞতা থাকলে ভালো হয়।

৫.৪ প্রথম নীতি থেকে ব্যাখ্যা

একটি Container আসলে একটি Process-এর জীবন — আর প্রতিটি Process-এর জীবনচক্র আছে: তৈরি হওয়া, চলা, থামা, মরে যাওয়া। Container-এর Lifecycle সেই জীবনচক্রেরই আনুষ্ঠানিক রূপ। **Created** মানে Container তৈরি, কিন্তু Process এখনো শুরু হয়নি — যেন একটি গাড়ি তৈরি, কিন্তু ইঞ্জিন বন্ধ। **Running** মানে PID 1 চলছে। **Paused** মানে Process-কে সাময়িকভাবে স্থগিত করা হয়েছে (SIGSTOP দিয়ে) — Container আছে, কিন্তু CPU ব্যবহার করছে না। **Exited/Stopped** মানে মূল Process শেষ হয়েছে — স্বেচ্ছায় (Exit Code 0) বা জোর করে (SIGKILL, Exit Code 137)। **Removed** মানে Container-এর Writable Layer ও Config মুছে ফেলা হয়েছে — Image থেকে যায়। এই অবস্থাগুলোর মধ্যে যাওয়া-আসা হয় নির্দিষ্ট কমান্ডে: `create`, `start`, `pause` / `unpause`, `stop`, `kill`, `restart`, `rm`। আর প্রতিটি পরিবর্তন একটি **Event** হিসেবে লেখা থাকে — `docker events` দিয়ে সেই ইতিহাস দেখা যায়। Resource Management-এর মূল নীতি: Container ডিফল্টভাবে সীমাহীন — Host-এর সবকিছু খেতে পারে — তাই প্রতিটি Container-এ সীমা বসানোই পরিচালনার প্রথম কাজ।

CONCEPT: Container-এর জীবন Process-এর জীবন — Image নয়। `docker run` মানে একটি Process শুরু করা; মূল Process মারা গেলে Container মারা যায়, যতই Image ভালো হোক। তাই অ্যাপের মূল Process-কে Foreground-এ চালাতে হয় — Background-এ পাঠিয়ে দিলে (যেমন `node server.js &`) মূল Process শেষ হয়ে Container বন্ধ হয়ে যায়। এটি নতুনদের সবচেয়ে সাধারণ ভুল।

৫.৫ অভ্যন্তরীণ আর্কিটেকচার ও চিত্র



EXIT CODE — মৃত্যুর কারণ পড়ার উপায়

0 = স্বেচ্ছায় সফলভাবে শেষ (স্বাভাবিক)

1 = অ্যাপের ত্রুটি (যেমন Exception)

137 = 128 + 9 (SIGKILL) — OOM-Killed বা docker kill

143 = 128 + 15 (SIGTERM) — ভদ্রভাবে বন্ধের সংকেত

125 = Docker Daemon-এর ত্রুটি (যেমন ভুল ফ্ল্যাগ)

126 = কমান্ড চালানো যায়নি (Permission)

127 = কমান্ড খুঁজে পাওয়া যায়নি (ভুল পথ)

docker stats — কী মাপা হয়

CPU % • MEM USAGE / LIMIT • MEM % • NET I/O • BLOCK I/O • PIDS

মনে রাখবেন: `docker ps` শুধু Running Container দেখায়; `docker ps -a` সব অবস্থার Container দেখায় — মৃত Container-এর Exit Code দেখতে `-a` লাগে।

৫.৬ বিস্তারিত প্রযুক্তিগত ব্যাখ্যা

Signal ও Graceful Shutdown

`docker stop` প্রথমে `SIGTERM` (15) পাঠায় — অ্যাপকে বলে "ভদ্রভাবে বন্ধ হও" — ডিফল্ট 10 সেকেন্ড অপেক্ষা করে, তারপর `SIGKILL` (9) দিয়ে জোর করে বন্ধ করে। Node.js অ্যাপে SIGTERM Handler লিখতে হয়: চলমান অনুরোধ শেষ করা, ডেটাবেস সংযোগ বন্ধ করা, তারপর `process.exit(0)`। Handler না থাকলে Node.js SIGTERM-এ সঙ্গে সঙ্গে মারা যায় — চলমান অনুরোধ মাঝপথে কেটে যায়। `docker kill` সরাসরি SIGKILL দেয় (বা `-s` দিয়ে অন্য Signal) — কোনো অপেক্ষা নেই। PID 1-এর বিশেষত্ব: Kernel PID 1-কে ডিফল্ট Signal Handler দেয় না — তাই PID 1 হিসেবে চলা Node.js অ্যাপকে Signal নিজে সামলাতে হয়, অথবা `--init` দিয়ে tini বসাতে হয়।

Restart Policy — কখন আবার চালু হবে

`--restart` ঠিক করে Container মারা গেলে Daemon কী করবে: `no` (ডিফল্ট — আবার চালাবে না), `on-failure[:max-retries]` (শুধু ভুল Exit Code-এ আবার চালাবে), `always` (সবসময় আবার চালাবে — এমনকি Daemon Restart-এর পরেও), `unless-stopped` (সবসময়, তবে আপনি নিজে Stop করলে আর চালাবে না)। প্রোডাকশনে সাধারণত `unless-stopped` ব্যবহার করা হয় — Host Reboot-এর পর Container নিজে থেকে ফিরে আসে, কিন্তু রক্ষণাবেক্ষণের জন্য Stop করলে আর চালু হয় না। সতর্কতা: Restart Policy অ্যাপের ভুল লুকিয়ে ফেলতে পারে — একটি Container বারবার Crash করে বারবার Restart হলে (Restart Loop) কারণটি খুঁজুন, Policy দিয়ে ঢাকবেন না।

Healthcheck — Container চলছে, কিন্তু অ্যাপ সুস্থ তো?

Running মানে Process চলছে — অ্যাপ সাড়া দিচ্ছে কি না তা বলে না। **Healthcheck** নিয়মিত একটি পরীক্ষা চালায় — যেমন API-এর `/health` Endpoint-এ অনুরোধ পাঠানো — আর ফল অনুযায়ী Container-কে `healthy`, `unhealthy` বা `starting` হিসেবে চিহ্নিত করে। Orchestrator (Swarm/Kubernetes) Unhealthy Container-কে সরিয়ে নতুনটি বসায়। Healthcheck-এর সেটিং: `--interval` (কতক্ষণ পর পর), `--timeout`, `--start-period` (শুরুর Grace সময় — অ্যাপ গরম হতে সময় লাগে), `--retries` (কতবার ব্যর্থ হলে Unhealthy)।

Resource Limit — Cgroup সীমার প্রয়োগ

`--memory` (যেমন `512m`) মেমরি সীমা; `--memory-swap` মেমরি + Swap-এর মোট সীমা; `--cpus` (যেমন `1.5`) CPU সীমা; `--cpu-shares` আপেক্ষিক গুরুত্ব (প্রতিযোগিতায় কে বেশি পাবে); `--pids-limit` Process সংখ্যা; `--device-read-bps` / `--device-write-bps` Disk I/O সীমা। সীমা বসানোর পর যাচাই করুন `docker inspect`-এ `HostConfig` দেখে ও `docker stats`-এ ব্যবহার দেখে। মনে রাখবেন: CPU সীমা ছাড়ালে Throttle, মেমরি ছাড়ালে OOM-Kill — দুটির লক্ষণ আলাদা (অধ্যায় ২)।

Log পরিচালনা

Container-এর STDOUT/STDERR Daemon সংগ্রহ করে — `docker logs` দিয়ে দেখা যায়, `-f` দিয়ে Live দেখা যায়, `--tail` দিয়ে শেষ কয়েক লাইন দেখা যায়। ডিফল্ট Log Driver `json-file` — লগ `/var/lib/docker/containers/<id>/<id>-json.log`-এ জমা হয়। Rotation না থাকলে লগ Disk ভরিয়ে ফেলে — তাই `daemon.json`-এ বা `--log-opt max-size=10m --log-opt max-file=3` দিয়ে সীমা বসান। প্রোডাকশনে লগ Central জায়গায় পাঠানো হয় (যেমন Loki, Elasticsearch) — Container-এর লগ কখনোই একমাত্র কপি হিসেবে রাখবেন না।

৫.৭ Docker কমান্ড — লাইন ধরে ধরে ব্যাখ্যা

```
# 1. সব অবস্থার Container দেখুন — Exit Code-সহ
docker ps -a
# ব্যাখ্যা: -a = সব Container (চলমান ও বন্ধ); STATUS কলামে Up/Exited ও Exit Code দেখায়।
docker ps --format "table {{.Names}}\t{{.Status}}\t{{.Ports}}"
# ব্যাখ্যা: --format = নিজের পছন্দের কলামে সাজানো আউটপুট।

# 2. Resource ব্যবহার Live দেখুন — সবচেয়ে গুরুত্বপূর্ণ পর্যবেক্ষণ কমান্ড
docker stats
# চিত্রণমূলক আউটপুট:
#   NAME           CPU %   MEM USAGE / LIMIT   MEM %   NET I/O       BLOCK I/O   PIDS
#   technova-api   2.15%   128MiB / 512MiB     25%     1.2MB / 800kB  0B / 0B      11
# ব্যাখ্যা: CPU % = CPU ব্যবহার; MEM USAGE/LIMIT = মেমরি ব্যবহার ও সীমা;
# NET I/O = নেটওয়ার্কে আসা/যাওয়া; BLOCK I/O = Disk পড়া/লেখা; PIDS = Process সংখ্যা।
docker stats --no-stream technova-api
# ব্যাখ্যা: --no-stream = একবার দেখিয়ে থামো (স্ক্রিপ্টে ব্যবহারের জন্য)।

# 3. Lifecycle Event দেখুন — কী ঘটেছে, কখন ঘটেছে
docker events --since "1h" --filter container=technova-api
# ব্যাখ্যা: events = Daemon-এর ঘটনার ইতিহাস; --since = শেষ ১ ঘণ্টা;
# --filter = শুধু একটি Container-এর ঘটনা; ঘটনা: create, start, die, stop, destroy।

# 4. সীমাসহ চালান ও Restart Policy বসান
docker run -d --name technova-api --memory 512m --memory-swap 512m --cpus 1.0 \
  --pids-limit 200 --restart unless-stopped -p 4000:4000 technova-api:1.0.0
# ব্যাখ্যা: --memory/--cpus = Cgroup সীমা; --restart unless-stopped = মারা গেলে
# আবার চালাও, তবে নিজে Stop করলে নয়; -p 4000:4000 = Host Port 4000 থেকে
# Container Port 4000-এ পাঠাও (Host:Container ক্রম)।

# 5. Healthcheck-সহ চালান
docker run -d --name technova-api --health-cmd "wget -q -O /dev/null http://localhost:4000/
health || exit 1" \
  --health-interval 30s --health-timeout 5s --health-start-period 10s --health-retries 3 \
  technova-api:1.0.0
# ব্যাখ্যা: --health-cmd = সুস্থতা পরীক্ষার কমান্ড (Exit 0 = সুস্থ);
# --health-interval 30s = ৩০ সেকেন্ড পর পর; --health-start-period 10s = শুরুর
# ১০ সেকেন্ড ব্যর্থতা গোনা হবে না; --health-retries 3 = ৩ বার ব্যর্থ হলে Unhealthy।

# 6. থামানো, বিরতি ও আবার চালু
docker pause technova-api
# ব্যাখ্যা: Process স্থগিত করো (SIGSTOP) — Container আছে, CPU ব্যবহার বন্ধ।
docker unpause technova-api
# ব্যাখ্যা: স্থগিত Process আবার চালাও (SIGCONT)।
docker stop --time 30 technova-api
# ব্যাখ্যা: SIGTERM, ৩০ সেকেন্ড অপেক্ষা, তারপর SIGKILL।
docker kill technova-api
# ব্যাখ্যা: সঙ্গে সঙ্গে SIGKILL — কোনো অপেক্ষা নেই; শুধু জরুরি ক্ষেত্রে।
docker restart technova-api
# ব্যাখ্যা: stop + start একসঙ্গে — কনফিগারেশন বদলের পর নয়, শুধু পুনরায় চালুর জন্য।

# 7. লগ দেখুন ও অনুসরণ করুন
docker logs --tail 50 -f technova-api
# ব্যাখ্যা: --tail 50 = শেষ ৫০ লাইন; -f = নতুন লগ আসতেই দেখাও (follow)।
docker inspect technova-api --format '{{ .State.Status }} {{ .State.ExitCode }}
```

```
{{ .State.OOMKilled }}'
# ব্যাখ্যা: অবস্থা, Exit Code ও OOM-Killed কি না — এক লাইনে নির্ণয়।
```

৫.৮ সম্পূর্ণ কনফিগারেশন উদাহরণ

```
# docker-compose.yml — TechNova MERN স্ট্যাক, সীমা ও Healthcheck-সহ (অংশবিশেষ)
services:
  api:
    image: technova-api:1.0.0
    restart: unless-stopped
    mem_limit: 512m
    cpus: 1.0
    pids_limit: 200
    user: "1000:1000"
    read_only: true
    ports:
      - "4000:4000"
    environment:
      - NODE_ENV=production
      - MONGO_URI=mongodb://db:27017/technova
      - JWT_SECRET=CHANGE-ME-SECRET
    healthcheck:
      test: ["CMD", "wget", "-q", "-O", "/dev/null", "http://localhost:4000/health"]
      interval: 30s
      timeout: 5s
      start_period: 10s
      retries: 3
    logging:
      driver: json-file
      options:
        max-size: "10m"
        max-file: "3"
    depends_on:
      db:
        condition: service_healthy
    networks:
      - technova-net
  db:
    image: mongo:7
    restart: unless-stopped
    mem_limit: 1g
    cpus: 1.0
    volumes:
      - mongo-data:/data/db
    healthcheck:
      test: ["CMD", "mongosh", "--eval", "db.adminCommand('ping')"]
      interval: 10s
      timeout: 5s
      retries: 5
    networks:
      - technova-net
volumes:
  mongo-data:
networks:
  technova-net:
    driver: bridge
# ব্যাখ্যা: depends_on + service_healthy = DB সুস্থ না হলে API চালু হবে না;
```

```
# JWT_SECRET এখানে প্লেসহোল্ডার — আসল Secret Environment বা Secret Manager থেকে দিতে হয়;
# mongo-data Named Volume = DB-এর ডেটা Container মুছলেও থেকে যায়।
```

৫.৯ বাস্তব প্রোডাকশন পরিস্থিতি

REAL WORLD — TechNova: TechNova-র API Container প্রতি রাতে ৩টায় Restart হচ্ছিল — কেউ বুঝতে পারছিল না কেন। `docker events` দেখাল প্রতিরাতে `die` Event, Exit Code 137; `docker inspect`-এ `OOMKilled: true`; আর `docker stats`-এর ইতিহাসে দেখা গেল রাতের Batch কাজে মেমরি ধীরে ধীরে বেড়ে 512 MB সীমা ছুঁয়ে ফেলছিল। কারণ: Batch কাজে একটি Array-তে সব রেকর্ড জমা করা হচ্ছিল — মেমরি লিক নয়, নকশার ভুল। সমাধান: Batch কাজ ছোট ছোট ভাগে ভাগ করা (Streaming), আর মেমরি সীমা 512 MB থেকে 768 MB করা — তবে সীমা বাড়ানোর আগে কারণ ঠিক করা হয়েছিল, নইলে লিক আরও বড় হয়ে ফিরত। শিক্ষা: Exit Code 137 দেখলে প্রথমে OOMKilled যাচাই করুন, তারপর `docker stats`-এ মেমরি-বৃদ্ধির ধারা দেখুন।

৫.১০ ধাপে ধাপে ব্যবহারিক বাস্তবায়ন

1. ৫.৮-এর Compose ফাইল দিয়ে স্ট্যাক চালু করুন (`docker compose up -d`) ও `docker ps`-এ অবস্থা দেখুন।
2. `docker stats --no-stream` চালিয়ে প্রতিটি Container-এর CPU, মেমরি ও PIDs লিখুন।
3. `docker exec technova-api wget -q -O /dev/null http://localhost:4000/health; echo $?` চালিয়ে Healthcheck-এর কমান্ডটি হাতে পরীক্ষা করুন।
4. API Container-কে `docker stop --time 30` দিয়ে থামান, `docker ps -a`-এ Exit Code লিখুন, তারপর `docker start` দিয়ে আবার চালান।
5. `docker events --since 10m` চালিয়ে আপনার করা প্রতিটি কাজের Event খুঁজুন ও লিখুন।
6. ইচ্ছাকৃতভাবে `--memory 64m` দিয়ে একটি Container চালিয়ে মেমরি-খাওয়া স্ক্রিপ্ট চালান — Exit 137 ও OOMKilled দেখুন।

৫.১১ সাধারণ ভুল ও ভুল ধারণা

- **ভুল:** মূল Process-কে Background-এ চালানো (`node server.js &`) — মূল Process শেষ হয়ে Container বন্ধ হয়ে যায়। Foreground-এ চালান।
- **ভুল ধারণা:** "Running মানেই সুস্থ।" — না; Process চললেও অ্যাপ সাড়া না দিতে পারে — Healthcheck লাগে।
- **ভুল:** সীমা না দেওয়া — একটি Container পুরো Host-এর RAM খেয়ে ফেলতে পারে।
- **ভুল:** `docker kill` দিয়ে নিয়মিত বন্ধ করা — চলমান অনুরোধ ও ডেটা নষ্ট হয়; `docker stop` ব্যবহার করুন।
- **ভুল:** Restart Policy দিয়ে Crash লুকানো — Restart Loop-এর কারণ খুঁজুন, Policy দিয়ে ঢাকবেন না।
- **ভুল:** লগ Rotation না করা — একটি Container-এর লগ Disk ভরিয়ে সব Container ফেলে দিতে পারে।
- **ভুল ধারণা:** "`docker restart` মানে নতুন Image নেওয়া।" — না; Restart একই Container-কে আবার চালায়, নতুন Image টানে না — নতুন Image চাইলে পুরনো Container মুছে নতুনটি তৈরি করতে হয়।

৫.১২ নিরাপত্তা বিবেচনা

WARNING: Resource Limit নিরাপত্তার অংশ — সীমা না থাকলে একটি আক্রান্ত Container Host-এর সব RAM/CPU খেয়ে অন্য Container-কে বন্ধ করে দিতে পারে (Denial of Service)। PIDs সীমা না থাকলে Fork Bomb পুরো Host অচল করে দিতে পারে। লগে Secret ছাপাবেন না — লগ Central জায়গায় যায়, সেখানে Secret ছড়িয়ে পড়ে। Healthcheck কমান্ডে Secret বা Token লিখবেন না। আর Restart Policy-এর কারণে একটি আক্রান্ত Container বারবার ফিরে আসতে পারে — সন্দেহজনক Container-কে প্রথমে `docker stop` করে তদন্ত করুন, Restart-এ ছেড়ে দেবেন না।

৫.১৩ কর্মক্ষমতা ও রিসোর্স বিবেচনা

সঠিক সীমা বের করার নিয়ম: Staging-এ সীমা ছাড়া ২৪ ঘণ্টা চালিয়ে `docker stats`-এ সর্বোচ্চ ব্যবহার মাপুন, তারপর তার ১.৫ গুণ সীমা বসান, আর ৮০% ব্যবহারে Alert বসান। CPU সীমা খুব কম দিলে Throttle হয়ে Response ধীর হয় — `docker stats`-এ CPU 100% দেখালে সীমা যাচাই করুন। মেমরি সীমা খুব কম দিলে OOM-Kill হয় — মেমরি-বৃদ্ধির ধারা দেখুন, হঠাৎ লাফ নাকি ধীরে বৃদ্ধি (লিক)। Disk I/O সীমা না দিলে একটি Container-এর Backup বা লগ লেখা অন্য Container-এর ডেটাবেসকে ধীর করে দিতে পারে। `docker stats` নিয়মিত দেখুন — এটি `top`-এর Container-সংস্করণ।

৫.১৪ সমস্যা সমাধান (Troubleshooting)

TROUBLESHOOTING: Container বারবার Restart হচ্ছে (Restart Loop) — ধাপ: (১) `docker ps -a`-এ Exit Code দেখুন; (২) `docker logs --tail 50`-এ শেষ ত্রুটি পড়ুন; (৩) `docker inspect`-এ OOMKilled যাচাই করুন; (৪) `docker events`-এ Restart-এর সময় দেখুন; (৫) কারণ না পেলে Container-কে `docker run --rm -it --entrypoint sh` দিয়ে হাতে চালিয়ে ভেতরে পরীক্ষা করুন। "Container চালু হয়েই বন্ধ" হলে প্রথমে দেখুন মূল Process কি সঙ্গে সঙ্গে শেষ হয়ে যাচ্ছে — `docker logs`-এ তার প্রমাণ থাকে।

1. **লক্ষণ:** Exit 137 — **কারণ:** OOM-Killed বা Kill। **পরীক্ষা:** `OOMKilled` ফিল্ড; **সমাধান:** সীমা বাড়ান বা লিক ঠিক করুন।
2. **লক্ষণ:** Exit 1, লগে `EADDRINUSE` — **কারণ:** Port ইতিমধ্যে ব্যবহৃত। **সমাধান:** Host Port বদলান বা পুরনো Container থামান।
3. **লক্ষণ:** Healthcheck Unhealthy, কিন্তু অ্যাপ চলে — **কারণ:** Healthcheck কমান্ড ভুল (যেমন `wget` Image-এ নেই) বা `start-period` খুব কম। **সমাধান:** কমান্ডটি `docker exec` দিয়ে হাতে পরীক্ষা করুন।
4. **লক্ষণ:** `docker stop` 10 সেকেন্ড আটকে থাকে — **কারণ:** অ্যাপ SIGTERM সামলায় না। **সমাধান:** SIGTERM Handler লিখুন বা `--time 30` দিন।

৫.১৫ সেরা অনুশীলন ও অ্যান্টি-প্যাটার্ন

সেরা অনুশীলন: প্রতিটি Container-এ CPU, মেমরি ও PIDs সীমা দিন; `unless-stopped` Restart Policy ব্যবহার করুন; Healthcheck লিখুন; SIGTERM Handler লিখুন (Graceful Shutdown); লগ Rotation বসান; `docker stats` ও `docker events` নিয়মিত পর্যবেক্ষণ করুন; Exit Code পড়ে নির্ণয় করুন। **অ্যান্টি-প্যাটার্ন:** সীমা ছাড়া চালানো;

`docker kill` দিয়ে নিয়মিত বন্ধ করা; Restart Policy দিয়ে Crash লুকানো; Healthcheck ছাড়া প্রোডাকশনে চালানো; লগে Secret ছাপানো; মূল Process Background-এ চালানো।

৫.১৬ অধ্যায়ের সারাংশ

Container-এর জীবনচক্র: created → running → (paused) → exited → removed — প্রতিটি পরিবর্তন একটি Event। মূল Process মারা গেলে Container মারা যায়; Exit Code মৃত্যুর কারণ বলে (137 = OOM-Kill, 143 = SIGTERM, 0 = স্বাভাবিক)। `docker stop` প্রথমে SIGTERM তারপর SIGKILL দেয়; Graceful Shutdown-এর জন্য অ্যাপে Signal Handler লাগে। Restart Policy ও Healthcheck Container-কে সুস্থ রাখে; `docker stats` CPU, মেমরি, নেটওয়ার্ক ও I/O মাপে; আর CPU, মেমরি, PIDs ও I/O সীমা একটি Container-এর ক্ষতির পরিধি বেঁধে রাখে।

৫.১৭ মূল প্রযুক্তিগত পরিভাষা

Lifecycle: Container-এর অবস্থার ক্রম — created, running, paused, exited, removed।

Exit Code: Process শেষ হওয়ার সংখ্যা-সংকেত — মৃত্যুর কারণ বোঝায় (যেমন 137 = SIGKILL)।

SIGTERM / SIGKILL: বন্ধের সংকেত — SIGTERM ভদ্র অনুরোধ, SIGKILL জোরপূর্বক বন্ধ।

Graceful Shutdown: চলমান কাজ শেষ করে, সংযোগ বন্ধ করে ভদ্রভাবে বন্ধ হওয়া।

Restart Policy: Container মারা গেলে Daemon আবার চালাবে কি না তার নিয়ম (no, on-failure, always, unless-stopped)।

Healthcheck: Container-এর অ্যাপ সুস্থ কি না তা নিয়মিত পরীক্ষা করার ব্যবস্থা।

docker stats: চলমান Container-এর CPU, মেমরি, নেটওয়ার্ক ও I/O ব্যবহার Live দেখার কমান্ড।

docker events: Daemon-এর ঘটনার ইতিহাস — create, start, die, stop, destroy।

OOM-Killed: মেমরি সীমা ছাড়িয়ে OOM Killer-এর হাতে Process মারা যাওয়া; Exit Code 137।

Throttling: CPU সীমা ছাড়িয়ে Process-কে ধীর করে দেওয়া; Kill করা হয় না।

৫.১৮ জ্ঞান যাচাই — প্রশ্ন

1. Lifecycle-এর পাঁচটি অবস্থা লিখুন ও প্রতিটিতে যাওয়ার কমান্ড বলুন।
2. Exit Code 0, 1, 137 ও 143 — প্রতিটির অর্থ কী?
3. `docker stop` ও `docker kill`-এর পার্থক্য কী? কখন কোনটি ব্যবহার করবেন?
4. Running Container সুস্থ নাও হতে পারে কেন? Healthcheck কীভাবে সাহায্য করে?
5. Restart Policy-এর চারটি বিকল্পের পার্থক্য কী? প্রোডাকশনে কোনটি সাধারণ?
6. `docker stats`-এর প্রতিটি কলামের অর্থ লিখুন।
7. মেমরি সীমা ছাড়ালে কী হয়, CPU সীমা ছাড়ালে কী হয় — পার্থক্যটি ব্যাখ্যা করুন।

৫.১৯ ব্যবহারিক অনুশীলন

1. **Lifecycle মানচিত্র:** ৫.১০-এর ধাপগুলো করে প্রতিটি অবস্থা, Exit Code ও Event-এর রিপোর্ট লিখুন।
2. **OOM পরীক্ষা:** `--memory 64m` দিয়ে Container চালিয়ে OOM-Killed (Exit 137) দেখুন ও `docker inspect`-এ প্রমাণ খুঁজুন।
3. **Healthcheck:** আপনার MERN API-এর জন্য `/health` Endpoint লিখুন ও ৫.৮-এর Healthcheck বসিয়ে `docker inspect`-এ Health Status দেখুন।

PART II • DEVOPS ENGINEERING MASTERY

Docker Image Engineering

Docker ইনস্টলেশন, CLI, Dockerfile, build context ও cache, multi-stage build, tagging ও registry।

অধ্যায় 6 • PART II — Docker Foundations and Image Engineering

Chapter 6 — Docker Installation and Development Environment

৬.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: (১) Container ও Virtual Machine-এর পার্থক্য প্রথম নীতি থেকে ব্যাখ্যা করতে; (২) Ubuntu 24.04 LTS-এ Docker Engine সঠিকভাবে ইনস্টল ও যাচাই করতে; (৩) Docker Daemon, Docker CLI ও Docker Hub-এর ভূমিকা আলাদা করতে; (৪) Rootless Docker কী এবং কখন তা ব্যবহার করবেন তা বলতে; (৫) একটি Node.js/Express অ্যাপের জন্য কাজের Development Environment সাজাতে; (৬) ইনস্টলেশনের সাধারণ ব্যর্থতা নিজে নির্ণয় করতে।

৬.২ কেন এই বিষয়টি DevOps-এ গুরুত্বপূর্ণ

Volume 05-এ আমরা শিখেছি: একই Artifact Staging ও Production — দুই জায়গায় চালাতে হয়, নইলে "আমার মেশিনে চলেছিল" সমস্যাটি ফিরে আসে। Docker সেই প্রতিশ্রুতি পূরণের যন্ত্র। একটি Docker Image হলো অ্যাপ, তার Runtime, নির্ভরতা ও কনফিগারেশনের একটি অপরিবর্তনীয় (Immutable) প্যাকেজ — ল্যাপটপে যা চলে, সার্ভারেও হুবহু তাই চলে। কিন্তু এই সুবিধা তখনই পাওয়া যায়, যখন Docker সঠিকভাবে ইনস্টল করা হয়: ভুলভাবে ইনস্টল করা Docker — যেমন পুরনো প্যাকেজ, অনুমতি (Permission) না বোঝা, Daemon না চলা — শুরুতেই এমন সব বিভ্রান্তি তৈরি করে যে অনেকে Docker-কে "জটিল" বলে ছেড়ে দেন। এই অধ্যায়ে আমরা ভিত্তিটি শক্ত করে বানাব, যাতে পরের অধ্যায়গুলোতে আপনি শুধু শিখতে পারেন, ইনস্টলেশন নিয়ে লড়তে না হয়।

৬.৩ পূর্বশর্ত জ্ঞান

Ubuntu Linux-এ টার্মিনাল ব্যবহার, `sudo`, প্যাকেজ ম্যানেজার `apt`, ফাইল অনুমতি (Permission) ও `systemd` সার্ভিসের প্রাথমিক ধারণা (Volume 04), Git-এর মৌলিক ব্যবহার এবং Node.js/Express অ্যাপ চালানোর অভিজ্ঞতা থাকলেই চলবে। Container, Image বা Kernel-এর পূর্ব অভিজ্ঞতা প্রয়োজন নেই — প্রতিটি ধারণা এখানেই প্রথম নীতি থেকে পরিচয় করিয়ে দেওয়া হবে।

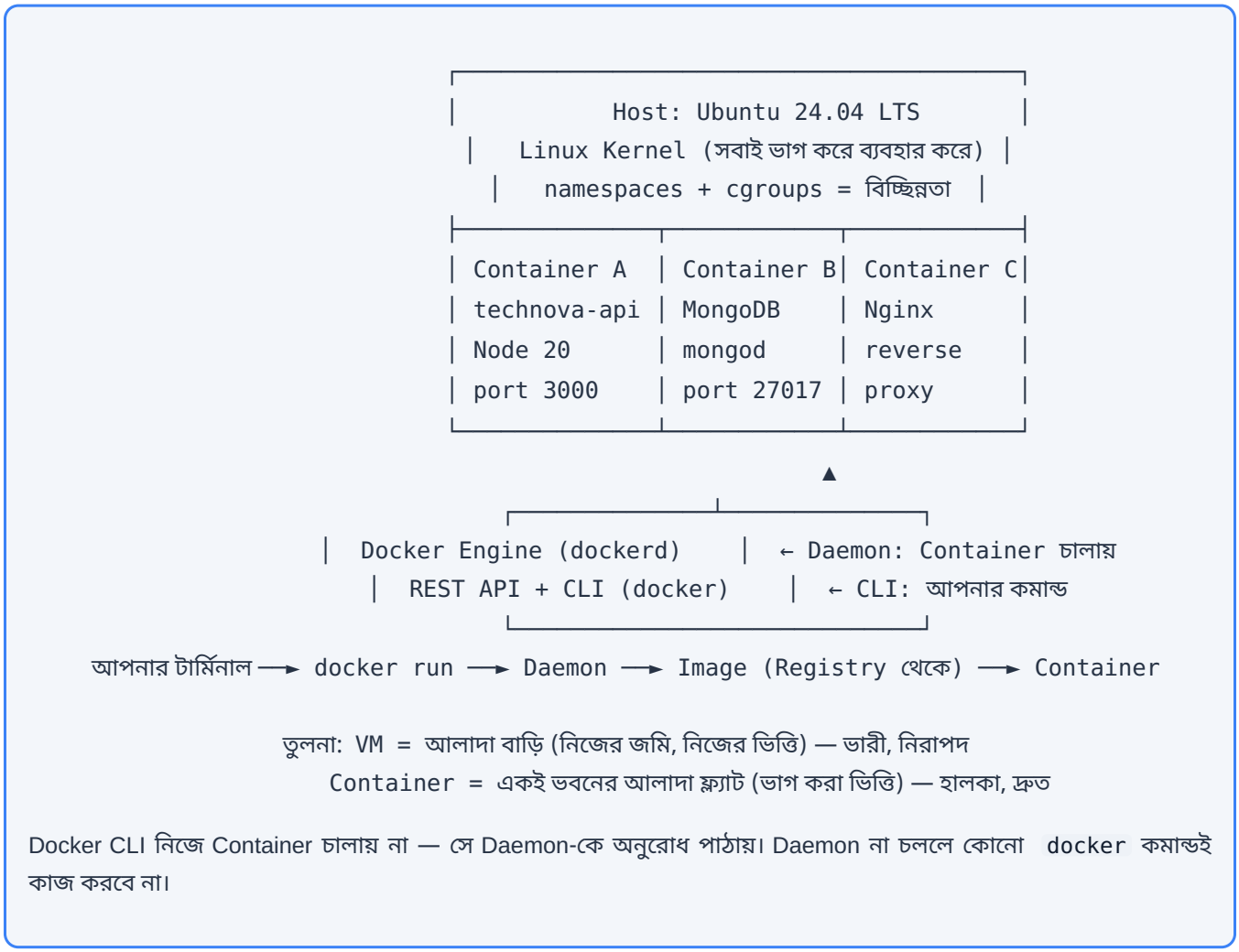
৬.৪ প্রথম নীতি থেকে ব্যাখ্যা

একটি কম্পিউটারে অ্যাপ চালাতে তিনটি জিনিস লাগে: **কোড**, **Runtime** (যেমন Node.js) এবং **পরিবেশ** (অপারেটিং সিস্টেমের লাইব্রেরি, ফাইল, নেটওয়ার্ক সেটিং)। সমস্যা হলো, পরিবেশ প্রতিটি মেশিনে আলাদা — আপনার ল্যাপটপে Node 20, সার্ভারে Node 18; ল্যাপটপে একটি লাইব্রেরি আছে, সার্ভারে নেই। **Virtual Machine (VM)** এই সমস্যার একটি সমাধান: পুরো একটি Guest Operating System-সহ আলাদা কম্পিউটার অনুকরণ করা। কিন্তু VM ভারী — প্রতিটি VM-এ আলাদা Kernel, আলাদা মেমোরি, মিনিটখানেক Boot সময়। **Container** হলো হালকা সমাধান: Host-এর Kernel-ই সব Container ভাগ করে ব্যবহার করে, কিন্তু প্রতিটি Container-এর জন্য আলাদা **Namespace** (প্রসেস, নেটওয়ার্ক, ফাইলসিস্টেমের আলাদা দৃষ্টিভঙ্গি) ও **Control Group (cgroup)** (CPU/মেমোরির সীমা) তৈরি করা হয়। ফলে Container দেখতে আলাদা মেশিনের মতো, কিন্তু আসলে Host Kernel-এর ওপর চলা একটি বিচ্ছিন্ন (Isolated) প্রসেস-গোষ্ঠী মাত্র — সেকেন্ডের ভগ্নাংশে চালু হয়, মেগাবাইটের বদলে মেগাবাইটের ভগ্নাংশ মেমোরি খায়। **Docker** হলো Container তৈরি,

চালানো ও বিতরণের সবচেয়ে প্রচলিত প্ল্যাটফর্ম: **Docker Engine** (Daemon + CLI) Container চালায়, **Docker Image** হলো Container-এর ছাঁচ (Template), আর **Docker Hub** হলো Image-এর ভান্ডার (Registry)।

> **CONCEPT:** Container কোনো "ছোট VM" নয়। VM নিজের Kernel চালায়; Container Host-এর Kernel ভাগ করে নেয়। এই কারণে Container হালকা ও দ্রুত, কিন্তু এই কারণেই Container-এর নিরাপত্তা মডেল VM-এর চেয়ে আলাদা — একটি Container থেকে Host Kernel-এ পৌঁছানো গেলে সব Container ঝুঁকিতে পড়ে। তাই Container-কে কখনোই পূর্ণ VM-এর মতো বিশ্বাস করবেন না।

৬.৫ আর্কিটেকচার / ধারণাগত চিত্র



Docker CLI নিজে Container চালায় না — সে Daemon-কে অনুরোধ পাঠায়। Daemon না চললে কোনো `docker` কমান্ডই কাজ করবে না।

৬.৬ বিস্তারিত প্রযুক্তিগত ব্যাখ্যা

Linux Namespace ও cgroup — Container-এর দুটি স্তম্ভ

Namespace হলো Kernel-এর একটি বৈশিষ্ট্য যা একটি প্রসেস-গোষ্ঠীকে সিস্টেমের একটি আলাদা "দৃষ্টিভঙ্গি" দেয়: **PID Namespace**-এ Container-এর ভেতরে প্রথম প্রসেসের PID হয় 1, যদিও Host-এ তার আসল PID অন্য কিছু; **Network Namespace**-এ Container নিজের IP ও Port পায়; **Mount Namespace**-এ Container নিজের ফাইলসিস্টেম দেখে; **User Namespace**-এ Container-এর ভেতরের root Host-এর সাধারণ ব্যবহারকারী হতে পারে। **cgroup** হলো সম্পদের

সীমা: একটি Container-কে বলা যায় "তুমি সর্বোচ্চ 512 MB মেমোরি ও 1টি CPU পাবে" — সীমা ছাড়ালে Kernel তাকে থামিয়ে দেয়। Docker এই দুটি Kernel বৈশিষ্ট্যের ওপর একটি সহজ ইন্টারফেস মাত্র। এজন্যই Docker মূলত Linux-এর প্রযুক্তি: Windows বা macOS-এ Docker Desktop আসলে পেছনে একটি হালকা Linux VM চালায়।

Docker Engine-এর উপাদান

উপাদান	কী করে	আপনি কীভাবে দেখবেন
Docker Daemon (`dockerd`)	Image, Container, Network, Volume পরিচালনা করে; আসল কাজটি করে	`sudo systemctl status docker`
Docker CLI (`docker`)	Daemon-এর REST API-তে অনুরোধ পাঠায়; আপনি যা টাইপ করেন	`docker version`
containerd	নিম্ন-স্তরের Container Runtime — Container-এর জীবনচক্র সামলায়	Daemon-এর ভেতরে চলে; সরাসরি ব্যবহার করবেন না
runc	Namespace ও cgroup তৈরি করে আসল Container প্রসেস চালু করে	Daemon-এর ভেতরে চলে
Docker Hub / Registry	Image সংরক্ষণ ও বিতরণ করে	`docker pull`, `docker push`

এই স্তরগুলো বোঝা গুরুত্বপূর্ণ, কারণ সমস্যা নির্ণয়ের সময় জানতে হয় সমস্যাটি কোন স্তরে: CLI-তে, Daemon-এ, নাকি Registry-তে। যেমন `Cannot connect to the Docker daemon` মানে Daemon চলছে না — Image-এর কোনো দোষ নেই।

ইনস্টলেশনের পথ — কোনটি বেছে নেবেন

Ubuntu-তে Docker ইনস্টলের তিনটি প্রধান পথ আছে: (১) **Docker-এর অফিসিয়াল apt Repository** — প্রস্তাবিত পথ; সবচেয়ে নতুন স্থিতিশীল সংস্করণ পাওয়া যায়, আপডেট `apt upgrade`-এর সঙ্গে আসে। (২) **Ubuntu-র নিজস্ব Repository** (`sudo apt install docker.io`) — সহজ, কিন্তু সংস্করণ প্রায়ই পুরনো থাকে; নতুন ফিচার (যেমন BuildKit-এর নতুন সুবিধা) নাও থাকতে পারে। (৩) **Docker Desktop** — ল্যাপটপে (Windows/macOS, এবং Linux-এও) GUI-সহ প্যাকেজ; শেখার জন্য সুবিধাজনক, তবে সার্ভারে নয়। এই বইয়ে আমরা অফিসিয়াল apt Repository পথটি ব্যবহার করব — এটাই Production সার্ভারের মানদণ্ড। **Convenience Script** (`get.docker.com` থেকে স্ক্রিপ্ট ডাউনলোড করে চালানো) শুধু পরীক্ষামূলক পরিবেশে ব্যবহার করবেন — Production-এ যাচাই না করা স্ক্রিপ্ট ইন্টারনেট থেকে চালানো নিরাপত্তা ঝুঁকি।

Rootless Docker — কী ও কেন

সাধারণভাবে Docker Daemon `root` হিসেবে চলে — অর্থাৎ একটি Container ভেঙে বেরিয়ে এলে সে Host-এর root হতে পারে। **Rootless Docker** হলো Daemon-কে সাধারণ (non-root) ব্যবহারকারী হিসেবে চালানো, User Namespace-এর সাহায্যে। এতে Daemon-এর ক্ষমতা কমে, তাই Container ভেঙে গেলেও Host-এর root পাওয়া যায় না। সীমাবদ্ধতা আছে: কিছু নেটওয়ার্ক ফিচার (যেমন 1024-এর নিচের Port সরাসরি ব্যবহার, নির্দিষ্ট Storage Driver) Rootless মোডে সীমিত বা অতিরিক্ত সেটআপ চায়। শেখার ল্যাপটপে ও নিরাপত্তা-সংবেদনশীল পরিবেশে Rootless ভালো পছন্দ; Production সার্ভারে অনেক দল এখনো root Daemon + কঠোর নীতি ব্যবহার করে, কারণ Rootless-এর সীমাবদ্ধতাগুলো সেখানে বাধা হয়। এই অধ্যায়ে আমরা root Daemon দিয়ে শুরু করব, আর Rootless সেটআপটিও দেখিয়ে দেব যাতে আপনি সিদ্ধান্ত নিতে পারেন।

Development বনাম Staging বনাম Production ইনস্টলেশন

Development ল্যাপটপে Docker Desktop বা Rootless — সুবিধা ও নিরাপত্তা প্রধান। Staging সার্ভারে Production-এর হুবহু একই ইনস্টলেশন পদ্ধতি — নইলে Staging-এর পরীক্ষার মূল্য কমে যায়। Production-এ অফিসিয়াল Repository থেকে নির্দিষ্ট সংস্করণ (Version Pinning) করে ইনস্টল করুন, স্বয়ংক্রিয় আপগ্রেড বন্ধ রাখুন, আর Daemon-এর কনফিগারেশন (`/etc/docker/daemon.json`) সংস্করণ-নিয়ন্ত্রণে (Git) রাখুন। Development-এ `latest` ঠিক আছে; Production-এ `latest` কখনো নয় — অধ্যায় ১১-এ এর কারণ বিস্তারিত আসবে।

৬.৭ Docker কমান্ড — লাইন বাই লাইন ব্যাখ্যাসহ ইনস্টলেশন ও যাচাই

নিচের ধাপগুলো Ubuntu 24.04 LTS-এর জন্য। প্রতিটি কমান্ডের নিচে তার কাজ লেখা আছে — না বুঝে কোনো কমান্ড চালাবেন না।

```
# ধাপ ১: পুরনো ও পরস্পর-বিরোধী প্যাকেজ সরান (থাকলে)
sudo apt remove docker docker-engine docker.io containerd runc
# apt remove = প্যাকেজ আনইনস্টল; পুরনো docker.io থাকলে অফিসিয়াল প্যাকেজের সঙ্গে সংঘাত হয়

# ধাপ ২: apt-এর HTTPS ও GPG সাপোর্ট ইনস্টল করুন
sudo apt update
sudo apt install -y ca-certificates curl
# ca-certificates = HTTPS যাচাইয়ের সার্টিফিকেট; curl = ফাইল ডাউনলোডের টুল

# ধাপ ৩: Docker-এর GPG কী নিরাপদ জায়গায় রাখুন
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc
# GPG কী = Docker-এর ডিজিটাল স্বাক্ষর যাচাইয়ের চাবি; এটি ছাড়া apt জানবে না
# প্যাকেজগুলো সত্যিই Docker-এর কি না। -fsSL = ব্যর্থ হলে চুপচাপ নয়, স্পষ্ট ত্রুটি দেখাও।

# ধাপ ৪: Docker-এর অফিসিয়াল Repository যোগ করুন
echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.asc] \
  https://download.docker.com/linux/ubuntu $(. /etc/os-release && echo "$VERSION_CODENAME")
stable" \
  | sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
# deb লাইন = নতুন সফটওয়্যার-ভান্ডারের ঠিকানা; signed-by = ওপরের GPG কী দিয়ে যাচাই করো;
# VERSION_CODENAME = noble (Ubuntu 24.04); stable = স্থিতিশীল শাখা

# ধাপ ৫: Docker Engine ইনস্টল করুন
sudo apt update
sudo apt install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-
plugin
# docker-ce = Community Edition Engine (Daemon + CLI)
# docker-ce-cli = শুধু CLI; containerd.io = নিম্ন-স্তরের Runtime
# docker-buildx-plugin = আধুনিক Build টুল (BuildKit); docker-compose-plugin = Compose V2

# ধাপ ৬: Daemon চলছে কি না যাচাই করুন
sudo systemctl status docker
sudo systemctl enable docker
# status = Daemon চলছে কি না দেখুন (active (running) হতে হবে)
# enable = সার্ভার রিস্টার্টের পর Daemon নিজে থেকে চালু হবে
```

যাচাই — প্রথম Container চালানো:

```
# ধাপ ৭: hello-world Image দিয়ে পরীক্ষা
sudo docker run hello-world
# docker run = Image থেকে Container তৈরি করে চালাও
# hello-world = Docker-এর পরীক্ষামূলক ছোট Image; না থাকলে Docker Hub থেকে ডাউনলোড হবে

# নমুনা আউটপুট (চিত্রণমূলক — আপনার আউটপুটের তারিখ/সংস্করণ আলাদা হবে):
# Hello from Docker!
# This message shows that your installation appears to be working correctly.
# ... (Image ডাউনলোডের লাইনগুলো প্রথমবার দেখা যাবে)

# ধাপ ৮: সংস্করণ ও সারাংশ দেখুন
docker version
docker info
# version = Client (CLI) ও Server (Daemon) — দুটোর সংস্করণ আলাদাভাবে দেখায়
# info = Daemon-এর সারাংশ: Container/Image সংখ্যা, Storage Driver, Rootless কি না
```

sudo ছাড়া `docker` চালানো (docker গ্রুপ):

```
sudo groupadd docker          # docker গ্রুপ তৈরি (না থাকলে)
sudo usermod -aG docker $USER # বর্তমান ব্যবহারকারীকে গ্রুপে যোগ করুন (-aG = append)
newgrp docker                 # নতুন গ্রুপ সক্রিয় করুন (অথবা লগআউট করে আবার লগইন করুন)
docker run hello-world        # এখন sudo ছাড়াই চলা উচিত
# সতর্কতা: docker গ্রুপের সদস্য হওয়া প্রায় root-এর সমান ক্ষমতা — কারণ Docker
# Daemon root হিসেবে চলে এবং যেকোনো Container-কে Host ফাইল মাউন্ট করতে দিতে পারে।
# তাই শুধু বিশ্বস্ত ব্যবহারকারীকেই এই গ্রুপে রাখুন।
```

Rootless Docker সেটআপ (ঐচ্ছিক, নিরাপত্তা-সচেতন পরিবেশে):

```
sudo apt install -y uidmap dbus-user-session
# uidmap = User Namespace-এর জন্য UID ম্যাপিং; dbus-user-session = ব্যবহারকারী-সেশন সার্ভিস
dockerd-rootless-setuptool.sh install
# এই স্ক্রিপ্টটি Rootless Daemon ইনস্টল করে (প্রথমে root Daemon বন্ধ করে নিন:
# sudo systemctl disable --now docker)
systemctl --user status docker
docker context use rootless
# context = CLI কোন Daemon-এর সঙ্গে কথা বলবে তা বদলায়; rootless = ব্যবহারকারীর Daemon
# সীমাবদ্ধতা: 1024-এর নিচের Port সরাসরি ব্যবহারে অতিরিক্ত সেটআপ লাগে; কিছু
# Storage Driver Rootless-এ ধীর হতে পারে।
```

> **WARNING:** কখনোই Docker Daemon-কে TCP পোর্টে (যেমন `2375`) প্রমাণীকরণ (Authentication) ছাড়া খুলবেন না। ইন্টারনেটে এমন খোলা Daemon খুঁজে আক্রমণকারীরা সার্ভার দখল করে — এটি Docker-এর সবচেয়ে পরিচিত ভয়াবহ ভুলগুলোর একটি। দূর থেকে Daemon ব্যবহার করতে হলে TLS সার্টিফিকেটসহ `2376` পোর্ট ব্যবহার করুন, অথবা SSH-এর মাধ্যমে সংযোগ করুন।

৬.৮ বাস্তব প্রকৌশল পরিস্থিতি

REAL WORLD — TechNova: TechNova-র নতুন ডেভেলপার নাহিদ তার Ubuntu ল্যাপটপে `sudo apt install docker.io` দিয়ে Docker ইনস্টল করেছিলেন — সংস্করণটি ছিল পুরনো, ফলে অধ্যায় ৯-এর BuildKit ফিচারগুলো কাজ করছিল না এবং দলের অন্যরা যে কমান্ড দিতেন তা তাঁর মেশিনে ভিন্ন আচরণ করত। দল সিদ্ধান্ত নিল: সবাই অফিসিয়াল Repository থেকে একই সংস্করণ ব্যবহার করবে, সংস্করণ নম্বর দলের `README`-তে লেখা থাকবে, আর নতুন কেউ যোগ দিলে এই অধ্যায়ের ধাপগুলো অনুসরণ করবে। Staging সার্ভারে (203.0.113.10) একই পদ্ধতিতে ইনস্টল করা হলো, যাতে ল্যাপটপ, Staging ও Production — তিন জায়গায় Docker-এর আচরণ এক হয়।

৬.৯ ধাপে ধাপে ব্যবহারিক উদাহরণ

উদাহরণ: TechNova API-র Development Environment সাজানো

1. এই অধ্যায়ের ধাপ ১–৬ অনুসরণ করে Docker ইনস্টল করুন এবং `docker run hello-world` দিয়ে যাচাই করুন।
2. `docker version` চালিয়ে Client ও Server সংস্করণ লিখে রাখুন — দলের `README`-তে এই সংস্করণটি নোট করুন।
3. একটি পরীক্ষামূলক Node.js অ্যাপ চালান: `docker run --rm -p 3000:3000 node:20-alpine node -e "require('http').createServer((q,s)=>s.end('TechNova OK')).listen(3000)"` — তারপর ব্রাউজারে `http://localhost:3000` খুলে "TechNova OK" দেখুন। (`--rm` মানে Container থামলে মুছে ফেলো; `-p 3000:3000` মানে Host-এর 3000 পোর্ট Container-এর 3000-এ জোড়া লাগাও — বিস্তারিত অধ্যায় ৭-এ।)
4. `docker info` চালিয়ে Storage Driver (`overlay2` হওয়া উচিত) ও Container সংখ্যা দেখুন।
5. ল্যাপটপে কাজ করলে সিদ্ধান্ত নিন: Rootless ব্যবহার করবেন কি না — নিরাপত্তা বনাম সুবিধার ভারসাম্য লিখে রাখুন।

৬.১০ সাধারণ ভুল ও ভ্রান্ত ধারণা

- **ভুল:** পুরনো `docker.io` ও অফিসিয়াল `docker-ce` একসঙ্গে ইনস্টল করা — সংঘাতে Daemon চালু হয় না। সমাধান: ধাপ ১-এ পুরনো প্যাকেজ সরান।
- **ভুল:** প্রতিটি `docker` কমান্ডে `sudo` ব্যবহার করা ও গ্রুপ সেটআপ না করা — কাজ করে, কিন্তু প্রতিদিনের বিরক্তি ও স্ক্রিপ্টে ভুলের উৎস।
- **ভুল ধারণা:** "Docker ইনস্টল মানেই Container নিরাপদ।" — ভুল; ডিফল্ট Docker Daemon root হিসেবে চলে। Rootless বা কঠোর নীতি ছাড়া Container ভেঙে Host ঝুঁকিতে পড়তে পারে।
- **ভুল ধারণা:** "Windows/macOS-এ Docker আর Linux-এ Docker একই।" — প্রায় একই, তবে Windows/macOS-এ পেছনে একটি Linux VM চলে বলে ফাইল শেয়ারিং ও নেটওয়ার্কিংয়ে সূক্ষ্ম পার্থক্য থাকে।
- **ভুল:** Convenience Script (`curl ... | sh`) Production সার্ভারে চালানো — যাচাই না করা স্ক্রিপ্টে যা খুশি থাকতে পারে।
- **ভুল:** Daemon-এর `2375` পোর্ট প্রমাণীকরণ ছাড়া খোলা — ইন্টারনেটে স্ক্যান করে আক্রমণকারীরা এটি খুঁজে পায়।

৬.১১ নিরাপত্তা বিবেচনা

ইনস্টলেশনের সময়ই নিরাপত্তার ভিত্তি তৈরি হয়। Docker Daemon root হিসেবে চলে — তাই `docker` গ্রুপে শুধু বিশ্বস্ত ব্যবহারকারী রাখুন; গ্রুপের সদস্য `docker run -v /:/host ...` দিয়ে Host-এর পুরো ফাইলসিস্টেম পড়তে পারে, অর্থাৎ কার্যত root-এর সমান ক্ষমতা পায়। Daemon-এর সকেট (`/var/run/docker.sock`) শুধু root ও docker গ্রুপের পড়ার

অনুমতি পায় — এর অনুমতি কখনো `777` করবেন না। Rootless Docker বিবেচনা করুন, বিশেষ করে শেয়ার্ড বা নিরাপত্তা-সংবেদনশীল মেশিনে। ইনস্টলের পর `docker info`-তে `live-restore` ও লগিং সেটিং দেখে নিন; Daemon-এর কনফিগারেশন ফাইল `/etc/docker/daemon.json` শুধু root-এর লেখার অনুমতিতে থাকা উচিত। আর সবচেয়ে গুরুত্বপূর্ণ: আসল Secret (API Key, পাসওয়ার্ড, ডেটাবেস Credential) কখনো Image-এ বা Dockerfile-এ লিখবেন না — এই নিয়ম এই বইয়ের প্রতিটি অধ্যায়ে বারবার আসবে।

৬.১২ কর্মক্ষমতা ও সম্পদ বিবেচনা

Docker নিজে হালকা, কিন্তু ভুল সেটআপে সম্পদ নষ্ট হয়। Storage Driver হিসেবে `overlay2` ব্যবহার করুন — এটি Ubuntu 24.04-এর ডিফল্ট ও দ্রুততম; পুরনো `devicemapper` বা `vfs` ধীর। `/var/lib/docker` (Image ও Container-এর ডেটা) যে ডিস্কে থাকে, সেখানে পর্যাপ্ত জায়গা রাখুন — Image জমে ডিস্ক ভরে যাওয়া সবচেয়ে সাধারণ Production সমস্যাগুলোর একটি; নিয়মিত `docker system prune` (অধ্যায় ৭) ব্যবহার করুন। Development ল্যাপটপে Docker Desktop-এর মেমোরি সীমা (ডিফল্ট প্রায়ই 2–4 GB) কম হলে বড় Build ব্যর্থ হয় — প্রয়োজনে বাড়ান। Daemon-এর লগ ড্রাইভার ডিফল্ট `json-file` — লগের আকার সীমিত না রাখলে (`daemon.json`-এ `log-opts`) ডিস্ক ভরে যেতে পারে।

৬.১৩ সমস্যা সমাধান (Troubleshooting)

TROUBLESHOOTING: `Cannot connect to the Docker daemon at unix:///var/run/docker.sock. Is the docker daemon running?` — সবচেয়ে সাধারণ ত্রুটি। ধাপ: (১) `sudo systemctl status docker` — Daemon চলছে কি না দেখুন; (২) না চললে `sudo systemctl start docker` ও `sudo journalctl -u docker --no-pager | tail -50` দিয়ে লগ দেখুন; (৩) `permission denied` হলে আপনি docker গ্রুপে আছেন কি না দেখুন (`groups`) এবং লগআউট-লগইন করুন — `usermod`-এর পর গ্রুপ সক্রিয় হতে নতুন সেশন লাগে।

সমস্যা	সম্ভাব্য কারণ	সমাধান
`docker: command not found`	ইনস্টল হয়নি বা PATH-এ নেই	ধাপ ৫ পুনরায় চালান; `which docker` দেখুন
Daemon চালু হয় না	পুরনো প্যাকেজের সংঘাত / ভুল `daemon.json`	`journalctl -u docker` লগ দেখুন; JSON সিনট্যাক্স যাচাই করুন (`python3 -m json.tool /etc/docker/daemon.json`)
`permission denied` সকেটে	ব্যবহারকারী docker গ্রুপে নেই বা সেশন পুরনো	`sudo usermod -aG docker $USER`, তারপর লগআউট-লগইন
`hello-world` ডাউনলোড ব্যর্থ	নেটওয়ার্ক / প্রক্সি / DNS সমস্যা	`curl -I https://hub.docker.com` দিয়ে সংযোগ যাচাই; কর্পোরেট প্রক্সি থাকলে Daemon-এ প্রক্সি সেট করুন
ডিস্ক দ্রুত ভরে যাচ্ছে	পুরনো Image/Container জমে আছে	`docker system df` ও `docker system prune -a` (সাবধানে — অধ্যায় ৭ দেখুন)

৬.১৪ সেরা অনুশীলন ও অ্যান্টি-প্যাটার্ন

সেরা অনুশীলন: অফিসিয়াল Repository থেকে নির্দিষ্ট সংস্করণ ইনস্টল করুন; সংস্করণ নম্বর দলের ডকুমেন্টে লিখুন; Staging ও Production-এ একই পদ্ধতি ব্যবহার করুন; Daemon কনফিগারেশন Git-এ রাখুন; নিয়মিত `docker system df` দিয়ে ডিস্ক ব্যবহার দেখুন। **অ্যান্টি-প্যাটার্ন:** প্রতিটি সার্ভারে আলাদা পদ্ধতিতে ইনস্টল করা ("Snowflake Server" — Volume 05-এর ধারণা); Convenience Script Production-এ চালানো; Daemon-এর সকেটের অনুমতি ঢিলা করা; ইনস্টলের পর আর কখনো আপডেট না করা — পুরনো Docker-এ পরিচিত নিরাপত্তা ত্রুটি থেকে যায়।

৬.১৫ অধ্যায়ের সারাংশ

Container হলো Host Kernel ভাগ করে নেওয়া, Namespace ও cgroup দিয়ে বিচ্ছিন্ন করা হালকা প্রসেস-গোষ্ঠী — VM-এর চেয়ে অনেক হালকা ও দ্রুত। Docker Engine-এর Daemon আসল কাজ করে, CLI শুধু অনুরোধ পাঠায়। Ubuntu 24.04-এ অফিসিয়াল apt Repository থেকে Docker ইনস্টল করাই প্রস্তাবিত পথ; `hello-world` দিয়ে যাচাই করুন, docker গ্রুপ সেটআপ করুন, আর Rootless মোডের সুবিধা-সীমাবদ্ধতা জেনে সিদ্ধান্ত নিন। ইনস্টলেশনের সময়ই নিরাপত্তা ও সম্পদের ভিত্তি তৈরি হয় — Daemon-এর সকেট সুরক্ষিত রাখুন, Storage Driver `overlay2` নিশ্চিত করুন।

৬.১৬ মূল প্রযুক্তিগত পরিভাষা

Container: Host Kernel ভাগ করে নেওয়া, বিচ্ছিন্ন পরিবেশে চলা অ্যাপ ও তার নির্ভরতার প্যাকেজ।

Image: Container তৈরির অপরিবর্তনীয় ছাঁচ — ফাইলসিস্টেমের স্তরসমূহ ও মেটাডেটা।

Docker Engine: Docker-এর মূল সফটওয়্যার — Daemon, CLI, API-সহ।

Docker Daemon (dockerd): ব্যাকগ্রাউন্ডে চলা সার্ভিস যা Container পরিচালনা করে।

Docker CLI: টার্মিনাল থেকে Daemon-কে নির্দেশ দেওয়ার টুল (`docker` কমান্ড)।

Namespace: Linux Kernel বৈশিষ্ট্য — প্রসেস-গোষ্ঠীকে সিস্টেমের আলাদা দৃষ্টিভঙ্গি দেয়।

cgroup (Control Group): Kernel বৈশিষ্ট্য — প্রসেস-গোষ্ঠীর CPU/মেমোরি ব্যবহার সীমিত করে।

containerd / runc: নিম্ন-স্তরের Runtime — Container-এর জীবনচক্র ও প্রসেস চালু করে।

Registry: Image সংরক্ষণ ও বিতরণের ভান্ডার (যেমন Docker Hub)।

Rootless Docker: Daemon-কে root নয়, সাধারণ ব্যবহারকারী হিসেবে চালানো।

Storage Driver (overlay2): Image স্তরগুলোকে ডিস্কে সাজিয়ে Container-এর ফাইলসিস্টেম তৈরির পদ্ধতি।

Docker Context: CLI কোন Daemon-এর সঙ্গে কথা বলবে, তার সেটিং।

৬.১৭ জ্ঞান যাচাই — প্রশ্ন

1. Container ও Virtual Machine-এর মূল পার্থক্য কী? Kernel-এর ভূমিকা ব্যাখ্যা করুন।
2. Namespace ও cgroup — প্রতিটি কী বিচ্ছিন্নতা বা সীমা দেয়? একটি করে উদাহরণ দিন।
3. Docker Daemon না চললে `docker ps` কী ত্রুটি দেয়, এবং কেন?
4. docker গ্রুপের সদস্য হওয়া কেন প্রায় root-এর সমান ক্ষমতা দেয়?
5. Rootless Docker-এর সুবিধা ও সীমাবদ্ধতা লিখুন। কখন Rootless বেছে নেবেন?
6. Ubuntu-র `docker.io` প্যাকেজের বদলে অফিসিয়াল Repository কেন প্রস্তাবিত?

৬.১৮ ব্যবহারিক অনুশীলন

1. **ইনস্টল ও যাচাই:** একটি Ubuntu VM-এ এই অধ্যায়ের ধাপগুলো অনুসরণ করে Docker ইনস্টল করুন; `docker version` ও `docker info`-এর আউটপুট সংরক্ষণ করুন।
2. **গ্রুপ পরীক্ষা:** নতুন ব্যবহারকারী তৈরি করে docker গ্রুপে যোগ করুন; লগআউট-লগইনের আগে ও পরে `docker ps` চালিয়ে পার্থক্য দেখুন।
3. **Rootless পরীক্ষা:** একটি পরীক্ষামূলক VM-এ Rootless Docker সেটআপ করুন এবং একটি Container চালিয়ে দেখুন; সীমাবদ্ধতাগুলো নোট করুন।
4. **ত্রুটি অনুশীলন:** ইচ্ছাকৃতভাবে Daemon বন্ধ করে (`sudo systemctl stop docker`) `docker ps` চালান; ত্রুটি বার্তা পড়ে নিজে ঠিক করুন ও ধাপগুলো লিখুন।

অধ্যায় 7 • PART II — Docker Foundations and Image Engineering

Chapter 7 — Docker CLI: Managing Images and Containers

৭.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: (১) Docker CLI-এর গঠন ও কমান্ডের ধরন (Management Command বনাম সংক্ষিপ্ত রূপ) ব্যাখ্যা করতে; (২) Image ডাউনলোড, তালিকা, পরিদর্শন ও মোছার কমান্ড ব্যবহার করতে; (৩) Container তৈরি, চালানো, থামানো, মুছে ফেলা ও পুনরায় চালু করতে; (৪) চলমান Container-এর লগ দেখতে ও তার ভেতরে কমান্ড চালাতে (`exec`); (৫) Port Publishing ও Environment Variable দিয়ে Container কনফিগার করতে; (৬) Container-এর অবস্থা (State) ও প্রস্থান কোড (Exit Code) পড়ে সমস্যা নির্ণয় করতে।

৭.২ কেন এই বিষয়টি DevOps-এ গুরুত্বপূর্ণ

Docker-এর প্রায় সব কাজ CLI দিয়েই হয় — CI Pipeline-এ, সার্ভারে SSH করে, সমস্যা নির্ণয়ের সময়। CLI না জানলে আপনি Image-টি কী ধারণ করে তা জানবেন না, Container কেন থেমে গেল তা বুঝবেন না, আর Production-এ একটি ভাঙা Deploy-এর সামনে অসহায় হয়ে পড়বেন। উল্টোদিকে, CLI-তে দক্ষ হলে আপনি যেকোনো পরিবেশে — ল্যাপটপ, Staging, Production — একই কমান্ডে একইভাবে কাজ করতে পারেন। এই অধ্যায়টি Docker-এর দৈনন্দিন ভাষা শেখায়; পরের অধ্যায়গুলো এই ভাষাতেই লেখা।

৭.৩ পূর্বশর্ত জ্ঞান

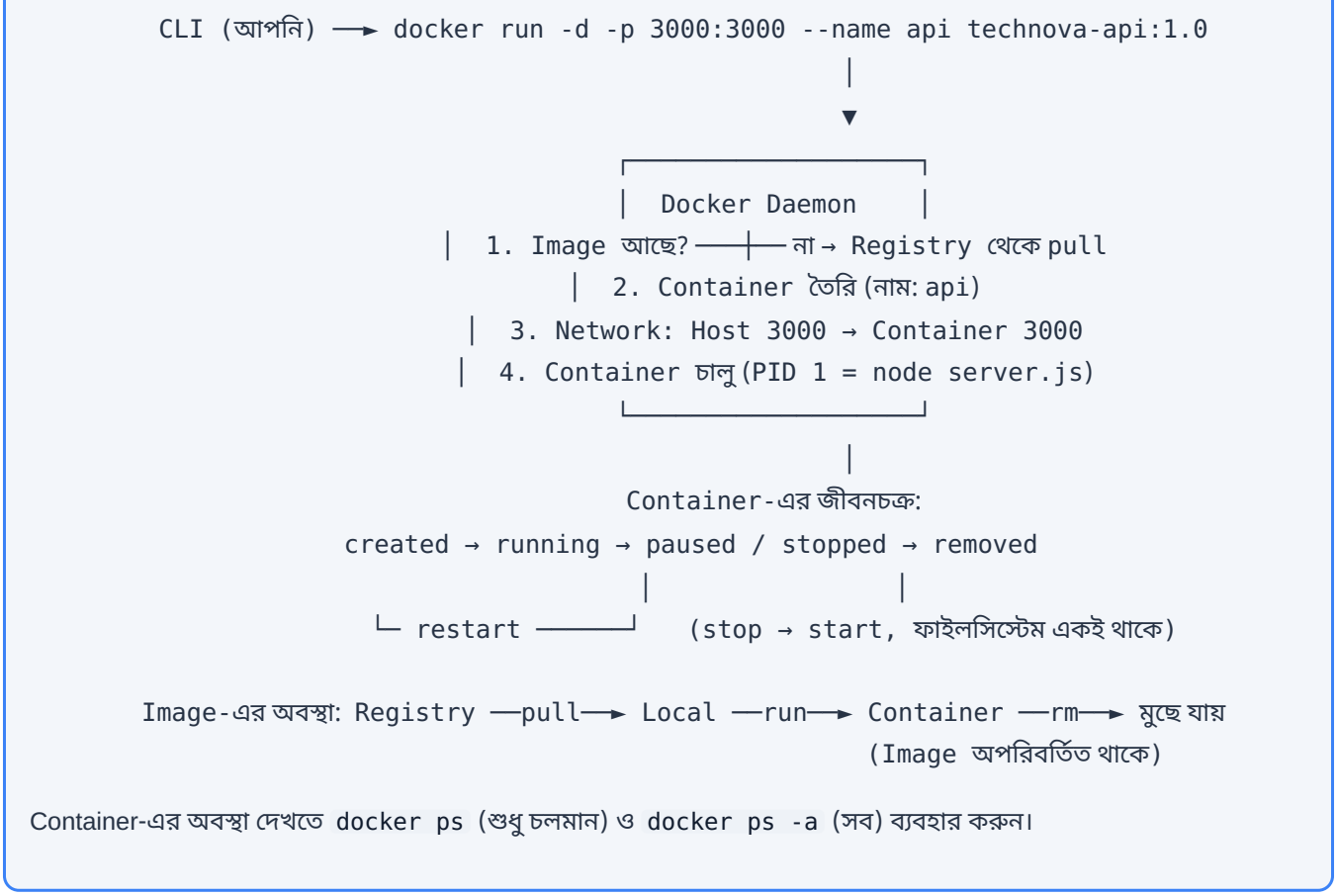
অধ্যায় ৬ — Docker ইনস্টলেশন, Daemon ও CLI-এর ধারণা, Image বনাম Container-এর পার্থক্য। Linux-এ প্রসেস, Port ও Environment Variable-এর প্রাথমিক ধারণা এবং Node.js অ্যাপে `PORT` Environment Variable ব্যবহারের অভিজ্ঞতা থাকলে ভালো হয়।

৭.৪ প্রথম নীতি থেকে ব্যাখ্যা

Image হলো একটি স্থির ছাঁচ — যেমন একটি রেসিপি বইয়ের পাতা; **Container** হলো সেই রেসিপি অনুযায়ী বানানো একটি খাবার — চালু, পরিবর্তনশীল, মুছে ফেলা যায়। একই Image থেকে অনেক Container বানানো যায়, আর Container মুছে ফেললে Image-এর কিছু হয় না। Container-এর নিজের ফাইলসিস্টেমে লেখা পরিবর্তন Container-এর সঙ্গে সঙ্গেই যায় — Container মুছলে তা হারিয়ে যায় (এই কারণেই স্থায়ী ডেটার জন্য Volume দরকার — Volume 07-এ বিস্তারিত)। CLI-এর প্রতিটি কমান্ড আসলে Daemon-কে একটি অনুরোধ: "এই Imageটি দাও" (`pull`), "এই Image থেকে Container বানাও" (`run`), "চলমান Container-গুলো দেখাও" (`ps`)। Docker CLI-এর আধুনিক রূপে কমান্ডগুলো গোষ্ঠীভুক্ত: `docker image ...`, `docker container ...`, `docker volume ...` — এগুলো **Management Command**; পুরনো সংক্ষিপ্ত রূপ (`docker ps`, `docker images`) এখনো কাজ করে, তবে নতুন লেখায় Management Command ব্যবহার করাই পরিষ্কার।

CONCEPT: `docker run` আসলে তিনটি কাজ একসঙ্গে করে: (১) Image না থাকলে `pull` করে, (২) Image থেকে Container `create` করে, (৩) Container `start` করে। এই তিনটি ধাপ আলাদা করে করলে আপনি প্রতিটি ধাপ নিয়ন্ত্রণ করতে পারেন — যেমন আগে Image তৈরি করে, তারপর কনফিগারেশন ঠিক করে চালু করা। Production-এ এই আলাদা করার অভ্যাসই পেশাদার পদ্ধতি।

৭.৫ আর্কিটেকচার / ধারণাগত চিত্র



Container-এর অবস্থা দেখতে `docker ps` (শুধু চলমান) ও `docker ps -a` (সব) ব্যবহার করুন।

৭.৬ বিস্তারিত প্রযুক্তিগত ব্যাখ্যা

Image কমান্ড — pull, list, inspect, remove

`docker pull node:20-alpine` — Registry (ডিফল্ট Docker Hub) থেকে Image ডাউনলোড করে। Image-এর নামের গঠন: `registry/namespace/name:tag` — যেমন `docker.io/library/node:20-alpine`-এর সংক্ষিপ্ত রূপ `node:20-alpine`; ট্যাগ না দিলে `latest` ধরা হয়। `docker images` বা `docker image ls` স্থানীয় Image-গুলোর তালিকা দেয়: Repository, Tag, Image ID, তৈরি হওয়ার সময়, আকার। `docker image inspect node:20-alpine` JSON আকারে Image-এর সম্পূর্ণ মেটাডেটা দেয় — স্তর (Layer), Environment Variable, ডিফল্ট কমান্ড, আর্কিটেকচার। `docker image rm node:20-alpine` Image মুছে ফেলে; চলমান Container যে Image ব্যবহার করছে তা মুছতে প্রথমে Container থামাতে ও মুছতে হয়।

Container কমান্ড — run, ps, stop, start, rm

`docker run`-এর গুরুত্বপূর্ণ ফ্ল্যাগ: `-d` (Detached — ব্যাকগ্রাউন্ডে চালাও, টার্মিনাল ফিরিয়ে দাও), `--name` (নাম দেওয়া — নাম ছাড়া Docker এলোমেলো নাম দেয়, যেমন `brave_morse`), `-p host:container` (Port Publishing), `-e KEY=VALUE` (Environment Variable), `-v` (Volume মাউন্ট — Volume 07), `--rm` (থামলে Container মুছে ফেলো), `-it` (Interactive Terminal — `-i` stdin খোলা + `-t` টার্মিনাল)। `docker ps` চলমান Container দেখায়; `docker ps -a` থেমে যাওয়া Container-সহ সব দেখায়। `docker stop api` Container-কে থামায় — প্রথমে `SIGTERM` পাঠায়, ১০ সেকেন্ডে না থামলে `SIGKILL`। `docker start api` থেমে যাওয়া Container আবার চালু করে (নতুন Container নয় — আগের ফাইলসিস্টেমসহ)। `docker rm api` থেমে যাওয়া Container মুছে ফেলে; চলমান Container মুছতে `docker rm -f api`।

লগ দেখা ও ভেতরে ঢোকা — logs, exec, inspect

`docker logs api` Container-এর stdout/stderr দেখায় — Node.js অ্যাপের `console.log` এখানেই আসে। `docker logs -f api` (Follow) লাইভ লগ দেখায়; `--tail 100` শেষ ১০০ লাইন দেখায়; `--since 10m` শেষ ১০ মিনিটের লগ দেখায়। `docker exec -it api sh` চলমান Container-এর ভেতরে একটি নতুন shell চালায় — ডিবাগিংয়ের প্রধান হাতিয়ার। মনে রাখবেন: `exec` দিয়ে তৈরি প্রসেস Container-এর মূল প্রসেস (PID 1) নয় — মূল প্রসেস মারা গেলে Container থেমে যায়, `exec`-এর শেল মারা গেলে নয়। `docker inspect api` Container-এর সম্পূর্ণ অবস্থা JSON-এ দেয় — IP ঠিকানা, মাউন্ট, Port ম্যাপিং, Environment Variable। `docker stats` চলমান Container-গুলোর CPU/মেমোরি ব্যবহার লাইভ দেখায়।

Port Publishing — -p কী করে

Container-এর নিজের Network Namespace-এ নিজের IP থাকে (ডিফল্ট `bridge` নেটওয়ার্কে সাধারণত `172.17.0.x`)। Host থেকে Container-এ পৌঁছাতে Port Publishing লাগে: `-p 3000:3000` মানে Host-এর 3000 পোর্টে আসা ট্রাফিক Container-এর 3000 পোর্টে যাবে। `-p 127.0.0.1:3000:3000` মানে শুধু localhost থেকে পৌঁছানো যাবে — নিরাপত্তার জন্য ভালো, বিশেষ করে ডেটাবেসের ক্ষেত্রে। `-p 8080:3000` মানে Host-এর 8080 → Container-এর 3000। `-P` (বড় হাতের) এলোমেলো Host Port বরাদ্দ করে — পরীক্ষায় কাজে লাগে, Production-এ নয়। `EXPOSE` (অধ্যায় ৮) শুধু ঘোষণা দেয়, Port খোলে না — `-p` ছাড়া বাইরে থেকে পৌঁছানো যায় না; এই পার্থক্যটি নতুনদের সবচেয়ে বেশি বিভ্রান্ত করে।

Environment Variable — কনফিগারেশন বনাম Secret

`docker run -e NODE_ENV=production -e PORT=3000 technova-api:1.0` — Container-এর ভেতরে Environment Variable সেট করে। সাধারণ কনফিগারেশন (যেমন `NODE_ENV`, `PORT`, `LOG_LEVEL`) `-e` দিয়ে দেওয়া স্বাভাবিক। কিন্তু **আসল Secret কখনো `-e` দিয়ে কমান্ড লাইনে লিখবেন না** — কমান্ড লাইন shell history-তে, প্রসেস তালিকায় (`ps aux`) ও `docker inspect`-এ দৃশ্যমান থাকে। Secret-এর জন্য Docker Secret, Environment ফাইল (`--env-file` — ফাইলটির অনুমতি সুরক্ষিত রেখে), বা Volume 05/07-এর Secret ব্যবস্থাপনা ব্যবহার করুন। `--env-file .env.production` ফাইল থেকে একসঙ্গে অনেক Variable নেয় — তবে ফাইলটি Git-এ commit করবেন না।

কমান্ড	কাজ	কখন ব্যবহার করবেন
`docker image ls`	স্থানীয় Image তালিকা	ডিস্কে কী আছে দেখতে
`docker pull / push`	Registry থেকে নেওয়া / Registry-তে দেওয়া	Image বিতরণে (অধ্যায় ১১)
`docker run -d --name`	Container তৈরি ও চালু	অ্যাপ চালাতে

`docker ps / ps -a`	চলমান / সব Container	অবস্থা দেখতে
`docker logs -f`	লাইভ লগ	ডিবাগিংয়ে
`docker exec -it`	Container-এর ভেতরে shell/কমান্ড	পরিদর্শন ও ডিবাগিংয়ে
`docker stop / start / restart`	থামানো / চালু / পুনরায় চালু	জীবনচক্র পরিচালনায়
`docker rm / image rm`	Container / Image মোছা	পরিষ্কারে
`docker system prune`	অব্যবহৃত সব কিছু মোছা	ডিস্ক পরিষ্কারে (সাবধানে)

৭.৭ Docker কমান্ড — লাইন বাই লাইন ব্যাখ্যাসহ সম্পূর্ণ উদাহরণ

```
# --- Image ব্যবস্থাপনা ---
docker pull node:20-alpine
# pull = Docker Hub থেকে node Image-এর 20-alpine ট্যাগ ডাউনলোড করো
# alpine = ছোট Linux বেস (অধ্যায় ১০-এ বিস্তারিত)

docker image ls
# স্থানীয় Image-গুলো দেখাও: REPOSITORY, TAG, IMAGE ID, CREATED, SIZE

docker image inspect node:20-alpine | head -60
# inspect = Image-এর মেটাডেটা (JSON); head -60 = প্রথম ৬০ লাইন

# --- TechNova API Container চালানো ---
docker run -d --name technova-api -p 3000:3000 \
  -e NODE_ENV=production -e PORT=3000 \
  --restart unless-stopped \
  technova-api:1.0
# -d           = detached — ব্যাকগ্রাউন্ডে চালাও
# --name       = Container-এর নাম technova-api (ID-র বদলে নামে ডাকতে পারবেন)
# -p 3000:3000 = Host 3000 → Container 3000
# -e ...       = Environment Variable সেট করো
# --restart unless-stopped = সার্ভার রিস্টার্ট হলে Container নিজে চালু হবে,
#                            তবে আপনি হাতে থামালে আর চালু হবে না
# technova-api:1.0 = Image-এর নাম:ট্যাগ

# --- অবস্থা, লগ ও পরিদর্শন ---
docker ps
# চলমান Container-গুলো: নাম, Image, Port, অবস্থা, চলার সময়
docker ps -a
# থেমে যাওয়া Container-সহ সব দেখাও
docker logs -f --tail 50 technova-api
# -f = follow (লাইভ); --tail 50 = শেষ ৫০ লাইন থেকে শুরু
docker exec -it technova-api sh
# exec = চলমান Container-এর ভেতরে নতুন প্রসেস চালাও; -it = ইন্টার‍্যাক্টিভ শেল
# ভেতরে: node --version ; ls /app ; env | grep NODE_ENV ; exit

docker inspect technova-api
# Container-এর সম্পূর্ণ অবস্থা: IP, মাউন্ট, Port ম্যাপিং, Env — JSON আকারে
docker stats --no-stream
# --no-stream = একবারের স্ন্যাপশট (লাইভ নয়); CPU/Mem ব্যবহার দেখাও

# --- থামানো, চালু, মোছা ---
```

```
docker stop technova-api     # SIGTERM পাঠাও, ১০ সেকেন্ডে না থামলে SIGKILL
docker start technova-api    # একই Container আবার চালু করো (নতুন নয়)
docker restart technova-api  # থামিয়ে আবার চালু করো
docker rm technova-api       # থেমে যাওয়া Container মুছে ফেলো
docker rm -f technova-api    # চলমান Container জোর করে থামিয়ে মুছে ফেলো
docker image rm technova-api:1.0  # Image মুছে ফেলো (Container না থাকলে)

# --- পরিষ্কার করা ---
docker system df             # Image/Container/Volume কত ডিস্ক খাচ্ছে দেখাও
docker system prune          # থেমে যাওয়া Container, অব্যবহৃত Network/Image মুছে ফেলো
docker system prune -a --volumes  # সব অব্যবহৃত Image + Volume-ও মুছে ফেলো — সাবধান!
# -a --volumes ডেটা মুছে ফেলতে পারে; Production-এ কখনো না ভেবে চালাবেন না।
```

নমুনা আউটপুট (চিত্রণমূলক):

```
$ docker ps
CONTAINER ID   IMAGE               COMMAND                  STATUS
PORTS                     NAMES
a1b2c3d4e5f6   technova-api:1.0    "node server.js"         Up 2 hours      0.0.0.0:3000-
>3000/tcp   technova-api

$ docker logs --tail 3 technova-api
Server listening on port 3000
Connected to MongoDB at 203.0.113.20:27017
TechNova API ready
```

৭.৮ বাস্তব প্রকৌশল পরিস্থিতি

REAL WORLD — TechNova: Staging সার্ভারে (203.0.113.10) TechNova API হঠাৎ সাড়া দেওয়া বন্ধ করল। ডেভেলপার প্রথমে সার্ভার রিস্টার্ট করতে চেয়েছিলেন — কিন্তু তার আগে `docker ps` চালিয়ে দেখলেন Container-টি `Exited (1) 5 minutes ago` অবস্থায় আছে; `docker logs --tail 50 technova-api` দেখাল MongoDB-র সঙ্গে সংযোগ ব্যর্থ হচ্ছে (`ECONNREFUSED 203.0.113.20:27017`) — MongoDB Container-টি থেমে গিয়েছিল। সমাধান ছিল API নয়, MongoDB Container `docker start` করা। এই ঘটনাটি দেখায়: লগ ও অবস্থা পড়তে জানলে সমস্যার মূল কারণ মিনিটে ধরা যায়; না জানলে ঘণ্টা নষ্ট হয়।

৭.৯ ধাপে ধাপে ব্যবহারিক উদাহরণ

উদাহরণ: TechNova API + MongoDB — দুটি Container চালানো

1. `docker pull mongo:7` ও `docker pull node:20-alpine` দিয়ে Image দুটি নামান।
2. MongoDB চালান: `docker run -d --name technova-mongo -p 127.0.0.1:27017:27017 -e MONGO_INITDB_ROOT_USERNAME=admin -e MONGO_INITDB_ROOT_PASSWORD=CHANGE-ME-DB-PASS mongo:7` — লক্ষ্য করুন Port শুধু localhost-এ বাঁধা হয়েছে, এবং পাসওয়ার্ডটি কাল্পনিক প্লেসহোল্ডার।
3. API চালান: `docker run -d --name technova-api -p 3000:3000 -e NODE_ENV=development -e PORT=3000 -e MONGO_URI=mongodb://admin:CHANGE-ME-DB-PASS@technova-mongo:27017/technova node:20-alpine sh -c "node -e \"console.log('API placeholder')\""` — এই ধাপটি ধারণাগত; আসল API Image অধ্যায় ৮-এ বানাবেন।

4. `docker ps`, `docker logs technova-mongo` ও `docker exec -it technova-mongo mongosh -u admin -p CHANGE-ME-DB-PASS --eval "db.runCommand({ping:1})"` দিয়ে যাচাই করুন।
5. দুটো Container থামান ও মুছুন, তারপর `docker system df` দিয়ে ডিস্ক ব্যবহার দেখুন।

৭.১০ সাধারণ ভুল ও ভ্রান্ত ধারণা

- **ভুল:** `docker run` প্রতিবার চালানো, অথচ পুরনো Container থেমে আছে — ফলে একই নামে নতুন Container বানাতে গিয়ে `Conflict. The container name is already in use` ত্রুটি। সমাধান: `docker ps -a` দেখে `docker rm` করুন, অথবা `docker start` ব্যবহার করুন।
- **ভুল:** `EXPOSE 3000` করলেই Port খুলে গেছে ভাবা — না, `-p` ছাড়া বাইরে থেকে পৌঁছানো যায় না।
- **ভুল ধারণা:** "Container মুছলে Image-ও মুছে যায়।" — না; Container ও Image আলাদা। `docker rm` Container মোছে, `docker image rm` Image মোছে।
- **ভুল ধারণা:** "Container থামানো মানেই ডেটা মুছে যাওয়া।" — থামানো Container-এর ফাইলসিস্টেম থাকে; `docker rm` করলেই তা যায়। তবে Production ডেটার জন্য Volume ছাড়া Container-এর ফাইলসিস্টেমে ভরসা করবেন না।
- **ভুল:** `docker exec` দিয়ে Container-এর ভেতরে ফাইল সম্পাদনা করে "ঠিক" করা — Container পুনরায় তৈরি হলে সেই পরিবর্তন হারিয়ে যায়। স্থায়ী সমাধান Image বা কনফিগারেশনে করুন।
- **ভুল:** `docker system prune -a --volumes` Production-এ না ভেবে চালানো — Volume-সহ ডেটা মুছে যেতে পারে।
- **ভুল:** Secret `-e` দিয়ে কমান্ড লাইনে লেখা — history ও `docker inspect`-এ দৃশ্যমান থাকে।

৭.১১ নিরাপত্তা বিবেচনা

Port Publishing-এ সতর্ক থাকুন: `-p 27017:27017` মানে MongoDB ইন্টারনেটে খোলা — ডেটাবেসের Port শুধু `127.0.0.1:27017:27017` দিয়ে localhost-এ বাঁধুন, অথবা কোনো Port Publish না করে Container নেটওয়ার্কের ভেতরেই রাখুন। `--privileged` ফ্ল্যাগ Container-কে Host-এর প্রায় সব ক্ষমতা দেয় — ডিবাগিং ছাড়া কখনো ব্যবহার করবেন না। `docker exec` দিয়ে Container-এর ভেতরে ঢোকা মানে সেই Container-এর অনুমতি পাওয়া — Production Container-এ `exec` করার অধিকার সীমিত রাখুন ও লগ করুন। Image শুধু বিশ্বস্ত Registry থেকে নিন; অপরিচিত Image-এ ম্যালওয়্যার থাকতে পারে — `docker image inspect` দিয়ে Image-এর ইতিহাস দেখে নেওয়ার অভ্যাস করুন।

৭.১২ কর্মক্ষমতা ও সম্পদ বিবেচনা

প্রতিটি Container CPU/মেমোরি খায় — সীমা না দিলে একটি Container পুরো Host-এর মেমোরি খেয়ে অন্য Container-কে থামিয়ে দিতে পারে। Production-এ `--memory=512m --cpus=1.0` ধরনের সীমা দিন (বিস্তারিত Volume 07-এ)। `docker stats` দিয়ে নিয়মিত ব্যবহার দেখুন। লগের পরিমাণ নিয়ন্ত্রণে রাখুন: অতিরিক্ত `console.log` ডিস্ক ও CPU — দুটোই খায়; Production-এ লগের স্তর (`LOG_LEVEL=info`) সেট করুন। থেমে যাওয়া Container ও অব্যবহৃত Image জমে ডিস্ক ভরে যায় — সাপ্তাহিক `docker system df` ও পরিকল্পিত `prune` (Volume-সহ নয়) রুটিনে রাখুন।

৭.১৩ সমস্যা সমাধান (Troubleshooting)

TROUBLESHOOTING: Container চালু হয়েই থেমে যাচ্ছে (`Exited (1)`)? ধাপ: (১) `docker ps -a` — Exit Code দেখুন; (২) `docker logs <name>` — শেষ ত্রুটি পড়ুন; (৩) `docker inspect <name>`-এর `State` অংশে `Error` ও `OOMKilled` দেখুন (OOMKilled = true মানে মেমোরি শেষ হয়ে Kernel Container মেরে ফেলেছে); (৪) সন্দেহ হলে `docker run --rm -it <image> sh` দিয়ে Image-টি হাতে চালিয়ে ভেতরে পরীক্ষা করুন।

সমস্যা	সম্ভাব্য কারণ	সমাধান
`port is already allocated`	Host-এর ওই Port অন্য প্রসেস/Container ব্যবহার করছে	`docker ps` দেখে পুরনো Container থামান, অথবা অন্য Host Port ব্যবহার করুন
`Conflict. The container name is already in use`	ওই নামে থেমে যাওয়া Container আছে	`docker rm <name>` বা `docker start <name>`
`Exited (0)` — সঙ্গে সঙ্গে থেমে যায়	মূল প্রসেস সঙ্গে সঙ্গে শেষ হয়ে গেছে (যেমন `sh` কোনো কমান্ড ছাড়া)	দীর্ঘস্থায়ী প্রসেস (সার্ভার) চালান; `docker logs` দেখুন
`Exited (137)`	OOMKilled — মেমোরি শেষ	`--memory` সীমা বাড়ান; অ্যাপের মেমোরি লিক খুঁজুন
লগে `ECONNREFUSED`	নির্ভরশীল সার্ভিস (MongoDB) চলছে না বা ভুল ঠিকানা	সার্ভিসটির Container চালু আছে কি না ও ঠিকানা যাচাই করুন
`docker exec`-এ `executable file not found`	Image-এ ওই শেল/কমান্ড নেই (alpine-এ `bash` নেই)	`sh` ব্যবহার করুন; প্রয়োজনে `apk add bash` (অস্থায়ী)

৭.১৪ সেরা অনুশীলন ও অ্যান্টি-প্যাটার্ন

সেরা অনুশীলন: প্রতিটি Container-কে `--name` দিন; `-d` ও `--restart unless-stopped` Production Container-এ ব্যবহার করুন; লগের জন্য `docker logs` ব্যবহার করুন, Container-এর ভেতরে ফাইল সম্পাদনা নয়; Management Command (`docker container ls`) লিখুন — স্ক্রিপ্টে তা পড়তে সহজ; `docker ps -a` দিয়ে নিয়মিত থেমে যাওয়া Container পরিষ্কার করুন। **অ্যান্টি-প্যাটার্ন:** নাম ছাড়া Container চালিয়ে পরে খুঁজে না পাওয়া; `latest` ট্যাগ দিয়ে চালিয়ে কোন সংস্করণ চলছে না জানা; Container-এর ভেতরে হাতে পরিবর্তন করে "ঠিক" করা; ডেটাবেসের Port ইন্টারনেটে খুলে দেওয়া; `prune --volumes` না ভেবে চালানো।

৭.১৫ অধ্যায়ের সারাংশ

Image হলো স্থির ছাঁচ, Container তার চলমান উদাহরণ। `pull`, `image ls`, `inspect`, `image rm` দিয়ে Image পরিচালনা করুন; `run`, `ps`, `stop`, `start`, `rm` দিয়ে Container-এর জীবনচক্র সামলান। `logs` দিয়ে লগ দেখুন, `exec` দিয়ে ভেতরে ঢুকুন, `-p` দিয়ে Port প্রকাশ করুন, `-e` দিয়ে কনফিগারেশন দিন — কিন্তু Secret কখনো কমান্ড লাইনে নয়। Exit Code ও লগ পড়তে জানলেই Container-এর বেশিরভাগ সমস্যা মিনিটে নির্ণয় করা যায়।

৭.১৬ মূল প্রযুক্তিগত পরিভাষা

Management Command: গোষ্ঠীভুক্ত Docker কমান্ড — যেমন `docker image ls`, `docker container ls`।

Detached Mode (-d): Container ব্যাকগ্রাউন্ডে চালানো — টার্মিনাল ফিরে পাওয়া যায়।

Port Publishing (-p): Host-এর Port Container-এর Port-এর সঙ্গে জোড়া লাগানো।

Exit Code: প্রসেস শেষ হওয়ার সময় ফেরত দেওয়া সংখ্যা — 0 মানে সফল, অন্য সংখ্যা মানে ত্রুটির ধরন।

SIGTERM / SIGKILL: প্রসেস থামানোর সংকেত — SIGTERM ভদ্রভাবে থামতে বলে, SIGKILL জোর করে মারে।

OOMKilled: মেমোরি শেষ হয়ে গেলে Kernel Container-কে মেরে ফেলেছে — এমন অবস্থা।

Restart Policy: Container থেমে গেলে বা সার্ভার রিস্টার্ট হলে Docker কী করবে — যেমন `unless-stopped`।

Bridge Network: Docker-এর ডিফল্ট ভার্চুয়াল নেটওয়ার্ক, যেখানে Container-গুলো নিজেদের IP পায়।

৭.১৭ জ্ঞান যাচাই — প্রশ্ন

1. Image ও Container-এর পার্থক্য নিজের ভাষায় ব্যাখ্যা করুন। একই Image থেকে দুটি Container বানালে কী ভাগ হয়, কী আলাদা হয়?
2. `docker run` আসলে কোন তিনটি কাজ একসঙ্গে করে?
3. `docker ps` ও `docker ps -a`-এর পার্থক্য কী?
4. `EXPOSE 3000` আর `-p 3000:3000`-এর পার্থক্য কী?
5. Exit Code 137 দেখলে আপনি প্রথমে কী সন্দেহ করবেন এবং কী যাচাই করবেন?
6. Secret `-e` দিয়ে কমান্ড লাইনে দেওয়া কেন বিপজ্জনক? নিরাপদ বিকল্প কী?

৭.১৮ ব্যবহারিক অনুশীলন

1. **জীবনচক্র:** `nginx:alpine` Image দিয়ে একটি Container চালান, `docker ps` দেখুন, থামান, আবার চালু করুন, লগ দেখুন, তারপর মুছে ফেলুন — প্রতিটি ধাপের আউটপুট নোট করুন।
2. **লগ অনুশীলন:** একটি Node.js Container-এ ইচ্ছাকৃত ত্রুটি (ভুল Port) দিয়ে চালান; `docker logs` থেকে ত্রুটিটি খুঁজে বের করুন।
3. **exec অনুশীলন:** চলমান Container-এ `docker exec -it` দিয়ে ঢুকে `env`, `ps` ও ফাইল তালিকা দেখুন।

4. **পরিষ্কার:** `docker system df` চালান, তারপর `docker system prune` (Volume ছাড়া) চালিয়ে কত জায়গা ফিরল তা লিখুন।

অধ্যায় 8 • PART II — Docker Foundations and Image Engineering

Chapter 8 — Dockerfile Fundamentals and Build Instructions

৮.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: (১) Dockerfile কী এবং `docker build` কীভাবে Image তৈরি করে তা ব্যাখ্যা করতে; (২) প্রতিটি নির্দেশনা (Instruction) — FROM, WORKDIR, COPY, ADD, RUN, ENV, ARG, EXPOSE, USER, CMD, ENTRYPOINT, HEALTHCHECK — লাইন বাই লাইন ব্যাখ্যা করতে; (৩) CMD বনাম ENTRYPOINT, COPY বনাম ADD, ARG বনাম ENV-এর পার্থক্য বলতে; (৪) একটি Node.js/Express অ্যাপের জন্য সম্পূর্ণ Dockerfile লিখতে; (৫) Build-এর সাধারণ ত্রুটি নির্ণয় করতে।

৮.২ কেন এই বিষয়টি DevOps-এ গুরুত্বপূর্ণ

Dockerfile হলো Image-এর রেসিপি — একটি টেক্সট ফাইল, যেখানে লেখা থাকে বেস Image কোনটি, কোড কোথায় কপি হবে, নির্ভরতা কীভাবে ইনস্টল হবে, আর Container চালু হলে কোন কমান্ড চলবে। Dockerfile ভুল লিখলে Image হয় বিশাল, ধীর, অনিরাপদ — কখনো Secret-সহ। Dockerfile ঠিক লিখলে Image হয় ছোট, দ্রুত, নিরাপদ ও পুনরাবৃত্তিযোগ্য। CI Pipeline-এ প্রতিটি Build এই ফাইলটি পড়ে — তাই Dockerfile লেখা জানা মানেই নিজের অ্যাপের ডেলিভারি নিয়ন্ত্রণ করা। এই অধ্যায়টি এই বইয়ের সবচেয়ে গুরুত্বপূর্ণ অধ্যায়গুলোর একটি।

৮.৩ পূর্বশর্ত জ্ঞান

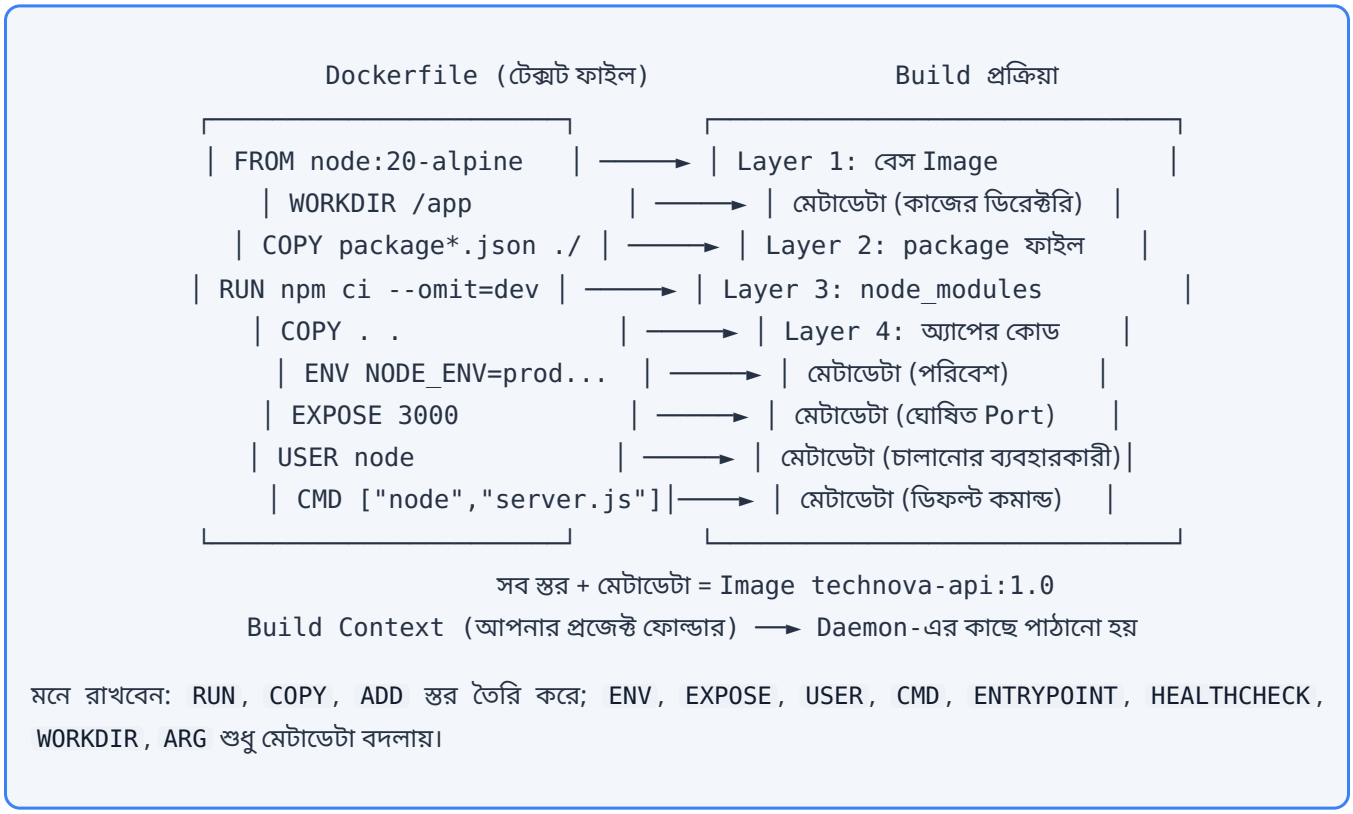
অধ্যায় ৬–৭ — Docker ইনস্টলেশন, Image বনাম Container, `docker run` ও `-p` / `-e`-এর ধারণা। Node.js-এ `package.json`, `npm ci` বনাম `npm install`-এর পার্থক্য, এবং Linux-এ ফাইল পাথ ও ব্যবহারকারী (User) ধারণা জানা থাকতে হবে।

৮.৪ প্রথম নীতি থেকে ব্যাখ্যা

Image তৈরি হয় স্তরে স্তরে (Layer)। Dockerfile-এর প্রতিটি নির্দেশনা একটি নতুন স্তর যোগ করে — যেমন `FROM node:20-alpine` একটি বেস স্তর নেয়, `COPY package.json ./` একটি স্তরে ফাইল যোগ করে, `RUN npm ci` একটি স্তরে নির্ভরতা ইনস্টল করে। এই স্তরগুলো একে অপরের ওপরে সাজানো হয়, আর প্রতিটি স্তর শুধু আগের স্তরের সঙ্গে তার পার্থক্যটুকু (Diff) সংরক্ষণ করে — তাই স্তরগুলো Cache করা যায় ও ভাগ করা যায় (বিস্তারিত অধ্যায় ৯-এ)। `docker build` Dockerfile পড়ে, প্রতিটি নির্দেশনা একটি অস্থায়ী Container-এ চালিয়ে, তার ফলাফলকে স্তর হিসেবে সংরক্ষণ করে, শেষে সব স্তর জুড়ে একটি Image তৈরি করে। Dockerfile-এর নির্দেশনাগুলো দুই ভাগে ভাগ করা যায়: **Build-সময়ের নির্দেশনা** (FROM, COPY, ADD, RUN, ARG) — Image তৈরির সময় কাজ করে; আর **Runtime-কনফিগারেশন নির্দেশনা** (ENV, EXPOSE, USER, CMD, ENTRYPOINT, HEALTHCHECK) — Image-এর মেটাডেটায় লেখা থাকে, Container চালু হলে কাজ করে।

> **CONCEPT:** Dockerfile একটি ঘোষণামূলক (Declarative) ফাইল — আপনি বলেন "কী চাই", Docker ঠিক করে "কীভাবে"। প্রতিটি নির্দেশনা একটি স্তর — তাই নির্দেশনার ক্রমই Image-এর গঠন, আকার ও Build-এর গতি নির্ধারণ করে। নির্দেশনা কমানো বা সাজানো মানে Image-কে ভালো করা — অধ্যায় ৯-এ এই শিল্পটি শিখবেন।

৮.৫ আর্কিটেকচার / ধারণাগত চিত্র



মনে রাখবেন: `RUN`, `COPY`, `ADD` স্তর তৈরি করে; `ENV`, `EXPOSE`, `USER`, `CMD`, `ENTRYPOINT`, `HEALTHCHECK`, `WORKDIR`, `ARG` শুধু মেটাডেটা বদলায়।

৮.৬ বিস্তারিত প্রযুক্তিগত ব্যাখ্যা — প্রতিটি নির্দেশনা লাইন বাই লাইন

FROM — বেস Image নির্বাচন

`FROM node:20-alpine` — Image-এর ভিত্তি। প্রতিটি Dockerfile-এর প্রথম নির্দেশনা (কমেন্ট ছাড়া) `FROM` হতেই হবে। ট্যাগ সবসময় নির্দিষ্ট করুন: `node:20-alpine` ভালো; `node:latest` খারাপ — কাল `latest` বদলে গেলে Build-এর ফল বদলে যাবে। বেস Image বেছে নেওয়ার নিয়ম: অফিসিয়াল Image (`node`, `nginx`) ব্যবহার করুন; ছোট বেস (alpine, slim) পছন্দ করুন — অধ্যায় ১০-এ আকারের হিসাব আসবে। Multi-stage Build-এ একাধিক `FROM` থাকে — প্রতিটি একটি নতুন Stage শুরু করে।

WORKDIR — কাজের ডিরেক্টরি

`WORKDIR /app` — পরের নির্দেশনাগুলোর জন্য বর্তমান ডিরেক্টরি নির্ধারণ করে; ডিরেক্টরি না থাকলে তৈরি করে। `RUN cd /app && npm ci` লেখার বদলে `WORKDIR /app` তারপর `RUN npm ci` লিখুন — পরিষ্কার ও নির্ভরযোগ্য। আপেক্ষিক পথ (`WORKDIR app`) আগের WORKDIR-এর সাপেক্ষে কাজ করে; পরম পথ (`/app`) ব্যবহার করাই নিরাপদ।

COPY ও ADD — ফাইল Image-এ নেওয়া

`COPY package*.json ./` — Build Context থেকে ফাইল Image-এ কপি করে। `COPY`-ই ডিফল্ট পছন্দ: স্বচ্ছ, শুধু কপি করে। `ADD` দুটি অতিরিক্ত কাজ পারে: URL থেকে ফাইল ডাউনলোড করা এবং tar আর্কাইভ স্বয়ংক্রিয়ভাবে খুলে ফেলা (Extract)। এই অতিরিক্ত ক্ষমতাই সমস্যা — কী হচ্ছে তা অস্পষ্ট থাকে। **নিয়ম: সবসময় COPY ব্যবহার করুন; ADD শুধু তখনই, যখন tar Extract করার দরকার।** URL থেকে ফাইল নিতে `ADD`-এর বদলে `RUN curl` ব্যবহার করুন — তাতে কী হচ্ছে স্পষ্ট থাকে ও যাচাই করা যায়।

RUN — Build-সময়ে কমান্ড চালানো

`RUN npm ci --omit=dev` — Image তৈরির সময় shell-এ কমান্ড চালায়, ফলাফল একটি স্তর হিসেবে সংরক্ষিত হয়। দুটি রূপ: Shell রূপ (`RUN npm ci` — `/bin/sh -c` দিয়ে চলে) ও Exec রূপ (`RUN ["npm", "ci"]` — shell ছাড়া সরাসরি চলে)। একাধিক কমান্ড `&&` দিয়ে এক `RUN`-এ জুড়ুন — প্রতিটি `RUN` একটি স্তর, তাই আলাদা আলাদা `RUN apt-get update` ও `RUN apt-get install` লিখলে Cache-এর কারণে পুরনো প্যাকেজ তালিকা ব্যবহার হতে পারে। সঠিক প্যাটার্ন: `RUN apt-get update && apt-get install -y --no-install-recommends curl && rm -rf /var/lib/apt/lists/*` — এক স্তরে আপডেট, ইনস্টল ও পরিষ্কার।

ENV ও ARG — Variable-এর পার্থক্য

`ENV NODE_ENV=production` — Image-এর মেটাডেটায় Environment Variable সেট করে; Container চালু হলে তা ভেতরে থাকে, `docker run -e` দিয়ে ওভাররাইড করা যায়। `ARG NODE_VERSION=20` — শুধু Build-সময়ের Variable; `docker build --build-arg NODE_VERSION=22` দিয়ে বদলানো যায়; Container-এ থাকে না। **পার্থক্য মনে রাখুন: ARG Build-সময়ে, ENV Runtime-এ।** সবচেয়ে গুরুত্বপূর্ণ নিয়ম: **ARG বা ENV — কোনোটিতেই Secret লিখবেন না।** ARG-এর মান Build ইতিহাসে (`docker history`) দেখা যায়; ENV-এর মান `docker inspect`-এ দেখা যায়। Secret দিতে BuildKit Secret Mount (`--mount=type=secret`) ব্যবহার করুন — অধ্যায় ৯-এ।

EXPOSE — Port ঘোষণা

`EXPOSE 3000` — Container যে Port-এ শোনে (Listen করে) তা ঘোষণা করে — ডকুমেন্টেশন ও টুলিংয়ের জন্য। এটি Port খোলে না — বাইরে থেকে পৌঁছাতে `docker run -p` লাগে (অধ্যায় ৭)। একাধিক Port: `EXPOSE 3000 8080`। `EXPOSE 3000/tcp` ডিফল্ট; UDP হলে `EXPOSE 53/udp`।

USER — কোন ব্যবহারকারী হিসেবে চলবে

`USER node` — পরের নির্দেশনা ও Container-এর মূল প্রসেস কোন ব্যবহারকারী হিসেবে চলবে তা নির্ধারণ করে। ডিফল্ট root — নিরাপত্তা ঝুঁকি (অধ্যায় ১০-এ বিস্তারিত)। `node:20-alpine` Image-এ `node` নামে non-root ব্যবহারকারী আগে থেকেই আছে। নিয়ম: ফাইল কপি ও ইনস্টলের পর, শেষ দিকে `USER node` লিখুন — যাতে অ্যাপ non-root হিসেবে চলে।

CMD বনাম ENTRYPOINT — Container চালু হলে কী চলবে

বিষয়	CMD	ENTRYPOINT
কাজ	Container চালুর ডিফল্ট কমান্ড	Container-এর মূল প্রোগ্রাম (সবসময় চলে)
`docker run`-এ আর্গুমেন্ট দিলে	CMD পুরোপুরি বদলে যায়	আর্গুমেন্ট ENTRYPOINT-এর পেছনে যোগ হয়
উদাহরণ	`CMD ["node", "server.js"]` — `docker run img sh` দিলে `sh` চলে	`ENTRYPOINT ["node"] + CMD ["server.js"]` — `docker run img app.js` দিলে `node app.js` চলে
কখন ব্যবহার করবেন	সাধারণ অ্যাপ — ডিফল্ট কমান্ড দিতে	Image একটি টুল হিসেবে ব্যবহার হলে (যেমন `node` Image)

দুটোই Exec রূপে (JSON array) লিখুন — Shell রূপ (`CMD node server.js`) shell-এর মাধ্যমে চলে, ফলে PID 1 হয় shell, Node নয়; তখন `SIGTERM` ঠিকমতো Node-এ পৌঁছায় না, Container থামতে দেরি হয়। **নিয়ম: Exec রূপ ব্যবহার**

করুন; একটিমাত্র CMD থাকতে পারে (শেষটি কার্যকর হয়); ENTRYPOINT ও CMD একসঙ্গে ব্যবহার করলে CMD ENTRYPOINT-এর ডিফল্ট আর্গুমেন্ট হয়।

HEALTHCHECK — Container সুস্থ কি না, Docker নিজে যাচাই করবে

`HEALTHCHECK --interval=30s --timeout=5s --start-period=10s --retries=3 CMD node -e "fetch('http://localhost:3000/health').then(r => process.exit(r.ok ? 0 : 1)).catch(() => process.exit(1))"` — Docker নিয়মিত Container-এর ভেতরে এই কমান্ড চালিয়ে স্বাস্থ্য যাচাই করে; কমান্ড সফল (Exit 0) হলে `healthy`, ব্যর্থ হলে কয়েকবার চেষ্টার পর `unhealthy`। `--start-period` হলো অ্যাপ চালু হওয়ার জন্য অপেক্ষার সময় — এই সময়ে ব্যর্থতা গোনা হয় না। HEALTHCHECK Container থামায় না — শুধু অবস্থা জানায়; Orchestrator (Volume 08) সেই অবস্থা দেখে সিদ্ধান্ত নেয়। আপনার Express অ্যাপে `/health` এন্ডপয়েন্ট রাখুন — এটি Production-এর মানদণ্ড।

অন্যান্য নির্দেশনা — সংক্ষেপে

`LABEL` — Image-এ মেটাডেটা (যেমন `LABEL maintainer="team@technova.example" version="1.0"`)। `VOLUME` — ডেটা সংরক্ষণের মাউন্ট পয়েন্ট ঘোষণা (বিস্তারিত Volume 07)। `SHELL` — `RUN`-এর ডিফল্ট shell বদলায় (Windows Image-এ বেশি ব্যবহৃত হয়)। `STOPSIGNAL` — থামানোর সময় কোন সংকেত পাঠাবে (ডিফল্ট SIGTERM)। `ONBUILD` — এই Image বেস হিসেবে ব্যবহার হলে পরে চালানোর নির্দেশনা — এখন এড়িয়ে চলাই ভালো, কারণ আচরণ অস্পষ্ট হয়।

৮.৭ সম্পূর্ণ কনফিগারেশন উদাহরণ — TechNova API-র Dockerfile

```
# Dockerfile — TechNova API (Node.js/Express)
# Build: docker build -t technova-api:1.0 .
# Run:   docker run -d --name technova-api -p 3000:3000 technova-api:1.0

FROM node:20-alpine
# FROM = বেস Image: Node.js 20, ছোট Alpine Linux; ট্যাগ নির্দিষ্ট — latest নয়

LABEL maintainer="team@technova.example"
LABEL version="1.0"
# LABEL = মেটাডেটা; Image-এর পরিচয়

WORKDIR /app
# WORKDIR = পরের সব নির্দেশনার কাজের ডিরেক্টরি; না থাকলে তৈরি হয়

COPY package.json package-lock.json ./
# COPY = Build Context থেকে শুধু package ফাইল দুটি নাও — কোড পরে নেওয়া হবে
# কারণ: package ফাইল বদলালেই শুধু নির্ভরতা আবার ইনস্টল হবে (Cache — অধ্যায় ৯)

RUN npm ci --omit=dev
# RUN = Build-সময়ে কমান্ড; npm ci = lock ফাইল অনুযায়ী হুবহু নির্ভরতা ইনস্টল
# --omit=dev = dev নির্ভরতা (পরীক্ষার টুল) বাদ — Production Image ছোট হয়

COPY . .
# COPY = এবার অ্যাপের বাকি কোড নাও (.dockerignore যা বাদ দিতে বলে, তা বাদ যাবে)

ENV NODE_ENV=production
ENV PORT=3000
# ENV = Runtime Environment Variable; docker run -e দিয়ে বদলানো যায়
# এখানে Secret নেই — Secret কখনো Dockerfile-এ লিখবেন না
```

```
EXPOSE 3000
# EXPOSE = ঘোষণা: অ্যাপ 3000 পোর্টে শোনে; Port খুলতে -p লাগে

USER node
# USER = এরপর থেকে non-root ব্যবহারকারী node হিসেবে চলবে

HEALTHCHECK --interval=30s --timeout=5s --start-period=10s --retries=3 \
  CMD node -e "fetch('http://localhost:3000/health').then(r => process.exit(r.ok ? 0 :
1)).catch(() => process.exit(1))"
# HEALTHCHECK = Docker নিয়মিত /health যাচাই করবে; ব্যর্থ হলে unhealthy দেখাবে

CMD ["node", "server.js"]
# CMD = Exec রূপে ডিফল্ট কমান্ড; docker run-এ আর্গুমেন্ট দিলে এটি বদলে যায়
```

Build ও চালানো:

```
docker build -t technova-api:1.0 .
# -t technova-api:1.0 = Image-এর নাম:ট্যাগ; . = Build Context (বর্তমান ডিরেক্টরি)
# আউটপুটে প্রতিটি ধাপ দেখা যাবে: Step 3/10 : COPY package.json ...

docker image ls technova-api
docker run -d --name technova-api -p 3000:3000 technova-api:1.0
docker ps
curl http://localhost:3000/health
# নমুনা আউটপুট (চিত্রণমূলক): {"status":"ok","uptime":42}
docker logs technova-api
docker exec -it technova-api sh
# ভেতরে: whoami (node দেখা উচিত — non-root প্রমাণ), ls /app
```

৮.৮ বাস্তব প্রকৌশল পরিস্থিতি

REAL WORLD — TechNova: TechNova-র প্রথম Dockerfile-এ লেখা ছিল `FROM node:latest`, `COPY . .` প্রথমেই, `RUN npm install`, আর Container root হিসেবে চলত। ফল: একদিন `latest` Node 20 থেকে 22 হয়ে গেল — Production Build ভেঙে গেল; প্রতিটি ছোট কোড পরিবর্তনে পুরো `npm install` আবার চলত (১০ মিনিটের Build); Image-এর আকার 1.2 GB। এই অধ্যায়ের Dockerfile-এ বদলে ফেলার পর: ট্যাগ বাঁধা, `package.json` আগে কপি, `npm ci --omit=dev`, non-root `USER node` — Image 180 MB, Build ৪০ সেকেন্ড (Cache থাকলে), আর Node সংস্করণ আর হঠাৎ বদলায় না।

৮.৯ ধাপে ধাপে ব্যবহারিক উদাহরণ

উদাহরণ: আপনার Express অ্যাপের Dockerfile লেখা

1. একটি ছোট Express অ্যাপ বানান (`server.js` — `/` ও `/health` এন্ডপয়েন্টসহ) ও `package.json` তৈরি করুন।
2. ওপরের Dockerfileটি কপি করে আপনার অ্যাপের নাম ও Port অনুযায়ী বদলান।
3. `docker build -t my-api:1.0 .` চালান; প্রতিটি Step-এর আউটপুট পড়ুন — কোন ধাপে Cache ব্যবহৃত হলো (`CACHED`) দেখুন।

4. Container চালিয়ে `curl http://localhost:3000/health` দিয়ে যাচাই করুন; `docker exec -it ... sh` দিয়ে `whoami` দেখুন।
5. ইচ্ছাকৃতভাবে `CMD`-এর Shell রূপ (`CMD node server.js`) লিখে `docker exec ... ps` দিয়ে PID 1 কী তা দেখুন — Exec রূপের সঙ্গে তুলনা করুন।

৮.১০ সাধারণ ভুল ও ভ্রান্ত ধারণা

- **ভুল:** `FROM node:latest` — পুনরাবৃত্তিযোগ্যতা নষ্ট হয়; সবসময় নির্দিষ্ট ট্যাগ বা Digest ব্যবহার করুন।
- **ভুল:** `COPY . .` প্রথমেই করা — কোডের ছোট পরিবর্তনেও নির্ভরতা আবার ইনস্টল হয় (অধ্যায় ৯-এ বিস্তারিত)।
- **ভুল:** `RUN npm install` — lock ফাইল উপেক্ষা করে; `npm ci` ব্যবহার করুন।
- **ভুল:** Dockerfile-এ Secret লেখা (`ENV API_KEY=...`, `ARG TOKEN=...`) — Image-এর ইতিহাসে চিরকাল থেকে যায়। কখনো নয়।
- **ভুল ধারণা:** "ADD আর COPY একই।" — ADD tar খোলে ও URL নেয়; স্বচ্ছতার জন্য COPY-ই ডিফল্ট।
- **ভুল ধারণা:** "CMD আর ENTRYPOINT একই।" — CMD বদলানো যায়, ENTRYPOINT Container-এর মূল প্রোগ্রাম হিসেবে থাকে।
- **ভুল ধারণা:** "EXPOSE করলেই Port খুলে যায়।" — না, `-p` লাগে।
- **ভুল:** Container root হিসেবে চালানো — `USER node` শেষে যোগ করুন।
- **ভুল:** আলাদা আলাদা `RUN apt-get update` ও `RUN apt-get install` — Cache-এর কারণে পুরনো প্যাকেজ তালিকা ব্যবহার হতে পারে; এক `RUN`-এ জুড়ুন।

৮.১১ নিরাপত্তা বিবেচনা

> **WARNING:** Dockerfile-এ Secret লেখা এই বইয়ের সবচেয়ে কঠোর নিষেধ। `ENV`, `ARG`, `RUN echo "password" > config` — সবকিছুই Image-এর স্তরে থেকে যায়; Image যার হাতে যাবে, Secret তার হাতে যাবে — এমনকি Image মুছে ফেলার পরও Registry-তে থেকে যেতে পারে। Build-সময়ে Secret দরকার হলে BuildKit Secret Mount ব্যবহার করুন (অধ্যায় ৯); Runtime Secret-এর জন্য Environment Variable (বাইরে থেকে দেওয়া), Docker Secret বা Secret Manager ব্যবহার করুন।

আরও: অফিসিয়াল বেস Image ব্যবহার করুন; বেস Image নিয়মিত আপডেট করুন (পুরনো বেসে পরিচিত দুর্বলতা থাকে); `USER node` দিয়ে non-root চালান; অপ্রয়োজনীয় টুল (যেমন `curl`, `git`) Production Image-এ রাখবেন না — আক্রমণকারীর হাতিয়ার কমে।

৮.১২ কর্মক্ষমতা ও সম্পদ বিবেচনা

Image-এর আকার সরাসরি Deploy-এর গতিকে প্রভাবিত করে: 1 GB Image ডাউনলোড করতে যে সময় লাগে, 180 MB Image-এ তার ভগ্নাংশ লাগে। `--omit=dev`, ছোট বেস Image (alpine), অপ্রয়োজনীয় ফাইল বাদ দেওয়া (`.dockerignore` — অধ্যায় ৯) — তিনটি মিলে Image ছোট হয়। `RUN` নির্দেশনা কম রাখুন — প্রতিটি স্তর ওভারহেড যোগ করে। Build-এর সময়ও সম্পদ: `npm ci`-এর Cache (অধ্যায় ৯) ব্যবহার করলে CI-তে Build মিনিট থেকে সেকেন্ডে নামে।

৮.১৩ সমস্যা সমাধান (Troubleshooting)

TROUBLESHOOTING: Build ব্যর্থ হলে Docker কোন ধাপে থেমেছে তা দেখায় (`ERROR: failed to solve: ... Step 5/10`)। প্রথমে ওই ধাপের নির্দেশনাটি পড়ুন। সাধারণ কারণ: ভুল ফাইল পাথ (`COPY` -এর ফাইল Build Context-এ নেই), নেটওয়ার্ক ব্যর্থতা (`npm ci` -এ registry-তে পৌঁছানো যায়নি), বা বেস Image-এর ট্যাগ ভুল। `docker build --no-cache` দিয়ে Cache ছাড়া Build করে দেখুন সমস্যাটি Cache-জনিত কি না।

সমস্যা	সম্ভাব্য কারণ	সমাধান
`failed to compute cache key: "/x" not found`	COPY-এর ফাইল Context-এ নেই বা `.dockerignore` -তে বাদ পড়েছে	ফাইল পাথ ও `.dockerignore` যাচাই করুন
`npm ci` ব্যর্থ — lock ফাইল নেই	`package-lock.json` কপি করা হয়নি	`COPY package.json package-lock.json ./` দুটোই কপি করুন
Container চালু হয়েই থেমে যায়	CMD ভুল, বা অ্যাপ ভুল Port-এ শুনছে	`docker logs` দেখুন; `PORT` ও `CMD` যাচাই করুন
`permission denied` ফাইল লিখতে গিয়ে	`USER node` -এর পর ফাইলের মালিক root	`COPY --chown=node:node` ব্যবহার করুন, অথবা `USER` -এর আগে কপি শেষ করুন
HEALTHCHECK সবসময় unhealthy	`/health` এন্ডপয়েন্ট নেই বা ভুল Port	এন্ডপয়েন্ট ও Port যাচাই করুন; `--start-period` বাড়ান

৮.১৪ সেরা অনুশীলন ও অ্যান্টি-প্যাটার্ন

সেরা অনুশীলন: নির্দিষ্ট ট্যাগের অফিসিয়াল বেস Image; `package.json` আগে, কোড পরে; `npm ci --omit=dev` ; Exec রূপে CMD; `USER node` ; HEALTHCHECK-সহ `/health` এন্ডপয়েন্ট; LABEL-এ সংস্করণ। **অ্যান্টি-প্যাটার্ন:** `latest` ট্যাগ; Dockerfile-এ Secret; root হিসেবে চালানো; `COPY . .` প্রথমে; `npm install` (lock ছাড়া); Shell রূপে CMD; `ADD` দিয়ে URL ডাউনলোড; একাধিক CMD (শুধু শেষটি চলে — বিভ্রান্তি তৈরি করে)।

৮.১৫ অধ্যায়ের সারাংশ

Dockerfile হলো Image-এর রেসিপি: প্রতিটি নির্দেশনা একটি স্তর বা মেটাডেটা যোগ করে। FROM বেস নির্ধারণ করে, WORKDIR কাজের ডিরেক্টরি ঠিক করে, COPY ফাইল নেয় (ADD শুধু tar-এর জন্য), RUN Build-সময়ে কমান্ড চালায়, ENV Runtime Variable দেয় (ARG শুধু Build-সময়ে), EXPOSE Port ঘোষণা করে, USER non-root চালায়, CMD ডিফল্ট কমান্ড দেয় (ENTRYPOINT মূল প্রোগ্রাম), HEALTHCHECK স্বাস্থ্য যাচাই করে। Secret কখনো Dockerfile-এ নয় — এই নিয়মটি এই অধ্যায়ের সবচেয়ে গুরুত্বপূর্ণ শিক্ষা।

৮.১৬ মূল প্রযুক্তিগত পরিভাষা

Dockerfile: Image তৈরির নির্দেশনাসমূহ লেখা টেক্সট ফাইল।

Build Context: `docker build`-এর সময় Daemon-এর কাছে পাঠানো ফাইলসমূহ (সাধারণত প্রজেক্ট ফোল্ডার)।

Layer (স্তর): Image-এর একটি ধাপ — প্রতিটি `RUN` / `COPY` / `ADD` একটি স্তর তৈরি করে।

Base Image: Dockerfile-এর `FROM`-এ নির্দেশিত শুরুর Image।

Exec Form: কমান্ড JSON array আকারে লেখা — যেমন `["node", "server.js"]` — shell ছাড়া চলে।

Shell Form: কমান্ড সাধারণ টেক্সট আকারে লেখা — shell-এর মাধ্যমে চলে।

HEALTHCHECK: Docker-এর নিয়মিত স্বাস্থ্য যাচাই — Container healthy না unhealthy তা জানায়।

PID 1: Container-এর প্রথম প্রসেস — সংকেত (Signal) গ্রহণ ও সন্তান প্রসেস সামলানোর দায়িত্ব তার।

৮.১৭ জ্ঞান যাচাই — প্রশ্ন

1. কোন নির্দেশনাগুলো স্তর তৈরি করে, আর কোনগুলো শুধু মেটাডেটা বদলায়? তালিকা করুন।
2. COPY বনাম ADD — কখন কোনটি ব্যবহার করবেন?
3. ARG বনাম ENV — পার্থক্য কী, এবং কোনটিতে Secret লেখা যাবে না কেন?
4. CMD বনাম ENTRYPOINT — `docker run img sh` দিলে দুটি ক্ষেত্রে কী ঘটে?
5. Shell রূপে CMD লিখলে PID 1-এর সমস্যাটি কী?
6. HEALTHCHECK কী করে, আর কী করে না?

৮.১৮ ব্যবহারিক অনুশীলন

1. **Dockerfile লেখা:** আপনার একটি Express অ্যাপের জন্য এই অধ্যায়ের Dockerfile লিখে Build করুন ও চালান।
2. **CMD বনাম ENTRYPOINT:** একটি Image-এ দুটি সংস্করণ বানান — একটিতে CMD, একটিতে ENTRYPOINT — এবং `docker run`-এ আর্গুমেন্ট দিয়ে পার্থক্য দেখুন।
3. **Secret পরীক্ষা:** একটি পরীক্ষামূলক Image-এ `ENV FAKE_SECRET=CHANGE-ME-TEST` লিখে `docker inspect` ও `docker history`-তে তা দেখা যায় কি না যাচাই করুন — তারপর কেন এটি বিপজ্জনক, তা লিখুন।
4. **HEALTHCHECK:** আপনার অ্যাপে `/health` এন্ডপয়েন্ট যোগ করে HEALTHCHECK চালু করুন; `docker ps`-এ `(healthy)` দেখা যায় কি না যাচাই করুন।

অধ্যায় 9 • PART II — Docker Foundations and Image Engineering

Chapter 9 — Build Context, Layers, Cache and Build Optimization

৯.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: (১) Build Context ও `.dockerignore`-এর কাজ ব্যাখ্যা করতে; (২) Image Layer কীভাবে তৈরি, সংরক্ষণ ও ভাগ করা হয় তা বলতে; (৩) Layer Cache কীভাবে কাজ করে ও কখন ভাঙে (Cache Invalidation) তা নির্ণয় করতে; (৪) BuildKit ও Build Cache Mount ব্যবহার করে Build দ্রুত করতে; (৫) Build-সময়ে Secret নিরাপদে দিতে (Secret Mount); (৬) একটি ধীর Build বিশ্লেষণ করে অপ্টিমাইজ করতে।

৯.২ কেন এই বিষয়টি DevOps-এ গুরুত্বপূর্ণ

একটি ধীর Build CI Pipeline-এর সবচেয়ে বড় বোতল-ঘাড় (Bottleneck): প্রতিটি Pull Request-এ ১০ মিনিটের Build মানে দিনে ২০টি Pull Request-এ ২০০ মিনিট নষ্ট। আর Build Context-এ ভুল ফাইল পাঠানো মানে Secret Image-এ ঢুকে যাওয়া বা Build ধীর হয়ে যাওয়া। Layer ও Cache বুঝলে আপনি Build-কে মিনিট থেকে সেকেন্ডে নামাতে পারেন — এবং একই সঙ্গে Image-কে নিরাপদ রাখতে পারেন। এই অধ্যায়টি Docker-কে "কাজ করে" থেকে "দ্রুত ও নিরাপদে কাজ করে"-তে নিয়ে যায়।

৯.৩ পূর্বশর্ত জ্ঞান

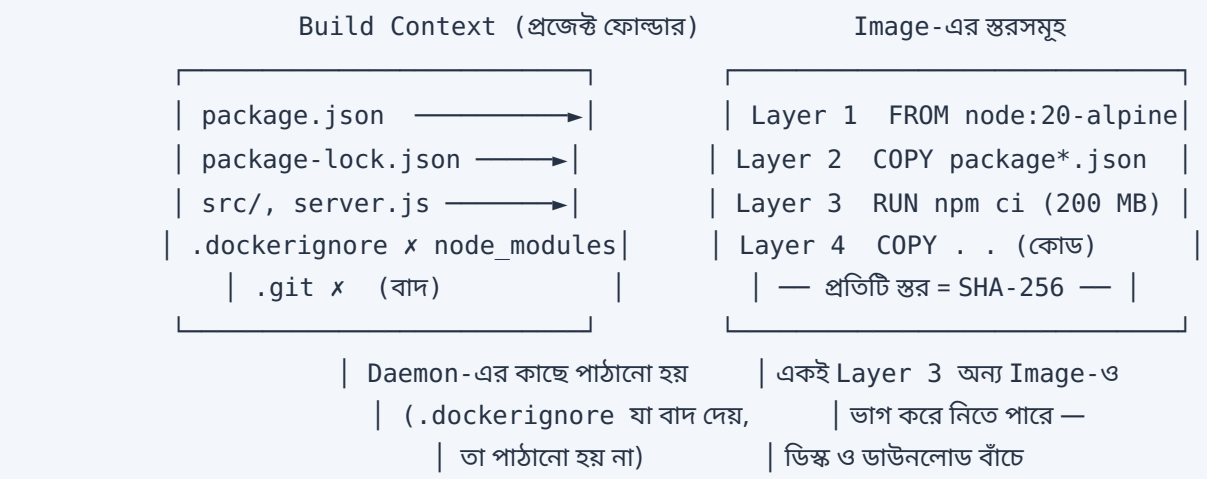
অধ্যায় ৮ — Dockerfile-এর সব নির্দেশনা, বিশেষ করে COPY, RUN, ARG, ENV। Git-এ `.gitignore`-এর ধারণা, `npm ci`-এর কাজ, এবং Linux-এ ফাইলের আকার (`du -sh`) দেখার অভিজ্ঞতা থাকলে ভালো হয়।

৯.৪ প্রথম নীতি থেকে ব্যাখ্যা

`docker build` চালালে প্রথমে যা ঘটে তা অনেকেই খেয়াল করেন না: Docker আপনার প্রজেক্ট ফোল্ডারের (Build Context) সব ফাইল Daemon-এর কাছে পাঠায় — তারপর Dockerfile পড়ে। Context বড় হলে (যেমন `node_modules`-সহ 500 MB) প্রতিটি Build-এর শুরুতেই মিনিটখানেক শুধু ফাইল পাঠাতে যায়। **Layer** হলো Image-এর নির্মাণ-খণ্ড: প্রতিটি `RUN` / `COPY` / `ADD` একটি স্তর তৈরি করে, আর প্রতিটি স্তর শুধু আগের স্তরের সঙ্গে তার পার্থক্য সংরক্ষণ করে। স্তরগুলো **Content-Addressable** — প্রতিটি স্তরের পরিচয় তার বিষয়বস্তুর Hash (SHA-256); একই বিষয়বস্তুর স্তর দুটি Image-এ থাকলে ডিস্কে একবারই সংরক্ষিত হয়, দুটি Image-ই তা ভাগ করে নেয়। **Layer Cache** এই নীতির ওপর দাঁড়িয়ে: Build-এর সময় Docker দেখে — এই ধাপের নির্দেশনা ও তার ইনপুট (যেমন কপি করা ফাইলের Hash) আগের Build-এর কোনো স্তরের সঙ্গে মিলছে কি না; মিললে নতুন করে চালানো হয় না, Cache করা স্তরটি ব্যবহার করা হয়। একটি ধাপের Cache ভাঙলে (Invalidation) তার পরের সব ধাপের Cache-ও ভাঙে — তাই নির্দেশনার ক্রমই Build-এর গতি নির্ধারণ করে।

> **CONCEPT:** Cache-এর মূল নীতি — "যা বদলায় না, তা আগে রাখুন; যা বদলায়, তা পরে রাখুন।" নির্ভরতা (`package.json`) কম বদলায়, কোড প্রতিদিন বদলায় — তাই `COPY package.json` ও `RUN npm ci` আগে, `COPY . .` পরে। উল্টো ক্রমে প্রতিটি কোড পরিবর্তনে `npm ci` আবার চলবে।

৯.৫ আর্কিটেকচার / ধারণাগত চিত্র



`docker history technova-api:1.0` দিয়ে প্রতিটি স্তরের আকার ও কমান্ড দেখুন — কোন স্তরটি সবচেয়ে বড়, তা এখানেই ধরা পড়ে।

৯.৬ বিস্তারিত প্রযুক্তিগত ব্যাখ্যা

Build Context ও .dockerignore

Context হলো `docker build`-এর শেষ আর্গুমেন্ট — সাধারণত `.` (বর্তমান ডিরেক্টরি)। Daemon Context-এর সব ফাইল পায়, তবে `COPY` শুধু Context-এর ভেতরের ফাইলই নিতে পারে — Context-এর বাইরের ফাইল (`../secret.txt`) কপি করা যায় না। `.dockerignore` ফাইলটি `.gitignore`-এর মতো কাজ করে — যে ফাইল Context-এ পাঠানো হবে না, তার তালিকা। TechNova-র `.dockerignore`-এ থাকা উচিত: `node_modules`, `.git`, `.env` (Secret!), `*.log`, `coverage`, `.next` (Next.js Build আউটপুট — Multi-stage-এ ভেতরে তৈরি হবে), `Dockerfile` নিজে নয় — তবে `.dockerignore`-তে `.env` রাখা মানে `COPY . .` তা কপি করবে না — Secret Image-এ ঢোকার একটি পথ বন্ধ হয়। সতর্কতা: `.dockerignore` ভুল লিখলে দরকারি ফাইলও বাদ পড়ে — `docker build`-এর শুরুতে "Sending build context" আকারটি দেখুন; অস্বাভাবিক ছোট বা বড় হলে সন্দেহ করুন।

Layer Cache — কখন ভাঙে, কখন থাকে

Cache মেলার শর্ত: একই নির্দেশনা, একই আগের স্তর, এবং `COPY`-এর ক্ষেত্রে ফাইলের বিষয়বস্তুর Hash একই। `RUN npm ci`-এর Cache ভাঙে যখন `package.json` বা `package-lock.json` বদলায় (কারণ তার আগের `COPY` স্তর বদলেছে)। `RUN apt-get update`-এর Cache বিপজ্জনক হতে পারে — Cache থাকলে পুরনো প্যাকেজ তালিকা ব্যবহার হয়; তাই `apt-get update && apt-get install` এক `RUN`-এ জুড়ুন (অধ্যায় ৮)। `ARG`-এর মান বদলালে তার পরের ধাপগুলোর Cache ভাঙে। Cache ইচ্ছাকৃতভাবে ভাঙতে `docker build --no-cache` ব্যবহার করুন — তবে CI-তে প্রতিবার `--no-cache` ব্যবহার করা সময়ের অপচয়।

BuildKit — আধুনিক Build Engine

BuildKit হলো Docker-এর আধুনিক Build Engine (Buildx-এর মাধ্যমে)। সুবিধা: সমান্তরাল (Parallel) ধাপ চালানো, উন্নত Cache, Secret Mount, Cache Mount, Multi-platform Build। যাচাই করুন: `docker buildx version` ও `docker buildx ls`। `DOCKER_BUILDKIT=1 docker build ...` বা Buildx ব্যবহার করলে BuildKit সক্রিয় হয় — Docker-এর নতুন সংস্করণে ডিফল্ট। `docker buildx build --cache-to type=local,dest=/tmp/cache --cache-from type=local,src=/tmp/cache` দিয়ে CI-তে Cache Runner-দের মধ্যে ভাগ করা যায়।

Cache Mount — নির্ভরতার Cache পুনরায় ব্যবহার

```
# syntax=docker/dockerfile:1
# এই লাইনটি Dockerfile-এর প্রথম লাইনে লিখলে BuildKit ফিচার সক্রিয় হয়
FROM node:20-alpine
WORKDIR /app
COPY package.json package-lock.json ./
RUN --mount=type=cache,target=/root/.npm npm ci --omit=dev
# --mount=type=cache = /root/.npm ডিরেক্টরিটি Build-গুলোর মধ্যে Cache হিসেবে থাকে
# ফলে npm-এর ডাউনলোড করা প্যাকেজ আবার ডাউনলোড হয় না — কিন্তু Cacheটি
# Image-এর স্তরে ঢোকে না, তাই Image বড় হয় না। এটাই Cache Mount-এর জাদু।
```

Secret Mount — Build-সময়ে Secret নিরাপদে দেওয়া

```
# syntax=docker/dockerfile:1
FROM node:20-alpine
WORKDIR /app
COPY package.json package-lock.json ./
RUN --mount=type=secret,id=npmrc,target=/root/.npmrc npm ci --omit=dev
# --mount=type=secret = Secret ফাইলটি শুধু এই RUN ধাপের সময় মাউন্ট হয়,
# তারপর মুছে যায় — Image-এর কোনো স্তরে থাকে না।
# Build কমান্ড: docker build --secret id=npmrc,src=$HOME/.npmrc -t technova-api:1.0 .
# ভুল পদ্ধতি: ARG NPM_TOKEN=... — Build ইতিহাসে (docker history) দেখা যায়। কখনো নয়।
```

> **WARNING:** `ARG`-এ Secret দেওয়া আর Dockerfile-এ Secret লেখা — দুটোই সমান বিপজ্জনক। `docker history` দিয়ে যে কেউ ARG-এর মান দেখতে পারে। প্রাইভেট npm Registry বা Git Repository থেকে নির্ভরতা আনতে হলে BuildKit Secret Mount ব্যবহার করুন, আর Runtime Secret-এর জন্য Environment Variable বা Secret Manager ব্যবহার করুন।

Image আকার বিশ্লেষণ

`docker history technova-api:1.0` — প্রতিটি স্তরের আকার দেখায়; `docker image ls` — মোট আকার; `docker system df -v` — বিস্তারিত ডিস্ক ব্যবহার। বড় স্তর সাধারণত `npm ci` (node_modules) বা `apt-get install` হয়। আকার কমানোর উপায়: `--omit=dev`, ছোট বেস, অপ্রয়োজনীয় ফাইল `.dockerignore`-তে বাদ, এক `RUN`-এ ইনস্টল + পরিষ্কার, আর Multi-stage Build (অধ্যায় ১০)।

৯.৭ সম্পূর্ণ কনফিগারেশন উদাহরণ — অপ্টিমাইজড Build

```
# .dockerignore — TechNova API
node_modules
```

```
.git
.env
.env.*
*.log
coverage
.next
dist
README.md
# এই ফাইলগুলো Build Context-এ যাবে না — Build দ্রুত হবে, Secret Image-এ ঢুকবে না

# Dockerfile (BuildKit) — অপ্টিমাইজড সংস্করণ
# syntax=docker/dockerfile:1
FROM node:20-alpine
WORKDIR /app
COPY package.json package-lock.json ./
RUN --mount=type=cache,target=/root/.npm npm ci --omit=dev
COPY . .
RUN npm run build
# npm run build = Next.js বা TypeScript প্রজেক্টে প্রয়োজন; সাধারণ Express অ্যাপে বাদ দিন
ENV NODE_ENV=production
USER node
EXPOSE 3000
CMD ["node", "server.js"]

# Build কমান্ড (Cache-সহ):
DOCKER_BUILDKIT=1 docker build -t technova-api:1.0 .
# প্রথম Build: সব ধাপ চলে (মিনিটখানেক)
# কোড বদলে আবার Build: শুধু COPY . . থেকে পরের ধাপগুলো চলে (সেকেন্ড)
# package.json বদলে আবার Build: npm ci থেকে পরের ধাপগুলো চলে
```

৯.৮ বাস্তব প্রকৌশল পরিস্থিতি

> **REAL WORLD — TechNova:** TechNova-র CI-তে প্রতিটি Build ৯ মিনিট নিত — কারণ `.dockerignore` ছিল না, ফলে 600 MB Context (node_modules ও .git-সহ) প্রতিবার Daemon-এ পাঠানো হতো, আর `COPY . .` প্রথমে থাকায় প্রতিটি commit-এ `npm ci` আবার চলত। `.dockerignore` যোগ করে Context 2 MB-এ নামল, `package.json` আগে কপি করে Cache ঠিক করা হলো, আর Cache Mount যোগ করা হলো — Build ৯ মিনিট থেকে ৪৫ সেকেন্ডে নামল। CI-এর মাসিক খরচও কমল, কারণ Runner-এর সময়ই খরচ।

৯.৯ ধাপে ধাপে ব্যবহারিক উদাহরণ

উদাহরণ: আপনার Build-এর Cache বিশ্লেষণ

1. `docker build -t my-api:1.0 .` চালিয়ে সময় মাপুন (`time docker build ...`)।
2. কোনো ফাইল না বদলে আবার Build করুন — সব ধাপ `CACHED` দেখা উচিত, সময় কয়েক সেকেন্ড।
3. শুধু `server.js`-এ একটি কমেন্ট বদলে আবার Build করুন — কোন ধাপ থেকে Cache ভাঙল তা লিখুন।
4. শুধু `package.json`-এ একটি নির্ভরতা যোগ করে আবার Build করুন — কোন ধাপ থেকে Cache ভাঙল তা লিখুন।
5. `.dockerignore` যোগ করার আগে ও পরে Context-এর আকার তুলনা করুন — Build-এর শুরুর "Sending build context" লাইনটি দেখুন।

৯.১০ সাধারণ ভুল ও ভ্রান্ত ধারণা

- **ভুল:** `.dockerignore` না থাকা — Context বিশাল, Build ধীর, `.env` Image-এ ঢুকে যাওয়ার ঝুঁকি।
- **ভুল:** `COPY . .` প্রথমে — প্রতিটি কোড পরিবর্তনে `npm ci` আবার চলে।
- **ভুল ধারণা:** "Cache থাকলে Build সবসময় সঠিক।" — না; `apt-get update`-এর Cache পুরনো প্যাকেজ দিতে পারে, আর `RUN curl ... | sh`-এর Cache পুরনো স্ক্রিপ্ট দিতে পারে। সন্দেহ হলে `--no-cache` দিয়ে যাচাই করুন।
- **ভুল:** ARG-এ Secret দেওয়া — `docker history`-তে দৃশ্যমান।
- **ভুল ধারণা:** "স্তর মুছলে Image ছোট হয়।" — না; পরের স্তরে ফাইল মুছলে আগের স্তরের ফাইল Image-এ থেকেই যায় — শুধু লুকিয়ে যায়। একই `RUN`-এ ইনস্টল ও পরিষ্কার করতে হয়।
- **ভুল:** `docker build --no-cache` CI-তে সবসময় ব্যবহার করা — সময় ও খরচের অপচয়।

৯.১১ নিরাপত্তা বিবেচনা

Build Context-এ Secret ফাইল (`.env`, `id_rsa`, `credentials.json`) থাকলে তা Daemon-এর কাছে পাঠানো হয় — `.dockerignore` দিয়ে বাদ দিন। Build-সময়ে Secret দরকার হলে শুধু Secret Mount ব্যবহার করুন। প্রাইভেট Registry থেকে Image টানার সময় Registry-র Credential `docker login`-এ সংরক্ষিত হয় (`~/.docker/config.json`) — এই ফাইলের অনুমতি সুরক্ষিত রাখুন। Build-এর সময় `curl | sh` প্যাটার্ন এড়িয়ে চলুন — যাচাই না করা স্ক্রিপ্ট Image-এ যা খুশি করতে পারে।

৯.১২ কর্মক্ষমতা ও সম্পদ বিবেচনা

Build-এর গতি তিনটি জিনিসে নির্ভর করে: Context-এর আকার, Cache-এর ব্যবহার, আর নেটওয়ার্ক ডাউনলোড। Context ছোট রাখুন (`.dockerignore`), নির্দেশনার ক্রম Cache-বান্ধব রাখুন, Cache Mount দিয়ে npm Cache পুনরায় ব্যবহার করুন, আর CI-তে Build Cache ভাগ করুন (`--cache-from`/`--cache-to`)। BuildKit-এর সমান্তরাল ধাপ চালানো Multi-stage Build-এ বিশেষভাবে কার্যকর। Disk-এর দিকেও নজর দিন: প্রতিটি Build পুরনো স্তর রেখে যায় — নিয়মিত `docker system prune` করুন (Volume-সহ নয়)।

৯.১৩ সমস্যা সমাধান (Troubleshooting)

TROUBLESHOOTING: Build হঠাৎ ধীর হয়ে গেলে প্রথমে Context-এর আকার দেখুন — Build-এর শুরুর লাইনে "Sending build context to Docker daemon 612.5MB" লেখা থাকলে `.dockerignore` যাচাই করুন। Cache কাজ করছে না মনে হলে `docker build --progress=plain` দিয়ে বিস্তারিত আউটপুট দেখুন — কোন ধাপে `CACHED` লেখা নেই, তা খুঁজুন।

সমস্যা	সম্ভাব্য কারণ	সমাধান
Build-এর শুরুতে মিনিটখানেক আটকে থাকে	Context বিশাল (node_modules/.git)	`.dockerignore` যোগ করুন
প্রতিবার `npm ci` আবার চলে	`COPY . .` `npm ci`-এর আগে	`package.json` আগে কপি করুন

CI-তে Cache কাজ করে না	প্রতিটি Runner নতুন মেশিন	Registry বা Local Cache Export/Import ব্যবহার করুন
`docker history`-তে Secret দেখা যায়	ARG/ENV-এ Secret দেওয়া হয়েছে	Image মুছে ফেলুন, Registry থেকে সরান, Secret বদলান; ভবিষ্যতে Secret Mount ব্যবহার করুন

৯.১৪ সেরা অনুশীলন ও অ্যান্টি-প্যাটার্ন

সেরা অনুশীলন: `.dockerignore` সবসময় রাখুন; কম-বদলানো জিনিস আগে, বেশি-বদলানো জিনিস পরে; Cache Mount দিয়ে npm Cache পুনরায় ব্যবহার করুন; Secret Mount দিয়ে Build-সময়ের Secret দিন; `docker history` দিয়ে স্তরের আকার নিয়মিত পরীক্ষা করুন। **অ্যান্টি-প্যাটার্ন:** Context-এ `node_modules` পাঠানো; `--no-cache` অভ্যাসে পরিণত করা; পরের স্তরে ফাইল মুছে Image ছোট করার চেষ্টা; ARG-এ Secret; `curl | sh` দিয়ে Build।

৯.১৫ অধ্যায়ের সারাংশ

Build Context হলো Daemon-এর কাছে পাঠানো ফাইলসমূহ — `.dockerignore` দিয়ে তা ছোট ও নিরাপদ রাখুন। Image স্তরে স্তরে তৈরি হয়; প্রতিটি স্তর Content-Addressable Hash দিয়ে চেনা যায় ও ভাগ করা যায়। Layer Cache নির্দেশনার ক্রমের ওপর নির্ভর করে — কম-বদলানো জিনিস আগে রাখুন। BuildKit Cache Mount ও Secret Mount দিয়ে Build দ্রুত ও নিরাপদ করুন; Secret কখনো ARG বা স্তরে নয়।

৯.১৬ মূল প্রযুক্তিগত পরিভাষা

Build Context: Build-এর সময় Daemon-এর কাছে পাঠানো ফাইলসমূহ।

.dockerignore: Context থেকে বাদ দেওয়ার ফাইলের তালিকা।

Layer Cache: আগের Build-এর স্তর পুনরায় ব্যবহার করে Build দ্রুত করার ব্যবস্থা।

Cache Invalidation: কোনো ধাপের ইনপুট বদলে যাওয়ায় তার Cache আর ব্যবহার করা যায় না — এমন অবস্থা।

Content-Addressable: বিষয়বস্তুর Hash দিয়ে চেনা যায় — এমন সংরক্ষণ পদ্ধতি।

BuildKit: Docker-এর আধুনিক Build Engine — সমান্তরাল Build, Secret/Cache Mount দেয়।

Cache Mount: Build-গুলোর মধ্যে Cache ডিরেক্টরি ভাগ করা — Image-এ ঢোকে না।

Secret Mount: Build-সময়ে Secret সাময়িকভাবে মাউন্ট করা — Image-এ থাকে না।

৯.১৭ জ্ঞান যাচাই — প্রশ্ন

1. Build Context বড় হলে Build কেন ধীর হয়? `.dockerignore` কীভাবে সাহায্য করে?
2. একটি ধাপের Cache ভাঙলে পরের ধাপগুলোর কী হয়, এবং কেন?
3. পরের স্তরে ফাইল মুছলে Image ছোট হয় না কেন?
4. Cache Mount ও Secret Mount-এর পার্থক্য কী? কোনটি Image-এ ঢোকে?
5. ARG-এ Secret দেওয়া কেন বিপজ্জনক? নিরাপদ বিকল্প কী?

৯.১৮ ব্যবহারিক অনুশীলন

1. **Context পরিমাপ:** আপনার প্রজেক্টে `.dockerignore` যোগ করার আগে ও পরে Build-এর "Sending build context" আকার তুলনা করুন।
2. **Cache পরীক্ষা:** অধ্যায় ৯.৯-এর ধাপগুলো অনুসরণ করে কোন পরিবর্তনে কোন ধাপ থেকে Cache ভাঙে, তা লিখুন।
3. **Cache Mount:** BuildKit Cache Mount যোগ করে প্রথম ও দ্বিতীয় Build-এর সময় তুলনা করুন।
4. **Secret Mount:** একটি কাল্পনিক টোকেন Secret Mount দিয়ে Build-এ ব্যবহার করুন; `docker history`-তে তা দেখা যায় কি না যাচাই করুন।

অধ্যায় 10 • PART II — Docker Foundations and Image Engineering

Chapter 10 — Multi-Stage Builds and Production Image Design

১০.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: (১) Multi-Stage Build কী এবং কেন তা Production Image-এর মানদণ্ড, তা ব্যাখ্যা করতে; (২) Build Stage ও Runtime Stage আলাদা করে Dockerfile লিখতে; (৩) `COPY --from` দিয়ে Stage-এর মধ্যে Artifact নিতে; (৪) Non-root Runtime User, Distroless ও ছোট বেস Image বেছে নিতে; (৫) Image-এর আকার পরিমাপ করে অপ্টিমাইজ করতে; (৬) Dev/Staging/Production-এর জন্য আলাদা Image ডিজাইনের সিদ্ধান্ত নিতে।

১০.২ কেন এই বিষয়টি DevOps-এ গুরুত্বপূর্ণ

একটি Single-Stage Image-এ Build-এর সব টুল (TypeScript কম্পাইলার, পরীক্ষার লাইব্রেরি, Git) Production-এও চলে যায় — Image বড় হয়, আক্রমণের পৃষ্ঠ (Attack Surface) বাড়ে, Deploy ধীর হয়। Multi-Stage Build এই সমস্যার সমাধান: প্রথম Stage-এ Build করুন, দ্বিতীয় Stage-এ শুধু চালানোর জন্য যা দরকার তাই নিন। ফল: 1 GB-র Image 150 MB-এ নামে, Production-এ কোনো কম্পাইলার থাকে না, আর আক্রমণকারী ভাঙলেও হাতিয়ার পায় না। Production Image ডিজাইন করা মানেই এই পার্থক্যটি বোঝা।

১০.৩ পূর্বশর্ত জ্ঞান

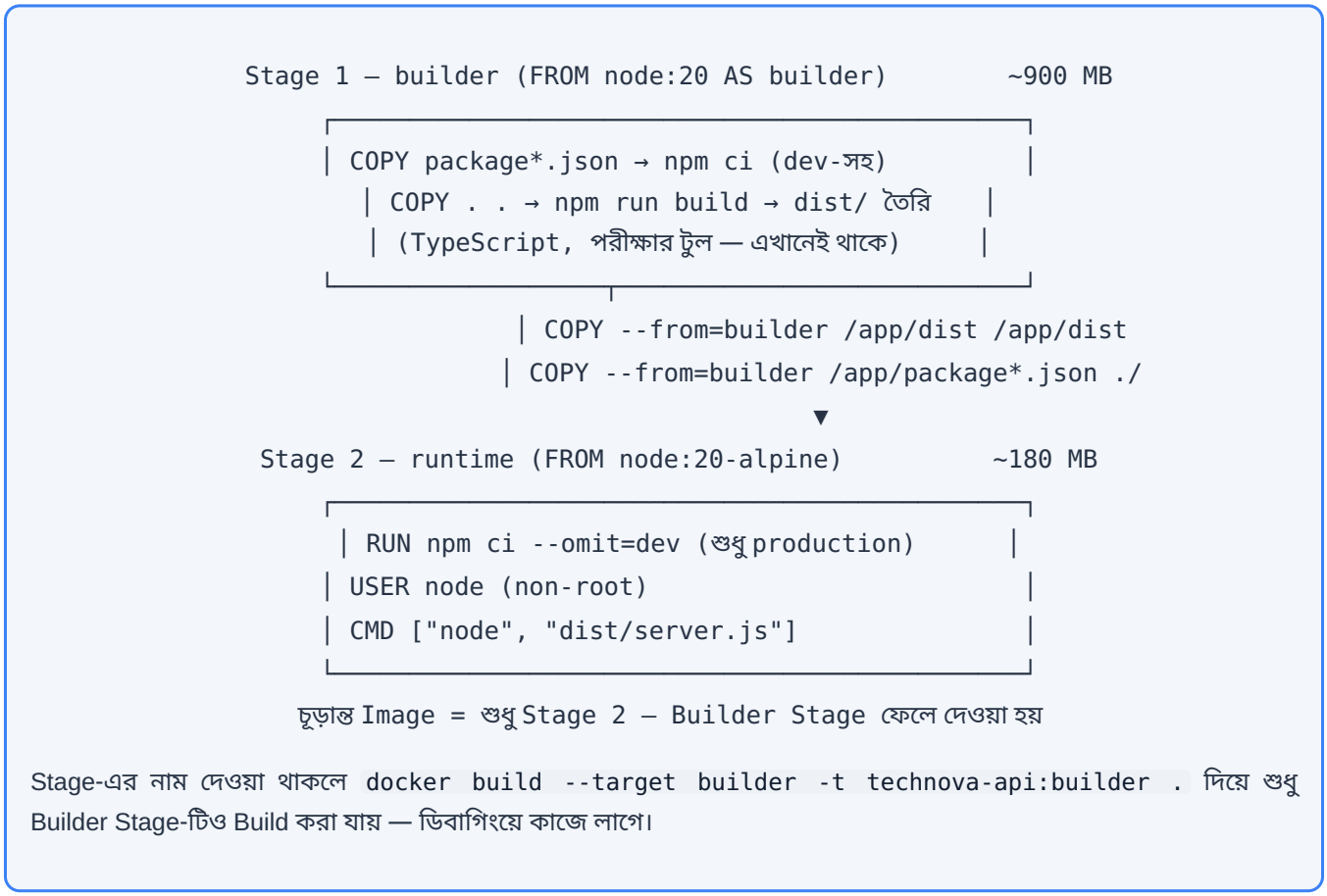
অধ্যায় ৮–৯ — Dockerfile নির্দেশনা, Layer, Cache, `.dockerignore`। TypeScript/Next.js Build-এর ধারণা (`npm run build`), `node_modules`-এ dev বনাম production নির্ভরতার পার্থক্য, এবং Linux ব্যবহারকারী ও অনুমতির ধারণা জানা থাকতে হবে।

১০.৪ প্রথম নীতি থেকে ব্যাখ্যা

একটি অ্যাপের জীবনে দুটি আলাদা প্রয়োজন থাকে: **Build-সময়ে** দরকার কম্পাইলার, পরীক্ষার টুল, সোর্স কোড; **Runtime-এ** দরকার শুধু চালানোর ফল — কম্পাইল করা JavaScript, production নির্ভরতা, Node.js Runtime। Single-Stage Dockerfile-এ এই দুটি প্রয়োজন একই Image-এ মিশে যায়। Multi-Stage Build-এ Dockerfile-এ একাধিক `FROM` থাকে — প্রতিটি একটি **Stage** শুরু করে, যার নিজের নাম থাকে (`FROM node:20 AS builder`)। প্রতিটি Stage আলাদা Image হিসেবে তৈরি হয়, কিন্তু চূড়ান্ত Image হয় শুধু শেষ Stage-টি। মাঝের Stage-গুলো থেকে শুধু দরকারি ফাইল `COPY --from=builder /app/dist /app/dist` দিয়ে নেওয়া হয় — বাকি সব ফেলে দেওয়া হয়। এভাবে Build-এর টুল Production Image-এ পৌঁছায় না।

> **CONCEPT:** Multi-Stage Build হলো কারখানা আর দোকানের পার্থক্য: কারখানায় (Builder Stage) যন্ত্রপাতি, কাঁচামাল, শ্রমিক থাকে; দোকানে (Runtime Stage) শুধু তৈরি পণ্যটি যায়। কেউ দোকানে কারখানার যন্ত্রপাতি রাখে না — Production Image-ও তাই।

১০.৫ আর্কিটেকচার / ধারণাগত চিত্র



Stage-এর নাম দেওয়া থাকলে `docker build --target builder -t technova-api:builder .` দিয়ে শুধু Builder Stage-টিও Build করা যায় — ডিবাগিংয়ে কাজে লাগে।

১০.৬ বিস্তারিত প্রযুক্তিগত ব্যাখ্যা

Multi-Stage Dockerfile — সম্পূর্ণ উদাহরণ

```
# syntax=docker/dockerfile:1
# Stage 1 — builder: TypeScript কম্পাইল ও Build
FROM node:20 AS builder
WORKDIR /app
COPY package.json package-lock.json ./
RUN --mount=type=cache,target=/root/.npm npm ci
COPY . .
RUN npm run build
# এই Stage-এ dist/ তৈরি হয়; dev নির্ভরতা ও সোর্স কোড এখানেই থাকে

# Stage 2 — runtime: শুধু চালানোর জন্য যা দরকার
FROM node:20-alpine AS runtime
WORKDIR /app
COPY package.json package-lock.json ./
RUN --mount=type=cache,target=/root/.npm npm ci --omit=dev
COPY --from=builder /app/dist ./dist
# --from=builder = Builder Stage থেকে শুধু dist/ নাও; সোর্স কোড নেওয়া হয় না
COPY --chown=node:node --from=builder /app/dist ./dist
# (ওপরের লাইনটির বদলে --chown দিয়ে নিলে ফাইলের মালিক node হয় — অনুমতি সমস্যা এড়াতে)
ENV NODE_ENV=production
ENV PORT=3000
USER node
```

```
EXPOSE 3000
HEALTHCHECK --interval=30s --timeout=5s --start-period=10s --retries=3 \
  CMD node -e "fetch('http://localhost:3000/health').then(r => process.exit(r.ok ? 0 :
1)).catch(() => process.exit(1))"
CMD ["node", "dist/server.js"]
```

লক্ষ্য করুন: Runtime Stage-এ `npm ci --omit=dev` আবার চালানো হয়েছে — কারণ Builder-এর `node_modules` নিলে dev নির্ভরতাও চলে আসত। বিকল্প: `COPY --from=builder /app/node_modules ./node_modules` — তবে তখন Builder-এ `npm ci --omit=dev` করে নিতে হয়, অথবা `npm prune --omit=dev` চালাতে হয়।

বেস Image নির্বাচন — আকার ও নিরাপত্তার ভারসাম্য

বেস Image	আকার (প্রায়)	বৈশিষ্ট্য	কখন ব্যবহার করবেন
`node:20` (Debian)	~350 MB	পূর্ণ Linux, shell ও টুলসহ	ডিবাগিং সহজ চাইলে; Development
`node:20-slim`	~200 MB	Debian slim — কম টুল	সাধারণ Production পছন্দ
`node:20-alpine`	~180 MB	Alpine Linux (musl libc) — খুব ছোট	ছোট Image চাইলে; তবে কিছু Native মডিউল (যেমন bcrypt-এর পুরনো সংস্করণ) musl-এ সমস্যা করতে পারে
Distroless (`gcr.io/distroless/nodejs20`)	~120 MB	শুধু Runtime — কোনো shell, প্যাকেজ ম্যানেজার নেই	সর্বোচ্চ নিরাপত্তা; তবে `exec` দিয়ে ডিবাগ করা যায় না

নিয়ম: Development-এ `node:20` (ডিবাগিং সহজ), Production-এ `node:20-alpine` বা `slim`; Distroless শুধু তখনই, যখন দল Distroless-এর সীমাবদ্ধতা (shell নেই, ডিবাগ কঠিন) মেনে নিতে প্রস্তুত। Alpine-এ Native মডিউল সমস্যা হলে `slim`-এ ফিরে যান — আকারের চেয়ে সঠিকভাবে চলা গুরুত্বপূর্ণ।

Non-Root Runtime User — Production-এর মানদণ্ড

Container root হিসেবে চললে আক্রমণকারী Container ভেঙে Host-এর root হওয়ার পথ পায়। নিয়ম: Runtime Stage-এ `USER node` লিখুন; ফাইলগুলো `COPY --chown=node:node` দিয়ে node-এর মালিকানায় নিন; অ্যাপ যেন 1024-এর নিচের Port-এ শুনতে না চায় (1024-এর নিচের Port বাঁধতে root লাগে — তাই অ্যাপ 3000 Port-এ শুনুক, বাইরের 80/443 Nginx বা Load Balancer সামলাবে)। `docker run --user node` দিয়েও Runtime User বদলানো যায় — তবে Dockerfile-এ লেখাই নির্ভরযোগ্য।

Dev বনাম Staging বনাম Production Image

Development: Single-Stage বা Multi-Stage-এর Builder Target — সোর্স কোড মাউন্ট করে হট-রিলোড, ডিবাগ টুলসহ। **Staging:** Production-এর হুবহু একই Image — নইলে Staging-এর পরীক্ষার মূল্য নেই। **Production:** Multi-Stage, ছোট বেস, non-root, HEALTHCHECK, `--omit=dev`, নির্দিষ্ট ট্যাগ/Digest। তিন পরিবেশে Image একই হওয়া উচিত — শুধু কনফিগারেশন (Environment Variable) আলাদা।

১০.৭ সম্পূর্ণ কনফিগারেশন উদাহরণ — Build, যাচাই ও তুলনা

```
# Single-Stage (তুলনার জন্য) বনাম Multi-Stage Build
docker build -t technova-api:single -f Dockerfile.single .
docker build -t technova-api:multi -f Dockerfile .
docker image ls technova-api
# নমুনা আউটপুট (চিত্রণমূলক):
# technova-api   single   1.42GB
# technova-api   multi    189MB

docker history technova-api:multi --no-trunc | head -20
# history = প্রতিটি স্তরের আকার ও কমান্ড; বড় স্তরটি চিহ্নিত করুন
docker build --target builder -t technova-api:builder .
# --target builder = শুধু Builder Stage বানাও (ডিবাগিংয়ের জন্য)
docker run --rm -it technova-api:builder sh -c "ls -la /app/dist && du -sh /app/node_modules"
docker run -d --name technova-api -p 3000:3000 technova-api:multi
docker exec technova-api whoami
# whoami = node (non-root প্রমাণ)
curl http://localhost:3000/health
docker ps --format "table {{.Names}}\t{{.Status}}"
# Status-এ (healthy) দেখা উচিত — HEALTHCHECK কাজ করছে
```

এই যাচাইগুলোই Production Image-এর গ্রহণযোগ্যতা পরীক্ষা (Acceptance Test): আকার 200 MB-এর নিচে, `whoami` non-root, `/health` সাড়া দেয়, আর `docker history`-তে কোনো Build টুলের স্তর নেই।

১০.১৮ বাস্তব প্রকৌশল পরিস্থিতি

REAL WORLD — TechNova: TechNova-র Next.js অ্যাপের Single-Stage Image ছিল 1.4 GB — TypeScript, ESLint, পরীক্ষার লাইব্রেরিসহ। প্রতিটি Deploy-এ Staging সার্ভারে (203.0.113.10) Image ডাউনলোডে ৪ মিনিট লাগত, আর নিরাপত্তা স্ক্যানে ৩০০+ পরিচিত দুর্বলতা ধরা পড়ত — বেশিরভাগই Build টুলের। Multi-Stage-এ বদলে Image 190 MB হলো, Deploy ৪০ সেকেন্ডে নামল, দুর্বলতা ৩০-এর নিচে এলো। আর `USER node` যোগ করার পর একটি ভুল কনফিগারেশনে অ্যাপ Host ফাইলে লিখতে চেয়ে ব্যর্থ হলো — যা আসলে নিরাপত্তার প্রমাণ: non-root Container Host-এ লিখতে পারে না।

১০.১৮ ধাপে ধাপে ব্যবহারিক উদাহরণ

উদাহরণ: Single-Stage থেকে Multi-Stage-এ রূপান্তর

1. আপনার TypeScript/Express অ্যাপের Single-Stage Dockerfile দিয়ে Image বানান; `docker image ls` দিয়ে আকার লিখুন।
2. ওপরের Multi-Stage Dockerfile লিখে আবার বানান; আকারের পার্থক্য লিখুন।
3. `docker history <image>` দিয়ে দুটি Image-এর স্তর তুলনা করুন — কোন স্তরগুলো হারিয়ে গেল?
4. `docker run --rm -it <image> sh` চালিয়ে দেখুন Runtime Image-এ shell আছে কি না (alpine-এ আছে, Distroless-এ নেই)।
5. `docker exec` দিয়ে `whoami` যাচাই করুন — `node` দেখা উচিত।

১০.১৮ সাধারণ ভুল ও ভ্রান্ত ধারণা

- **ভুল:** Builder Stage থেকে `node_modules` পুরোটা কপি করা — dev নির্ভরতাও Production-এ চলে আসে।
- **ভুল:** Runtime Stage-এ আবার সোর্স কোড কপি করা — `dist/` ছাড়া আর কিছু দরকার নেই।
- **ভুল ধারণা:** "Multi-Stage মানেই Image ছোট।" — না; ভুলভাবে লিখলে (যেমন Runtime-এ আবার সব কপি করলে) আকার একই থাকে।
- **ভুল:** `USER node` লিখে `COPY`-এর ফাইলের মালিক root রাখা — অ্যাপ ফাইল পড়তে/লিখতে পারে না; `--chown=node:node` ব্যবহার করুন।
- **ভুল:** Distroless বেছে নিয়ে ডিবাগিংয়ের উপায় না রাখা — Production সমস্যায় `exec` করা যায় না; ডিবাগ Image আলাদা রাখুন।

১০.১৮ নিরাপত্তা বিবেচনা

WARNING: Production Image-এ Build টুল, Git ইতিহাস, `.env` ফাইল বা পরীক্ষার ডেটা রাখবেন না — প্রতিটি অতিরিক্ত ফাইল আক্রমণের পৃষ্ঠ বাড়ায়। Image স্ক্যান করুন (`docker scout` বা Trivy — Volume 09-এ বিস্তারিত); বেস Image নিয়মিত আপডেট করুন। Non-root User বাধ্যতামূলক করুন; `--privileged` ও `--cap-add` Production-এ এড়িয়ে চলুন।

১০.১৮ কর্মক্ষমতা ও সম্পদ বিবেচনা

Image-এর আকার Deploy-এর গতি, Registry-র খরচ ও ডিস্ক ব্যবহার — তিনটিই নির্ধারণ করে। লক্ষ্য: Node.js API-র Production Image 200 MB-এর নিচে। `docker image ls` ও `docker history` দিয়ে নিয়মিত পরিমাপ করুন। Build-এর সময়ও খেয়াল রাখুন: Builder Stage-এর Cache CI-তে ভাগ করুন, যাতে প্রতিটি Build শূন্য থেকে শুরু না হয়।

১০.১৮ সমস্যা সমাধান (Troubleshooting)

TROUBLESHOOTING: Multi-Stage Build-এ "file not found" হলে প্রথমে দেখুন `COPY --from`-এর পাথটি Builder Stage-এর ভেতরের পাথের সঙ্গে মিলছে কি না — Builder-এ `WORKDIR /app` থাকলে `dist/` আসলে `/app/dist`। ডিবাগ করতে `docker build --target builder -t debug:builder .` দিয়ে শুধু Builder Stage বানিয়ে `docker run --rm -it debug:builder sh` দিয়ে ভেতরে দেখুন।

সমস্যা	সম্ভাব্য কারণ	সমাধান
`Cannot find module '...'` Runtime-এ	Production নির্ভরতা Runtime Stage-এ ইনস্টল হয়নি	Runtime Stage-এ `npm ci --omit=dev` আছে কি না দেখুন
Image আশানুরূপ ছোট হয়নি	Builder-এর ফাইল Runtime-এ কপি হয়েছে	`docker history` দিয়ে বড় স্তর খুঁজুন
`permission denied` Runtime-এ	ফাইলের মালিক root, User node	`COPY --chown=node:node` ব্যবহার করুন

Alpine-এ Native মডিউল ব্যর্থ	musl libc-র সঙ্গে অসামঞ্জস্য	`node:20-slim` ব্যবহার করুন

১০.১৮ সেরা অনুশীলন ও অ্যান্টি-প্যাটার্ন

সেরা অনুশীলন: Build ও Runtime Stage আলাদা করুন; শুধু Artifact কপি করুন; ছোট বেস Image; non-root User; HEALTHCHECK; নির্দিষ্ট ট্যাগ। **অ্যান্টি-প্যাটার্ন:** Single-Stage Production Image; Build টুল Production-এ রাখা; root হিসেবে চালানো; `latest` বেস; Builder-এর `node_modules` পুরোটা কপি করা।

১০.১৮ অধ্যায়ের সারাংশ

Multi-Stage Build-এ Builder Stage-এ Build হয়, Runtime Stage-এ শুধু চালানোর ফল যায় — Image ছোট, নিরাপদ ও দ্রুত Deploy হয়। `COPY --from` দিয়ে Stage-এর মধ্যে Artifact নিন; Runtime Stage-এ ছোট বেস Image, `--omit=dev`, non-root `USER node` ও HEALTHCHECK ব্যবহার করুন। Development, Staging ও Production-এ একই Image, শুধু কনফিগারেশন আলাদা — এটাই Production Image ডিজাইনের মূল নীতি।

১০.১৮ মূল প্রযুক্তিগত পরিভাষা

Multi-Stage Build: একাধিক `FROM` Stage-সহ Dockerfile — Build ও Runtime আলাদা করা যায়।

Builder Stage: কম্পাইল ও Build করার Stage — চূড়ান্ত Image-এ থাকে না।

Runtime Stage: চূড়ান্ত Stage — শুধু অ্যাপ চালানোর জন্য যা দরকার, তাই থাকে।

COPY --from: এক Stage থেকে অন্য Stage-এ ফাইল কপি করা।

Distroless Image: শুধু Runtime-সহ, shell ছাড়া অতি-ছোট বেস Image।

Attack Surface: আক্রমণকারী যে পথগুলো দিয়ে ঢুকতে পারে — তার মোট পরিমাণ।

musl libc: Alpine Linux-এর হালকা C লাইব্রেরি — কিছু Native মডিউলের সঙ্গে অসামঞ্জস্য হতে পারে।

১০.১৮ জ্ঞান যাচাই — প্রশ্ন

1. Multi-Stage Build কেন Single-Stage-এর চেয়ে নিরাপদ ও ছোট Image দেয়?
2. `COPY --from=builder` কী করে? Builder Stage-এর বাকি ফাইলগুলোর কী হয়?
3. Runtime Stage-এ `npm ci --omit=dev` আবার চালানো কেন দরকার হতে পারে?
4. Alpine বনাম slim বনাম Distroless — কখন কোনটি বেছে নেবেন?

5. Non-root User ব্যবহার না করলে কী ঝুঁকি থাকে?

১০.১৮ ব্যবহারিক অনুশীলন

1. **রূপান্তর:** আপনার অ্যাপের Single-Stage Dockerfile-কে Multi-Stage-এ বদলে আকারের পার্থক্য পরিমাপ করুন।
2. **Builder ডিবাগ:** `--target builder` দিয়ে শুধু Builder Stage বানিয়ে তার ভেতরে `dist/` যাচাই করুন।
3. **বেস তুলনা:** একই অ্যাপ alpine, slim ও Distroless বেসে বানিয়ে আকার ও চালু হওয়ার সময় তুলনা করুন।

অধ্যায় 11 • PART II — Docker Foundations and Image Engineering

Chapter 11 — Image Tagging, Registries and Image Distribution

১১.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: (১) Tag বনাম Digest-এর পার্থক্য ব্যাখ্যা করতে; (২) Semantic Versioning অনুযায়ী Tag দেওয়ার কৌশল বলতে; (৩) Docker Hub-এ Image Push/Pull করতে; (৪) প্রাইভেট Registry (Docker Registry, GitHub Container Registry) চালাতে ও ব্যবহার করতে; (৫) Image Distribution, Retention ও পরিষ্কারের নীতি নির্ধারণ করতে; (৬) CI থেকে স্বয়ংক্রিয় Push-এর প্রবাহ বর্ণনা করতে।

১১.২ কেন এই বিষয়টি DevOps-এ গুরুত্বপূর্ণ

Image বানানো অর্ধেক কাজ; সেই Image-কে নির্ভরযোগ্যভাবে Staging ও Production-এ পৌঁছে দেওয়া বাকি অর্ধেক। Tag ভুল হলে Production-এ ভুল সংস্করণ চলে — Volume 05-এর "কোন সংস্করণ চলছে জানি না" সমস্যাটি ফিরে আসে। Registry ছাড়া Image শুধু একটি মেশিনে থাকে — অন্য সার্ভারে পৌঁছায় না। Tagging, Registry ও Distribution বোঝা মানেই ডেলিভারি প্রবাহের শেষ অংশটি সম্পূর্ণ করা।

১১.৩ পূর্বশর্ত জ্ঞান

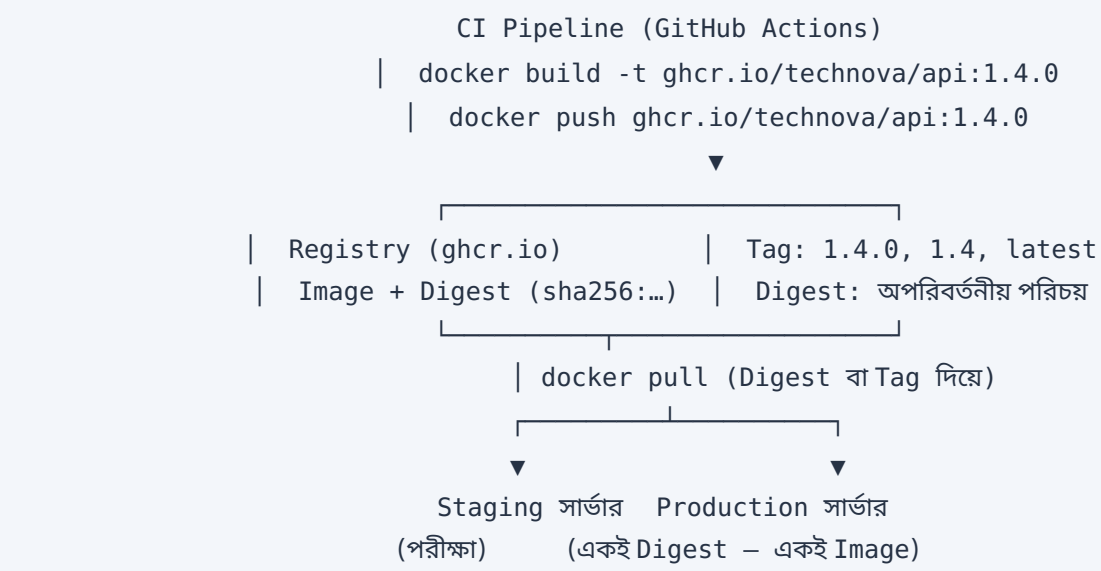
অধ্যায় ৭–১০ — Image, Tag, `docker pull/push`, Dockerfile ও Multi-Stage Build। Semantic Versioning-এর ধারণা (Volume 05), Git-এ Commit Hash, এবং CI Pipeline-এর প্রাথমিক ধারণা জানা থাকতে হবে।

১১.৪ প্রথম নীতি থেকে ব্যাখ্যা

Image-এর পরিচয় দুটি জিনিসে: **Tag** হলো মানুষের পড়ার মতো নাম — যেমন `technova-api:1.4.0`; **Digest** হলো Image-এর বিষয়বস্তুর SHA-256 Hash — যেমন `sha256:9f86d081...`। Tag বদলানো যায় — একই Tag (`latest`) কাল অন্য Image-কে নির্দেশ করতে পারে; Digest বদলানো যায় না — একই Digest সবসময় একই Image-কে নির্দেশ করে। তাই Tag সুবিধাজনক, Digest নির্ভরযোগ্য। **Registry** হলো Image-এর ভান্ডার: `docker push` Image Registry-তে তোলে, `docker pull` Registry থেকে নামায়। Docker Hub হলো পাবলিক Registry; প্রাইভেট Registry হলো নিজের বা ক্লাউডের ভান্ডার — যেমন Docker-এর অফিসিয়াল Registry Image, GitHub Container Registry (ghcr.io), AWS ECR, Google Artifact Registry। Image-এর পুরো নাম: `registry/namespace/name:tag` — যেমন `ghcr.io/technova/api:1.4.0`। Registry না লিখলে Docker Hub (`docker.io`) ধরা হয়।

> **CONCEPT:** Tag হলো একটি বইয়ের শিরোনাম, Digest হলো তার বিষয়বস্তুর আঙুলের ছাপ। শিরোনাম বদলে যেতে পারে, আঙুলের ছাপ কখনো বদলায় না। Production-এ Digest দিয়ে Deploy করলে আপনি নিশ্চিত থাকেন — Staging-এ পরীক্ষিত Image-টিই Production-এ চলছে।

১১.৫ আর্কিটেকচার / ধারণাগত চিত্র



একই Image, একই Digest — Staging ও Production-এ হুবহু একই Artifact। এটাই Volume 05-এর প্রতিশ্রুতি।

১১.৬ বিস্তারিত প্রযুক্তিগত ব্যাখ্যা

Tagging কৌশল — Semantic Versioning

Tag দেওয়ার নিয়ম: `1.4.0` (Major.Minor.Patch — Semantic Versioning), সঙ্গে `1.4` ও `latest` Tag-ও দিতে পারেন — তবে Production Deploy-এ `latest` ব্যবহার করবেন না। CI-তে প্রতিটি Build-এ Git Commit Hash-এর Tag দিন — যেমন `technova-api:1.4.0-a1b2c3d` — যাতে কোন Commit থেকে Image তৈরি হয়েছে, তা জানা যায়। `docker tag technova-api:1.4.0 ghcr.io/technova/api:1.4.0` — একটি Image-কে আরেকটি নাম (Registry-সহ) দেয়; Image কপি হয় না, শুধু নাম যোগ হয়।

Digest দিয়ে Deploy — অপরিবর্তনীয়তার গ্যারান্টি

`docker pull technova-api@sha256:9f86d081884c7d659a2feaa0c55ad015a3bf4f1b2b0b822cd15d6c15b0f00a08` — Digest দিয়ে Image টানা। Production-এ Digest ব্যবহার করলে Registry-তে Tag বদলে গেলেও ভুল Image Deploy হয় না। CI-তে Build-এর পর Digest সংরক্ষণ করুন (`docker buildx build --push` Digest দেখায়) এবং Deploy-এর সময় সেই Digest ব্যবহার করুন।

Docker Hub — পাবলিক Registry

`docker login` — Docker Hub-এ লগইন (Credential `~/.docker/config.json`-এ সংরক্ষিত হয় — ফাইলের অনুমতি সুরক্ষিত রাখুন)। পাবলিক Repository-তে Image দিলে যে কেউ তা টানতে পারে — নিজের অ্যাপের Image কখনো পাবলিক করবেন না; প্রাইভেট Repository ব্যবহার করুন। Docker Hub-এর ফ্রি অ্যাকাউন্টে Pull-এর হার-সীমা (Rate Limit) আছে — CI-তে হঠাৎ Pull ব্যর্থ হলে এই সীমাটি সন্দেহ করুন।

প্রাইভেট Registry — নিজের ভান্ডার

```
# Docker-এর অফিসিয়াল Registry Image দিয়ে প্রাইভেট Registry চালানো
docker run -d --name registry -p 127.0.0.1:5000:5000 \
  -v registry-data:/var/lib/registry \
  --restart unless-stopped registry:2
# registry:2 = Docker-এর অফিসিয়াল Registry Image
# -v registry-data:/var/lib/registry = Image ডেটা Volume-এ সংরক্ষণ (Volume 07)
# 127.0.0.1 = শুধু localhost থেকে পৌঁছানো যাবে — প্রমাণীকরণ ছাড়া ইন্টারনেটে খুলবেন না

# Registry-তে Image Push করা
docker tag technova-api:1.0 localhost:5000/technova-api:1.0
docker push localhost:5000/technova-api:1.0
docker pull localhost:5000/technova-api:1.0
# tag = Registry-এর নামসহ নতুন নাম দেওয়া; push = Registry-তে তোলা
# Production Registry-তে TLS ও প্রমাণীকরণ (htpasswd বা ক্লাউড IAM) বাধ্যতামূলক
```

GitHub Container Registry (ghcr.io) ব্যবহার করলে GitHub-এর Token দিয়ে লগইন করতে হয় — `echo $GHCR_TOKEN | docker login ghcr.io -u USERNAME --password-stdin`। Token-টি GitHub-এর Secret-এ রাখুন, কখনো কোডে নয়।

Distribution ও Retention — পুরনো Image কী করবেন

প্রতিটি Build নতুন Image তৈরি করে — Registry দ্রুত ভরে যায়। **Retention Policy** নির্ধারণ করুন: যেমন শেষ ২০টি সংস্করণ ও সব Release Tag রাখুন, বাকি মুছে ফেলুন। Docker Hub ও ghcr.io-তে Retention নীতি সেট করা যায়; নিজের Registry-তে নিয়মিত `registry garbage-collect` চালাতে হয়। Multi-architecture Image (ARM + x86) Build করতে Buildx ব্যবহার করুন — `docker buildx build --platform linux/amd64,linux/arm64 --push -t ...` — তবে CI-তে সময় বাড়ে।

১১.৭ Docker কমান্ড — লাইন বাই লাইন সম্পূর্ণ উদাহরণ

```
# --- Tag দেওয়া ও Push করা ---
docker build -t technova-api:1.4.0 -t technova-api:latest .
# এক Build-এ দুটি Tag: সংস্করণ Tag + latest
docker tag technova-api:1.4.0 ghcr.io/technova/api:1.4.0
docker tag technova-api:1.4.0 ghcr.io/technova/api:1.4
# tag = একই Image-কে Registry-এর নামসহ নতুন নাম দেওয়া

echo "CHANGE-ME-GHCR-TOKEN" | docker login ghcr.io -u nahid-hasan --password-stdin
# --password-stdin = পাসওয়ার্ড কমান্ড লাইনে নয়, stdin দিয়ে — history-তে থাকে না
# (আসল Token কখনো উদাহরণে লিখবেন না — এটি কাল্পনিক প্লেসহোল্ডার)
docker push ghcr.io/technova/api:1.4.0
docker push ghcr.io/technova/api:1.4
# push = Registry-তে Image তোলা; প্রথমবার স্তরগুলো আপলোড হয়, পরে শুধু নতুন স্তর

# --- Digest যাচাই ও Digest দিয়ে Deploy ---
docker buildx imagetools inspect ghcr.io/technova/api:1.4.0
# imagetools inspect = Registry-তে Image-এর Digest ও Manifest দেখায়
docker pull ghcr.io/technova/
api@sha256:9f86d081884c7d659a2feaa0c55ad015a3bf4f1b2b0b822cd15d6c15b0f00a08
# Digest দিয়ে Pull — Tag বদলে গেলেও এই Image-টিই আসবে
```

```
# --- CI-তে স্বয়ংক্রিয় Build ও Push (ধারণাগত GitHub Actions ধাপ) ---
# - uses: docker/login-action@v3 (Registry-তে লগইন — Secret থেকে Token নেয়)
# - uses: docker/build-push-action@v6 (Build করে Push করে, Digest আউটপুট দেয়)
# Deploy ধাপে সেই Digest ব্যবহার করা হয় — Tag নয়
```

১১.৮ বাস্তব প্রকৌশল পরিস্থিতি

REAL WORLD — TechNova: TechNova একবার Production-এ `technova-api:latest` Deploy করেছিল। একদিন CI-তে একটি ভাঙা Build-ও `latest` Tag পেয়ে Registry-তে উঠে গেল — পরের Deploy-এ ভাঙা Image Production-এ চলে গেল, যদিও কেউ ইচ্ছাকৃতভাবে Deploy করেনি। এরপর দল নিয়ম করল: Production-এ শুধু সংস্করণ Tag (`1.4.0`) বা Digest; `latest` শুধু Development-এ। আর প্রতিটি Release-এর Digest Deploy লগে লেখা হয়, যাতে কোনো সমস্যা হলে ঠিক কোন Image চলছে, তা জানা যায়।

১১.৯ ধাপে ধাপে ব্যবহারিক উদাহরণ

উদাহরণ: নিজের প্রাইভেট Registry চালানো

1. ওপরের কমান্ডে `localhost:5000`-এ Registry চালান।
2. আপনার API Image-কে `localhost:5000/technova-api:1.0` নামে Tag করে Push করুন।
3. স্থানীয় Image মুছে (`docker image rm`) Registry থেকে আবার Pull করে চালান — Distribution-এর পুরো চক্রটি দেখুন।
4. `docker buildx imagetools inspect localhost:5000/technova-api:1.0` দিয়ে Digest লিখে রাখুন; Digest দিয়ে Pull করে দেখুন।
5. একই Image-কে `1.0`, `1` ও `latest` — তিনটি Tag দিয়ে Push করুন; `docker image ls`-এ Image ID একই দেখা উচিত।

১১.১০ সাধারণ ভুল ও ভ্রান্ত ধারণা

- **ভুল:** Production-এ `latest` Tag ব্যবহার করা — কোন সংস্করণ চলছে, তা জানা যায় না।
- **ভুল ধারণা:** "Tag বদলানো যায় না।" — Tag যেকোনো সময় অন্য Image-কে নির্দেশ করতে পারে; Digest-ই অপরিবর্তনীয়।
- **ভুল:** নিজের অ্যাপের Image পাবলিক Registry-তে তোলা — Secret বা ব্যবসায়িক কোড ফাঁস হতে পারে।
- **ভুল:** প্রমাণীকরণ ছাড়া Registry ইন্টারনেটে খোলা — আক্রমণকারীরা Image চুরি করতে বা ম্যালওয়্যার-সহ Image তুলতে পারে।
- **ভুল:** Retention নীতি না থাকা — Registry ভরে যায়, খরচ বাড়ে, পুরনো দুর্বল Image থেকে যায়।
- **ভুল:** Token কমান্ড লাইনে লেখা (`docker login -p SECRET`) — history-তে থাকে; `--password-stdin` ব্যবহার করুন।

১১.১১ নিরাপত্তা বিবেচনা

WARNING: Registry-র Credential ফাঁস হলে আক্রমণকারী আপনার Image-এ ম্যালওয়্যার ঢুকিয়ে দিতে পারে — আর সেই Image আপনার Production-এ চলবে। Registry-তে সবসময় TLS ও প্রমাণীকরণ ব্যবহার করুন; Token-এর মেয়াদ ও অনুমতি সীমিত রাখুন; Image-এ স্বাক্ষর (Docker Content Trust / Cosign) বিবেচনা করুন; আর Image নিয়মিত স্ক্যান করুন। প্রাইভেট Image কখনো পাবলিক করবেন না।

১১.১২ কর্মক্ষমতা ও সম্পদ বিবেচনা

Image Pull-এর গতি Deploy-এর গতি নির্ধারণ করে — ছোট Image (অধ্যায় ১০) Registry থেকে দ্রুত নামে। Registry-র কাছাকাছি সার্ভার বেছে নিন (যেমন সার্ভার AWS-এ হলে ECR ব্যবহার করুন)। Layer ভাগ করা হয় বলে একই বেস Image-এর অনেক সংস্করণ Pull করলে শুধু নতুন স্তরগুলোই নামে — এজন্যই বেস Image স্থিতিশীল রাখা লাভজনক। Retention নীতি দিয়ে Registry-র Storage খরচ নিয়ন্ত্রণে রাখুন।

১১.১৩ সমস্যা সমাধান (Troubleshooting)

TROUBLESHOOTING: `denied: requested access to the resource is denied` — Push-এর সময় সবচেয়ে সাধারণ ত্রুটি। কারণ: লগইন করা হয়নি (`docker login` করুন), Token-এর অনুমতি নেই, অথবা Image-এর নামে Registry/Namespace ভুল। `unauthorized: authentication required` হলে Token-এর মেয়াদ বা অনুমতি যাচাই করুন। `toomanyrequests` হলে Docker Hub-এর Rate Limit — লগইন করুন বা প্রাইভেট Registry ব্যবহার করুন।

সমস্যা	সম্ভাব্য কারণ	সমাধান
`denied: requested access`	লগইন নেই / অনুমতি নেই / নাম ভুল	`docker login` ; নাম ও Namespace যাচাই করুন
`manifest unknown`	Tag/Digest Registry-তে নেই	Tag-এর বানান ও Registry যাচাই করুন
Pull হঠাৎ ধীর	Registry দূরে বা Rate Limit	কাছের Registry ব্যবহার করুন; লগইন করুন
Registry ডিস্ক ভরে গেছে	পুরনো Image জমে আছে	Retention নীতি ও Garbage Collection চালান

১১.১৪ সেরা অনুশীলন ও অ্যান্টি-প্যাটার্ন

সেরা অনুশীলন: Semantic Versioning Tag + Commit Hash Tag; Production Deploy-এ Digest; প্রাইভেট Registry-তে TLS ও প্রমাণীকরণ; Retention নীতি; CI থেকে স্বয়ংক্রিয় Push; Token `--password-stdin` দিয়ে দেওয়া। **অ্যান্টি-প্যাটার্ন:** Production-এ `latest` ; পাবলিক Registry-তে প্রাইভেট Image; প্রমাণীকরণ ছাড়া Registry; Retention নীতি না থাকা; Token কমান্ড লাইনে লেখা।

১১.১৫ অধ্যায়ের সারাংশ

Tag মানুষের পড়ার নাম, Digest অপরিবর্তনীয় পরিচয় — Production-এ Digest বা নির্দিষ্ট সংস্করণ Tag ব্যবহার করুন, `latest` নয়। Registry Image-এর ভান্ডার: Docker Hub পাবলিক, প্রাইভেট Registry নিজের বা ক্লাউডের। `docker tag`, `push`, `pull` দিয়ে Image বিতরণ করুন; Retention নীতি দিয়ে Registry পরিষ্কার রাখুন; Registry-তে TLS ও প্রমাণীকরণ বাধ্যতামূলক করুন।

১১.১৬ মূল প্রযুক্তিগত পরিভাষা

Tag: Image-এর মানুষের পড়ার মতো সংস্করণ-নাম — যেমন `1.4.0`।

Digest: Image-এর বিষয়বস্তুর SHA-256 Hash — অপরিবর্তনীয় পরিচয়।

Registry: Image সংরক্ষণ ও বিতরণের ভান্ডার — যেমন Docker Hub, ghcr.io।

Semantic Versioning (SemVer): Major.Minor.Patch সংস্করণ পদ্ধতি — যেমন `1.4.0`।

Manifest: Image-এর স্তর ও কনফিগারেশনের তালিকা — Registry-তে সংরক্ষিত হয়।

Retention Policy: Registry-তে কতদিন বা কত সংস্করণ Image রাখা হবে — তার নীতি।

Rate Limit: Registry-তে নির্দিষ্ট সময়ে কতবার Pull করা যাবে — তার সীমা।

১১.১৭ জ্ঞান যাচাই — প্রশ্ন

1. Tag বনাম Digest — পার্থক্য কী, এবং Production-এ Digest কেন নিরাপদ?
2. `latest` Tag দিয়ে Production Deploy করা কেন বিপজ্জনক? একটি উদাহরণ দিন।
3. Image-এর পুরো নামের অংশগুলো (`registry/namespace/name:tag`) ব্যাখ্যা করুন।
4. প্রাইভেট Registry-তে TLS ও প্রমাণীকরণ ছাড়া Image রাখলে কী ঝুঁকি থাকে?
5. Retention Policy না থাকলে Registry-তে কী সমস্যা হয়?

১১.১৮ ব্যবহারিক অনুশীলন

1. **Registry চক্র:** অধ্যায় ১১.৯-এর ধাপগুলো অনুসরণ করে নিজের Registry-তে Push/Pull-এর পুরো চক্রটি করুন।
2. **Tag কৌশল:** আপনার অ্যাপের জন্য Semantic Versioning Tag + Commit Hash Tag-এর তালিকা লিখুন।
3. **Digest Deploy:** একটি Image-এর Digest বের করে Digest দিয়ে Pull করুন; Tag দিয়ে Pull-এর সঙ্গে তুলনা করুন।

PART III • DEVOPS ENGINEERING MASTERY

Container Networking and Storage

Docker নেটওয়ার্কিং — bridge, host, none, custom network — DNS ও service discovery, volume, bind mount ও ডেটা ব্যবস্থাপনা।

অধ্যায় 12 • PART III — Container Networking and Persistent Storage

Chapter 12 — Docker Networking Architecture

১২.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: (১) Container Network Model (CNM) — Docker-এর নেটওয়ার্কিংয়ের মূল কাঠামো — ব্যাখ্যা করতে; (২) Sandbox, Endpoint ও Network — এই তিনটি মৌলিক ধারণার পার্থক্য ও সম্পর্ক বোঝাতে; (৩) Network Driver কী এবং কেন একাধিক Driver দরকার তা বলতে; (৪) একটি Container-এর ভেতর থেকে বাইরের জগতের সঙ্গে যোগাযোগের পুরো পথটি আঁকতে; (৫) `docker network inspect` ও `docker network ls` দিয়ে একটি চলমান সিস্টেমের নেটওয়ার্ক মানচিত্র পড়তে; (৬) Container Port ও Published Host Port-এর পার্থক্য স্পষ্ট করতে।

১২.২ কেন এই বিষয়টি DevOps-এ গুরুত্বপূর্ণ

একটি Container একা চললে তার কোনো মূল্য নেই — মূল্য তৈরি হয় যখন সে অন্য Container, Database, বাইরের API ও ব্যবহারকারীর সঙ্গে কথা বলতে পারে। TechNova-র MERN অ্যাপে Next.js Frontend-কে Express API-র সঙ্গে, API-কে MongoDB ও PostgreSQL-এর সঙ্গে কথা বলতে হয়; কোনো এক জায়গায় নেটওয়ার্কিং ভুল হলে অ্যাপটি নিখুঁতভাবে Build হয়েও একটিও Request-এর উত্তর দিতে পারে না। প্রোডাকশনে সবচেয়ে বেশি সময় নষ্ট হয় এমন সমস্যায় যেখানে Container চলছে, Log-এ কোনো Error নেই, কিন্তু দুটি Container পরস্পরকে খুঁজে পাচ্ছে না — অথবা Database Port হোস্টে ভুলভাবে প্রকাশ (Expose) হয়ে গেছে বলে বাইরে থেকে আক্রমণের সুযোগ তৈরি হয়েছে। নেটওয়ার্ক আর্কিটেকচার বোঝা মানেই এই সমস্যাগুলোকে অনুমান নয়, মানচিত্র ধরে সমাধান করা।

১২.৩ পূর্বশর্ত জ্ঞান

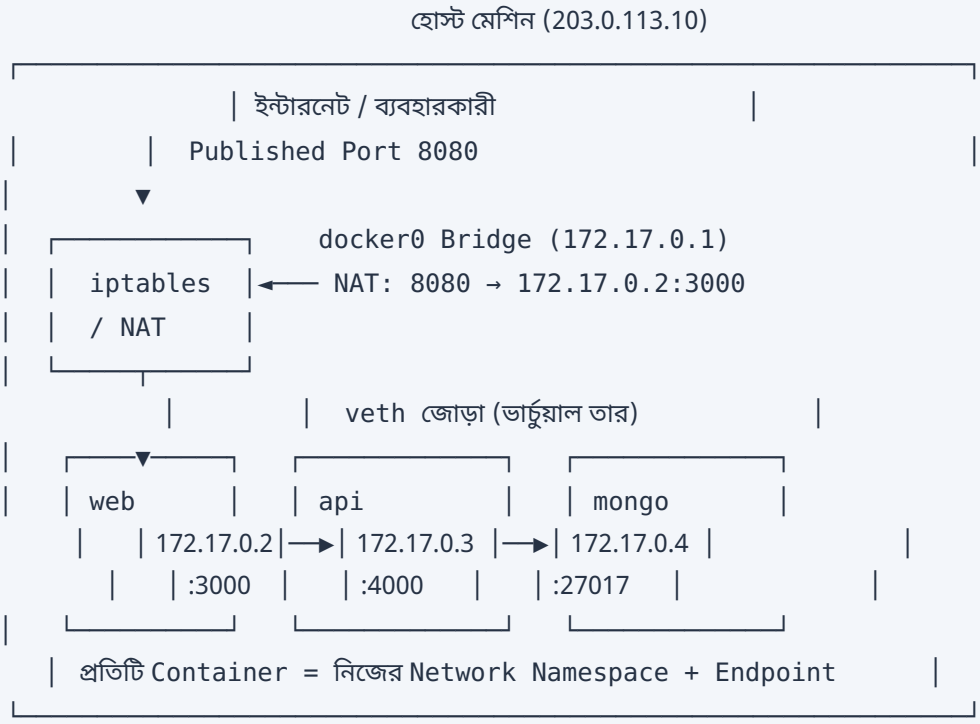
IP Address, Subnet, Port, DNS ও HTTP Request-এর প্রাথমিক ধারণা, Linux-এ `ip addr` ও `ping` চালানোর অভিজ্ঞতা, এবং Volume 06-এর আগের Part-গুলো থেকে Container তৈরি, Image চালানো ও `docker run`-এর মৌলিক ব্যবহার জানা থাকতে হবে। NAT (Network Address Translation), Bridge ও Firewall-এর ধারণা থাকলে ভালো হয়, তবে এই অধ্যায়ে প্রতিটি ধারণা প্রথম নীতি থেকে ব্যাখ্যা করা হবে।

১২.৪ প্রথম নীতি থেকে ব্যাখ্যা

একটি Container আসলে হোস্ট Linux কার্নেলের ওপর একটি বিচ্ছিন্ন (Isolated) প্রক্রিয়া-দল — তার নিজের Network Namespace আছে। **Network Namespace** মানে কার্নেলের একটি আলাদা নেটওয়ার্ক-জগৎ: নিজের Interface, নিজের IP Address, নিজের Routing Table। দুটি Container দুটি আলাদা Namespace-এ থাকলে তারা পরস্পরকে দেখতে পায় না, যতক্ষণ না কেউ তাদের মধ্যে একটি সেতু (Bridge) বানিয়ে দেয়। Docker সেই সেতু বানানোর কাজটিই করে — এবং সেই কাজের নকশাকে বলে **Container Network Model (CNM)**। CNM-এর তিনটি মৌলিক অংশ: **Sandbox** — একটি Container-এর নিজস্ব নেটওয়ার্ক পরিবেশ (তার Namespace, Interface, DNS সেটিং); **Endpoint** — Sandbox-কে একটি Network-এর সঙ্গে যুক্ত করার সংযোগ-বিন্দু (একটি ভার্চুয়াল তারের এক প্রান্ত); **Network** — Endpoint-গুলোকে পরস্পরের সঙ্গে যুক্ত করার যৌক্তিক গোষ্ঠী (যেমন একটি Bridge Network)। একটি Container একটি Network-এ যুক্ত হওয়া মানে তার Sandbox-এ একটি Endpoint তৈরি হওয়া। আর **Network Driver** হলো সেই যন্ত্রপাতি যা Network-টি আসলে কীভাবে কাজ করবে তা নির্ধারণ করে — Bridge Driver একটি ভার্চুয়াল সুইচ বানায়, Host Driver Container-কে হোস্টের নেটওয়ার্কই ব্যবহার করতে দেয়, None Driver কোনো নেটওয়ার্কই দেয় না। এই তিনটি ধারণা মনে রাখলে Docker-এর যেকোনো নেটওয়ার্কিং আচরণ ব্যাখ্যা করা যায়।

CONCEPT: Container-এর নেটওয়ার্ককে একটি অ্যাপার্টমেন্ট ভবন ভাবুন। প্রতিটি ফ্ল্যাট (Container) নিজের দরজা ও ঠিকানা (IP) পায়; ভবনের করিডর ও লিফট (Bridge Network) ফ্ল্যাটগুলোকে পরস্পরের সঙ্গে যুক্ত করে; মূল ফটক (Published Port) দিয়ে শুধু অনুমোদিত অতিথিরা ভেতরে আসতে পারে। কেউ যদি ফটক খুলে সব দরজা রাস্তার দিকে খুলে দেয়, তবে গোপনীয়তা থাকে না — ঠিক তেমনি ভুল Port Publishing পুরো Database-কে ইন্টারনেটে উন্মুক্ত করে দেয়।

১২.৫ আর্কিটেকচার / ধারণাগত চিত্র



Container থেকে বাইরে: Container → Bridge → NAT (MASQUERADE) → হোস্ট eth0 → ইন্টারনেট
বাইরে থেকে ভেতরে: শুধু Published Port (-p) দিয়ে; না ছাপালে ভেতরে আসা যায় না।

Bridge হলো হোস্টের ভেতরে একটি ভার্চুয়াল সুইচ; veth জোড়া হলো সেই সুইচের সঙ্গে প্রতিটি Container-কে যুক্ত করা ভার্চুয়াল তার।

১২.৬ বিস্তারিত প্রযুক্তিগত ব্যাখ্যা

Container Network Model (CNM) — তিনটি অংশের গভীরে

Sandbox: Docker যখন একটি Container তৈরি করে, তখন কার্নেলকে বলে একটি নতুন Network Namespace বানাতে। এই Namespace-এর ভেতরে থাকে একটি Loopback Interface (`lo`) ও একটি Ethernet Interface (`eth0`)। `eth0` -র অন্য প্রান্তটি হোস্টের দিকে থাকে — একে বলে **veth (Virtual Ethernet) জোড়া**। একটি জোড়ার এক প্রান্ত Container-এর ভেতরে `eth0` , অন্য প্রান্ত হোস্টে `vethXXXX` নামে দেখা যায় এবং সেটি Bridge-এর সঙ্গে যুক্ত থাকে। Container মুছে ফেললে তার Namespace ও veth জোড়া — দুটোই মুছে যায়। **Endpoint:** একটি Network-এ যুক্ত হওয়ার সময় Docker Container-এর Interface-কে একটি IP Address, Subnet Mask, Gateway ও DNS Server দেয়।

একটি Container একই সঙ্গে একাধিক Network-এ যুক্ত হতে পারে — তখন তার একাধিক Interface ও একাধিক Endpoint থাকে, যেমন একটি কম্পিউটারে দুটি Network Card থাকা। **Network:** Network হলো Endpoint-গুলোর গোষ্ঠী ও তাদের যোগাযোগের নিয়ম। একই Network-এর Container-গুলো পরস্পরের সঙ্গে সরাসরি কথা বলতে পারে; ভিন্ন Network-এর Container-গুলো সাধারণত পারে না — এটাই নেটওয়ার্ক বিচ্ছিন্নতা (Isolation)।

Network Driver — কেন একাধিক ধরন

Driver নির্ধারণ করে Network-টি বাস্তবে কীভাবে প্যাকেট পৌঁছে দেবে। **Bridge** — ডিফল্ট; হোস্টের ভেতরে ভার্চুয়াল সুইচ, Container-গুলো ব্যক্তিগত (Private) IP পায়, বাইরে যেতে NAT লাগে। **Host** — Container হোস্টের Network Namespace-ই ব্যবহার করে; কোনো বিচ্ছিন্নতা নেই, কিন্তু গতি সর্বোচ্চ ও Port Mapping লাগে না। **None** — কোনো Interface ছাড়া (শুধু `lo`); সম্পূর্ণ বিচ্ছিন্ন, শুধু নিজের সঙ্গে কথা বলা যায়। **Overlay** — একাধিক হোস্টের Container-কে একটিই ভার্চুয়াল নেটওয়ার্কে যুক্ত করে (Swarm/Kubernetes-এর ভিত্তি)। **Macvlan** — Container-কে হোস্টের ফিজিক্যাল নেটওয়ার্কে একটি আসল MAC Address দেয়, যেন সে একটি আলাদা ফিজিক্যাল মেশিন। প্রতিটি Driver-এর ব্যবহার আলাদা — বিস্তারিত অধ্যায় ১৩-এ।

Container Port বনাম Published Host Port

বিষয়	Container Port	Published Host Port
এটি কী	Container-এর ভেতরে অ্যাপ যে Port-এ শোনে (Listen করে)	হোস্টের যে Port-টি বাইরে থেকে পৌঁছানো যায়
উদাহরণ	Next.js অ্যাপ `:3000`-এ শোনে	`-p 8080:3000` — হোস্টের 8080 → Container-এর 3000
কে দেখতে পায়	একই Network-এর অন্য Container	হোস্টের নেটওয়ার্কে পৌঁছাতে পারে এমন যে কেউ
ঘোষণা	Dockerfile-এ `EXPOSE 3000` (শুধু তথ্য, Port খোলে না)	`docker run -p` বা Compose-এ `ports:`
ভুলের ফল	ভুল Port-এ শুনলে Container নিজেই ব্যর্থ	ভুলভাবে ছাপালে Database ইন্টারনেটে উন্মুক্ত হয়

মনে রাখার নিয়ম: `EXPOSE` শুধু ডকুমেন্টেশন — এটি কোনো Port খোলে না, Firewall বদলায় না। আসল প্রকাশ ঘটে শুধু `-p` দিয়ে। আর Container-এর ভেতরে অ্যাপকে `0.0.0.0`-তে শুনতে হয়, `localhost`-এ নয় — `localhost`-এ শুনলে শুধু সেই Container-এর ভেতর থেকেই পৌঁছানো যায়, অন্য Container বা হোস্ট থেকে নয়।

বাইরে যাওয়ার পথ: NAT ও DNS

Container যখন বাইরের কোনো সার্ভারে (যেমন npm Registry বা Stripe API) যেতে চায়, তখন তার ব্যক্তিগত IP (যেমন 172.17.0.2) ইন্টারনেটে বৈধ নয়। Docker হোস্টের iptables-এ **MASQUERADE** নিয়ম বসায় — বেরোনোর সময় Source IP-কে হোস্টের IP-তে বদলে দেওয়া হয়, আর উত্তরের প্যাকেট ফেরত এলে আবার আসল Container-এ পাঠানো হয়। একে বলে **Source NAT (SNAT)**। আর নাম থেকে IP বের করার কাজটি করে Docker-এর **Embedded DNS Server** (127.0.0.11) — Container-এর `/etc/resolv.conf`-এ এই ঠিকানাটিই লেখা থাকে। User-defined Network-এ এই DNS Container-এর নাম থেকে IP বের করতে পারে; ডিফল্ট Bridge-এ পারে না — এই পার্থক্যটি অধ্যায় ১৪-এর মূল বিষয়।

Docker-এর ডিফল্ট নেটওয়ার্কসমূহ

নতুন Docker ইনস্টলেশনে তিনটি Network থাকে: **bridge** (ডিফল্ট; `docker0` Interface, 172.17.0.0/16), **host** (হোস্টের নেটওয়ার্ক ভাগ করে নেওয়া), **none** (কোনো নেটওয়ার্ক নেই)। `docker run`-এ কোনো Network না বললে Container ডিফল্ট Bridge-এ যায়। ডিফল্ট Bridge-এর সীমাবদ্ধতা — নাম দিয়ে DNS Resolution হয় না, আর সব Container একই ভাগাভাগি করা নেটওয়ার্কে থাকে — এই কারণে প্রোডাকশনে সবসময় নিজের নামে একটি User-defined Bridge Network বানানো উচিত।

১২.৭ Docker কমান্ড — লাইন-বাই-লাইন বিশ্লেষণ

```
$ docker network ls
# docker network ls — হোস্টের সব Docker Network-এর তালিকা দেখায়।
# NETWORK ID: সংক্ষিপ্ত শনাক্তকারী; NAME: bridge/host/none বা নিজের বানানো নাম;
# DRIVER: কোন Driver দিয়ে তৈরি; SCOPE: local (একটিমাত্র হোস্ট) না swarm।

$ docker network inspect bridge
# docker network inspect bridge — নাম ধরে একটি Network-এর সম্পূর্ণ বিবরণ দেখায়।
# আউটপুট JSON: Subnet (172.17.0.0/16), Gateway (172.17.0.1), সংযুক্ত Container
# ও তাদের IPv4Address, Options (যেমন com.docker.network.bridge.name=docker0)।

$ docker run -d --name web -p 8080:3000 technova-web:1.0
# docker run -d — Detached (পেছনে) মোডে Container চালাও।
# --name web — Container-এর নাম web (পরে DNS ও পরিচালনায় লাগবে)।
# -p 8080:3000 — Published Host Port 8080-কে Container Port 3000-এর সঙ্গে যুক্ত করো।
# technova-web:1.0 — চালানোর Image ও সংস্করণ।

$ docker run -d --name api --network technova-net technova-api:1.0
# --network technova-net — Container-কে technova-net নামের Network-এ যুক্ত করো।

$ docker network connect technova-net web
# docker network connect — চলমান Container web-কে technova-net Network-এ
# অতিরিক্তভাবে যুক্ত করো; এখন web-এর দুটি Interface (দুটি Network) থাকবে।

$ docker network disconnect technova-net web
# docker network disconnect — web-কে technova-net থেকে বিচ্ছিন্ন করো।

$ docker exec web cat /etc/resolv.conf
# docker exec web — চলমান web Container-এর ভেতরে কমান্ড চালাও।
# cat /etc/resolv.conf — DNS সেটিং দেখো; এখানে nameserver 127.0.0.11 থাকবে।

$ docker exec web ip addr show eth0
# ip addr show eth0 — Container-এর eth0 Interface-এর IP ও MAC দেখো।
```

নমুনা আউটপুট (চিত্রণমূলক — আপনার IP ও ID আলাদা হবে):

```
$ docker network ls
NETWORK ID     NAME      DRIVER    SCOPE
a1b2c3d4e5f6   bridge    bridge    local
f6e5d4c3b2a1   host      host      local
9a8b7c6d5e4f   none      null      local
$ docker exec web ip addr show eth0
```

```
2: eth0@if7: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500
    inet 172.17.0.2/16 brd 172.17.0.255 scope global eth0
```

১২.৮ সম্পূর্ণ কনফিগারেশন উদাহরণ

TechNova-র তিন-স্তরের অ্যাপের জন্য একটি User-defined Network বানিয়ে সব Container তাতে চালানো — এটি এই Part-এর ভিত্তি-উদাহরণ, পরের অধ্যায়গুলোতে এর ওপরেই গড়ে তোলা হবে:

```
# 1. নিজের নামে একটি Bridge Network বানান (সাবনেট নিজে ঠিক করে দেওয়া যায়)
docker network create \
  --driver bridge \
  --subnet 172.28.0.0/16 \
  --gateway 172.28.0.1 \
  technova-net

# 2. Database প্রথমে চালান (কোনো Published Port নয় — শুধু ভেতরের Network)
docker run -d --name technova-mongo \
  --network technova-net \
  -e MONGO_INITDB_ROOT_USERNAME=admin \
  -e MONGO_INITDB_ROOT_PASSWORD=CHANGE-ME-MONGO-DB-PASS \
  -v mongo-data:/data/db \
  mongo:7

# 3. API চালান — শুধু একই Network-এর ভেতর থেকে পৌঁছানো যাবে
docker run -d --name technova-api \
  --network technova-net \
  -e MONGODB_URI=mongodb://admin:CHANGE-ME-MONGO-DB-PASS@technova-mongo:27017/technova \
  -e PORT=4000 \
  technova-api:1.0

# 4. Web Frontend — এটিই একমাত্র Container যার Port হোস্টে প্রকাশ করা হয়েছে
docker run -d --name technova-web \
  --network technova-net \
  -p 8080:3000 \
  -e NEXT_PUBLIC_API_URL=http://technova-api:4000 \
  technova-web:1.0

# যাচাই: API থেকে Mongo-র নাম Resolution হচ্ছে কি না
docker exec technova-api getent hosts technova-mongo
# প্রত্যাশিত আউটপুট (চিত্রণমূলক): 172.28.0.2  technova-mongo
```

লক্ষ্য করুন: MongoDB-র কোনো `-p` নেই — অর্থাৎ Database-টি হোস্টের বাইরে থেকে একেবারেই পৌঁছানো যায় না, শুধু `technova-net`-এর ভেতরের Container-গুলোই তাকে দেখতে পায়। এটাই সঠিক নকশা।

১২.৯ বাস্তব প্রোডাকশন পরিস্থিতি

REAL WORLD — TechNova: TechNova-র Staging সার্ভারে (203.0.113.10) একদিন সকালে দেখা গেল Frontend চলছে, কিন্তু API থেকে "Connection refused" আসছে। Log-এ কোনো Error নেই, Container-গুলো সব `Up`। পরীক্ষায় দেখা গেল API Container-টি ভুল করে ডিফল্ট Bridge-এ চালানো হয়েছিল, আর MongoDB ছিল `technova-net`-এ — দুটি ভিন্ন Network, তাই তারা পরস্পরকে দেখতেই পাচ্ছে না। সমাধান ছিল এক লাইনের: `docker network connect technova-net technova-api`। এই ঘটনার পর দলের নিয়ম হলো — প্রতিটি Deploy-এর আগে `docker network inspect technova-net` দিয়ে যাচাই করা হবে কোন Container কোন Network-এ আছে। নেটওয়ার্ক সমস্যার ৮০% কারণই হলো "ভুল Network-এ চালানো" বা "ভুল Port-এ শোনা"।

আরেকটি প্রোডাকশন দৃশ্য: একটি দল `-p 27017:27017` দিয়ে MongoDB-কে হোস্টে প্রকাশ করেছিল "ডিবাগ করার সুবিধার জন্য" — এবং সেটা ভুলে গিয়েছিল। কয়েক সপ্তাহ পর স্ক্যানে ধরা পড়ল তাদের Database ইন্টারনেট থেকে খোলা। শিক্ষা: Published Port একটি নিরাপত্তা-সিদ্ধান্ত, সুবিধার সিদ্ধান্ত নয়; ডিবাগের জন্য `docker exec` বা সাময়িক Port Forward ব্যবহার করুন, স্থায়ী `-p` নয়।

১২.১০ ধাপে ধাপে ব্যবহারিক উদাহরণ

উদাহরণ: আপনার প্রথম নেটওয়ার্ক মানচিত্র তৈরি করা

1. `docker network ls` চালিয়ে ডিফল্ট তিনটি Network দেখুন ও তাদের Driver লিখে রাখুন।
2. `docker network create lab-net` দিয়ে নিজের Network বানান।
3. দুটি Container চালান: `docker run -d --name lab-a --network lab-net nginx:alpine` ও একইভাবে `lab-b`।
4. `docker exec lab-a ping -c 3 lab-b` চালান — নাম দিয়ে Ping সফল হওয়া উচিত (User-defined Network-এর DNS)।
5. এবার ডিফল্ট Bridge-এ দুটি Container চালিয়ে একই পরীক্ষা করুন — নাম দিয়ে Ping ব্যর্থ হবে, IP দিয়ে সফল হবে। পার্থক্যটি নিজে দেখুন।
6. `docker network inspect lab-net` দিয়ে Subnet, Gateway ও সংযুক্ত Container-গুলোর IP লিখে একটি মানচিত্র আঁকুন।
7. শেষে `docker rm -f lab-a lab-b` ও `docker network rm lab-net` দিয়ে পরিষ্কার করুন।

১২.১১ সাধারণ ভুল ও ভুল ধারণা

TROUBLESHOOTING: "Container চলছে, কিন্তু curl করলে Connection refused" — প্রথমে দেখুন অ্যাপটি Container-এর ভেতরে কোন Interface-এ শুনছে: `docker exec <name> ss -tlnp`। যদি শুধু `127.0.0.1:3000` দেখায়, তবে অ্যাপ `localhost`-এ বাঁধা — `0.0.0.0`-তে শুনতে হবে। দ্বিতীয়ত দেখুন `-p` mapping ঠিক আছে কি না: `docker port <name>`। তৃতীয়ত, Firewall (UFW/iptables) হোস্ট Port আটকাচ্ছে কি না দেখুন।

- **ভুল:** `EXPOSE 3000` দিলেই Port খুলে যায় ভাবা। `EXPOSE` শুধু তথ্য; প্রকাশ করতে `-p` লাগে।

- **ভুল:** অ্যাপকে `localhost`-এ শোনানো — অন্য Container বা হোস্ট থেকে পৌঁছানো যায় না। সবসময় `0.0.0.0` ব্যবহার করুন।
- **ভুল:** সব Container ডিফল্ট Bridge-এ চালানো — নাম দিয়ে DNS কাজ করে না, বিচ্ছিন্নতাও নেই।
- **ভুল:** Database-র Port হোস্টে প্রকাশ করা (`-p 27017:27017`) "সুবিধার জন্য"।
- **ভুল ধারণা:** "দুটি Container একই হোস্টে থাকলেই পরস্পরকে দেখতে পায়।" — না, একই Network-এ থাকতে হয়।
- **ভুল ধারণা:** "Container-এর IP স্থায়ী।" — না, Container পুনরায় তৈরি হলে IP বদলে যেতে পারে; তাই সবসময় নাম ব্যবহার করুন, IP নয়।

১২.১২ নিরাপত্তা বিবেচনা

> **WARNING:** `-p 27017:27017` বা `-p 5432:5432` দিয়ে Database Port প্রকাশ করলে সেই Database ইন্টারনেট থেকে পৌঁছানো যায় — দুর্বল পাসওয়ার্ড থাকলে কয়েক ঘণ্টার মধ্যে আক্রমণ হতে পারে। শুধুমাত্র Frontend বা Reverse Proxy (Nginx)-এর Port প্রকাশ করুন; Database, Cache (Redis) ও অভ্যন্তরীণ API শুধু অভ্যন্তরীণ Network-এ রাখুন। প্রয়োজনে `-p 127.0.0.1:27017:27017` দিয়ে শুধু হোস্টের Loopback-এ বাঁধুন — এতে বাইরের নেটওয়ার্ক থেকে পৌঁছানো যাবে না।

নেটওয়ার্ক বিচ্ছিন্নতাই প্রথম নিরাপত্তা-স্তর: Frontend, API ও Database — তিনটি আলাদা Network-এ ভাগ করলে আক্রমণকারী Frontend ভেদ করলেও সরাসরি Database-এ পৌঁছাতে পারে না (বিস্তারিত অধ্যায় ১৩)। Docker-এর iptables নিয়ম হোস্টের UFW Firewall-কে পাশ কাটিয়ে যেতে পারে — `-p` দিয়ে প্রকাশ করা Port UFW-তে বন্ধ থাকলেও পৌঁছানো যেতে পারে। প্রোডাকশনে এই আচরণটি জেনে রাখুন এবং প্রয়োজনে `--iptables=false` বা Docker-এর `daemon.json`-এ `iptables` সেটিং পরীক্ষা করুন।

১২.১৩ কর্মক্ষমতা ও রিসোর্স বিবেচনা

Bridge Network-এর মধ্য দিয়ে প্যাকেট যাওয়ার সময় NAT ও ভার্চুয়াল সুইচের ছোট একটি খরচ (Overhead) থাকে — সাধারণ ওয়েব অ্যাপের জন্য তা নগণ্য, কিন্তু প্রতি সেকেন্ডে হাজার হাজার ছোট Request বা বড় ফাইল স্থানান্তরে তা চোখে পড়তে পারে। উচ্চ-গতির কাজে (যেমন বড় ফাইল সার্ভ করা, লো-লেটেন্সি API) Host Network বিবেচনা করা যায়, তবে বিচ্ছিন্নতা হারানোর মূল্য দিতে হয়। প্রতিটি Container-এর veth Interface ও IP হোস্টের রিসোর্স নেয়; শত শত Container চালালে Subnet-এর আকার (`/16` বনাম `/24`) ও হোস্টের `ip_conntrack` সীমা পরীক্ষা করুন। DNS Lookup-এর জন্য Docker-এর Embedded DNS (127.0.0.11) দ্রুত, তবে Container তৈরির সময় DNS ব্যর্থ হলে `--dns` দিয়ে বাইরের DNS (যেমন 1.1.1.1) দেওয়া যায়।

১২.১৪ সমস্যা সমাধান পদ্ধতি

1. **Container চলছে কি না:** `docker ps` — Status `Up` কি না, Port Mapping দেখুন।
2. **কোন Network-এ আছে:** `docker inspect <name> --format '{{json .NetworkSettings.Networks}}'` — Network-এর নাম ও IP দেখুন।
3. **নাম Resolution:** `docker exec <name> getent hosts <target>` বা `nslookup <target>` — ব্যর্থ হলে Network আলাদা কি না, বা ডিফল্ট Bridge ব্যবহার হচ্ছে কি না দেখুন।
4. **Port পৌঁছানো যায় কি না:** একই Network-এর Container থেকে `docker exec <name> curl -v http://<target>:<port>/health`; হোস্ট থেকে `curl -v http://localhost:<published-port>`।

5. **অ্যাপ শুনছে কি না:** `docker exec <name> ss -tlnp` — `0.0.0.0:<port>` দেখা যাচ্ছে কি না।
6. **বাইরে যাওয়া যায় কি না:** `docker exec <name> ping -c 2 8.8.8.8` (IP দিয়ে) ও `ping -c 2 registry.example` (নাম দিয়ে) — IP চলে, নাম চলে না মানে DNS সমস্যা; দুটোই না চললে Gateway/NAT সমস্যা।
7. **iptables দেখুন:** `sudo iptables -t nat -L DOCKER -n` — Published Port-এর DNAT নিয়ম আছে কি না।

১২.১৫ সেরা অনুশীলন ও অ্যান্টি-প্যাটার্ন

সেরা অনুশীলন: প্রতিটি অ্যাপের জন্য নাম-সহ একটি User-defined Network বানান; Database-র Port কখনো প্রকাশ করবেন না; অ্যাপকে `0.0.0.0`-তে শোনান; Container-এর নাম দিয়ে যোগাযোগ করুন, IP দিয়ে নয়; Network-এর নাম ও Subnet ডকুমেন্ট করুন; Deploy-এর আগে `docker network inspect` দিয়ে যাচাই করুন। **অ্যান্টি-প্যাটার্ন:** সবকিছু ডিফল্ট Bridge-এ চালানো; `-p` দিয়ে সব Port প্রকাশ করা; IP Address হার্ডকোড করা (Container পুনরায় তৈরি হলে IP বদলায়); `--link` ব্যবহার করা — এটি পুরনো (Legacy) পদ্ধতি, এখন User-defined Network-এর DNS ব্যবহার করুন।

১২.১৬ অধ্যায়ের সারাংশ

Docker নেটওয়ার্কিংয়ের মূলে আছে Container Network Model: Sandbox (Container-এর নিজস্ব Network Namespace), Endpoint (Network-এর সঙ্গে সংযোগ-বিন্দু) ও Network (Endpoint-গুলোর গোষ্ঠী)। Bridge Driver হোস্টের ভেতরে ভার্চুয়াল সুইচ বানায়, veth জোড়া Container-কে সেই সুইচের সঙ্গে যুক্ত করে, আর iptables NAT বাইরের জগতের সঙ্গে যোগাযোগ ঘটায়। Container Port হলো অ্যাপের শোনার Port, Published Host Port হলো বাইরে থেকে পৌঁছানোর দরজা — `EXPOSE` শুধু তথ্য, প্রকাশ করে `-p`। প্রোডাকশনে সবসময় User-defined Network ব্যবহার করুন, Database-র Port প্রকাশ করবেন না, আর সমস্যা হলে Network, DNS, Port ও শোনার Interface — এই চারটি ধাপে পরীক্ষা করুন।

১২.১৭ মূল প্রযুক্তিগত পরিভাষা

Container Network Model (CNM): Docker-এর নেটওয়ার্কিং কাঠামো — Sandbox, Endpoint ও Network-এর সমন্বয়।

Sandbox: একটি Container-এর নিজস্ব নেটওয়ার্ক পরিবেশ — Network Namespace, Interface ও DNS সেটিংসহ।

Endpoint: একটি Sandbox-কে একটি Network-এর সঙ্গে যুক্ত করার সংযোগ-বিন্দু।

Network Driver: Network-টি বাস্তবে কীভাবে প্যাকেট পৌঁছে দেবে তা নির্ধারণকারী চালক (যেমন bridge, host, none, overlay)।

Network Namespace: Linux কার্নেলের আলাদা নেটওয়ার্ক-জগৎ — নিজের Interface, IP ও Routing Tableসহ।

veth (Virtual Ethernet): ভার্চুয়াল তারের জোড়া — এক প্রান্ত Container-এর ভেতরে, অন্য প্রান্ত হোস্টের Bridge-এ।

Bridge: হোস্টের ভেতরে ভার্চুয়াল সুইচ যা একই Network-এর Container-গুলোকে যুক্ত করে।

NAT (Network Address Translation): ব্যক্তিগত IP-কে হোস্টের IP-তে বদলে বাইরে যোগাযোগের ব্যবস্থা; MASQUERADE হলো তার একটি রূপ।

Container Port: Container-এর ভেতরে অ্যাপ যে Port-এ শোনে।

Published Host Port: হোস্টের যে Port-টি `-p` দিয়ে Container Port-এর সঙ্গে যুক্ত করে বাইরে প্রকাশ করা হয়।

Embedded DNS: Docker-এর ভেতরের DNS সার্ভার (127.0.0.11) যা User-defined Network-এ Container-এর নাম থেকে IP বের করে।

EXPOSE: Dockerfile-এর নির্দেশ যা শুধু জানায় অ্যাপ কোন Port-এ শুনবে — Port খোলে না।

১২.১৮ জ্ঞান যাচাই — প্রশ্ন

1. CNM-এর তিনটি অংশ — Sandbox, Endpoint, Network — নিজের ভাষায় ব্যাখ্যা করুন ও একটি উদাহরণ দিন।
2. Container Port ও Published Host Port-এর পার্থক্য কী? `-p 8080:3000` বলতে কী বোঝায়?
3. অ্যাপকে `localhost`-এর বদলে `0.0.0.0`-তে শোনাতে হয় কেন?
4. `EXPOSE 3000` দিলেই কি বাইরে থেকে Port 3000-এ পৌঁছানো যায়? কেন, ব্যাখ্যা করুন।
5. Container থেকে বাইরের ইন্টারনেটে যাওয়ার পথে NAT কী ভূমিকা রাখে?
6. ডিফল্ট Bridge Network-এর দুটি সীমাবদ্ধতা লিখুন।

১২.১৯ ব্যবহারিক অনুশীলন

1. **মানচিত্র আঁকা:** ১২.১০-এর ধাপগুলো অনুসরণ করে একটি দুই-Container নেটওয়ার্কের মানচিত্র (Subnet, Gateway, IP, Port) আঁকুন।
2. **Port পরীক্ষা:** একটি Nginx Container `-p 8081:80` দিয়ে চালিয়ে হোস্ট থেকে `curl localhost:8081` করুন; এবার `-p` ছাড়া চালিয়ে দেখুন কী হয়।
3. **DNS তুলনা:** ডিফল্ট Bridge ও User-defined Bridge — দুটোতেই দুটি করে Container চালিয়ে নাম দিয়ে Ping-এর পার্থক্য নথিভুক্ত করুন।
4. **নিরাপত্তা নিরীক্ষা:** আপনার হোস্টে `docker ps` দিয়ে দেখুন কোন Port প্রকাশ করা আছে; কোনো Database Port প্রকাশ থাকলে তা বন্ধ করার পরিকল্পনা লিখুন।

অধ্যায় 13 • PART III — Container Networking and Persistent Storage

Chapter 13 — Bridge, Host, None and Custom Networks

১৩.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: (১) Bridge, Host, None, Overlay ও Macvlan Driver-এর কাজ, সুবিধা ও ঝুঁকি তুলনা করে বলতে; (২) ডিফল্ট Bridge ও User-defined Bridge-এর পার্থক্য ব্যাখ্যা করতে — বিশেষ করে DNS, Isolation ও Hot-plug সংযোগের ক্ষেত্রে; (৩) নিজের Subnet, Gateway, IP Range ও DNS ঠিক করে একটি Custom Network বানাতে; (৪) একটি Container-কে একাধিক Network-এ যুক্ত করে Frontend, API ও Database-কে আলাদা স্তরে ভাগ করতে; (৫) Host ও None Network কখন ব্যবহার করা নিরাপদ আর কখন নয় তা বিচার করতে; (৬) Network তৈরি, পরিদর্শন, সংযোগ, বিচ্ছিন্নতা ও মুছে ফেলার পুরো জীবনচক্র পরিচালনা করতে।

১৩.২ কেন এই বিষয়টি DevOps-এ গুরুত্বপূর্ণ

অধ্যায় ১২-এ আপনি নেটওয়ার্কিংয়ের মানচিত্র শিখেছেন; এই অধ্যায়ে আপনি সেই মানচিত্রের প্রতিটি রাস্তা বেছে নেওয়ার সিদ্ধান্ত শিখবেন। ভুল Driver বেছে নিলে হয় অ্যাপ ধীর হয়, নয়তো নিরাপত্তা ভেঙে যায়, নয়তো দুটি সার্ভিস পরস্পরকে খুঁজেই পায় না। TechNova-র প্রোডাকশনে একবার Host Network ব্যবহার করে একটি পরীক্ষামূলক Container চালানো হয়েছিল — সেটি হোস্টের Port 80 দখল করে নেওয়ায় আসল Nginx থেমে গিয়েছিল। আবার আরেকবার সব সার্ভিস একটিই Bridge-এ রাখায় একটি আক্রান্ত Frontend Container থেকে সরাসরি Database-এ পৌঁছানো গিয়েছিল। এই অধ্যায়ের সিদ্ধান্তগুলোই সেই দুটি বিপদ থেকে বাঁচায়: কোন কাজে কোন Network, আর স্তরগুলোকে কীভাবে আলাদা রাখতে হয়।

১৩.৩ পূর্বশর্ত জ্ঞান

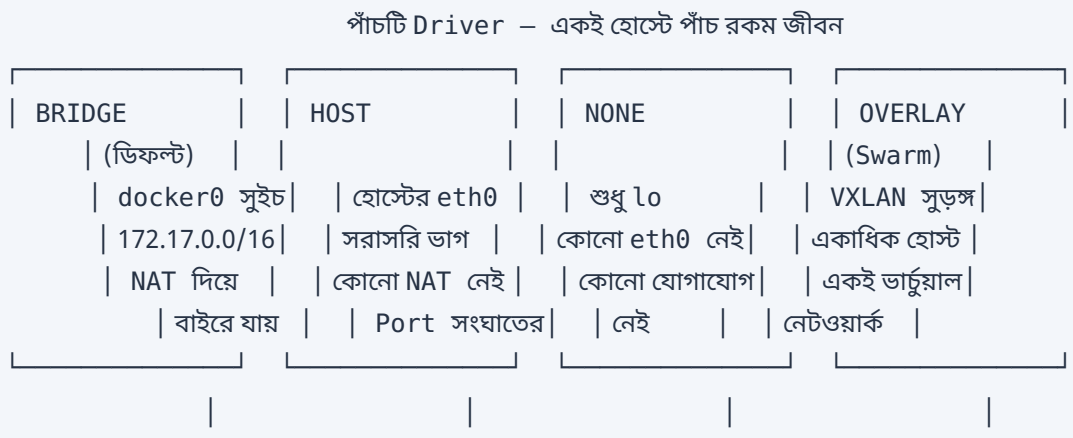
অধ্যায় ১২-এর CNM (Sandbox, Endpoint, Network), Bridge, NAT, Container Port বনাম Published Port, ও Embedded DNS-এর ধারণা জানা থাকতে হবে। IP Subnet লেখার ধরন (যেমন 172.28.0.0/16, /24 মানে ২৫৬টি ঠিকানা) ও `docker network` কমান্ডের মৌলিক ব্যবহার জানা থাকলে ভালো হয়। Overlay ও Macvlan-এর জন্য Swarm বা ফিজিক্যাল নেটওয়ার্কের অভিজ্ঞতা প্রয়োজন নেই — ধারণাগতভাবে ব্যাখ্যা করা হবে।

১৩.৪ প্রথম নীতি থেকে ব্যাখ্যা

একটি Network Driver আসলে একটি প্রশ্নের উত্তর: "এই Container-গুলোর প্যাকেট কীভাবে এক জায়গা থেকে আরেক জায়গায় যাবে?" **Bridge Driver** বলে: "আমি হোস্টের ভেতরে একটি ভার্চুয়াল সুইচ বানাব, Container-গুলো সেই সুইচে তার দিয়ে যুক্ত হবে, আর বাইরে যেতে NAT ব্যবহার হবে।" **Host Driver** বলে: "কোনো সুইচ নয় — Container হোস্টের নেটওয়ার্কই ব্যবহার করবে, যেন সে হোস্টেরই একটি প্রক্রিয়া।" **None Driver** বলে: "কোনো তার নয় — Container সম্পূর্ণ একা থাকবে।" **Overlay Driver** বলে: "আমি একাধিক হোস্টের সুইচগুলোকে একটি ভার্চুয়াল সুড়ঙ্গ (Tunnel) দিয়ে জুড়ে দেব, যাতে দূরের Container-ও মনে করে তারা একই সুইচে আছে।" **Macvlan Driver** বলে: "আমি Container-কে ফিজিক্যাল নেটওয়ার্কে একটি আসল ঠিকানা দেব, যেন সে একটি আলাদা কম্পিউটার।" প্রতিটি উত্তরই একটি বিনিময় (Trade-off): বিচ্ছিন্নতা বনাম গতি, সরলতা বনাম ব্যাপ্তি, নিরাপত্তা বনাম সুবিধা। সঠিক পছন্দ নির্ভর করে প্রশ্নটির ওপর — একটি মেশিনে ছোট অ্যাপের জন্য Bridge-ই যথেষ্ট; একাধিক সার্ভারে ছড়ানো অ্যাপের জন্য Overlay দরকার।

CONCEPT: Driver বেছে নেওয়া অনেকটা পরিবহন বেছে নেওয়ার মতো: Bridge হলো ভবনের ভেতরের করিডর — নিরাপদ, কিন্তু ভবনের বাইরে যেতে ফটক (NAT) লাগে। Host হলো করিডর ভেঙে রাস্তায় নেমে পড়া — দ্রুত, কিন্তু আর কোনো দরজা নেই। None হলো ঘরে তালা দিয়ে বসে থাকা — সম্পূর্ণ নিরাপদ, কিন্তু কারও সঙ্গে কথা বলা যায় না। Overlay হলো দুটি ভবনের মধ্যে সুড়ঙ্গ — দূরের বন্ধুর সঙ্গেও একই করিডরের অনুভূতি।

১৩.৫ আর্কিটেকচার / ধারণাগত চিত্র



একটি Container একাধিক Network-এ থাকতে পারে — এই ক্ষমতাই স্তর-ভিত্তিক বিচ্ছিন্নতার চাবিকাঠি।

১৩.৬ বিস্তারিত প্রযুক্তিগত ব্যাখ্যা

Bridge Network — গভীরে

Bridge Driver একটি Linux Bridge Interface বানায় (ডিফল্ট নাম `docker0`, নিজের Network-এ `br-xxxx`)। প্রতিটি Container-এর veth-এর হোস্ট-প্রান্ত এই Bridge-এর একটি Port হয়। Bridge-টি একটি Layer-2 সুইচের মতো কাজ করে: একই Bridge-এর Container-গুলো পরস্পরের MAC Address শিখে সরাসরি প্যাকেট পাঠায়। বাইরে যাওয়ার সময় iptables-এর `MASQUERADE` নিয়ম Source IP বদলে দেয়, আর `-p` দিয়ে প্রকাশ করা Port-এর জন্য `DNAT` নিয়ম বাইরের প্যাকেটকে সঠিক Container-এ পাঠায়। Bridge-এর দুটি গুরুত্বপূর্ণ সেটিং: `--subnet` (কোন ঠিকানা-পরিসর), `--gateway` (Bridge-এর নিজের IP, সাধারণত পরিসরের প্রথম ঠিকানা), `--ip-range` (Container-দের দেওয়ার পরিসর) ও `--internal` (বাইরে যাওয়া সম্পূর্ণ বন্ধ — শুধু ভেতরের যোগাযোগ)। `--internal` দিয়ে বানানো Network Database-স্তরের জন্য আদর্শ: Database-র Container বাইরের ইন্টারনেটে যেতেই পারে না, এমনকি ভুল করে `-p` দিলেও নয়।

ডিফল্ট Bridge বনাম User-defined Bridge

বিষয়	ডিফল্ট Bridge (bridge)	User-defined Bridge
DNS — নাম দিয়ে খোঁজা	কাজ করে না; শুধু IP দিয়ে পৌঁছানো যায়	কাজ করে; Container-এর নাম ও Alias দিয়ে Resolution হয়
চলমান Container যুক্ত করা	যায় না (শুধু তৈরি করার সময়)	যায় — `docker network connect`
বিচ্ছিন্নতা	সব Container একই নেটওয়ার্কে	প্রতিটি অ্যাপের আলাদা নেটওয়ার্ক সম্ভব
নাম	একটাই, সবার ভাগাভাগি	নিজের নাম, Subnet, Gateway ঠিক করা যায়
ব্যবহার	দ্রুত পরীক্ষা, একক Container	প্রোডাকশনের মানদণ্ড

এই পার্থক্যের কারণ: User-defined Network-এ Docker প্রতিটি Container-এর `/etc/hosts` ও Embedded DNS-এ তার নাম নিবন্ধন করে; ডিফল্ট Bridge-এ এই নিবন্ধন হয় না। তাই প্রোডাকশনে ডিফল্ট Bridge ব্যবহার করা কার্যত একটি ভুল।

Host Network — গতি বনাম বিচ্ছিন্নতা

`--network host` দিলে Container-এর কোনো নিজস্ব Namespace থাকে না — সে হোস্টের Interface, IP ও Port-গুলো সরাসরি ব্যবহার করে। অ্যাপ `:3000`-এ শুনলে সেটি হোস্টের `:3000`-ই হয়; `-p` দেওয়ার দরকার নেই, দিলেও তা উপেক্ষিত হয়। সুবিধা: NAT-এর খরচ নেই, সর্বোচ্চ গতি, আর হোস্টের নেটওয়ার্ক সেবা (যেমন স্থানীয় DNS বা VPN) সরাসরি পাওয়া যায়। ঝুঁকি: Container হোস্টের সব Port দেখতে ও দখল করতে পারে, দুটি Container একই Port চাইলে সংঘাত হয়, আর নেটওয়ার্ক-স্তরের বিচ্ছিন্নতা শূন্য হয়ে যায়। ব্যবহার: শুধু তখনই, যখন গতি সত্যিই জরুরি (যেমন নেটওয়ার্ক মনিটরিং টুল, উচ্চ-গতির প্রক্সি) অথবা Container-কে হোস্টের নেটওয়ার্ক সেবা সরাসরি দরকার। ওয়েব অ্যাপের জন্য প্রায় কখনোই নয়।

None Network — সম্পূর্ণ বিচ্ছিন্নতা

`--network none` দিলে Container শুধু Loopback Interface (`lo`) পায় — কোনো বাইরের যোগাযোগ নেই। এটি অকেজো মনে হলেও এর ব্যবহার আছে: শুধু ফাইল প্রক্রিয়াকরণ বা গণনার কাজ (Batch Job) যেখানে নেটওয়ার্কের দরকারই নেই, অথবা নিরাপত্তা-পরীক্ষায় একটি সন্দেহজনক Image-কে সম্পূর্ণ বিচ্ছিন্নভাবে চালানো। পরে প্রয়োজনে `docker network connect` দিয়ে একটি Network যুক্ত করা যায় — অর্থাৎ None দিয়ে শুরু করে পরে ধাপে ধাপে যোগাযোগ দেওয়া যায়।

Overlay ও Macvlan — সংক্ষেপে

Overlay: Docker Swarm-এ একাধিক হোস্টের Container-কে একটি ভার্চুয়াল নেটওয়ার্কে যুক্ত করে। প্যাকেট VXLAN Tunnel-এর ভেতর দিয়ে যায় (UDP Port 4789)। এক হোস্টের Container অন্য হোস্টের Container-কে নাম দিয়ে খুঁজে পায়। একক হোস্টের অ্যাপে Overlay দরকার হয় না; Kubernetes-এ এর সমতুল্য ধারণা CNI Plugin দিয়ে হয়। **Macvlan:** Container-কে হোস্টের ফিজিক্যাল Interface-এর ওপর একটি ভার্চুয়াল Interface দেয়, নিজের MAC Addressসহ। রাউটার Container-কে একটি আলাদা ফিজিক্যাল ডিভাইস ভাবে, তাই সে সরাসরি LAN-এর IP পায় — পুরনো সিস্টেম (Legacy) বা বিশেষ নেটওয়ার্ক যন্ত্রের সঙ্গে যুক্ত হতে কাজে লাগে। সীমাবদ্ধতা: Macvlan Container সাধারণত হোস্টের সঙ্গে সরাসরি কথা বলতে পারে না, আর প্রতিটি Container একটি MAC Address খরচ করে।

Custom Network বানানো — সিদ্ধান্তের তালিকা

নতুন Network বানানোর আগে ঠিক করুন: (১) **নাম:** অ্যাপ ও স্তর বোঝায় এমন নাম (যেমন `technova-frontend`, `technova-db`); (২) **Subnet:** হোস্টের LAN বা অন্য Docker Network-এর সঙ্গে সংঘাত করে না এমন পরিসর — ব্যক্তিগত পরিসর 172.16.0.0/12, 10.0.0.0/8 বা 192.168.0.0/16 থেকে বাছুন; (৩) **Gateway:** সাধারণত Subnet-এর প্রথম ঠিকানা; (৪) **Internal কি না:** Database-স্তরে `--internal` বিবেচনা করুন; (৫) **DNS:** প্রয়োজনে `--dns` দিয়ে কোম্পানির DNS সার্ভার দিন; (৬) **MTU:** VPN বা Cloud পরিবেশে প্যাকেটের আকারের সমস্যা হলে `--opt com.docker.network.driver.mtu=1450` বিবেচনা করুন।

১৩.৭ Docker কমান্ড — লাইন-বাই-লাইন বিশ্লেষণ

```
$ docker network create --driver bridge \
    --subnet 172.28.0.0/16 --gateway 172.28.0.1 technova-net
# docker network create — নতুন Network বানাও।
# --driver bridge — Bridge Driver ব্যবহার করো (না বললেও এটাই ডিফল্ট)।
# --subnet 172.28.0.0/16 — Container-দের IP দেওয়ার পরিসর।
# --gateway 172.28.0.1 — Bridge Interface-এর নিজের IP।
# technova-net — Network-এর নাম।

$ docker network create --driver bridge --internal technova-db-net
# --internal — এই Network থেকে বাইরের ইন্টারনেটে যাওয়া যাবে না।
# Database-স্তরের জন্য উপযুক্ত; Container-গুলো শুধু পরস্পরের সঙ্গে কথা বলবে।

$ docker network create --driver bridge \
    --subnet 172.30.0.0/24 --ip-range 172.30.0.128/25 technova-limited
# --ip-range — Container-দের শুধু 172.30.0.128–172.30.0.255 পরিসর থেকে IP দাও।

$ docker run -d --name web --network technova-net nginx:alpine
# Container তৈরির সময় Network ঠিক করে দেওয়া — সবচেয়ে সাধারণ পদ্ধতি।

$ docker run -d --name monitor --network host prom/node-exporter
# --network host — হোস্টের নেটওয়ার্ক সরাসরি ব্যবহার করো; -p লাগে না।

$ docker run -d --name batch-job --network none alpine sleep 3600
# --network none — কোনো নেটওয়ার্ক ছাড়া চালাও।

$ docker run -d --name api --network technova-net \
    --network-alias api-v1 --network-alias backend technova-api:1.0
# --network-alias — একই Container-কে একাধিক নামে ডাকা যাবে;
# অন্য Container api-v1 বা backend — যেকোনো নামেই তাকে খুঁজে পাবে।

$ docker network connect --ip 172.28.0.50 technova-net web
# docker network connect --ip — চলমান Container-কে Network-এ যুক্ত করো
# এবং একটি নির্দিষ্ট (Static) IP দাও। Static IP শুধু বিশেষ ক্ষেত্রে ব্যবহার করুন।

$ docker network inspect technova-net --format '{{json .Containers}}'
# --format — শুধু সংযুক্ত Container-গুলোর তথ্য দেখাও (পুরো JSON নয়)।

$ docker network prune
# docker network prune — কোনো Container ব্যবহার করছে না এমন সব Network মুছে ফেলো।
# সাবধানে: নাম-ভুলে বানানো অপ্রয়োজনীয় Network পরিষ্কার করতে কাজে লাগে।
```

```
$ docker network rm technova-net
# docker network rm — Network মুছে ফেলো; আগে সব Container বিচ্ছিন্ন করতে হবে।
```

১৩.৮ সম্পূর্ণ কনফিগারেশন উদাহরণ

TechNova-র স্তর-ভিত্তিক নকশা — তিনটি Network, প্রতিটি স্তর শুধু প্রয়োজনীয় স্তরের সঙ্গে যুক্ত। এটি এই Part-এর সবচেয়ে গুরুত্বপূর্ণ নেটওয়ার্ক নকশা:

```
# ১. তিনটি Network বানান
docker network create --driver bridge --subnet 172.28.0.0/24 technova-frontend
docker network create --driver bridge --subnet 172.29.0.0/24 technova-backend
docker network create --driver bridge --subnet 172.30.0.0/24 --internal technova-db

# ২. Database — শুধু technova-db Network-এ (বাইরে যাওয়া বন্ধ, কোনো -p নয়)
docker run -d --name technova-mongo --network technova-db \
  -e MONGO_INITDB_ROOT_USERNAME=admin \
  -e MONGO_INITDB_ROOT_PASSWORD=CHANGE-ME-MONGO-DB-PASS \
  -v mongo-data:/data/db mongo:7

docker run -d --name technova-postgres --network technova-db \
  -e POSTGRES_USER=technova \
  -e POSTGRES_PASSWORD=CHANGE-ME-POSTGRES-PASS \
  -e POSTGRES_DB=technova \
  -v postgres-data:/var/lib/postgresql/data postgres:16

# ৩. API — backend ও db — দুই Network-এ (দুটি Interface)
docker run -d --name technova-api --network technova-backend \
  -e MONGODB_URI=mongodb://admin:CHANGE-ME-MONGO-DB-PASS@technova-mongo:27017/technova \
  -e DATABASE_URL=postgres://technova:CHANGE-ME-POSTGRES-PASS@technova-postgres:5432/technova
\
  -e PORT=4000 technova-api:1.0
docker network connect technova-db technova-api

# ৪. Web — শুধু frontend ও backend Network-এ; একমাত্র Published Port এখানে
docker run -d --name technova-web --network technova-frontend \
  -p 8080:3000 \
  -e NEXT_PUBLIC_API_URL=http://technova-api:4000 technova-web:1.0
docker network connect technova-backend technova-web

# ৫. যাচাই — Frontend থেকে Database-এ সরাসরি যাওয়া যায় কি না দেখুন (যাওয়া উচিত নয়)
docker exec technova-web getent hosts technova-mongo || echo "সঠিক: Frontend Mongo-কে খুঁজে পায় না"
docker exec technova-api getent hosts technova-mongo
# প্রত্যাশিত: API থেকে technova-mongo-র IP আসবে; Web থেকে আসবে না।
```

এই নকশার ফল: আক্রমণকারী Web Container ভেদ করলেও Database-র নামই খুঁজে পাবে না, কারণ Web `technova-db` Network-এ নেই। Database-র Container বাইরের ইন্টারনেটে যেতেও পারে না (`--internal`) — তাই আক্রান্ত হলেও তথ্য বাইরে পাঠানো কঠিন।

১৩.৯ বাস্তব প্রোডাকশন পরিস্থিতি

REAL WORLD — TechNova: TechNova-র প্রোডাকশন সার্ভারে (203.0.113.10) একবার একটি নতুন Microservice Host Network-এ চালানো হয়েছিল "গতি বাড়ানোর" জন্য। সেটি Port 4000 দখল করে নেওয়ায় আসল API Container চালু হতে পারছিল না — দুটি প্রক্রিয়া একই Port চাইছিল। আরও খারাপ: Host Network-এর Container হোস্টের সব Port দেখতে পাচ্ছিল, ফলে নিরাপত্তা নিরীক্ষায় সেটি ধরা পড়ে। সমাধান: Microservice-টিকে নিজের Bridge Network-এ ফেরানো হলো, আর গতি-সমস্যার আসল কারণ — API-র ধীর Database Query — আলাদাভাবে ঠিক করা হলো। শিক্ষা: Host Network একটি নিরাপত্তা-সিদ্ধান্ত, কর্মক্ষমতা-সিদ্ধান্ত নয়; গতি বাড়াতে হলে প্রথমে আসল বাধা (Bottleneck) খুঁজুন।

আরেকটি দৃশ্য: একটি দল Subnet না ঠিক করে অনেক Network বানিয়েছিল। একসময় নতুন Network-এর Subnet কোম্পানির VPN-এর Subnet-এর সঙ্গে মিলে গেল — ফলে VPN-এ যুক্ত ডেভেলপাররা Container-গুলোতে পৌঁছাতে পারছিলেন না। সমাধান: সব Network-এর Subnet একটি তালিকায় লিখে রাখা হলো, আর নতুন Network বানানোর আগে সেই তালিকা পরীক্ষা করা বাধ্যতামূলক হলো।

১৩.১০ ধাপে ধাপে ব্যবহারিক উদাহরণ

উদাহরণ: তিন-স্তরের নেটওয়ার্ক নিজে বানানো

1. ১৩.৮-এর তিনটি Network বানান ও `docker network ls` দিয়ে যাচাই করুন।
2. MongoDB শুধু `technova-db`-এ, API `technova-backend` ও `technova-db`-তে, Web `technova-frontend` ও `technova-backend`-তে চালান।
3. `docker exec technova-web ping -c 2 technova-api` — সফল হওয়া উচিত (একই backend Network)।
4. `docker exec technova-web getent hosts technova-mongo` — ব্যর্থ হওয়া উচিত (ভিন্ন Network)। এই ব্যর্থতাটিই নিরাপত্তা।
5. `docker exec technova-api ping -c 2 technova-mongo` — সফল হওয়া উচিত।
6. `docker network inspect technova-backend` দিয়ে কোন Container কোন IP পেয়েছে লিখুন।
7. পরীক্ষা শেষে সব Container থামিয়ে Network-গুলো মুছে ফেলুন: `docker network rm` করার আগে `docker rm -f` দিয়ে Container সরাতে হবে।

১৩.১১ সাধারণ ভুল ও ভুল ধারণা

TROUBLESHOOTING: "দুটি Container একই হোস্টে, কিন্তু নাম দিয়ে Ping হয় না" — প্রথমে `docker inspect` দিয়ে দেখুন দুটি Container একই Network-এ আছে কি না। দ্বিতীয়ত, ডিফল্ট Bridge ব্যবহার হচ্ছে কি না দেখুন — ডিফল্ট Bridge-এ নাম Resolution হয় না। তৃতীয়ত, Container-এর নামের বানান ও `--network-alias` পরীক্ষা করুন। চতুর্থত, `docker network inspect`-এ Container-টি আদৌ তালিকায় আছে কি না দেখুন — কখনো কখনো Container তৈরি হয়েছে, কিন্তু Network যুক্ত হয়নি।

- **ভুল:** সব সার্ভিস একটিই Network-এ রাখা — স্তর-ভিত্তিক বিচ্ছিন্নতা হারানো।
- **ভুল:** Host Network দিয়ে "সব সমস্যা সমাধান" করা — Port সংঘাত ও নিরাপত্তা-ঝুঁকি।

- **ভুল:** Static IP (`--ip`) সব জায়গায় ব্যবহার করা — IP বদলের ঝুঁকি ও পরিচালনার জটিলতা; নাম ব্যবহার করুন।
- **ভুল:** Subnet না ঠিক করে অনেক Network বানানো — একসময় VPN বা অন্য Network-এর সঙ্গে সংঘাত।
- **ভুল:** `--link` ব্যবহার করা — পুরনো পদ্ধতি; User-defined Network-এর DNS ব্যবহার করুন।
- **ভুল ধারণা:** "Host Network মানেই দ্রুত অ্যাপ।" — সাধারণ ওয়েব অ্যাপে Bridge-এর খরচ নগণ্য; বাধা সাধারণত অন্য জায়গায় থাকে।
- **ভুল ধারণা:** "None Network অকেজো।" — Batch Job ও নিরাপত্তা-পরীক্ষায় এটি সঠিক পছন্দ।

১৩.১২ নিরাপত্তা বিবেচনা

> **WARNING:** Host Network-এ চলা Container হোস্টের Network Namespace ভাগ করে নেয় — অর্থাৎ সে হোস্টের সব Port দেখতে পারে, হোস্টের Loopback সেবা (যেমন স্থানীয় Database বা Admin প্যানেল) স্পর্শ করতে পারে, আর হোস্টের Firewall নিয়মকে পাশ কাটাতে পারে। অপরিচিত বা তৃতীয় পক্ষের Image কখনো Host Network-এ চালাবেন না। একইভাবে, Database-স্তর সবসময় `--internal` Network-এ রাখুন, যাতে Database Container বাইরে তথ্য পাঠাতে না পারে।

Network-স্তরের নিরাপত্তার তিনটি নিয়ম: (১) **Least Privilege:** প্রতিটি Container শুধু সেই Network-এই থাকবে যা তার কাজের জন্য দরকার; (২) **কোনো Database Port প্রকাশ নয়:** Database শুধু অভ্যন্তরীণ Network-এ; (৩) **নিয়মিত নিরীক্ষা:** `docker network ls` ও `docker ps` দিয়ে দেখুন কোন Network-এ কে আছে, কোন Port প্রকাশ করা আছে — অপ্রয়োজনীয় Network ও Port মুছে ফেলুন।

১৩.১৩ কর্মক্ষমতা ও রিসোর্স বিবেচনা

Bridge-এর খরচ মূলত দুটি জায়গায়: NAT-এর প্যাকেট-বদল ও veth-এর মধ্য দিয়ে যাওয়া। সাধারণ ওয়েব ট্রাফিকে এটি প্রতি Request-এ মাইক্রোসেকেন্ডের ভগ্নাংশ — পরিমাপ না করে Host Network-এ যাওয়া ঠিক নয়। তবে বড় ফাইল স্থানান্তর, ভিডিও স্ট্রিমিং বা প্রতি সেকেন্ডে হাজার হাজার ছোট প্যাকেটের ক্ষেত্রে Host Network পরিমাপযোগ্য সুবিধা দিতে পারে — তখন Benchmark করে সিদ্ধান্ত নিন। MTU-র দিকে নজর দিন: Cloud বা VPN পরিবেশে বাইরের নেটওয়ার্কের MTU 1500-এর কম হলে (যেমন 1450) Container-এর প্যাকেট ভেঙে যেতে পারে বা হারিয়ে যেতে পারে — তখন Network তৈরির সময় `--opt com.docker.network.driver.mtu=1450` দিন। প্রতিটি Network একটি Bridge Interface ও Subnet খরচ করে; অপ্রয়োজনীয় Network `docker network prune` দিয়ে পরিষ্কার করুন।

১৩.১৪ সমস্যা সমাধান পদ্ধতি

1. **Network-এর তালিকা:** `docker network ls` — কোন Network আছে, কোন Driver।
2. **কে কোথায়:** `docker network inspect <net>` — সংযুক্ত Container, তাদের IP ও Alias।
3. **Container-এর দিক থেকে:** `docker inspect <name>` — `NetworkSettings.Networks` অংশে সব Network ও IP।
4. **নাম Resolution:** `docker exec <a> getent hosts <b>` — ব্যর্থ হলে Network আলাদা কি না, ডিফল্ট Bridge কি না দেখুন।
5. **Subnet সংঘাত:** হোস্টে `ip route` চালিয়ে দেখুন Docker-এর Subnet হোস্টের LAN/VPN-এর সঙ্গে মিলছে কি না।
6. **Host Network-এ Port সংঘাত:** `ss -tlnp` দিয়ে দেখুন কোন প্রক্রিয়া Port দখল করে আছে।

7. **Network মুছতে না পারলে:** `docker network inspect`-এ দেখুন কোন Container এখনো যুক্ত আছে; আগে `docker network disconnect` বা Container মুছুন।

১৩.১৫ সেরা অনুশীলন ও অ্যান্টি-প্যাটার্ন

সেরা অনুশীলন: প্রতিটি অ্যাপের জন্য নাম-সহ User-defined Network; স্তর অনুযায়ী আলাদা Network (frontend/backend/db); Database-স্তরে `--internal`; নাম ও Alias দিয়ে যোগাযোগ, IP দিয়ে নয়; Subnet-এর তালিকা ডকুমেন্ট করা; অপ্রয়োজনীয় Network নিয়মিত পরিষ্কার করা। **অ্যান্টি-প্যাটার্ন:** ডিফল্ট Bridge-এ প্রোডাকশন চালানো; সবকিছু এক Network-এ রাখা; Host Network-কে ডিফল্ট সমাধান বানানো; Static IP-র ওপর নির্ভর করা; `--link` ব্যবহার করা; Subnet না ভেবে Network বানানো।

১৩.১৬ অধ্যায়ের সারাংশ

Bridge হলো প্রোডাকশনের মানদণ্ড — হোস্টের ভেতরে ভার্চুয়াল সুইচ, NAT দিয়ে বাইরের যোগাযোগ, আর User-defined Bridge-এ নাম দিয়ে DNS Resolution। Host Network সর্বোচ্চ গতি দেয়, কিন্তু বিচ্ছিন্নতা কেড়ে নেয় — শুধু বিশেষ ক্ষেত্রে ব্যবহার করুন। None Network সম্পূর্ণ বিচ্ছিন্ন — Batch Job ও নিরাপত্তা-পরীক্ষায় উপযুক্ত। Overlay একাধিক হোস্টকে এক নেটওয়ার্কে জোড়ে, Macvlan Container-কে ফিজিক্যাল LAN-এর সদস্য বানায়। সবচেয়ে গুরুত্বপূর্ণ নকশা-নীতি: স্তর অনুযায়ী আলাদা Network, Database-স্তর `--internal`, আর যোগাযোগ সবসময় নাম দিয়ে — IP দিয়ে নয়।

১৩.১৭ মূল প্রযুক্তিগত পরিভাষা

Bridge Driver: হোস্টের ভেতরে ভার্চুয়াল সুইচ বানিয়ে Container-গুলোকে যুক্ত করার Driver — ডিফল্ট পছন্দ।

User-defined Bridge: নিজের নামে বানানো Bridge Network — নাম দিয়ে DNS Resolution ও চলমান Container যুক্ত করার সুবিধাসহ।

Host Network: Container হোস্টের Network Namespace সরাসরি ব্যবহার করে — কোনো বিচ্ছিন্নতা নেই, সর্বোচ্চ গতি।

None Network: কোনো নেটওয়ার্ক Interface ছাড়া চালানো — সম্পূর্ণ বিচ্ছিন্ন।

Overlay Network: একাধিক হোস্টের Container-কে VXLAN Tunnel দিয়ে এক ভার্চুয়াল নেটওয়ার্কে যুক্ত করার Driver।

Macvlan: Container-কে ফিজিক্যাল নেটওয়ার্কে নিজের MAC Address ও IP দেওয়ার Driver।

Subnet: IP ঠিকানার একটি পরিসর, যেমন 172.28.0.0/24 — একটি Network-এর ঠিকানা-সীমানা।

Gateway: একটি Network থেকে অন্য Network-এ যাওয়ার দরজা — Bridge-এর নিজের IP।

Network Alias: একটি Container-এর অতিরিক্ত নাম, যা দিয়ে অন্য Container তাকে খুঁজে পায়।

--internal: Network তৈরির বিকল্প যা সেই Network থেকে বাইরের ইন্টারনেটে যাওয়া বন্ধ করে দেয়।

MTU (Maximum Transmission Unit): একটি প্যাকেটের সর্বোচ্চ আকার; ভুল MTU প্যাকেট হারানোর কারণ হতে পারে।

VXLAN: Overlay Network-এ প্যাকেটকে সুড়ঙ্গের ভেতর দিয়ে পাঠানোর প্রযুক্তি (UDP Port 4789)।

১৩.১৮ জ্ঞান যাচাই — প্রশ্ন

1. ডিফল্ট Bridge ও User-defined Bridge-এর মধ্যে DNS-সংক্রান্ত পার্থক্যটি ব্যাখ্যা করুন।
2. Host Network-এর দুটি সুবিধা ও তিনটি ঝুঁকি লিখুন। কখন এটি ব্যবহার করা যুক্তিযুক্ত?
3. None Network কখন ব্যবহার করবেন? একটি বাস্তব উদাহরণ দিন।
4. Frontend Container থেকে Database Container-কে সরাসরি খুঁজে পাওয়া যাওয়া উচিত নয় কেন? কীভাবে তা আটকাবেন?
5. `--internal` Network কী করে? Database-স্তরে এটি কেন উপযোগী?
6. Overlay ও Macvlan — কে কখন দরকার, একটি করে উদাহরণসহ ব্যাখ্যা করুন।
7. Static IP ব্যবহারের বদলে নাম ব্যবহার করা উচিত কেন?

১৩.১৯ ব্যবহারিক অনুশীলন

1. **স্তর-ভিত্তিক নেটওয়ার্ক:** ১৩.১০-এর উদাহরণটি সম্পূর্ণ করুন ও প্রতিটি ধাপের ফলাফল লিখুন।
2. **Host বনাম Bridge:** একই Nginx Image একবার Bridge-এ (`-p 8081:80`) ও একবার Host Network-এ চালিয়ে `curl` ও `ss -tlnp` -এর পার্থক্য নথিভুক্ত করুন।
3. **Internal Network পরীক্ষা:** একটি `--internal` Network-এ একটি Container চালিয়ে ভেতর থেকে `ping 8.8.8.8` করুন — কী হয়, লিখুন।
4. **Subnet পরিকল্পনা:** TechNova-র জন্য তিনটি স্তরের Subnet (কোনো সংঘাত ছাড়া) পরিকল্পনা করে একটি তালিকা লিখুন।

অধ্যায় 14 • PART III — Container Networking and Persistent Storage

Chapter 14 — Container-to-Container Communication and DNS

১৪.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: (১) দুটি Container কীভাবে নাম দিয়ে পরস্পরকে খুঁজে পায় — Embedded DNS-এর পুরো প্রবাহ ব্যাখ্যা করতে; (২) User-defined Network-এ Service Discovery কীভাবে কাজ করে তা বলতে; (৩) Docker Compose-এ Service-গুলো কীভাবে Service নাম দিয়ে একে অপরকে আবিষ্কার করে তা ব্যাখ্যা করতে (Compose-এর বিস্তারিত অধ্যায় পরে আসবে — এখানে শুধু আবিষ্কারের নীতিটি শিখবেন); (৪) `/etc/hosts`, `/etc/resolv.conf` ও `--add-host`-এর ভূমিকা বোঝাতে; (৫) Network Alias ও Link-এর পার্থক্য বলতে; (৬) DNS ব্যর্থতা, নাম-সংঘাত ও Startup-ক্রম (একটি সার্ভিস আরেকটির আগে চালু হওয়া) সমস্যার সমাধান করতে; (৭) Healthcheck ও Retry-ভিত্তিক নির্ভরযোগ্য সংযোগের নকশা করতে।

১৪.২ কেন এই বিষয়টি DevOps-এ গুরুত্বপূর্ণ

আধুনিক অ্যাপ একটিমাত্র প্রক্রিয়া নয় — এটি সার্ভিসের একটি দল। Frontend API-কে ডাকে, API Database-কে ডাকে, Worker Queue থেকে কাজ নেয়, Cache-কে জিজ্ঞেস করে। এই ডাকাডাকির প্রতিটি ধাপে একটি নামকে একটি ঠিকানায় বদলাতে হয় — আর সেই বদলানোর কাজটি ব্যর্থ হলে পুরো অ্যাপ থেমে যায়, যদিও প্রতিটি Container আলাদাভাবে সুস্থ। TechNova-র সবচেয়ে বিরক্তিকর প্রোডাকশন-ঘটনাগুলোর প্রায় সবই এই শ্রেণির: API চালু হয়েছে MongoDB-র আগে, তাই প্রথম কয়েক মিনিট সব Request ব্যর্থ; অথবা নতুন সংস্করণে Container-এর নাম বদলানো হয়েছে, কিন্তু Connection String-এ পুরনো নামই রয়ে গেছে। Service Discovery বোঝা মানেই এই নির্ভরতাগুলোকে নাম দিয়ে, পুনরায় চেষ্টা (Retry) দিয়ে ও Healthcheck দিয়ে শক্ত করা — যাতে একটি সার্ভিসের পুনরায় চালু হওয়া পুরো সিস্টেমকে না ভাঙে।

১৪.৩ পূর্বশর্ত জ্ঞান

অধ্যায় ১২–১৩-এর CNM, Bridge Network, Embedded DNS (127.0.0.11), User-defined বনাম ডিফল্ট Bridge, Network Alias ও `docker network connect` জানা থাকতে হবে। DNS-এর সাধারণ ধারণা (নাম → IP), `/etc/hosts` ফাইলের কাজ, ও Environment Variable দিয়ে Connection String দেওয়ার অভিজ্ঞতা থাকলে ভালো হয়। Docker Compose-এর পূর্ব অভিজ্ঞতা প্রয়োজন নেই — এই অধ্যায়ে Compose-এর Service Discovery শুধু নীতিগতভাবে পরিচয় করানো হবে।

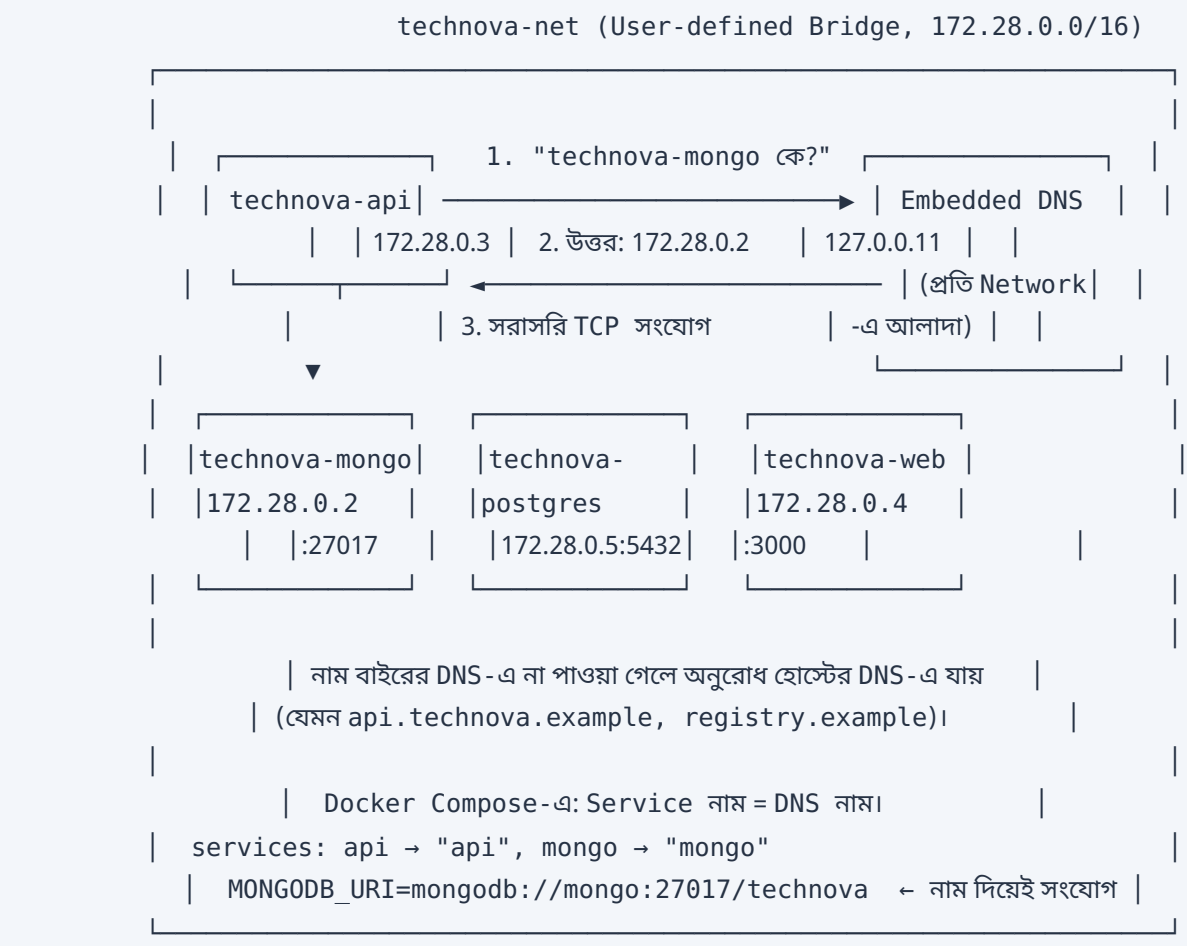
১৪.৪ প্রথম নীতি থেকে ব্যাখ্যা

মানুষ নাম মনে রাখে, কম্পিউটার ঠিকানা মনে রাখে। আপনি যখন ব্রাউজারে `app.technova.example` লেখেন, DNS সেই নামকে IP-তে বদলে দেয়। Container-দের জগতেও একই প্রয়োজন: API-কে MongoDB-র সঙ্গে কথা বলতে হলে `technova-mongo` নামটির IP জানতে হয়। কিন্তু Container-এর IP স্থায়ী নয় — Container মুছে আবার তৈরি হলে IP বদলে যায়। তাই Docker একটি অভ্যন্তরীণ ফোন-বই (Phone Book) রাখে: প্রতিটি User-defined Network-এর জন্য একটি **Embedded DNS Server** (127.0.0.11) থাকে, যা জানে এই Network-এ কোন নামের Container-এর বর্তমান IP কী। একটি Container যখন `technova-mongo` নামটি খোঁজে, তখন তার DNS অনুরোধ প্রথমে যায় 127.0.0.11-এ; সার্ভারটি দেখে নামটি এই Network-এর কোনো Container-এর কি না — হলে তার বর্তমান IP ফেরত দেয়; না হলে অনুরোধটি হোস্টের DNS-এ (বাইরের DNS) পাঠিয়ে দেয়। এই ব্যবস্থার নাম **Service Discovery** — সার্ভিসগুলো

নিজেরাই পরস্পরকে খুঁজে পায়, কোনো হার্ডকোড করা IP ছাড়াই। Docker Compose এই একই ব্যবস্থার ওপরে একটি সুবিধা যোগ করে: Compose ফাইলে Service-এর নাম (যেমন `api`, `mongo`) দিলেই Compose সেই নামে Network তৈরি করে ও প্রতিটি Service-কে সেই নামে নিবন্ধন করে — ফলে `mongodb://mongo:27017` লিখলেই API Mongo-কে খুঁজে পায়।

> **CONCEPT:** Embedded DNS-কে একটি ভবনের রিসেপশন ভাবুন। আপনি "মি. মঙ্গো" বলে জিজ্ঞেস করলে রিসেপশনিস্ট তার বর্তমান রুম-নম্বর বলে দেয় — রুম বদলালেও নাম একই থাকে। কিন্তু ডিফল্ট Bridge-এ কোনো রিসেপশনিস্ট নেই — সেখানে শুধু রুম-নম্বর (IP) দিয়ে খুঁজতে হয়, আর রুম বদলালে ঠিকানাটিও বদলে যায়। এজন্যই প্রোডাকশনে নাম ব্যবহার করা বাধ্যতামূলক।

১৪.৫ আর্কিটেকচার / ধারণাগত চিত্র



DNS Lookup প্রথমে ভেতরের ফোন-বইয়ে, না পাওয়া গেলে বাইরে — এই ক্রমটি বোঝা DNS সমস্যার সমাধানের চাবিকাঠি।

১৪.৬ বিস্তারিত প্রযুক্তিগত ব্যাখ্যা

Embedded DNS — ভেতরের কাজ

প্রতিটি Container-এর `/etc/resolv.conf`-এ লেখা থাকে `nameserver 127.0.0.11`। এটি কোনো বাইরের সার্ভার নয় — Docker Engine-এর ভেতরে চলা একটি DNS সার্ভার, যা প্রতিটি Container-এর Network Namespace-এ 127.0.0.11 ঠিকানায় পৌঁছানো যায়। যখন একটি Container কোনো নাম খোঁজে, তখন এই সার্ভার প্রথমে দেখে নামটি একই Network-এর কোনো Container, Network Alias, বা Service নাম কি না। হলে, সে Container-এর বর্তমান IP ফেরত দেয় — একাধিক Container একই নামে (যেমন একই Service-এর একাধিক Replica) থাকলে একাধিক IP ফেরত দেয়, আর ক্লায়েন্টটি একটি বেছে নেয় (Round-Robin-এর মতো আচরণ)। না হলে, অনুরোধটি হোস্টের কনফিগার করা DNS সার্ভারে (যেমন হোস্টের `/etc/resolv.conf` থেকে নেওয়া) পাঠানো হয়। এই কারণে একই Network-এর বাইরের নাম (যেমন `google.com`) ও ভেতরের নাম — দুটোই একই DNS দিয়ে কাজ করে।

নাম Resolution-এর তিনটি উৎস

উৎস	কোথায় থাকে	কী করে	কখন ব্যবহার করবেন
Embedded DNS (127.0.0.11)	প্রতিটি Container-এর resolv.conf	একই Network-এর Container/ Service-এর নাম → IP	সব Container-সংযোগে — ডিফল্ট পদ্ধতি
/etc/hosts	Container-এর ভেতরে	নাম → IP-র স্থির তালিকা (Docker নিজে Container-এর নাম লেখে)	সাধারণত হাত দেবেন না; পরীক্ষায় `--add-host` দিয়ে যোগ করা যায়
বাইরের DNS	হোস্টের DNS সার্ভার	ইন্টারনেটের নাম → IP	বাইরের API, Registry, প্যাকেজ সার্ভার

মনে রাখবেন: `/etc/hosts`-এ Docker নিজে Container-এর নিজের নাম ও তার Alias লেখে, কিন্তু অন্য Container-এর নাম লেখে না — অন্য Container-কে খুঁজতে Embedded DNS-ই ব্যবহৃত হয়। `--add-host mydb:203.0.113.20` দিয়ে আপনি নিজে একটি নাম-ঠিকানা যোগ করতে পারেন — এটি হোস্টের বাইরের কোনো সার্ভারকে নাম দিয়ে ডাকতে কাজে লাগে, তবে Container-এর নামের জন্য ব্যবহার করবেন না।

Docker Compose-এ Service Discovery — সেটআপ

Docker Compose-এ আপনি Service-গুলোকে নাম দেন — যেমন `web`, `api`, `mongo`, `postgres`। Compose চালু হলে সে একটি User-defined Network বানায় (নাম সাধারণত প্রজেক্টের নাম দিয়ে, যেমন `technova_default`) ও প্রতিটি Service-কে সেই Network-এ যুক্ত করে। তখন প্রতিটি Service-এর নামই একটি DNS নাম হয়ে যায়: `api` Service-এর Container-গুলোকে `api` নামে খুঁজে পাওয়া যায়, `mongo`-কে `mongo` নামে। API-র Connection String হয় `mongodb://mongo:27017/technova` — এখানে `mongo` কোনো হোস্টনাম নয়, Compose Service-এর নাম। একইভাবে Frontend থেকে API-কে ডাকা হয় `http://api:4000` দিয়ে — ব্রাউজার থেকে নয়, Frontend Container-এর ভেতর থেকে (Server-side)। সতর্কতা: ব্রাউজারে চলা JavaScript `http://api:4000` দেখতে পায় না — ব্রাউজার ব্যবহারকারীর মেশিনে চলে, Container Network-এ নয়; তাই ব্রাউজারের জন্য Published URL (`https://api.technova.example`) ব্যবহার করতে হয়। এই পার্থক্যটি Compose-এর সবচেয়ে সাধারণ ভুলের কারণ। Compose-এর পূর্ণ ব্যাখ্যা — Compose ফাইলের গঠন, Volume, Healthcheck, `depends_on` — পরের Part-এর Compose অধ্যায়ে আসবে; এখানে শুধু আবিষ্কারের নীতিটি শিখলেন।

Startup ক্রমের সমস্যা: "Database এখনো তৈরি নয়"

একটি সাধারণ ব্যর্থতা: API Container MongoDB-র আগেই চালু হয়ে সংযোগ করতে গিয়ে ব্যর্থ হয়, আর থেমে যায়। Container-এর নাম Resolution সফল হলেও Database-টি তখনো প্রস্তুত নয় — DNS ঠিক আছে, কিন্তু সার্ভিসটি এখনো শুনছে না। এই সমস্যার নাম **Startup Ordering** সমস্যা। সমাধান তিনটি: (১) **Retry:** API-র সংযোগ-কোডে পুনরায় চেষ্টার যুক্তি রাখা — ব্যর্থ হলে কয়েক সেকেন্ড অপেক্ষা করে আবার চেষ্টা করা; (২) **Healthcheck:** Database Container-এ Healthcheck দিয়ে তার প্রস্তুতি পরীক্ষা করা, আর API-কে সেই Healthcheck-এর ওপর নির্ভরশীল করা (Compose-এ `depends_on: condition: service_healthy`); (৩) **Wait Script:** API চালুর আগে একটি ছোট স্ক্রিপ্ট (যেমন `wait-for-it.sh`) দিয়ে Database-র Port খোলা কি না পরীক্ষা করা। `depends_on` একা যথেষ্ট নয় — এটি শুধু Container চালুর ক্রম ঠিক করে, Database-র ভেতরে প্রস্তুতি নিশ্চিত করে না।

একাধিক Network ও Alias — নামের পরিসর

একটি Container একাধিক Network-এ থাকলে প্রতিটি Network-এ তার নাম আলাদাভাবে নিবন্ধিত হয়। যেমন API `technova-backend` ও `technova-db` — দুই Network-এ থাকলে backend Network-এর Container-গুলো তাকে `technova-api` নামে খুঁজে পায়, আর db Network-এর Container-গুলোও একই নামে পায় — কিন্তু Frontend Network-এর Container পায় না, কারণ API সেই Network-এ নেই। **Network Alias** দিয়ে একটি Container-কে একাধিক নাম দেওয়া যায়: যেমন একই API-কে `api`, `api-v1` ও `backend` — তিন নামেই ডাকা যায়। Alias বিশেষভাবে কাজে লাগে যখন একটি Service-এর নতুন সংস্করণ পুরনো নামেই চালাতে হয়, অথবা একই Image-কে ভিন্ন ভিন্ন নামে ডাকতে হয়।

১৪.৭ Docker কমান্ড — লাইন-বাই-লাইন বিশ্লেষণ

```
$ docker exec technova-api cat /etc/resolv.conf
# cat /etc/resolv.conf — Container-এর DNS সেটিং দেখো।
# প্রত্যাশিত: nameserver 127.0.0.11 (Embedded DNS) ও search ডোমেইন।

$ docker exec technova-api getent hosts technova-mongo
# getent hosts technova-mongo — নামটির IP বের করো (DNS পরীক্ষা)।
# সফল হলে: 172.28.0.2  technova-mongo। ব্যর্থ হলে: কোনো আউটপুট নয়।

$ docker exec technova-api nslookup technova-mongo
# nslookup — DNS সার্ভার কোনটি উত্তর দিচ্ছে, বিস্তারিতসহ দেখো।
# Server: 127.0.0.11 দেখা যাওয়া উচিত।

$ docker exec technova-api ping -c 2 technova-mongo
# ping -c 2 — নাম দিয়ে দুটি প্যাকেট পাঠিয়ে সংযোগ পরীক্ষা করো।
# নাম Resolution ও নেটওয়ার্ক পথ — দুটোই একসঙ্গে পরীক্ষা হয়।

$ docker exec technova-api curl -s http://technova-mongo:27017/ | head -c 200
# curl — MongoDB-র Port-এ HTTP অনুরোধ পাঠিয়ে দেখো সার্ভিসটি শুনছে কি না।
# MongoDB HTTP নয়, তাই উত্তর অদ্ভুত হতে পারে — তবে সংযোগ স্থাপিত হওয়াই প্রমাণ।

$ docker run -d --name api --network technova-net \
    --add-host legacy-db:203.0.113.20 technova-api:1.0
# --add-host legacy-db:203.0.113.20 — Container-এর /etc/hosts-এ একটি
# নিজের নাম-ঠিকানা যোগ করো (হোস্টের বাইরের পুরনো সার্ভারের জন্য)।

$ docker run -d --name api --network technova-net \
    --dns 1.1.1.1 --dns-search technova.example technova-api:1.0
# --dns 1.1.1.1 — বাইরের DNS সার্ভার নিজে ঠিক করে দাও (ডিফল্ট: হোস্টের DNS)।
```

```
# --dns-search technova.example — ছোট নামের সঙ্গে এই ডোমেইন জুড়ে খোঁজো।

$ docker exec technova-api cat /etc/hosts
# cat /etc/hosts — Container-এর নিজের নাম ও Alias-এর তালিকা দেখো।
```

১৪.৮ সম্পূর্ণ কনফিগারেশন উদাহরণ

Service Discovery-সহ একটি সম্পূর্ণ Compose সেটআপ — এই উদাহরণটি শুধু আবিষ্কারের নীতিটি দেখায়; Compose ফাইলের পূর্ণ ব্যাখ্যা পরের Part-এ আসবে:

```
# docker-compose.yml — TechNova (Service Discovery-র উদাহরণ)
services:
  mongo:
    image: mongo:7
    container_name: technova-mongo
    environment:
      MONGO_INITDB_ROOT_USERNAME: admin
      MONGO_INITDB_ROOT_PASSWORD: CHANGE-ME-MONGO-DB-PASS
    volumes:
      - mongo-data:/data/db
    networks:
      - db-net
    healthcheck:
      # mongosh দিয়ে Database প্রস্তুত কি না পরীক্ষা করো
      test: ["CMD", "mongosh", "--quiet", "--eval", "db.adminCommand('ping')"]
      interval: 10s
      timeout: 5s
      retries: 5

  postgres:
    image: postgres:16
    container_name: technova-postgres
    environment:
      POSTGRES_USER: technova
      POSTGRES_PASSWORD: CHANGE-ME-POSTGRES-PASS
      POSTGRES_DB: technova
    volumes:
      - postgres-data:/var/lib/postgresql/data
    networks:
      - db-net
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U technova -d technova"]
      interval: 10s
      timeout: 5s
      retries: 5

  api:
    image: technova-api:1.0
    container_name: technova-api
    environment:
      # নাম দিয়েই সংযোগ — কোনো IP নেই
      MONGODB_URI: mongodb://admin:CHANGE-ME-MONGO-DB-PASS@mongo:27017/technova
      DATABASE_URL: postgres://technova:CHANGE-ME-POSTGRES-PASS@postgres:5432/technova
      PORT: "4000"
    networks:
      - backend-net
```

```
      - db-net
    depends_on:
      mongo:
        condition: service_healthy
      postgres:
        condition: service_healthy

  web:
    image: technova-web:1.0
    container_name: technova-web
    environment:
      # Server-side থেকে API-কে Service নামে ডাকা হয়
      NEXT_PUBLIC_API_URL: http://api:4000
    ports:
      - "8080:3000"
    networks:
      - frontend-net
      - backend-net
    depends_on:
      - api

volumes:
  mongo-data:
  postgres-data:

networks:
  frontend-net:
  backend-net:
  db-net:
    internal: true   # Database Network থেকে বাইরে যাওয়া বন্ধ
```

এই ফাইলে লক্ষ্য করুন: কোনো জায়গায় IP Address নেই — সব সংযোগ Service নাম দিয়ে। `depends_on` Healthcheck-এর সঙ্গে যুক্ত, তাই API চালু হয় Database প্রস্তুত হওয়ার পর। আর `db-net` `internal: true` — Database Container-গুলো বাইরের ইন্টারনেটে যেতেই পারে না।

১৪.৯ বাস্তব প্রোডাকশন পরিস্থিতি

REAL WORLD — TechNova: TechNova-র API-র একটি নতুন সংস্করণ Deploy করার পর দেখা গেল সব Request "MongoNetworkError: failed to connect to server" দিয়ে ব্যর্থ হচ্ছে। Container-গুলো সব চলছে, Network ঠিক আছে। তদন্তে দেখা গেল Connection String-এ লেখা ছিল `mongodb://mongo:27017` — কিন্তু নতুন Deploy-এ Container-এর নাম বদলে `technova-mongo-v2` করা হয়েছিল। নাম বদলানোয় DNS আর `mongo` নামটি খুঁজে পাচ্ছিল না। সমাধান: Connection String-এ Service-এর স্থির নাম ব্যবহার করা হলো, আর Container-এর নাম বদলানোর নিয়ম বাতিল করা হলো — নাম এখন Deploy-এর অংশ নয়, Service-এর পরিচয়। শিক্ষা: Connection String-এ সবসময় Service-এর স্থির নাম লিখুন, Container-এর অস্থায়ী নাম নয়।

আরেকটি দৃশ্য: API চালু হওয়ার প্রথম ৩০ সেকেন্ডে সব Request ব্যর্থ হচ্ছিল, তারপর নিজে থেকেই ঠিক হয়ে যাচ্ছিল। কারণ: API MongoDB-র আগেই চালু হচ্ছিল, আর প্রথম সংযোগ ব্যর্থ হলে সে আর চেষ্টা করত না। সমাধান: API-র সংযোগ-কোডে Retry যুক্ত করা হলো (ব্যর্থ হলে ৫ সেকেন্ড অপেক্ষা করে আবার চেষ্টা, মোট ১০ বার) ও MongoDB-তে Healthcheck যোগ করা হলো। এরপর থেকে Deploy-এর সময় আর কোনো ব্যর্থ Request আসে না।

১৪.১০ ধাপে ধাপে ব্যবহারিক উদাহরণ

উদাহরণ: DNS ব্যর্থতা নিজে তৈরি করে ঠিক করা

1. ১৪.৮-এর Compose ফাইল দিয়ে স্ট্যাক চালান: `docker compose up -d`।
2. `docker exec technova-api getent hosts mongo` — IP আসা উচিত। `docker exec technova-api getent hosts postgres` — একইভাবে।
3. `docker exec technova-web getent hosts mongo` — ব্যর্থ হওয়া উচিত (Web `db-net`-এ নেই)। এই ব্যর্থতাটিই নকশার প্রমাণ।
4. `docker exec technova-api cat /etc/resolv.conf` ও `cat /etc/hosts` দেখে DNS সেটিং লিখুন।
5. MongoDB Container থামান (`docker stop technova-mongo`), আবার চালান (`docker start technova-mongo`), তারপর `getent hosts mongo` আবার চালান — IP বদলাতে পারে, নাম একই থাকে।
6. API-র Log দেখুন: `docker logs technova-api` — সংযোগ সফল হয়েছে কি না, Retry হয়েছে কি না।
7. পরীক্ষা শেষে `docker compose down -v` দিয়ে পরিষ্কার করুন (সতর্কতা: `-v` Volume-ও মুছে ফেলে — পরীক্ষার ডেটা হারাবে, প্রোডাকশনে ব্যবহার করবেন না)।

১৪.১১ সাধারণ ভুল ও ভুল ধারণা

TROUBLESHOOTING: "getent hosts mongo — কোনো আউটপুট নেই" — ধাপে ধাপে পরীক্ষা করুন: (১) দুটি Container একই Network-এ আছে কি না (`docker inspect`); (২) Network-টি User-defined কি না (ডিফল্ট Bridge-এ নাম কাজ করে না); (৩) নামের বানান — Container-এর নাম, Service-এর নাম, না Alias — কোনটি ব্যবহার করছেন; (৪) `/etc/resolv.conf`-এ 127.0.0.11 আছে কি না; (৫) Container-টি চালু হওয়ার পর Network-এ যুক্ত করা হয়েছে কি না (`docker network inspect`)।

- **ভুল:** Connection String-এ Container-এর IP হার্ডকোড করা — Container পুনরায় তৈরি হলে IP বদলায়।
- **ভুল:** ব্রাউজারের JavaScript থেকে `http://api:4000` ডাকা — ব্রাউজার Container Network-এ নেই; ব্রাউজারের জন্য Published URL ব্যবহার করুন।
- **ভুল:** `depends_on` একা ব্যবহার করে ভাবা Database প্রস্তুত — `depends_on` শুধু চালুর ক্রম দেয়, প্রস্তুতি নয়; Healthcheck দরকার।
- **ভুল:** API-র সংযোগ-কোডে Retry না রাখা — প্রথম সংযোগ ব্যর্থ হলেই অ্যাপ থেমে যায়।
- **ভুল:** ডিফল্ট Bridge-এ নাম দিয়ে সংযোগের চেষ্টা — সেখানে DNS কাজ করে না।
- **ভুল ধারণা:** "DNS ঠিক থাকলেই সংযোগ হবে।" — DNS শুধু নাম থেকে IP বের করে; সার্ভিসটি শুনছে কি না, Port খোলা কি না, Firewall আটকাচ্ছে কি না — সেগুলো আলাদা পরীক্ষা।
- **ভুল ধারণা:** "Compose Service নাম আর Container নাম একই জিনিস।" — Service নাম স্থির পরিচয়; Container নাম বদলাতে পারে।

১৪.১২ নিরাপত্তা বিবেচনা

> **WARNING:** Service Discovery-র সুবিধা যেন নিরাপত্তার ফাঁক না হয়: একটি Container শুধু সেই Network-এই থাকবে যা তার কাজের জন্য দরকার — Frontend-কে Database Network-এ যুক্ত করবেন না, কারণ তাতে Frontend ভেদ হলে আক্রমণকারী Database-র নাম Resolution করতে পারবে। DNS নামগুলো অভ্যন্তরীণ — সেগুলো কখনো Public DNS-এ প্রকাশ করবেন না, আর Connection String-এর পাসওয়ার্ড কখনো Log বা Error Message-এ ছাপাবেন না।

আরও একটি নিরাপত্তা-বিষয়: `--add-host` দিয়ে হোস্টের বাইরের সার্ভারের নাম যোগ করলে সেই নামটি Container-এর ভেতরে স্থির হয়ে যায় — হোস্টের IP বদলালে Container-টি ভুল ঠিকানায় যাবে। বাইরের সার্ভারের জন্য DNS নাম ব্যবহার করুন, `--add-host` নয় — শুধু পরীক্ষা বা জরুরি ক্ষেত্রে `--add-host` ব্যবহার করুন।

১৪.১৩ কর্মক্ষমতা ও রিসোর্স বিবেচনা

Embedded DNS Lookup দ্রুত — সাধারণত এক মিলিসেকেন্ডের কম — তবে প্রতিটি নতুন সংযোগে DNS Lookup হয়। সংযোগ-পুল (Connection Pool) ব্যবহার করলে Lookup-এর সংখ্যা কমে: API Database-র সঙ্গে একটি স্থায়ী পুল খুলে রাখে, প্রতিটি Request-এ নতুন সংযোগ খোলে না। DNS Cache-এর দিকে নজর দিন: Node.js-এর DNS Lookup অপারেটিং সিস্টেমের Cache ব্যবহার করে, তবে Container পুনরায় তৈরি হলে Cache মুছে যায় — তাতে সমস্যা নেই, কারণ নতুন IP-টি তখনই DNS থেকে আসে। একটি সার্ভিসের অনেক Replica থাকলে Embedded DNS একাধিক IP ফেরত দেয় — ক্লায়েন্টটি সাধারণত প্রথমটি ব্যবহার করে, ফলে লোড সব Replica-তে সমানভাবে ভাগ হয় না; সত্যিকারের Load Balancing-এর জন্য Reverse Proxy (Nginx) বা Orchestrator-এর সেবা দরকার।

১৪.১৪ সমস্যা সমাধান পদ্ধতি

1. **নাম Resolution:** `docker exec <src> getent hosts <dst>` — ব্যর্থ হলে Network, নামের বানান ও User-defined Bridge পরীক্ষা করুন।
2. **DNS সার্ভার:** `docker exec <src> cat /etc/resolv.conf` — 127.0.0.11 আছে কি না দেখুন।
3. **Port শুনছে কি না:** `docker exec <dst> ss -tlnp` — সার্ভিসটি `0.0.0.0:<port>`-এ শুনছে কি না।
4. **সংযোগ পরীক্ষা:** `docker exec <src> curl -v http://<dst>:<port>/health` বা `nc -zv <dst> <port>`।
5. **বাইরের নাম:** `docker exec <src> getent hosts registry.example` — বাইরের নাম না মিললে হোস্টের DNS পরীক্ষা করুন (`cat /etc/resolv.conf` হোস্টে)।
6. **Startup ক্রম:** `docker logs <api>`-এ সংযোগ-ব্যর্থতার সময় দেখুন — Database-র Log-এর সময়ের সঙ্গে মেলান।
7. **Healthcheck:** `docker inspect <db> --format '{{json .State.Health}}'` — Database সুস্থ কি না দেখুন।

১৪.১৫ সেরা অনুশীলন ও অ্যান্টি-প্যাটার্ন

সেরা অনুশীলন: সব সংযোগ নাম দিয়ে, IP দিয়ে নয়; Service-এর স্থির নাম Connection String-এ ব্যবহার করুন; Database-এ Healthcheck দিন ও API-কে তার ওপর নির্ভরশীল করুন; সংযোগ-কোডে Retry ও Timeout রাখুন;

ব্রাউজারের জন্য Published URL, Server-side-এর জন্য Service নাম — এই পার্থক্য মনে রাখুন; প্রতিটি সার্ভিস শুধু প্রয়োজনীয় Network-এ রাখুন। **অ্যান্টি-প্যাটার্ন:** IP হার্ডকোড করা; `depends_on`-কে প্রস্তুতির প্রমাণ ভাবা; Retry ছাড়া সংযোগ-কোড; ব্রাউজার থেকে অভ্যন্তরীণ Service নাম ডাকা; সব সার্ভিস এক Network-এ রাখা।

১৪.১৬ অধ্যায়ের সারাংশ

Container-দের মধ্যে যোগাযোগের মূলে আছে নাম — IP নয়। Embedded DNS (127.0.0.11) User-defined Network-এ Container ও Service-এর নাম থেকে বর্তমান IP বের করে দেয়; Container পুনরায় তৈরি হলে IP বদলালেও নাম একই থাকে। Docker Compose-এ Service-এর নামই DNS নাম হয় — `mongodb://mongo:27017`-এর `mongo` একটি Service নাম। কিন্তু DNS ঠিক থাকলেই সংযোগ হয় না — সার্ভিসটি প্রস্তুত থাকতে হয়, তাই Healthcheck, `depends_on` (service_healthy) ও Retry — তিনটি একসঙ্গে দরকার। আর মনে রাখবেন: ব্রাউজার Container Network-এ নেই, তাই ব্রাউজারের জন্য Published URL ব্যবহার করতে হয়।

১৪.১৭ মূল প্রযুক্তিগত পরিভাষা

Service Discovery: সার্ভিসগুলো নাম দিয়ে পরস্পরকে খুঁজে পাওয়ার ব্যবস্থা — কোনো হার্ডকোড করা IP ছাড়াই।

Embedded DNS: Docker-এর ভেতরের DNS সার্ভার (127.0.0.11) যা একই Network-এর Container/Service-এর নাম থেকে IP বের করে।

Service Name: Docker Compose-এ একটি Service-এর নাম — যা একই সঙ্গে তার DNS নাম হয়।

Healthcheck: Container-এর ভেতরে নিয়মিত চালানো পরীক্ষা যা জানায় সার্ভিসটি আসলে প্রস্তুত কি না।

depends_on: Compose-এর নির্দেশ যা Service চালুর ক্রম ঠিক করে; Healthcheck-এর সঙ্গে যুক্ত হলে প্রস্তুতির অপেক্ষাও করে।

Startup Ordering: সার্ভিসগুলো কোন ক্রমে চালু হয় ও প্রস্তুত হয় — তার সমস্যা ও সমাধান।

Retry: ব্যর্থ সংযোগের পর অপেক্ষা করে আবার চেষ্টা করার যুক্তি।

Connection Pool: Database-র সঙ্গে আগে থেকে খোলা সংযোগের দল, যা প্রতিটি Request-এ নতুন সংযোগ খোলার খরচ বাঁচায়।

/etc/resolv.conf: একটি সিস্টেমের DNS সেটিংয়ের ফাইল — Container-এ এখানে 127.0.0.11 লেখা থাকে।

--add-host: Container-এর /etc/hosts-এ নিজের নাম-ঠিকানা যোগ করার `docker run` বিকল্প।

১৪.১৮ জ্ঞান যাচাই — প্রশ্ন

1. Embedded DNS কীভাবে কাজ করে — একটি API `mongo` নামটি খুঁজলে ভেতরে কী ঘটে, ধাপে ধাপে ব্যাখ্যা করুন।
2. ডিফল্ট Bridge-এ নাম দিয়ে সংযোগ হয় না কেন, অথচ User-defined Bridge-এ হয়?
3. Docker Compose-এ `mongodb://mongo:27017`-এর `mongo` আসলে কী? এটি কি একটি হোস্টনাম?
4. ব্রাউজারের JavaScript থেকে `http://api:4000` ডাকা যায় না কেন? সঠিক পদ্ধতি কী?
5. `depends_on` একা কেন Database-র প্রস্তুতি নিশ্চিত করে না? কী যোগ করতে হয়?
6. Connection String-এ IP হার্ডকোড করার ঝুঁকি কী?
7. একটি Container একাধিক Network-এ থাকলে তার নাম কীভাবে নিবন্ধিত হয়?

১৪.১৯ ব্যবহারিক অনুশীলন

1. **DNS পরীক্ষা:** ১৪.১০-এর ধাপগুলো অনুসরণ করে DNS ব্যর্থতা তৈরি করে ঠিক করুন ও প্রতিটি ধাপের ফলাফল লিখুন।
2. **Retry যোগ করা:** একটি ছোট Node.js স্ক্রিপ্ট লিখুন যা MongoDB-তে সংযোগ ব্যর্থ হলে ৫ সেকেন্ড অপেক্ষা করে ৫ বার চেষ্টা করে।
3. **Healthcheck লেখা:** PostgreSQL-এর জন্য `pg_isready` দিয়ে একটি Healthcheck লিখুন ও `docker inspect` দিয়ে তার অবস্থা দেখুন।
4. **নাম বনাম IP:** একটি Container মুছে আবার তৈরি করে দেখুন তার IP বদলেছে কি না, আর নাম দিয়ে সংযোগ আগের মতোই কাজ করে কি না।

অধ্যায় 15 • PART III — Container Networking and Persistent Storage

Chapter 15 — Volumes, Bind Mounts and Persistent Storage

১৫.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: (১) Container-এর Writable Layer কেন ক্ষণস্থায়ী (Ephemeral) তা ব্যাখ্যা করতে — এবং কেন গুরুত্বপূর্ণ ডেটা সেখানে রাখা যাবে না; (২) Named Volume, Anonymous Volume, Bind Mount ও Tmpfs Mount-এর পার্থক্য, সুবিধা ও ঝুঁকি তুলনা করতে; (৩) MongoDB ও PostgreSQL-এর ডেটা Volume-এ স্থায়ীভাবে রাখতে; (৪) Volume-এর পুরো জীবনচক্র — তৈরি, যুক্ত করা, পরিদর্শন, Backup, মুছে ফেলা — পরিচালনা করতে; (৫) ফাইল মালিকানা (Ownership) ও অনুমতি (Permission)-সংক্রান্ত সমস্যা নির্ণয় ও সমাধান করতে; (৬) Development-এ Bind Mount ও Production-এ Named Volume — কখন কোনটি — সিদ্ধান্ত নিতে; (৭) ডেটা Migration-এর পরিকল্পনা করতে।

১৫.২ কেন এই বিষয়টি DevOps-এ গুরুত্বপূর্ণ

Container-এর সবচেয়ে বিপজ্জনক ভুল ধারণাটি হলো: "Container-এর ভেতরে ফাইল লিখলেই তা থেকে যাবে।" থাকবে না। Container মুছে ফেললেই তার ভেতরের সব লেখা মুছে যায় — Database-র সব রেকর্ড, ব্যবহারকারীর আপলোড করা সব ছবি, সবকিছু। TechNova-র এক সহকর্মী একবার Database Container-টি "আপডেট করার জন্য" মুছে ফেলেছিলেন — Volume ছাড়া চালানো হয়েছিল বলে Staging-এর সব পরীক্ষার ডেটা এক মুহূর্তে হারিয়ে গিয়েছিল। এই অধ্যায়ের মূল শিক্ষা এক লাইনে: **Container ক্ষণস্থায়ী, ডেটা স্থায়ী — তাই ডেটাকে Container-এর বাইরে রাখতে হয়।** Volume, Bind Mount ও Tmpfs Mount — এই তিনটি উপায়ের মধ্যে কোনটি কখন ব্যবহার করতে হয়, তা না জানলে প্রোডাকশনে ডেটা হারানো শুধু সময়ের ব্যাপার।

১৫.৩ পূর্বশর্ত জ্ঞান

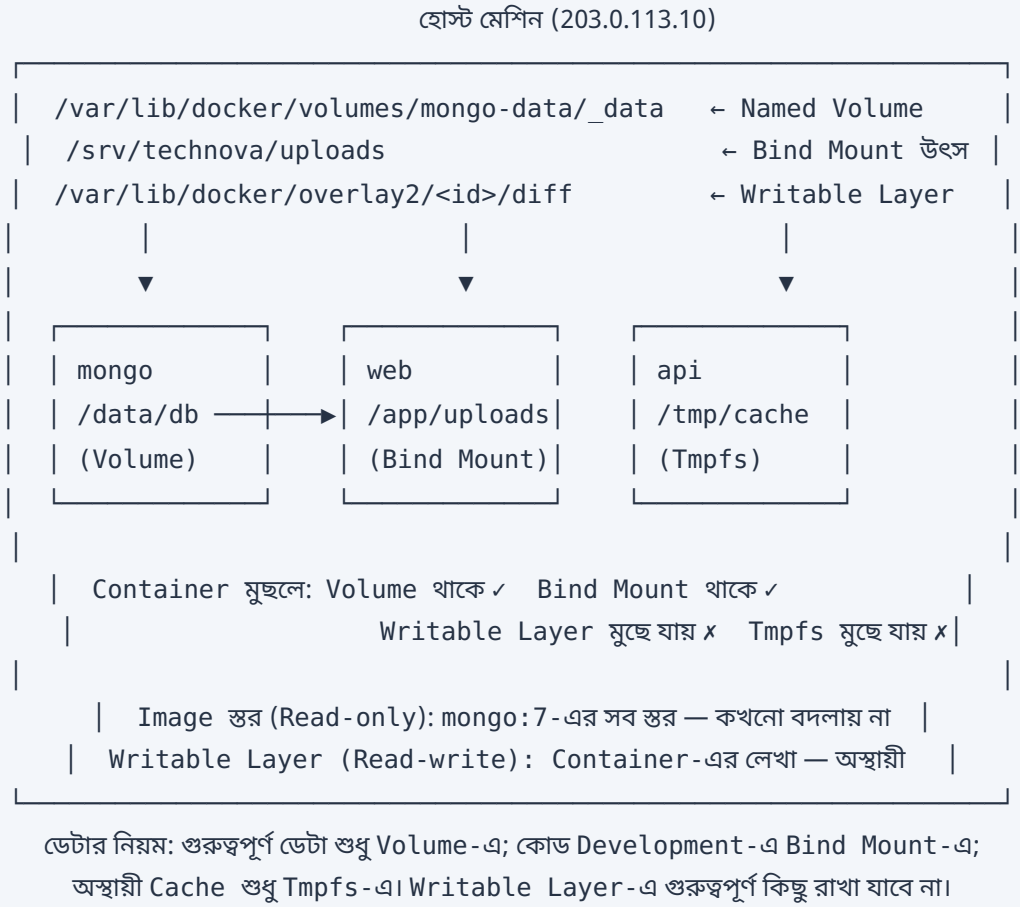
Linux ফাইলসিস্টেমের ধারণা (Path, Mount, Permission, Owner), `docker run -v`-এর মৌলিক ব্যবহার, MongoDB ও PostgreSQL-এর ডেটা কোথায় থাকে (`/data/db` ও `/var/lib/postgresql/data`) — এইটুকু জানা থাকলেই চলবে। OverlayFS বা Copy-on-Write-এর পূর্ব জ্ঞান প্রয়োজন নেই — এই অধ্যায়ে প্রথম নীতি থেকে ব্যাখ্যা করা হবে। অধ্যায় ১২–১৪-এর Network ধারণা জানা থাকলে উদাহরণগুলো বুঝতে সুবিধা হবে।

১৫.৪ প্রথম নীতি থেকে ব্যাখ্যা

একটি Docker Image হলো শুধু-পড়ার (Read-only) স্তরগুলোর স্তূপ — প্রতিটি `RUN` বা `COPY` নির্দেশ একটি স্তর যোগ করে। Container চালু হলে Image-এর ওপরে একটি পাতলা **Writable Layer** বসে — Container-এর ভেতরে আপনি যা লেখেন, বদলান বা মুছেন, সব এই স্তরেই ঘটে। এই স্তরটি Container-এর সঙ্গেই জন্মায় ও মরে: Container মুছে ফেললে Writable Layer-ও মুছে যায়। এই ব্যবস্থার নাম **Copy-on-Write**: কোনো ফাইল বদলাতে চাইলে প্রথমে তার একটি কপি Writable Layer-এ তোলা হয়, তারপর বদলানো হয় — মূল Image অপরিবর্তিত থাকে। এই নকশা Container-কে দ্রুত ও হালকা করে, কিন্তু ডেটা সংরক্ষণের জন্য এটি ভয়াবহ: Database-র ডেটা Writable Layer-এ থাকলে Container মুছলেই ডেটা শেষ। তাই Docker তিনটি পথ দেয় ডেটাকে Container-এর জীবন থেকে আলাদা করার জন্য: **Volume** — Docker-এর পরিচালিত হোস্টের একটি ডিরেক্টরি (`/var/lib/docker/volumes/`), যা Container মুছলেও থেকে যায়; **Bind Mount** — হোস্টের যেকোনো ফাইল বা ডিরেক্টরিকে Container-এর ভেতরে বসানো; **Tmpfs Mount** — শুধু মেমরিতে থাকা অস্থায়ী সংরক্ষণ, যা Container থামলেই মুছে যায়।

CONCEPT: Writable Layer-কে হোটেলের ঘর ভাবুন — আপনি যতক্ষণ আছেন ততক্ষণ আপনার জিনিস ঘরে থাকে, কিন্তু চেক-আউট করলেই ঘর পরিষ্কার হয়ে যায়। Volume হলো হোটেলের লকার — চেক-আউট করলেও আপনার জিনিস লকারে থাকে, পরের বার এসে আবার নিতে পারেন। Bind Mount হলো নিজের বাড়ির আলমারি হোটেলের ঘরে এনে রাখা — জিনিস আপনার বাড়িতেই থাকে। Tmpfs হলো ঘরের টেবিলে রাখা কাগজ — ঘর ছাড়লেই উড়ে যায়।

১৫.৫ আর্কিটেকচার / ধারণাগত চিত্র



ডেটার নিয়ম: গুরুত্বপূর্ণ ডেটা শুধু `Volume`-এ; কোড `Development`-এ `Bind Mount`-এ; অস্থায়ী `Cache` শুধু `Tmpfs`-এ। `Writable Layer`-এ গুরুত্বপূর্ণ কিছু রাখা যাবে না।

Volume Docker-এর পরিচালনায় থাকে — হোস্টের পথটি আপনার জানার দরকার নেই; Bind Mount-এ হোস্টের পথটি আপনিই ঠিক করেন।

১৫.৬ বিস্তারিত প্রযুক্তিগত ব্যাখ্যা

Writable Layer ও Copy-on-Write — গভীরে

Container-এর ভেতরে একটি ফাইল পড়া হলে তা Image-এর স্তর থেকে পড়া হয়। ফাইলটি বদলাতে গেলে OverlayFS প্রথমে তার একটি কপি Writable Layer-এ তোলে, তারপর বদলায় — একে বলে **Copy-on-Write**। বড় ফাইল বদলালে পুরো ফাইলটি কপি হয়, তাই Writable Layer-এ বড় ফাইল লেখা ধীর ও জায়গা-খরুচে। আরেকটি সমস্যা: Writable Layer হোস্টের ফাইলসিস্টেমে OverlayFS-এর ভেতরে থাকে — হোস্ট থেকে সরাসরি সেখানে হাত দেওয়া কঠিন ও বিপজ্জনক।

এই কারণে Database-র মতো ক্রমাগত লেখা হয় এমন ডেটা Writable Layer-এ রাখা দ্বিগুণ ভুল: ধীরও বটে, অস্থায়ীও বটে। Volume ব্যবহার করলে লেখা সরাসরি হোস্টের ফাইলসিস্টেমে যায় — দ্রুততর ও স্থায়ী।

চারটি সংরক্ষণ-পদ্ধতির তুলনা

বিষয়	Named Volume	Anonymous Volume	Bind Mount	Tmpfs Mount
তৈরি হয়	`docker volume create` বা `-v নাম:/পথ`	`-v /পথ` (নাম ছাড়া)	`-v /হোস্ট/পথ:/কন্টেইনার/পথ`	`--tmpfs /পথ`
কোথায় থাকে	Docker-এর পরিচালিত এলাকা	Docker-এর পরিচালিত এলাকা	হোস্টের যেকোনো পথ	শুধু মেমরি (RAM)
Container মুছলে	থাকে ✓	থাকে, কিন্তু নাম না থাকায় খুঁজে পাওয়া কঠিন	থাকে ✓	মুছে যায় ✗
সেরা ব্যবহার	Database ডেটা, প্রোডাকশন	এড়িয়ে চলুন	Development-এ কোড, কনফিগ ফাইল	অস্থায়ী Cache, Secret-এর মতো সংবেদনশীল অস্থায়ী ডেটা
ঝুঁকি	ভুলে মুছে ফেলা (`volume rm`)	অপ্রয়োজনীয় Volume জমে যাওয়া	হোস্টের পথের ওপর নির্ভরতা, Permission সমস্যা	মেমরি ভরে যাওয়া

Anonymous Volume এড়িয়ে চলুন: নাম না থাকায় কোন Volume কোন Container-এর তা বোঝা যায় না, আর `docker volume prune` চালালে সেগুলো মুছে যেতে পারে। সবসময় নাম দিন।

Database Persistence — MongoDB ও PostgreSQL

MongoDB তার ডেটা রাখে `/data/db`-তে, PostgreSQL রাখে `/var/lib/postgresql/data`-তে। এই দুটি পথে Volume না দিলে Container মুছলেই সব ডেটা হারাবে। সঠিক পদ্ধতি: `-v mongo-data:/data/db` ও `-v postgres-data:/var/lib/postgresql/data`। প্রথমবার চালু হলে Docker খালি Volume-টিকে Image-এর ওই পথের বিষয়বস্তু দিয়ে ভরে দেয় (Population) — তাই Database-র প্রাথমিক ফাইলগুলো ঠিকমতো তৈরি হয়। কিন্তু সতর্কতা: Volume-এ ইতিমধ্যে ডেটা থাকলে Image-এর বিষয়বস্তু আর কপি হয় না — অর্থাৎ নতুন Image-এর ডিফল্ট ডেটা পুরনো Volume-এর ওপর চাপিয়ে দেওয়া হয় না। আরেকটি সতর্কতা: PostgreSQL-এর Volume-টি খালি না থাকলে তার Initialization (প্রথমবার User ও Database তৈরি) আর চলে না — তাই Environment Variable বদলে আবার চালু করলে তা কাজ করবে না, যদি না Volume মুছে নতুন করে শুরু করেন।

Read-only Mount ও মালিকানা (Ownership)

Volume বা Bind Mount-কে শুধু-পড়ার (Read-only) করা যায়: `-v mongo-data:/data/db:ro` — এতে Container ডেটা পড়তে পারবে, বদলাতে পারবে না। কনফিগ ফাইল বা TLS Certificate Mount করার সময় `:ro` ব্যবহার করা নিরাপত্তার জন্য ভালো। **মালিকানার সমস্যা:** Container-এর ভেতরে প্রক্রিয়াটি যে User ID (UID) দিয়ে চলে, হোস্টের ফাইলের মালিকানা সেই UID-র সঙ্গে মিলতে হবে। যেমন PostgreSQL Container-এর ভেতরে `postgres` User (UID 999) ডেটা লেখে — হোস্টের Bind Mount ডিরেক্টরির মালিক যদি অন্য কেউ হয়, তবে "Permission denied" হবে। সমাধান: হোস্টে ডিরেক্টরির মালিকানা বদলে দিন (`sudo chown 999:999 /srv/technova/pgdata`) অথবা Container-

কে `--user` দিয়ে হোস্টের UID-তে চালান। Named Volume-এ এই সমস্যা কম হয়, কারণ Docker নিজে Volume-এর মালিকানা ঠিক করে দেয় — Bind Mount-এর এটি একটি বড় সুবিধা-অসুবিধার পার্থক্য।

Volume Lifecycle — জীবনচক্র

Volume-এর জীবন Container-এর জীবন থেকে আলাদা: (১) **তৈরি:** `docker volume create` বা প্রথম `docker run -v নাম:/পথ`-এ স্বয়ংক্রিয়ভাবে; (২) **যুক্ত:** Container চালুর সময় `-v` বা `--mount` দিয়ে; (৩) **ব্যবহার:** Container ডেটা লেখে, Volume-টি হোস্টে বাড়তে থাকে; (৪) **পরিদর্শন:** `docker volume inspect` দিয়ে হোস্টের পথ ও Driver দেখা; (৫) **Backup:** Volume-এর বিষয়বস্তু অন্য Container দিয়ে কপি করা (অধ্যায় ১৬); (৬) **মুছে ফেলা:** `docker volume rm` — শুধু তখনই, যখন কোনো Container তা ব্যবহার করছে না। `docker compose down -v` Volume-ও মুছে ফেলে — প্রোডাকশনে এই ফ্ল্যাগটি ভুলে ব্যবহার করলে ডেটা হারাবে।

Bind Mount বনাম Volume — কখন কোনটি

Development-এ Bind Mount শ্রেষ্ঠ: হোস্টে কোড বদলালেই Container-এর ভেতরে তা দেখা যায় — Hot Reload কাজ করে। যেমন `-v $(pwd)/src:/app/src` দিয়ে Next.js কোড Mount করলে ফাইল বাঁচালেই ব্রাউজারে পরিবর্তন দেখা যায়। Production-এ Named Volume শ্রেষ্ঠ: হোস্টের পথের ওপর নির্ভরতা নেই, Docker নিজে মালিকানা ও Backup পরিচালনা করে, আর হোস্টের ফাইলসিস্টেমের গঠন বদলালেও অ্যাপ চলে। কনফিগ ফাইলের জন্য Bind Mount ভালো: `-v /srv/technova/nginx.conf:/etc/nginx/nginx.conf:ro` — হোস্টে ফাইলটি সম্পাদনা করা যায়, Container-এ তা শুধু-পড়া হিসেবে দেখা যায়।

১৫.৭ Docker কমান্ড — লাইন-বাই-লাইন বিশ্লেষণ

```
$ docker volume create mongo-data
# docker volume create mongo-data — নাম-সহ একটি Volume বানাও।
# হোস্টে এর পথ হবে /var/lib/docker/volumes/mongo-data/_data।

$ docker volume ls
# docker volume ls — সব Volume-এর তালিকা: DRIVER (local) ও NAME।

$ docker volume inspect mongo-data
# docker volume inspect — Volume-এর Mountpoint (হোস্টের আসল পথ),
# Driver, তৈরি হওয়ার সময় ও Label দেখো।

$ docker run -d --name technova-mongo \
    -v mongo-data:/data/db mongo:7
# -v mongo-data:/data/db — mongo-data Volume-কে Container-এর /data/db
# পথে Mount করো। এই পথে যা লেখা হবে, সব Volume-এ যাবে।

$ docker run -d --name technova-mongo \
    --mount source=mongo-data,target=/data/db mongo:7
# --mount — -v-র চেয়ে স্পষ্ট (Verbose) রূপ; source=Volume-এর নাম,
# target=Container-এর ভেতরের পথ। ভুল পথ দিলে --mount স্পষ্ট Error দেয়।

$ docker run -d --name technova-web \
    -v $(pwd)/src:/app/src \
    -v /app/src/node_modules \
    technova-web:1.0
# -v $(pwd)/src:/app/src — হোস্টের src ডিরেক্টরি Bind Mount করো (Development)।
# -v /app/src/node_modules — Anonymous Volume দিয়ে node_modules-কে
# Bind Mount-এর আড়াল থেকে রক্ষা করো (নইলে হোস্টের খালি node_modules
# Container-এর ইনস্টল করা প্যাকেজ ঢেকে ফেলবে)।
```

```
$ docker run -d --name technova-api --tmpfs /tmp/cache technova-api:1.0
# --tmpfs /tmp/cache — /tmp/cache শুধু মেমরিতে রাখো; Container থামলেই মুছবে।

$ docker run -d --name technova-web \
    -v /srv/technova/nginx.conf:/etc/nginx/nginx.conf:ro nginx:alpine
# :ro — শুধু-পড়া Mount; Container কনফিগ ফাইল বদলাতে পারবে না।

$ docker volume rm mongo-data
# docker volume rm — Volume মুছে ফেলো। শুধু তখনই চলবে যখন কোনো
# Container তা ব্যবহার করছে না। মুছলে ভেতরের সব ডেটা চিরতরে হারাবে।

$ docker volume prune
# docker volume prune — কোনো Container ব্যবহার করছে না এমন সব Volume
# মুছে ফেলো। চালানোর আগে docker volume ls দিয়ে দেখে নিন কী মুছবে।
```

WARNING: `docker volume rm` ও `docker volume prune` — দুটোই ধ্বংসাত্মক (Destructive) কমান্ড। Volume মুছে ফেললে তার ভেতরের Database-র সব ডেটা, ব্যবহারকারীর সব আপলোড — সবকিছু চিরতরে হারিয়ে যাবে, কোনো Recycle Bin নেই। `docker compose down -v`-ও একই কাজ করে। যেকোনো Volume মুছে ফেলার আগে Backup নিন (অধ্যায় ১৬) ও `docker volume inspect` দিয়ে নিশ্চিত হোন এটি সঠিক Volume। প্রোডাকশনে `prune` চালানোর আগে দুবার ভাবুন।

১৫.৮ সম্পূর্ণ কনফিগারেশন উদাহরণ

TechNova-র সম্পূর্ণ Storage নকশা — Compose ফাইলে Volume, Bind Mount ও Tmpfs — তিনটি একসঙ্গে:

```
# docker-compose.yml — TechNova Storage নকশা
services:
  mongo:
    image: mongo:7
    container_name: technova-mongo
    environment:
      MONGO_INITDB_ROOT_USERNAME: admin
      MONGO_INITDB_ROOT_PASSWORD: CHANGE-ME-MONGO-DB-PASS
    volumes:
      # Named Volume — Database ডেটা স্থায়ীভাবে থাকবে
      - mongo-data:/data/db
      # Bind Mount (শুধু-পড়া) — Initialization স্ক্রিপ্ট
      - ./mongo-init:/docker-entrypoint-initdb.d:ro
    networks:
      - db-net

  postgres:
    image: postgres:16
    container_name: technova-postgres
    environment:
      POSTGRES_USER: technova
      POSTGRES_PASSWORD: CHANGE-ME-POSTGRES-PASS
      POSTGRES_DB: technova
    volumes:
      - postgres-data:/var/lib/postgresql/data
    networks:
      - db-net
```

```
  api:
    image: technova-api:1.0
    container_name: technova-api
    environment:
      MONGODB_URI: mongodb://admin:CHANGE-ME-MONGO-DB-PASS@mongo:27017/technova
      DATABASE_URL: postgres://technova:CHANGE-ME-POSTGRES-PASS@postgres:5432/technova
    volumes:
      # ব্যবহারকারীর আপলোড — Named Volume (স্থায়ী)
      - uploads-data:/app/uploads
      # অস্থায়ী Cache — Tmpfs (মেমরি, Container থামলেই মুছবে)
      - type: tmpfs
        target: /app/cache
        tmpfs:
          size: 104857600   # 100 MB
    networks:
      - backend-net
      - db-net

  web:
    image: technova-web:1.0
    container_name: technova-web
    ports:
      - "8080:3000"
    volumes:
      # Development-এ কোড Bind Mount (Production-এ এই লাইনটি সরিয়ে দিন)
      - ./web/src:/app/src
      - /app/src/node_modules
    networks:
      - frontend-net
      - backend-net

volumes:
  mongo-data:
    name: technova-mongo-data
  postgres-data:
    name: technova-postgres-data
  uploads-data:
    name: technova-uploads-data

networks:
  frontend-net:
  backend-net:
  db-net:
    internal: true
```

এই ফাইলে তিনটি Volume-এর নাম স্পষ্টভাবে দেওয়া হয়েছে (`name:`) — যাতে `docker volume ls`-এ সেগুলো সহজে চেনা যায়। আপলোডের Volume আলাদা রাখা হয়েছে, যাতে Database Backup আর আপলোড Backup আলাদাভাবে করা যায়।

১৫.৯ বাস্তব প্রোডাকশন পরিস্থিতি

> **REAL WORLD — TechNova:** TechNova-র Staging সার্ভারে একদিন MongoDB Container-টি নতুন Image-এ আপডেট করার জন্য মুছে ফেলা হলো — কিন্তু কেউ খেয়াল করেনি যে Container-টি Volume ছাড়া চালানো হয়েছিল। নতুন Container চালু হয়ে খালি Database দেখাল — তিন সপ্তাহের পরীক্ষার ডেটা হারিয়ে গেল। ঘটনার পর দলের নিয়ম হলো: Database Container চালুর আগে `docker inspect` দিয়ে যাচাই করা হবে Volume যুক্ত আছে কি না, আর প্রতিটি Deploy-এর আগে স্বয়ংক্রিয় Backup চলবে। আরও একটি শিক্ষা: `docker compose down` (`-v` ছাড়া) Volume মুছে না — কিন্তু `down -v` মুছে ফেলে। দলের Deploy স্ক্রিপ্ট থেকে `-v` ফ্ল্যাগটি চিরতরে সরিয়ে দেওয়া হলো।

আরেকটি দৃশ্য: Development-এ Bind Mount দিয়ে কোড চালানো হচ্ছিল, কিন্তু `node_modules` ঢেকে যাওয়ায় অ্যাপ চালু হচ্ছিল না — হোস্টের খালি `node_modules` ডিরেক্টরি Container-এর ইনস্টল করা প্যাকেজগুলোকে ঢেকে ফেলেছিল। সমাধান: ১৫.৭-এর মতো Anonymous Volume দিয়ে `/app/src/node_modules`-কে রক্ষা করা হলো। এই কৌশলটি Development-এ Bind Mount ব্যবহারের সময় প্রায় সবসময় দরকার হয়।

১৫.১০ ধাপে ধাপে ব্যবহারিক উদাহরণ

উদাহরণ: Volume ছাড়া ও Volume-সহ Database — পার্থক্য নিজে দেখা

1. Volume ছাড়া MongoDB চালান: `docker run -d --name mongo-test mongo:7`। ভেতরে একটি Database ও একটি Collection তৈরি করুন (`docker exec -it mongo-test mongosh`)।
2. Container মুছে আবার চালান:
   `docker rm -f mongo-test && docker run -d --name mongo-test mongo:7`। Collection-টি আর নেই — ডেটা হারিয়েছে।
3. এবার Volume-সহ চালান: `docker run -d --name mongo-test -v mongo-test-data:/data/db mongo:7`। আবার Collection তৈরি করুন।
4. Container মুছে আবার একই Volume দিয়ে চালান: `docker rm -f mongo-test && docker run -d --name mongo-test -v mongo-test-data:/data/db mongo:7`। Collection-টি এখনো আছে — ডেটা বেঁচে গেছে।
5. `docker volume inspect mongo-test-data` দিয়ে হোস্টের পথ দেখুন ও `sudo ls` দিয়ে ভেতরের ফাইল দেখুন।
6. পরীক্ষা শেষে Volume-টি মুছে ফেলুন — তবে আগে মনে রাখুন: `docker volume rm mongo-test-data` ডেটা চিরতরে মুছে ফেলবে।

১৫.১১ সাধারণ ভুল ও ভুল ধারণা

TROUBLESHOOTING: "Permission denied" — Database Container চালু হচ্ছে না: প্রথমে `docker logs <name>` দেখুন — PostgreSQL প্রায়ই বলে `data directory has wrong ownership`। হোস্টে Bind Mount ডিরেক্টরির মালিকানা পরীক্ষা করুন (`ls -ld /srv/technova/pgdata`) ও Container-এর User-এর UID-র সঙ্গে মেলান (`docker exec <name> id postgres`)। সমাধান: `sudo chown 999:999 /srv/technova/pgdata` অথবা Named Volume ব্যবহার করুন।

- **ভুল:** Database Volume ছাড়া চালানো — Container মুছলেই ডেটা হারানো।
- **ভুল:** `docker compose down -v` প্রোডাকশনে চালানো — Volume-সহ সব ডেটা মুছে যায়।
- **ভুল:** Anonymous Volume ব্যবহার করা — নাম না থাকায় পরে খুঁজে পাওয়া যায় না।
- **ভুল:** Development-এর Bind Mount Production-এ নিয়ে যাওয়া — হোস্টের পথের ওপর নির্ভরতা তৈরি হয়।
- **ভুল:** Bind Mount-এ `node_modules` ঢেকে ফেলা — Anonymous Volume দিয়ে রক্ষা করতে হয়।
- **ভুল:** Volume-এর ভেতরে হোস্ট থেকে হাত দিয়ে ফাইল বদলানো — চলমান Database-র ফাইল সরাসরি বদলালে ডেটা নষ্ট হতে পারে।
- **ভুল ধারণা:** "Container-এর ভেতরে লিখলেই ডেটা থেকে যায়।" — না, Writable Layer Container-এর সঙ্গেই মুছে যায়।
- **ভুল ধারণা:** "Image আপডেট করলে Volume-এর ডেটাও আপডেট হয়।" — না, Volume-এর ডেটা Image থেকে আলাদা; নতুন Image পুরনো ডেটার ওপরেই চলে।

১৫.১২ নিরাপত্তা বিবেচনা

WARNING: Bind Mount দিয়ে হোস্টের সংবেদনশীল ডিরেক্টরি (যেমন `/etc` বা হোস্টের SSH Key) Container-এর ভেতরে Mount করবেন না — Container ভেদ হলে আক্রমণকারী হোস্টের ফাইল পড়তে বা বদলাতে পারবে। Docker Socket (`/var/run/docker.sock`) Mount করা আরও বিপজ্জনক — তা Container-কে হোস্টের Docker-এর সম্পূর্ণ নিয়ন্ত্রণ দেয়। শুধু প্রয়োজনীয় পথই Mount করুন, আর সম্ভব হলে `:ro` (শুধু-পড়া) ব্যবহার করুন।

Volume-এর ডেটাও সুরক্ষিত রাখতে হয়: Database-র Volume হোস্টের ফাইলসিস্টেমে থাকে — হোস্টের Backup ও Disk Encryption সেই ডেটাকেও রক্ষা করে। সংবেদনশীল ফাইল (যেমন TLS Private Key) Tmpfs-এ রাখা ভালো — Container থামলেই তা মেমরি থেকে মুছে যায়, ডিস্কে কোনো চিহ্ন থাকে না। আর Volume-এর Permission পরীক্ষা করুন: হোস্টে `/var/lib/docker/volumes/`-এর ভেতরে শুধু `root` ও Docker Group-এর সদস্যরা ঢুকতে পারে — এই সুরক্ষাটি ভেঙে Volume-এর পথ সবার জন্য খুলে দেবেন না।

১৫.১৩ কর্মক্ষমতা ও রিসোর্স বিবেচনা

Volume-এ লেখা Writable Layer-এর চেয়ে দ্রুত — কারণ OverlayFS-এর Copy-on-Write খরচ নেই, লেখা সরাসরি হোস্টের ফাইলসিস্টেমে যায়। Database-র জন্য এটি গুরুত্বপূর্ণ: প্রতি সেকেন্ডে শত শত লেখা হলে Writable Layer-এর খরচ চোখে পড়ে। Bind Mount-এর গতি হোস্টের ফাইলসিস্টেমের গতির সমান — তবে macOS বা Windows-এ Docker Desktop-এ Bind Mount ধীর হতে পারে (ফাইল শেয়ারিংয়ের খরচ), সেখানে Development-এও Named Volume

বিবেচনা করুন। Tmpfs-এর গতি সর্বোচ্চ (RAM), তবে মেমরি ভরে গেলে Container থেমে যেতে পারে — Tmpfs-এর আকার সীমা (`size:`) সবসময় দিন। Volume-এর আকার নিয়মিত পরীক্ষা করুন (`docker system df -v`) — Database-র Volume বাড়তে থাকলে ডিস্ক ভরে যাওয়ার আগেই সতর্ক হোন।

১৫.১৪ সমস্যা সমাধান পদ্ধতি

1. **ডেটা হারিয়ে গেছে:** `docker volume ls` — Volume-টি এখনো আছে কি না দেখুন; `docker inspect <container>`-এ Mount অংশে Volume যুক্ত ছিল কি না দেখুন।
2. **Permission denied:** `docker logs` দেখুন, হোস্টে ডিরেক্টরির মালিকানা (`ls -ld`) ও Container-এর UID (`docker exec <name> id`) মেলান।
3. **Volume খালি দেখাচ্ছে:** `docker volume inspect`-এ Mountpoint দেখে হোস্টে ফাইল আছে কি না পরীক্ষা করুন; ভুল Volume নামে Mount করা হয়েছে কি না দেখুন।
4. **Bind Mount ঢেকে ফেলেছে:** Container-এর ভেতরে `ls -la /app/src` দেখুন — হোস্টের ডিরেক্টরি Image-এর ফাইল ঢেকে ফেলেছে কি না।
5. **ডিস্ক ভরে গেছে:** `docker system df` ও `du -sh /var/lib/docker/volumes/*` দিয়ে কোন Volume বড় হয়েছে দেখুন।
6. **Volume মুছতে পারছি না:** `docker ps -a` দেখুন কোনো Container (এমনকি থেমে থাকা) এখনো Volume ব্যবহার করছে কি না।

১৫.১৫ সেরা অনুশীলন ও অ্যান্টি-প্যাটার্ন

সেরা অনুশীলন: গুরুত্বপূর্ণ ডেটা সবসময় Named Volume-এ; Volume-এর নাম স্পষ্ট ও স্থির রাখুন; Database-র Volume আলাদা, আপলোডের Volume আলাদা; কনফিগ ফাইল `:ro` Bind Mount-এ; Development-এ Bind Mount, Production-এ Named Volume; Volume মুছে ফেলার আগে Backup নিন; `docker compose down`-এ `-v` দেবেন না। **অ্যান্টি-প্যাটার্ন:** Writable Layer-এ গুরুত্বপূর্ণ ডেটা রাখা; Anonymous Volume ব্যবহার করা; Database Volume ছাড়া চালানো; হোস্টের সংবেদনশীল পথ Bind Mount করা; Docker Socket Mount করা; Volume-এর ভেতরে হাত দিয়ে ফাইল বদলানো।

১৫.১৬ অধ্যায়ের সারাংশ

Container-এর Writable Layer ক্ষণস্থায়ী — Container মুছলেই তার ভেতরের সব লেখা মুছে যায়। তাই গুরুত্বপূর্ণ ডেটা কখনো Writable Layer-এ রাখা যাবে না — রাখতে হয় Named Volume-এ (প্রোডাকশনের মানদণ্ড), Development-এ কোডের জন্য Bind Mount-এ, আর অস্থায়ী Cache-এর জন্য Tmpfs-এ। MongoDB-র `/data/db` ও PostgreSQL-র `/var/lib/postgresql/data` — এই দুটি পথে Volume দেওয়া বাধ্যতামূলক। Volume মুছে ফেলা ধ্বংসাত্মক — আগে Backup নিন। আর মালিকানার সমস্যা হলে হোস্টের UID ও Container-এর UID মেলান।

১৫.১৭ মূল প্রযুক্তিগত পরিভাষা

Writable Layer: Image-এর ওপরে Container-এর নিজস্ব লেখার স্তর — Container মুছলে মুছে যায়।

Copy-on-Write: ফাইল বদলানোর সময় প্রথমে তার কপি Writable Layer-এ তোলার কৌশল — মূল Image অপরিবর্তিত থাকে।

Named Volume: নাম-সহ Docker-পরিচালিত স্থায়ী সংরক্ষণ — Container মুছলেও থেকে যায়।

Anonymous Volume: নাম ছাড়া Volume — খুঁজে পাওয়া কঠিন, এড়িয়ে চলা উচিত।

Bind Mount: হোস্টের যেকোনো ফাইল বা ডিরেক্টরিকে Container-এর ভেতরে বসানো।

Tmpfs Mount: শুধু মেমরিতে থাকা অস্থায়ী সংরক্ষণ — Container থামলেই মুছে যায়।

Mountpoint: হোস্টে Volume-এর আসল পথ (`/var/lib/docker/volumes/<নাম>/_data`)।

Ephemeral: ক্ষণস্থায়ী — Container-এর জীবনের সঙ্গেই শেষ হয়ে যায় এমন।

Read-only Mount (:ro): Container শুধু পড়তে পারে, বদলাতে পারে না — এমন Mount।

Volume Lifecycle: Volume-এর জীবনচক্র — তৈরি, যুক্ত করা, ব্যবহার, Backup ও মুছে ফেলা।

১৫.১৮ জ্ঞান যাচাই — প্রশ্ন

1. Writable Layer কেন ক্ষণস্থায়ী? গুরুত্বপূর্ণ ডেটা সেখানে রাখলে কী হবে?
2. Named Volume, Anonymous Volume, Bind Mount ও Tmpfs Mount — প্রতিটির একটি করে ব্যবহার লিখুন।
3. MongoDB ও PostgreSQL-এর ডেটা কোন পথে থাকে? Volume না দিলে কী হবে?
4. Development-এ Bind Mount ভালো, Production-এ Named Volume ভালো — কেন?
5. `docker compose down -v` কেন বিপজ্জনক?
6. PostgreSQL Container "Permission denied" দিলে কী পরীক্ষা করবেন?
7. Bind Mount-এ `node_modules` ঢেকে যাওয়ার সমস্যাটি কীভাবে সমাধান করবেন?

১৫.১৯ ব্যবহারিক অনুশীলন

1. **ডেটা বাঁচানো:** ১৫.১০-এর পরীক্ষাটি সম্পূর্ণ করুন ও ফলাফল লিখুন।
2. **Bind Mount Development:** একটি Node.js অ্যাপের কোড Bind Mount করে চালান ও হোস্টে ফাইল বদলে Hot Reload দেখুন।

3. **Read-only কনফিগ:** একটি Nginx কনফিগ ফাইল `:ro` Bind Mount করে চালান; Container-এর ভেতর থেকে ফাইলটি বদলানোর চেষ্টা করে দেখুন কী হয়।
4. **Volume নিরীক্ষা:** `docker system df -v` চালিয়ে আপনার হোস্টের সব Volume-এর আকারের তালিকা তৈরি করুন।

অধ্যায় 16 • PART III — Container Networking and Persistent Storage

Chapter 16 — Storage Backup, Restore and Data Management

১৬.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: (১) একটি Backup কৌশল নকশা করতে — কী Backup নেবেন, কত ঘন ঘন, কোথায় রাখবেন; (২) MongoDB (`mongodump` / `mongorestore`) ও PostgreSQL (`pg_dump` / `pg_restore`)-এর সম্পূর্ণ Backup ও Restore করতে; (৩) Volume-এর ফাইল-স্তরের Backup ও Restore করতে (`tar` ও অস্থায়ী Container ব্যবহার করে); (৪) RPO ও RTO ব্যাখ্যা করতে ও TechNova-র জন্য উপযুক্ত মান ঠিক করতে; (৫) Backup যাচাই (Restore পরীক্ষা) করতে — যাতে প্রয়োজনের সময় Backup কাজ করে; (৬) ডেটা Migration করতে — এক হোস্ট থেকে আরেক হোস্টে, এক সংস্করণ থেকে আরেক সংস্করণে; (৭) পুরনো Backup মুছে ফেলার নীতি (Retention) ঠিক করতে।

১৬.২ কেন এই বিষয়টি DevOps-এ গুরুত্বপূর্ণ

Backup নেওয়া হয়নি এমন ডেটা হারানো শুধু সময়ের ব্যাপার — ডিস্ক নষ্ট হয়, ভুল কমান্ড চলে, আক্রমণ হয়। কিন্তু আরও বড় সত্য: **Backup নেওয়া হয়নি — এমন নয়; Backup Restore করে দেখা হয়নি — সেটাই আসল ব্যর্থতা।** অনেক দল নিয়মিত Backup নেয়, কিন্তু প্রয়োজনের সময় দেখে Backup ফাইলটি নষ্ট, বা Restore-এর পদ্ধতি কেউ জানে না। TechNova-র নিয়ম তাই দুটি: প্রতিটি Backup-এর সঙ্গে একটি Restore পরীক্ষা, আর প্রতিটি Restore-এর সঙ্গে একটি লিখিত পদ্ধতি। এই অধ্যায়ে আপনি সেই পদ্ধতিটি শিখবেন — MongoDB ও PostgreSQL-এর জন্য, Volume-এর জন্য, আর পুরো স্ট্যাকের জন্য।

১৬.৩ পূর্বশর্ত জ্ঞান

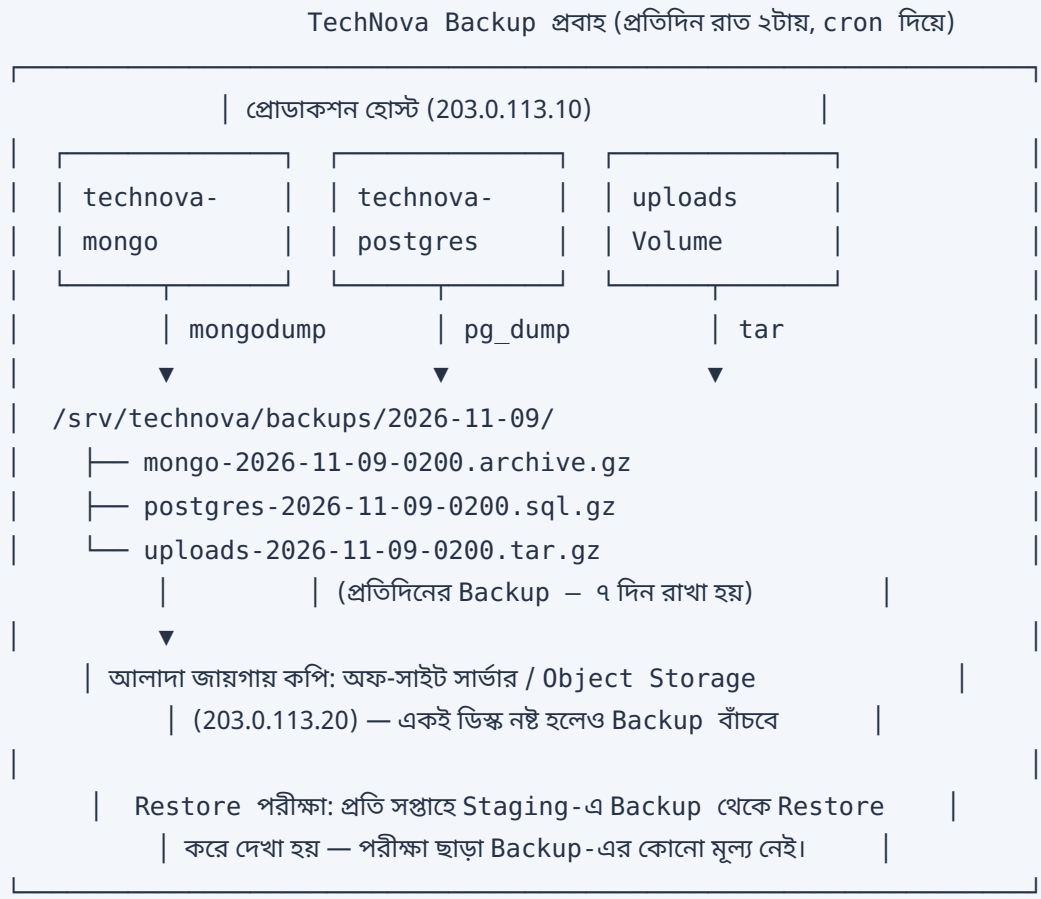
অধ্যায় ১৫-এর Volume, Bind Mount, Named Volume ও Volume Lifecycle জানা থাকতে হবে। MongoDB ও PostgreSQL-এর মৌলিক ব্যবহার (Database, Collection/Table, User) ও Linux-এ `tar`, `gzip` ও `cron`-এর প্রাথমিক ধারণা থাকলে ভালো হয়। Backup-এর জন্য `mongodump` ও `pg_dump`-এর পূর্ব অভিজ্ঞতা প্রয়োজন নেই — এই অধ্যায়ে প্রতিটি কমান্ড লাইন-বাই-লাইন ব্যাখ্যা করা হবে।

১৬.৪ প্রথম নীতি থেকে ব্যাখ্যা

Backup মানে ডেটার একটি কপি — এমন একটি কপি যা মূল ডেটা হারিয়ে গেলে ফিরিয়ে আনা যায়। কিন্তু "কপি" বললেই সব বলা হয় না: কপিটি কি চলমান Database-র সঙ্গে সামঞ্জস্যপূর্ণ (Consistent)? কপিটি কি পরীক্ষা করা হয়েছে? কপিটি কি মূল ডেটার কাছেই রাখা আছে — অর্থাৎ একই ডিস্ক নষ্ট হলে কপিটিও হারাবে? ভালো Backup-এর তিনটি গুণ: **সামঞ্জস্য (Consistency)** — Backup-এর মুহূর্তে ডেটা একটি স্থির অবস্থায় ছিল, অর্ধেক লেখা অবস্থায় নয়; **পরীক্ষিত (Tested)** — Restore করে দেখা হয়েছে যে কপিটি কাজ করে; **আলাদা (Separate)** — কপিটি মূল ডেটা থেকে আলাদা জায়গায় রাখা আছে (অন্য ডিস্ক, অন্য সার্ভার, অন্য অঞ্চল)। Database Backup-এর দুটি পদ্ধতি: **Logical Backup** — Database-র বিষয়বস্তুকে কমান্ডের তালিকা হিসেবে রপ্তানি করা (`mongodump`, `pg_dump`) — ছোট, বহনযোগ্য, সংস্করণ-বদলে কাজ করে; **Physical Backup** — ডেটা ফাইলের সরাসরি কপি (Volume-এর `tar`) — দ্রুত, কিন্তু একই সংস্করণেই Restore করতে হয়। দুটি পদ্ধতিই জানতে হবে — Logical Backup Migration-এর জন্য, Physical Backup দ্রুত Restore-এর জন্য।

CONCEPT: RPO (Recovery Point Objective) ও RTO (Recovery Time Objective) — Backup কৌশলের দুটি সংখ্যা। RPO বলে: "সবচেয়ে বেশি কতটুকু ডেটা হারানো চলবে?" — RPO ১ ঘণ্টা মানে প্রতি ঘণ্টায় Backup নিতে হবে। RTO বলে: "সবচেয়ে বেশি কতক্ষণ থেমে থাকা চলবে?" — RTO ৩০ মিনিট মানে Restore পদ্ধতিটি ৩০ মিনিটের মধ্যে শেষ করতে হবে। TechNova-র জন্য RPO ১ ঘণ্টা ও RTO ৩০ মিনিট একটি যুক্তিযুক্ত লক্ষ্য — ছোট দলের জন্য অর্জনযোগ্য, আবার ব্যবসার জন্য নিরাপদ।

১৬.৫ আর্কিটেকচার / ধারণাগত চিত্র



Backup-এর নিয়ম: ৩-২-১ — ডেটার ৩টি কপি, ২টি ভিন্ন মাধ্যমে, ১টি আলাদা জায়গায়।

১৬.৬ বিস্তারিত প্রযুক্তিগত ব্যাখ্যা

Logical Backup বনাম Physical Backup

বিষয়	Logical Backup	Physical Backup
পদ্ধতি	`mongodump`, `pg_dump` — ডেটাকে কমান্ড/নথি হিসেবে রপ্তানি	Volume-এর ফাইলের `tar` কপি

আকার	ছোট (সংকুচিত করা যায়)	বড় (পুরো ডেটা ফাইল)
Restore-এর গতি	ধীর (প্রতিটি রেকর্ড আবার লিখতে হয়)	দ্রুত (ফাইল বসিয়ে দিলেই হয়)
সংস্করণ-বদল	সাধারণত নতুন সংস্করণেও Restore হয়	একই সংস্করণেই Restore করতে হয়
কখন ব্যবহার করবেন	নিয়মিত Backup, Migration, ছোট Database	বড় Database, দ্রুত Restore, Disaster Recovery

TechNova-র নিয়ম: প্রতিদিন Logical Backup (সহজে বহনযোগ্য) ও সপ্তাহে একবার Physical Backup (দ্রুত Restore-এর জন্য) — দুটোই।

MongoDB Backup ও Restore

`mongodump` Database-র সব Collection-কে BSON ফাইলে রপ্তানি করে। চালানোর সময় Database চলমান থাকতে পারে — তবে বড় লেখার সময় Backup নিলে সামঞ্জস্যের জন্য `--oplog` (Replica Set-এ) ব্যবহার করতে হয়। ছোট দলের জন্য সহজ নিয়ম: কম ট্রাফিকের সময় (রাত ২টা) Backup নিন। `mongorestore` সেই ফাইলগুলোকে আবার Database-এ লেখে। সতর্কতা: `mongorestore` ডিফল্টভাবে বিদ্যমান Collection মুছে ফেলে না — পুরনো ডেটার সঙ্গে নতুন ডেটা মিশে যেতে পারে। সম্পূর্ণ প্রতিস্থাপন চাইলে `--drop` ফ্ল্যাগ দিন।

PostgreSQL Backup ও Restore

`pg_dump` Database-কে SQL কমান্ডের তালিকা হিসেবে রপ্তানি করে — টেবিল তৈরি, ডেটা ঢোকানো, Index বানানো — সবই SQL। Restore করতে `psql` দিয়ে সেই SQL ফাইলটি চালালেই হয়। `pg_dump`-এর সুবিধা: Backup ফাইলটি মানুষের পড়ার মতো, আর যেকোনো PostgreSQL সংস্করণে (সাধারণত) Restore করা যায়। সতর্কতা: `pg_dump` শুধু একটি Database-র Backup নেয় — সব Database চাইলে `pg_dumpall` ব্যবহার করুন। আর Restore করার আগে লক্ষ্য Database-টি খালি থাকতে হবে, নইলে "already exists" Error আসবে।

Volume-এর ফাইল-স্তরের Backup

Volume-এর Backup নেওয়ার সঠিক পদ্ধতি: চলমান Database Container-এর ভেতর থেকে ফাইল কপি করবেন না — Database ফাইল লিখছে এমন সময় কপি করলে Backup নষ্ট হতে পারে। সঠিক পদ্ধতি দুটি: (১) Database-র নিজের Backup টুল ব্যবহার করা (Logical Backup — উপরে দেখুন); (২) একটি অস্থায়ী Container চালিয়ে Volume-কে Mount করে `tar` করা — তবে তার আগে Database Container থামিয়ে নিন, অথবা নিশ্চিত হোন Database ফাইল লিখছে না। আপলোডের মতো সাধারণ ফাইলের Volume-এর জন্য `tar` পদ্ধতিটি নিরাপদ, কারণ সেখানে Database-এর মতো জটিল লেখা হয় না।

Backup Retention — কতদিন রাখবেন

সব Backup চিরকাল রাখা যায় না — ডিস্ক ভরে যাবে। একটি সাধারণ নীতি: দৈনিক Backup ৭ দিন, সাপ্তাহিক Backup ৪ সপ্তাহ, মাসিক Backup ১২ মাস। পুরনো Backup মুছে ফেলার কাজটি স্বয়ংক্রিয় করুন (`find ... -mtime +7 -delete`) — হাতে মুছলে ভুলে গুরুত্বপূর্ণ Backup মুছে যেতে পারে। আর Retention নীতি লিখে রাখুন — কোন Backup কতদিন থাকবে, তা দলের সবাই জানবে।

Data Migration — এক হোস্ট থেকে আরেক হোস্টে

Migration-এর ধাপ: (১) উৎস হোস্টে Logical Backup নিন; (২) Backup ফাইলটি নতুন হোস্টে পাঠান (`scp` বা Object Storage); (৩) নতুন হোস্টে Database Container চালান (একই সংস্করণ); (৪) Restore করুন; (৫) নতুন হোস্টে অ্যাপ

চালিয়ে যাচাই করুন — ডেটার সংখ্যা মিলছে কি না; (৬) DNS বা Load Balancer নতুন হোস্টের দিকে ঘুরিয়ে দিন; (৭) পুরনো হোস্টটি কিছুদিন রেখে দিন — ফিরে যাওয়ার (Rollback) পথ হিসেবে। Migration-এর সময় ডেটা লেখা বন্ধ রাখুন (Maintenance Window) অথবা নতুন লেখাগুলো পরে মিলিয়ে নিন — নইলে কিছু ডেটা হারিয়ে যাবে।

১৬.৭ Docker কমান্ড — লাইন-বাই-লাইন বিশ্লেষণ

```
# --- MongoDB Logical Backup ---
$ docker exec technova-mongo mongodump \
    --username admin --password CHANGE-ME-MONGO-DB-PASS \
    --authenticationDatabase admin \
    --db technova --archive=/tmp/mongo-backup.archive --gzip
# docker exec technova-mongo — চলমান MongoDB Container-এর ভেতরে কমান্ড চালাও।
# mongodump — MongoDB-র Backup টুল।
# --username admin --password ... — প্রমাণীকরণের তথ্য (কাল্পনিক পাসওয়ার্ড)।
# --authenticationDatabase admin — User-টি কোন Database-এ আছে।
# --db technova — শুধু technova Database-র Backup নাও।
# --archive=/tmp/mongo-backup.archive — সব Collection একটি ফাইলে লেখো।
# --gzip — ফাইলটি সংকুচিত করো (আকার ছোট হবে)।

$ docker cp technova-mongo:/tmp/mongo-backup.archive \
    /srv/technova/backups/mongo-2026-11-09.archive.gz
# docker cp — Container-এর ভেতরের ফাইল হোস্টে কপি করো।
# এটিই Backup ফাইলটি হোস্টে আনার পদ্ধতি।

# --- MongoDB Restore ---
$ docker cp /srv/technova/backups/mongo-2026-11-09.archive.gz \
    technova-mongo:/tmp/restore.archive.gz
# docker cp — হোস্টের Backup ফাইল Container-এর ভেতরে কপি করো।

$ docker exec technova-mongo mongorestore \
    --username admin --password CHANGE-ME-MONGO-DB-PASS \
    --authenticationDatabase admin \
    --archive=/tmp/restore.archive.gz --gzip --drop
# mongorestore — Backup থেকে Database ফিরিয়ে আনো।
# --drop — Restore-এর আগে বিদ্যমান Collection মুছে ফেলো (সম্পূর্ণ প্রতিস্থাপন)।

# --- PostgreSQL Logical Backup ---
$ docker exec technova-postgres pg_dump \
    -U technova -d technova > /srv/technova/backups/postgres-2026-11-09.sql
# pg_dump -U technova -d technova — technova User হিসেবে technova
# Database-র SQL Backup নাও; আউটপুট হোস্টের ফাইলে লেখা হচ্ছে (>)।

$ gzip /srv/technova/backups/postgres-2026-11-09.sql
# gzip — SQL ফাইলটি সংকুচিত করো (.sql.gz হবে)।

# --- PostgreSQL Restore ---
$ gunzip -c /srv/technova/backups/postgres-2026-11-09.sql.gz | \
    docker exec -i technova-postgres psql -U technova -d technova
# gunzip -c — সংকুচিত ফাইলটি খুলে Standard Output-এ পাঠাও।
# docker exec -i — Container-এর ভেতরে psql চালিয়ে সেই SQL ঢোকাও।

# --- Volume-এর ফাইল-স্তরের Backup (আপলোডের মতো সাধারণ ফাইলের জন্য) ---
$ docker run --rm \
    -v technova-uploads-data:/data \
    -v /srv/technova/backups:/backup \
```

```
    alpine tar czf /backup/uploads-2026-11-09.tar.gz -C /data .
# docker run --rm — অস্থায়ী Container চালিয়ে শেষে মুছে ফেলো।
# -v technova-uploads-data:/data — Backup নেওয়ার Volume Mount করো।
# -v /srv/technova/backups:/backup — Backup ফাইল রাখার হোস্টের পথ Mount করো।
# alpine tar czf ... — Alpine Image দিয়ে tar করে সংকুচিত ফাইল বানাও।

# --- Volume Restore ---
$ docker run --rm \
    -v technova-uploads-data:/data \
    -v /srv/technova/backups:/backup \
    alpine sh -c "rm -rf /data/* && tar xzf /backup/uploads-2026-11-09.tar.gz -C /data"
# sh -c "rm -rf /data/* && tar xzf ..." — প্রথমে Volume-এর পুরনো ফাইল
# মুছে ফেলো, তারপর Backup থেকে ফাইল ফিরিয়ে আনো।

# --- Backup যাচাই ও পরিষ্কার ---
$ ls -lh /srv/technova/backups/
# ls -lh — Backup ফাইলগুলোর আকার ও তারিখ দেখো; শূন্য আকারের ফাইল সন্দেহজনক।

$ find /srv/technova/backups -name "*.gz" -mtime +7 -delete
# find ... -mtime +7 -delete — ৭ দিনের পুরনো Backup ফাইল মুছে ফেলো (Retention)।
```

WARNING: Restore-এর আগে দুবার ভাবুন — `mongorestore --drop` ও Volume Restore-এর `rm -rf /data/*` বর্তমান ডেটা মুছে ফেলে। ভুল Backup ফাইল দিয়ে Restore করলে বর্তমান ডেটা হারিয়ে যাবে। Restore করার আগে বর্তমান ডেটারও একটি Backup নিন, আর প্রথমবার Restore সবসময় Staging-এ পরীক্ষা করুন — প্রোডাকশনে নয়। `docker volume rm` ও `docker compose down -v`-এর মতো ধ্বংসাত্মক কমান্ডের পাশাপাশি Restore-ও একটি ধ্বংসাত্মক অপারেশন — কারণ এটি বর্তমান অবস্থা বদলে দেয়।

১৬.৮ সম্পূর্ণ কনফিগারেশন উদাহরণ

TechNova-র স্বয়ংক্রিয় Backup স্ক্রিপ্ট ও Cron সেটআপ — প্রতিদিন রাত ২টায় চলে, ৭ দিনের Retention রাখে:

```
#!/bin/bash
# /srv/technova/scripts/backup.sh — TechNova দৈনিক Backup স্ক্রিপ্ট
set -e  # কোনো ধাপ ব্যর্থ হলে এখানেই থামো
DATE=$(date +%Y-%m-%d-%H%M)
BACKUP_DIR=/srv/technova/backups
mkdir -p "$BACKUP_DIR"

echo "[$DATE] MongoDB Backup শুরু..."
docker exec technova-mongo mongodump \
  --username admin --password CHANGE-ME-MONGO-DB-PASS \
  --authenticationDatabase admin \
  --db technova --archive=/tmp/mongo.archive --gzip
docker cp technova-mongo:/tmp/mongo.archive \
  "$BACKUP_DIR/mongo-$DATE.archive.gz"

echo "[$DATE] PostgreSQL Backup শুরু..."
docker exec technova-postgres pg_dump -U technova -d technova \
  | gzip > "$BACKUP_DIR/postgres-$DATE.sql.gz"

echo "[$DATE] Uploads Volume Backup শুরু..."
docker run --rm \
  -v technova-uploads-data:/data \
```

```
  -v "$BACKUP_DIR:/backup" \
  alpine tar czf "/backup/uploads-$DATE.tar.gz" -C /data .

echo "[$DATE] পুরনো Backup পরিষ্কার (৭ দিনের বেশি পুরনো)..."
find "$BACKUP_DIR" -name "*.gz" -mtime +7 -delete

echo "[$DATE] Backup সম্পন্ন।"
ls -lh "$BACKUP_DIR"

# Cron-এ যোগ করুন (crontab -e):
# 0 2 * * * /srv/technova/scripts/backup.sh >> /var/log/technova-backup.log 2>&1
# ব্যাখ্যা: প্রতিদিন রাত ২টায় (0 2 * * *) স্ক্রিপ্টটি চালাও,
# আউটপুট /var/log/technova-backup.log ফাইলে লেখো।
```

Restore পরীক্ষার স্ক্রিপ্ট (প্রতি সপ্তাহে Staging-এ চালান):

```
#!/bin/bash
# /srv/technova/scripts/restore-test.sh — Staging-এ Restore পরীক্ষা
set -e
LATEST_MONGO=$(ls -t /srv/technova/backups/mongo-*.archive.gz | head -1)
echo "পরীক্ষা করা হচ্ছে: $LATEST_MONGO"
# Staging MongoDB Container-এ Backup ফাইল কপি করে Restore করুন
docker cp "$LATEST_MONGO" technova-mongo-staging:/tmp/test-restore.archive.gz
docker exec technova-mongo-staging mongorestore \
  --username admin --password CHANGE-ME-MONGO-DB-PASS \
  --authenticationDatabase admin \
  --archive=/tmp/test-restore.archive.gz --gzip --drop
# যাচাই: Collection-এর সংখ্যা গুনুন
docker exec technova-mongo-staging mongosh --quiet \
  --username admin --password CHANGE-ME-MONGO-DB-PASS \
  --authenticationDatabase admin \
  --eval 'db.getSiblingDB("technova").getCollectionNames().length'
echo "Restore পরীক্ষা সম্পন্ন — Collection-এর সংখ্যা ওপরে দেখুন।"
```

১৬.৯ বাস্তব প্রোডাকশন পরিস্থিতি

REAL WORLD — TechNova: TechNova-র প্রোডাকশন সার্ভারের ডিস্ক একদিন হঠাৎ নষ্ট হয়ে গেল। সৌভাগ্যবশত দৈনিক Backup ছিল — তবে আসল পরীক্ষা তখনই এলো যখন Restore করতে গিয়ে দেখা গেল গত তিন দিনের Backup ফাইলগুলো শূন্য আকারের (0 bytes) — Backup স্ক্রিপ্টটি ব্যর্থ হচ্ছিল, কিন্তু কেউ Log দেখেনি। সৌভাগ্যবশত চার দিন আগের Backupটি ভালো ছিল, আর সেটি দিয়েই সার্ভিস ফিরিয়ে আনা হলো — তিন দিনের ডেটা হারিয়ে গেল। এই ঘটনার পর দলের নিয়ম হলো: (১) প্রতিটি Backup-এর পর ফাইলের আকার পরীক্ষা করা হয় (শূন্য হলে সতর্কবার্তা যায়); (২) প্রতি সপ্তাহে Staging-এ Restore পরীক্ষা করা হয়; (৩) Backup ফাইল অফ-সাইটেও কপি করা হয়। শিক্ষা: Backup-এর মূল্য তার Restore-এ — আর Restore পরীক্ষা না করলে Backup শুধু একটি আশা, নিরাপত্তা নয়।

১৬.১০ ধাপে ধাপে ব্যবহারিক উদাহরণ

উদাহরণ: সম্পূর্ণ Backup ও Restore চক্র

1. Volume-সহ MongoDB চালান ও কিছু পরীক্ষার ডেটা তৈরি করুন (দুটি Collection, কয়েকটি Document)।

2. `mongodump` দিয়ে Backup নিন ও হোস্টে কপি করুন; ফাইলের আকার পরীক্ষা করুন (`ls -lh`) — শূন্য নয় তো?
3. Database-র সব Collection মুছে ফেলুন (পরীক্ষার জন্য) — `mongosh` দিয়ে `db.dropDatabase()`।
4. `mongorestore --drop` দিয়ে Backup থেকে ফিরিয়ে আনুন; Collection ও Document-এর সংখ্যা মিলছে কি না যাচাই করুন।
5. একই চক্র PostgreSQL-র জন্য করুন: `pg_dump` → টেবিল মুছে ফেলা → `psql` দিয়ে Restore → সারি (Row) গুনে যাচাই।
6. Uploads Volume-এর জন্য `tar` Backup ও Restore করুন — Restore-এর আগে Volume-এর ফাইল মুছে দেখুন ফিরে আসে কি না।
7. ১৬.৮-এর Backup স্ক্রিপ্টটি আপনার পরিবেশে বসিয়ে একবার হাতে চালান ও Log পড়ুন।

১৬.১১ সাধারণ ভুল ও ভুল ধারণা

TROUBLESHOOTING: "Restore করলাম, কিন্তু ডেটা আসেনি" — প্রথমে Backup ফাইলের আকার দেখুন (শূন্য হলে Backup-ই হয়নি); দ্বিতীয়ত, সঠিক Database নামে Restore করেছেন কি না দেখুন (`--db` ও `-d`); তৃতীয়ত, `mongorestore`-এ `--drop` না দিলে পুরনো ডেটার সঙ্গে মিশে যেতে পারে; চতুর্থত, PostgreSQL Restore-এ "already exists" Error মানে লক্ষ্য Database খালি ছিল না — আগে Database মুছে আবার তৈরি করুন।

- **ভুল:** Backup নেওয়া, কিন্তু Restore করে না দেখা — প্রয়োজনের সময় Backup নষ্ট বেরোয়।
- **ভুল:** চলমান Database-র Volume ফাইল সরাসরি কপি করা — অসামঞ্জস্যপূর্ণ Backup হয়।
- **ভুল:** Backup শুধু একই সার্ভারে রাখা — সার্ভার নষ্ট হলে Backup-ও হারায়।
- **ভুল:** Backup স্ক্রিপ্টের Log না দেখা — ব্যর্থ Backup নীরবে চলতে থাকে।
- **ভুল:** `docker compose down -v` চালিয়ে Backup না নিয়ে Volume মুছে ফেলা।
- **ভুল:** Migration-এর সময় ডেটা লেখা চালু রাখা — কিছু লেখা হারিয়ে যায়।
- **ভুল ধারণা:** "Volume থাকলেই ডেটা নিরাপদ।" — Volume শুধু Container মুছলে ডেটা বাঁচায়; ডিস্ক নষ্ট হলে, ভুল কমান্ডে বা আক্রমণে Volume-ও হারায়।
- **ভুল ধারণা:** "Backup ফাইল থাকলেই Restore হবে।" — ফাইল নষ্ট হতে পারে, সংস্করণ মিলতে পারে না, পদ্ধতি ভুল হতে পারে — পরীক্ষা ছাড়া নিশ্চিত হওয়া যায় না।

১৬.১২ নিরাপত্তা বিবেচনা

WARNING: Backup ফাইলে পুরো Database থাকে — ব্যবহারকারীর তথ্য, পাসওয়ার্ডের Hash, সবকিছু। Backup ফাইলটি মূল Database-র মতোই সংবেদনশীল: এটি কখনো Public জায়গায় (যেমন Git Repository বা খোলা Object Storage) রাখবেন না, আর সম্ভব হলে Backup ফাইলটি এনক্রিপ্ট করুন (যেমন `gpg` দিয়ে)। Backup ফাইলের অনুমতি পরীক্ষা করুন (`chmod 600`) — শুধু প্রয়োজনীয় ব্যবহারকারীই পড়তে পারবে। আর পুরনো Backup মুছে ফেলার সময় নিশ্চিত হোন সেটি সত্যিই অপ্রয়োজনীয় — মুছে ফেলা Backup ফিরিয়ে আনা যায় না।

Restore-এর নিরাপত্তাও গুরুত্বপূর্ণ: অপরিচিত উৎসের Backup ফাইল Restore করবেন না — তাতে ক্ষতিকর ডেটা বা স্ক্রিপ্ট থাকতে পারে। আর Production-এ Restore করার আগে সবসময় Staging-এ পরীক্ষা করুন — Restore শুধু ডেটা ফেরায় না, বর্তমান ডেটাও বদলে দেয়।

১৬.১৩ কর্মক্ষমতা ও রিসোর্স বিবেচনা

Backup-এর সময় Database-র ওপর চাপ পড়ে — `mongodump` ও `pg_dump` ডেটা পড়ার সময় CPU ও Disk I/O ব্যবহার করে। তাই Backup কম ট্রাফিকের সময় (রাত ২টা) নিন। বড় Database-র Logical Backup ধীর — তখন Physical Backup (Volume `tar`) বা Database-র নিজের Incremental Backup বিবেচনা করুন। Backup ফাইলের আকার নিয়মিত পরীক্ষা করুন — Database বাড়ার সঙ্গে সঙ্গে Backup-এর জায়গাও বাড়ে; Retention নীতি দিয়ে পুরনো Backup মুছে জায়গা বাঁচান। Restore-এর সময়ও চাপ পড়ে — প্রোডাকশনে Restore করার সময় ট্রাফিক কম থাকা ভালো, আর বড় Restore-এর জন্য Maintenance Window নিন।

১৬.১৪ সমস্যা সমাধান পদ্ধতি

1. **Backup ফাইল শূন্য আকারের:** Backup স্ক্রিপ্টের Log দেখুন — `mongodump` বা `pg_dump` Error দিয়েছে কি না (ভুল পাসওয়ার্ড, ভুল Database নাম, Container থেমে আছে কি না)।
2. **Backup চলছে না (Cron):** `crontab -l` দিয়ে Cron Entry আছে কি না, স্ক্রিপ্টটি চালানোর অনুমতি (`chmod +x`) আছে কি না, ও Log ফাইলে কী লেখা আছে দেখুন।
3. **Restore ব্যর্থ:** Backup ফাইলের আকার, Database-র নাম, সংস্করণের মিল ও Error বার্তা পরীক্ষা করুন; `pg_restore`-এর বদলে `psql` দিয়ে SQL Backup চালানোর চেষ্টা করুন।
4. **ডিস্ক ভরে গেছে:** `df -h` ও `du -sh /srv/technova/backups/*` দিয়ে দেখুন কোন Backup বড়; Retention নীতি চালান।
5. **Volume Restore-এর পর ফাইল নেই:** সঠিক Volume নামে Mount করেছেন কি না, `tar`-এর `-C` পথ ঠিক ছিল কি না দেখুন।
6. **Migration-এর পর ডেটা মিলছে না:** উৎস ও লক্ষ্য Database-এ Collection/Table ও Row-এর সংখ্যা গুনে তুলনা করুন; Migration-এর সময় লেখা বন্ধ ছিল কি না দেখুন।

১৬.১৫ সেরা অনুশীলন ও অ্যান্টি-প্যাটার্ন

সেরা অনুশীলন: প্রতিদিন স্বয়ংক্রিয় Backup (Cron); প্রতিটি Backup-এর পর ফাইলের আকার পরীক্ষা; প্রতি সপ্তাহে Staging-এ Restore পরীক্ষা; ৩-২-১ নিয়ম (৩ কপি, ২ মাধ্যম, ১ আলাদা জায়গা); RPO ও RTO লিখে রাখা; Retention নীতি স্বয়ংক্রিয় করা; Restore-এর আগে বর্তমান ডেটার Backup নেওয়া; Backup ফাইল এনক্রিপ্ট করা ও অনুমতি সীমিত রাখা। **অ্যান্টি-প্যাটার্ন:** Backup নিয়ে Restore না দেখা; চলমান Database-র ফাইল সরাসরি কপি করা; Backup শুধু একই সার্ভারে রাখা; Backup Log না দেখা; `down -v` দিয়ে Backup ছাড়া Volume মুছে ফেলা; Migration-এর সময় লেখা চালু রাখা।

১৬.১৬ অধ্যায়ের সারাংশ

Backup-এর মূল্য তার Restore-এ — পরীক্ষা না করা Backup শুধু একটি আশা। MongoDB-র জন্য `mongodump`/`mongorestore`, PostgreSQL-র জন্য `pg_dump`/`psql` — Logical Backup; Volume-এর জন্য অস্থায়ী Container দিয়ে `tar` — Physical Backup। প্রতিদিন স্বয়ংক্রিয় Backup নিন (Cron), ৭ দিনের Retention রাখুন, Backup ফাইল অফ-সাইটেও কপি করুন, আর প্রতি সপ্তাহে Staging-এ Restore করে দেখুন। RPO বলে কতটুকু ডেটা হারানো চলবে,

RTO বলে কতক্ষণ থেমে থাকা চলবে — এই দুটি সংখ্যাই আপনার Backup কৌশল নির্ধারণ করে। আর মনে রাখবেন: Volume থাকলেই ডেটা নিরাপদ নয় — ডিস্ক নষ্ট হলে Volume-ও হারায়, শুধু Backup-ই বাঁচায়।

১৬.১৭ মূল প্রযুক্তিগত পরিভাষা

Backup: ডেটার একটি কপি, যা মূল ডেটা হারিয়ে গেলে ফিরিয়ে আনা যায়।

Restore: Backup থেকে ডেটা ফিরিয়ে আনার প্রক্রিয়া।

Logical Backup: Database-র বিষয়বস্তুকে কমান্ড বা নথি হিসেবে রপ্তানি করা (`mongodump`, `pg_dump`)।

Physical Backup: ডেটা ফাইলের সরাসরি কপি (Volume-এর `tar`)।

RPO (Recovery Point Objective): সবচেয়ে বেশি কতটুকু ডেটা হারানো চলবে — Backup-এর ঘনত্ব নির্ধারণ করে।

RTO (Recovery Time Objective): সবচেয়ে বেশি কতক্ষণ থেমে থাকা চলবে — Restore পদ্ধতির গতি নির্ধারণ করে।

Retention Policy: Backup কতদিন রাখা হবে — তার নিয়ম (যেমন দৈনিক ৭ দিন)।

3-2-1 Rule: ডেটার ৩টি কপি, ২টি ভিন্ন মাধ্যমে, ১টি আলাদা জায়গায় রাখার নিয়ম।

mongodump / mongorestore: MongoDB-র Backup ও Restore টুল।

pg_dump / pg_restore: PostgreSQL-র Backup ও Restore টুল (`psql` দিয়ে SQL Backup চালানো যায়)।

Consistency: Backup-এর মুহূর্তে ডেটা একটি স্থির, সামঞ্জস্যপূর্ণ অবস্থায় থাকার গুণ।

Data Migration: ডেটাকে এক হোস্ট বা এক সংস্করণ থেকে আরেকটিতে সরানোর প্রক্রিয়া।

১৬.১৮ জ্ঞান যাচাই — প্রশ্ন

1. Logical Backup ও Physical Backup-এর পার্থক্য কী? প্রতিটির একটি করে ব্যবহার লিখুন।

2. RPO ও RTO কী? TechNova-র জন্য RPO ১ ঘণ্টা হলে Backup কত ঘন ঘন নিতে হবে?
3. চলমান Database-র Volume ফাইল সরাসরি কপি করা কেন বিপজ্জনক?
4. `mongorestore --drop`-এর `--drop` কী করে? এটি না দিলে কী হতে পারে?
5. ৩-২-১ নিয়মটি ব্যাখ্যা করুন।
6. Backup নেওয়া হলেও Restore পরীক্ষা করা কেন জরুরি? একটি বাস্তব উদাহরণ দিন।
7. Migration-এর সময় ডেটা লেখা চালু রাখলে কী সমস্যা হতে পারে?

১৬.১৯ ব্যবহারিক অনুশীলন

1. **সম্পূর্ণ চক্র:** ১৬.১০-এর Backup ও Restore চক্রটি MongoDB ও PostgreSQL — দুটোর জন্যই সম্পূর্ণ করুন।
2. **স্বয়ংক্রিয় Backup:** ১৬.৮-এর স্ক্রিপ্টটি আপনার পরিবেশে বসিয়ে Cron-এ যোগ করুন ও একদিন Log পরীক্ষা করুন।
3. **Restore পরীক্ষা:** Staging-এ (বা পরীক্ষার Container-এ) একটি Backup থেকে Restore করে ডেটার সংখ্যা যাচাই করুন।
4. **Migration পরিকল্পনা:** TechNova-র MongoDB-কে এক হোস্ট থেকে আরেক হোস্টে সরানোর ধাপে ধাপে পরিকল্পনা (Rollback-সহ) লিখুন।
5. **Retention নীতি:** আপনার Backup ডিরেক্টরির জন্য একটি Retention নীতি (দৈনিক/সাপ্তাহিক/মাসিক) লিখুন ও `find` কমান্ড দিয়ে তা বাস্তবায়ন করুন।

PART IV • DEVOPS ENGINEERING MASTERY

Multi-Container Application Engineering

Docker Compose, মাল্টি-কনটেইনার MERN অ্যাপ্লিকেশন, environment variable ও secret, health check ও service recovery।

অধ্যায় 17 • PART IV — Docker Compose and Multi-Container Systems

Chapter 17 — Docker Compose Architecture and Configuration

১৭.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: (১) Docker Compose কী, কেন এটি একটিমাত্র কনটেইনার চালানোর চেয়ে আলাদা সমস্যার সমাধান করে তা ব্যাখ্যা করতে; (২) Compose ফাইলের গঠন — `services`, `networks`, `volumes`, `configs` ও `secrets` — পড়তে ও লিখতে; (৩) YAML-এর ইন্ডেন্টেশন, ম্যাপ, তালিকা ও ভেরিয়েবল প্রতিস্থাপনের নিয়ম প্রয়োগ করতে; (৪) `docker compose` v2 কমান্ডের জীবনচক্র — `up`, `down`, `ps`, `logs`, `exec` — পরিচালনা করতে; (৫) একাধিক Compose ফাইল, `profiles` ও dev/prod ওভাররাইড দিয়ে পরিবেশভেদে কনফিগারেশন সাজাতে; (৬) Compose-এর সীমা — এটি পূর্ণাঙ্গ অর্কেস্ট্রেটর নয় — স্পষ্টভাবে বলতে।

১৭.২ কেন এই বিষয়টি DevOps-এ গুরুত্বপূর্ণ

একটি বাস্তব অ্যাপ কখনো একটিমাত্র কনটেইনারে চলে না। TechNova-র MERN অ্যাপেই চারটি চলমান অংশ আছে: React/Next.js ফ্রন্টএন্ড, Node/Express ব্যাকএন্ড, MongoDB ডেটাবেস ও Nginx রিভার্স প্রক্সি। প্রতিটি অংশের নিজস্ব ইমেজ, পোর্ট, পরিবেশ চলক, ভলিউম ও নেটওয়ার্ক নিয়ম আছে। এগুলো আলাদা আলাদা `docker run` কমান্ড দিয়ে চালালে প্রতিটি ডেভেলপারের মেশিনে, প্রতিটি সার্ভারে কমান্ডের ক্রম, ফ্ল্যাগ ও টাইপো ভিন্ন হবে — "আমার মেশিনে চলেছিল" সমস্যাটি আবার ফিরে আসবে। Docker Compose এই পুরো মাল্টি-কনটেইনার ব্যবস্থাটিকে একটি ঘোষণামূলক (Declarative) YAML ফাইলে লিখে দেয়: আপনি বলেন কী চাই, Compose বোঝে কীভাবে চালাতে হবে। ফলে ডেভেলপমেন্ট পরিবেশ, Staging ও ছোট প্রোডাকশন সার্ভারে একই সংজ্ঞা থেকে একই ব্যবস্থা তৈরি হয়, সংস্করণ নিয়ন্ত্রণে (Git) থাকে, পর্যালোচনা করা যায় এবং বারবার চালানো যায়। এটাই DevOps-এর মূল প্রতিশ্রুতি — অবকাঠামোকে কোড হিসেবে লেখা (Infrastructure as Code) — Compose তার সবচেয়ে সহজ ও সরাসরি রূপ।

১৭.৩ পূর্বশর্ত জ্ঞান

আপনাকে জানতে হবে: Docker Image ও Container-এর পার্থক্য, `docker run`, `docker ps`, `docker logs` ও `docker build`-এর মৌলিক ব্যবহার (Volume 06-এর প্রথম অংশগুলোতে শেখা হয়েছে), YAML-এর প্রাথমিক ধারণা, Linux টার্মিনাল, পোর্ট ও নেটওয়ার্কের মৌলিক ধারণা (যেমন 3000 ও 27017 পোর্ট কী), এবং MERN স্ট্যাকের চারটি অংশ কীভাবে একে অপরের সঙ্গে কথা বলে। Compose-এর পূর্ব অভিজ্ঞতা প্রয়োজন নেই। `docker compose` (স্পেস-সহ, v2) ও পুরনো `docker-compose` (হাইফেন-সহ, v1) যে আলাদা — v1 এখন অপ্রচলিত — এই পার্থক্যটি এই অধ্যায়ে বারবার মনে করিয়ে দেওয়া হবে।

১৭.৪ প্রথম নীতি থেকে ব্যাখ্যা

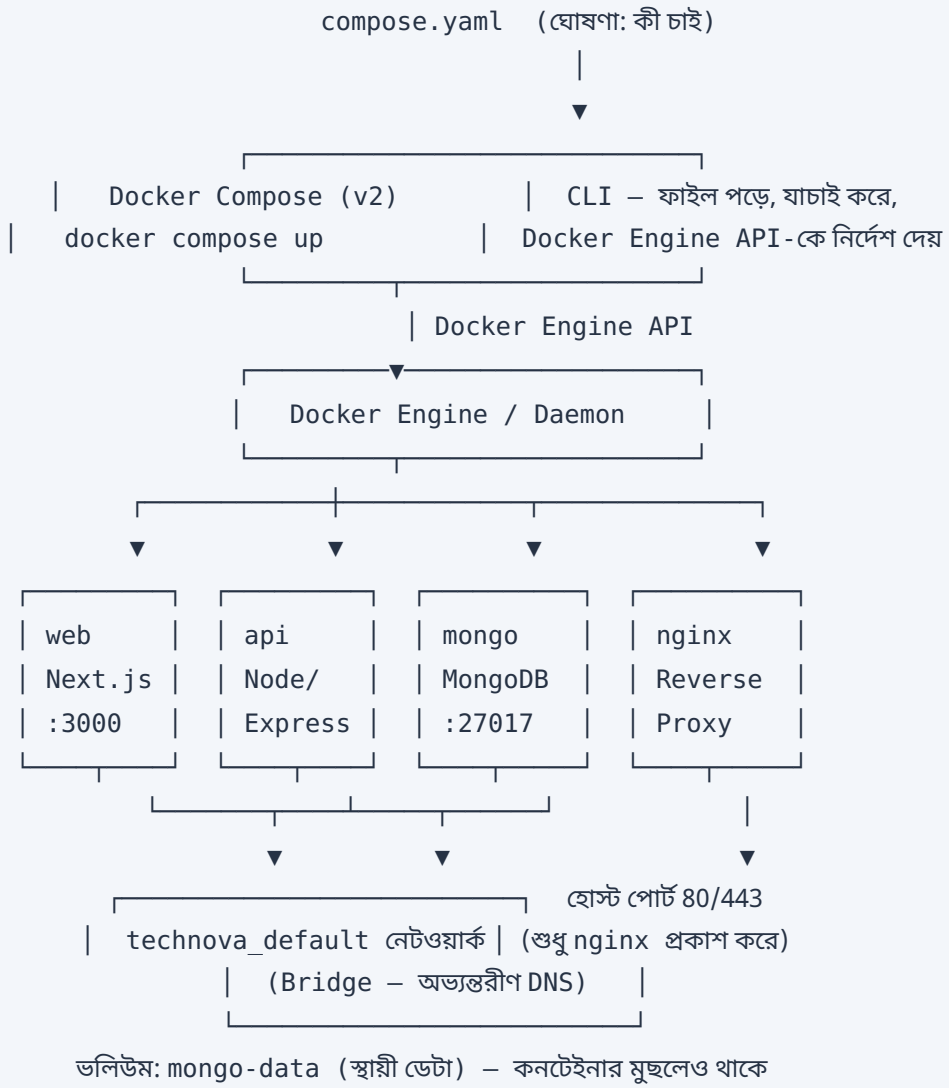
প্রথম নীতি থেকে ভাবুন: একটি অ্যাপ চালাতে হলে কম্পিউটারকে জানতে হয় — কোন প্রোগ্রাম চালাব, কোন পোর্ট খুলব, কোন ফাইল কোথায় রাখব, কোন সেবা কার ওপর নির্ভর করে, আর কোন গোপন মান (Secret) কোথা থেকে আসবে। `docker run` এই সব তথ্য একটিমাত্র দীর্ঘ কমান্ড-লাইনে দেয়, একটি কনটেইনারের জন্য। দুটি কনটেইনার হলে দুটি লম্বা কমান্ড; পাঁচটি হলে পাঁচটি — আর সেগুলোর ক্রম, নেটওয়ার্ক নাম ও ভলিউমের নাম মানুষকে মনে রাখতে হয়। Compose এই সমস্যাটিকে উল্টে দেয়: তথ্যগুলোকে একটি ফাইলে — `compose.yaml` (বা `docker-compose.yml`) — সাজিয়ে লেখা হয়, যেখানে প্রতিটি **Service** একটি চলমান অংশের সংজ্ঞা। একটি Service একটি ধরন বোঝায় (যেমন `api`

সার্ভিস), আর সেই সার্ভিস থেকে তৈরি হওয়া চলমান বস্তুটি হলো **Container**। Service বলে দেয় — কোন Image থেকে চলবে বা কোন Dockerfile থেকে Build হবে, কোন পোর্ট প্রকাশ করবে, কোন Network-এ যুক্ত হবে, কোন Volume মাউন্ট করবে, কোন Environment Variable পাবে, আর কখন Healthy বলে গণ্য হবে। Compose এই ফাইলটি পড়ে প্রথমে একটি **Project** তৈরি করে (সাধারণত ডিরেক্টরির নামে, যেমন `technova`), তারপর প্রতিটি সার্ভিসের জন্য নেটওয়ার্ক ও ভলিউম বানায়, কনটেইনার তৈরি করে এবং নির্ভরতার ক্রমে চালু করে। এটাই Compose-এর মূল ধারণা: *Imperative* কমান্ডের সারি নয়, *Declarative* ঘোষণা — যা চাই তা লেখো, Compose বাস্তবায়ন করুক।

> **CONCEPT:** Service বনাম Container — এই পার্থক্যটি না বুঝলে Compose কখনোই পরিষ্কার হবে না। Service হলো নকশা (যেমন "ব্যাকএন্ড সার্ভিসটি Node 20 ইমেজ থেকে চলবে, 3000 পোর্টে, MongoDB-এর ওপর নির্ভর করে")। Container হলো সেই নকশার চলমান দৃষ্টান্ত (Instance)। একই Service থেকে একাধিক Container চলতে পারে (`docker compose up --scale api=3`) — যেমন একটি নকশা থেকে অনেকগুলো একই রকম বাড়ি বানানো যায়। `docker compose ps` দেখায় Container, `compose.yaml` দেখায় Service।

Project শব্দটিও প্রথমবারে বুঝে নিন: Compose একটি ফাইলকে একটি প্রজেক্ট হিসেবে দেখে। প্রজেক্টের নাম দিয়ে নেটওয়ার্ক, ভলিউম ও কনটেইনারের নামের আগে উপসর্গ (Prefix) বসে — যেমন প্রজেক্ট `technova` হলে নেটওয়ার্কের নাম হয় `technova_default`, ভলিউম হয় `technova_mongo-data`। এই নামকরণের কারণেই একই হোস্টে দুটি আলাদা প্রজেক্টের একই নামের সার্ভিস পরস্পরের সঙ্গে গুলিয়ে যায় না। প্রজেক্টের নাম বদলাতে চাইলে `-p` ফ্ল্যাগ বা `COMPOSE_PROJECT_NAME` পরিবেশ চলক ব্যবহার করা যায় — বিস্তারিত ১৭.৭-এ।

১৭.৫ আর্কিটেকচার / ধারণাগত চিত্র



লক্ষ্য করুন: বাইরের জগতের সঙ্গে কথা বলে কেবল Nginx (পোর্ট 80/443); `api` ও `mongo` শুধু অভ্যন্তরীণ নেটওয়ার্কে দৃশ্যমান। এটাই ১৭.১২-এর Network Isolation-এর ভিত্তি।

Compose নিজে কোনো কনটেইনার চালায় না — এটি Docker Engine-এর একজন পরিকল্পক ও সমন্বয়কারী। Engine-ই আসল কাজ করে: Image টানা, Container তৈরি, নেটওয়ার্ক বানানো। Compose শুধু বলে দেয় কী, কোন ক্রমে, কোন নামে। এই কারণে Compose বন্ধ করলেও (`docker compose down`) Engine চলতে থাকে; আর Engine বন্ধ থাকলে Compose কিছুই করতে পারে না। এই স্তর-বিভাজন বোঝা গুরুত্বপূর্ণ, কারণ সমস্যা সমাধানের সময় প্রশ্ন করতে হয় — সমস্যাটি Compose-এর ঘোষণায়, নাকি Engine/Image/নেটওয়ার্কে?

১৭.৬ বিস্তারিত প্রযুক্তিগত ব্যাখ্যা

Compose ফাইলের শীর্ষ স্তরের কীগুলো

আধুনিক Compose Specification-এ ফাইলের শীর্ষে পাঁচটি প্রধান কী থাকতে পারে। সবচেয়ে গুরুত্বপূর্ণ `services` — এ ছাড়া ফাইল অর্থহীন। `networks` ও `volumes` নাম-দেওয়া (Named) নেটওয়ার্ক ও ভলিউম ঘোষণা করে, যেগুলো

সার্ভিসগুলো উল্লেখ করে। `configs` ও `secrets` ফাইলভিত্তিক কনফিগারেশন ও গোপন মান সরবরাহের ঘোষণামূলক উপায় — তবে একক হোস্টে Compose-এর Secrets মূলত ফাইল মাউন্ট; সত্যিকারের গোপন মান ব্যবস্থাপনা (Vault, ক্লাউড Secret Manager) অধ্যায় ১৯-এ বিস্তারিত। আর একটি কথা স্পষ্ট করে বলা দরকার: পুরনো উদাহরণে ফাইলের শুরুতে `version: "3.8"` লেখা থাকত। আধুনিক Compose Specification-এ এই কীটি **অপ্রচলিত (Obsolete)** — লিখলে সতর্কবার্তা আসে এবং তা উপেক্ষা করা হয়। নতুন ফাইলে এটি লিখবেন না।

শীর্ষ-স্তরের কী	কাজ	উদাহরণ
`services`	চলমান অংশগুলোর সংজ্ঞা (বাধ্যতামূলক)	`web`, `api`, `mongo`, `nginx`
`networks`	নাম-দেওয়া নেটওয়ার্ক ঘোষণা	`frontend`, `backend` (আলাদা অঞ্চল)
`volumes`	নাম-দেওয়া স্থায়ী ভলিউম ঘোষণা	`mongo-data`, `uploads`
`configs`	ফাইল থেকে কনফিগারেশন সরবরাহ	Nginx কনফিগারেশন ফাইল
`secrets`	ফাইল থেকে গোপন মান সরবরাহ (সীমিত)	`db_password` ফাইল
`name`	প্রজেক্টের নাম নির্ধারণ (ঐচ্ছিক)	`name: technova`

একটি Service-এর ভেতরে কী কী থাকে

প্রতিটি সার্ভিসের সংজ্ঞায় কয়েকটি ক্ষেত্র বারবার আসে। `image` বলে কোন Image থেকে চলবে; `build` বলে Dockerfile থেকে বানাতে হবে (দুটো একসঙ্গে থাকলে Build হওয়া Image-টিকে `image`-এর নাম দেওয়া হয় — ট্যাগ করার সুবিধাজনক উপায়)। `container_name` নির্দিষ্ট নাম দেয়, তবে সাধারণত না দেওয়াই ভালো — কারণ নির্দিষ্ট নাম দিলে একই সার্ভিসকে Scale করা যায় না। `ports` হোস্ট পোর্টকে কনটেইনার পোর্টের সঙ্গে বাঁধে (`"8080:80"` মানে হোস্টের 8080 → কনটেইনারের 80); `expose` শুধু একই নেটওয়ার্কের অন্য কনটেইনারের কাছে পোর্ট জানায়, হোস্টে প্রকাশ করে না। `environment` ও `env_file` পরিবেশ চলক দেয়। `volumes` ডেটা মাউন্ট করে। `depends_on` শুরুর ক্রম বলে। `healthcheck` সুস্থতা পরীক্ষা করে। `restart` ব্যর্থতার পর আচরণ নির্ধারণ করে। `networks` বলে কোন নেটওয়ার্কে যুক্ত হবে। এই ক্ষেত্রগুলোর প্রতিটি পরের অধ্যায়গুলোতে আলাদাভাবে গভীরে শেখানো হবে; এখানে শুধু মানচিত্রটি মনে রাখুন।

YAML — যে ভাষায় Compose লেখা হয়

YAML (YAML Ain't Markup Language) মানুষের পড়ার মতো একটি ডেটা ভাষা। এর তিনটি নিয়ম না জানলে Compose ফাইল লেখা যাবে না: (১) **ইন্ডেন্টেশনই গঠন** — স্পেস দিয়ে ভেতরের স্তর বোঝানো হয়; ট্যাব ব্যবহার করা যায় না, কেবল স্পেস। দুই স্পেসের ইন্ডেন্টেশন প্রচলিত। (২) **কী: মান** — কোলনের পর স্পেস আবশ্যক (`image: mongo:7` সঠিক, `image:mongo:7` ভুল)। (৩) **তালিকা** হয় `- আইটেম` দিয়ে (ড্যাশ ও স্পেস) অথবা `[a, b]` বন্ধনীতে। আরেকটি সূক্ষ্ম নিয়ম: কোলন বা বিশেষ চিহ্ন-সহ মান (যেমন পোর্ট ম্যাপিং `8080:80`) উদ্ধৃতি চিহ্নে (`"8080:80"`) লেখা নিরাপদ, না হলে YAML সেটিকে সময় বা সংখ্যা ভেবে ভুল ব্যাখ্যা করতে পারে। স্ট্রিং হিসেবে ব্যাখ্যা নিশ্চিত করতে পোর্ট ম্যাপিং সবসময় উদ্ধৃত করুন — এটি নবীনদের সবচেয়ে সাধারণ ভুলগুলোর একটি।

পরিবেশ চলক প্রতিস্থাপন (Variable Interpolation)

Compose ফাইলে `${VAR}` লিখলে Compose প্রথমে `.env` ফাইল (প্রজেক্ট ডিরেক্টরিতে) ও শেলের পরিবেশ চলক থেকে মান খোঁজে। `${VAR:-default}` মানে চলক না থাকলে `default` ব্যবহার করো; `${VAR:?error message}` মানে চলক না থাকলে ভুল দেখিয়ে থেমে যাও। এই ব্যবস্থাটি দিয়ে একই Compose ফাইল বিভিন্ন পরিবেশে (dev/staging/prod) বিভিন্ন Image ট্যাগ, পোর্ট বা ডেটাবেস নাম নিয়ে চলতে পারে। তবে মনে রাখবেন: `.env` ফাইলটি Compose ফাইলের নিজের

চলক পূরণের জন্য; কনটেইনারের ভেতরের অ্যাপের চলক দেওয়ার জন্য `env_file` ব্যবহার করা হয় — দুটির কাজ আলাদা, গুলিয়ে ফেলবেন না (অধ্যায় ১৯-এ বিস্তারিত)।

নেটওয়ার্ক: Service Discovery কীভাবে কাজ করে

একই Compose প্রজেক্টের সার্ভিসগুলো স্বয়ংক্রিয়ভাবে একটি Bridge নেটওয়ার্কে যুক্ত হয় এবং একটি অভ্যন্তরীণ DNS পায়। এর ফলে `api` কনটেইনার থেকে `mongo` নামে (সার্ভিসের নামে) MongoDB-কে ডাকা যায় — কোনো IP মুখস্থ করতে হয় না: সংযোগ স্ট্রিং হয় `mongodb://mongo:27017/technova`। Service-এর নামই Hostname। এই DNS নাম কনটেইনার মুছে আবার তৈরি করলেও একই থাকে, কারণ নামটি Service-এর, কোনো নির্দিষ্ট Container-এর নয়। আলাদা নেটওয়ার্ক ঘোষণা করে ফ্রন্টএন্ড ও ব্যাকএন্ডকে আলাদা অঞ্চলে ভাগ করা যায় — যেমন `mongo` শুধু `backend` নেটওয়ার্কে থাকবে, `nginx` থাকবে `frontend` ও `backend` দুটিতেই — ফলে বাইরের দিক থেকে ডেটাবেসে পৌঁছানোর কোনো পথই থাকে না।

ভলিউম: Named Volume বনাম Bind Mount

দুটি মাউন্ট পদ্ধতির পার্থক্য মৌলিক। **Named Volume** (`mongo-data:/data/db`) Docker-এর নিজের পরিচালিত স্টোরেজ — হোস্টে কোথায় আছে তা নিয়ে ভাবতে হয় না, `docker compose down` করলেও ডেটা থেকে যায়, শুধু `down -v` করলেই মুছে যায়। ডেটাবেসের ডেটা এখানেই রাখা উচিত। **Bind Mount** (`./src:/app/src`) হোস্টের একটি নির্দিষ্ট ডিরেক্টরিকে কনটেইনারে বাঁধে — ডেভেলপমেন্টে সোর্স কোড সরাসরি দেখানোর জন্য উপযোগী, কিন্তু প্রোডাকশনে বিপজ্জনক (হোস্টের ফাইল কনটেইনার থেকে বদলে যাওয়া যায়, এবং কনটেইনার হোস্টের ফাইল দেখতে পায় — অধ্যায় ২১-এ বিস্তারিত)। নিয়ম: ডেটার জন্য Named Volume, ডেভেলপমেন্টে লাইভ কোডের জন্য Bind Mount, কনফিগারেশনের জন্য শুধু-পড়ার (read-only) Bind Mount।

depends_on — ক্রম বনাম প্রস্তুতি

`depends_on` শুধু বলে কোন কনটেইনার আগে শুরু হবে। এটি কখনোই বলে না যে আগের সার্ভিসের অ্যাপ্লিকেশনটি প্রস্তুত। MongoDB কনটেইনার শুরু হওয়ার পরও ডেটাবেসের প্রথমবার চালু হতে (WiredTiger ইনিশিয়ালাইজ, ইনডেক্স তৈরি) কয়েক সেকেন্ড লাগে; সেই সময়ের মধ্যে API সংযোগ করতে গেলে ব্যর্থ হবে। আসল প্রস্তুতির জন্য `depends_on`-এর সঙ্গে `condition: service_healthy` এবং ডেটাবেসের `healthcheck` একসঙ্গে লিখতে হয় — এবং তারপরও অ্যাপের নিজের পুনঃচেষ্টা (Retry) যুক্তি দরকার। এই পার্থক্যটি এত গুরুত্বপূর্ণ যে অধ্যায় ২০ পুরোটাই এটি নিয়ে।

Restart Policy ও Profiles

`restart`-এর চারটি মান: `no` (কখনো নয় — ডিফল্ট), `always` (সবসময় চালু করার চেষ্টা — হোস্ট রিবুটের পরও), `unless-stopped` (সবসময়, তবে মানুষ ইচ্ছা করে থামালে আর নয় — প্রোডাকশনে এটিই সবচেয়ে উপযোগী), এবং `on-failure` (শুধু ব্যর্থ হয়ে বন্ধ হলে, `on-failure:3` মানে সর্বোচ্চ তিনবার)। **Profiles** দিয়ে কিছু সার্ভিসকে ঐচ্ছিক করা যায়: যেমন `profiles: ["tools"]` দেওয়া `mongo-express` (ডেটাবেস দেখার টুল) সাধারণ `docker compose up`-এ চালু হবে না; চালাতে হলে `docker compose --profile tools up` লিখতে হয়। এর ফলে একটিমাত্র ফাইলে মূল অ্যাপ ও ঐচ্ছিক টুল — দুটোই থাকে, অথচ প্রোডাকশনে টুলটি ভুলে চালু হওয়ার ঝুঁকি থাকে না।

Compose v2 বনাম v1 এবং `docker compose` কমান্ডের প্রকৃতি

পুরনো `docker-compose` ছিল পাইথনে লেখা আলাদা প্রোগ্রাম; আধুনিক `docker compose` হলো Docker CLI-এর একটি প্লাগইন, Go ভাষায় লেখা। ফলে v2 দ্রুত, একই `docker` বাইনারির সঙ্গে আসে এবং Compose Specification অনুসরণ করে। পুরনো টিউটোরিয়ালে `docker-compose up` দেখলে বুঝবেন সেটি পুরনো নির্দেশ; নতুন পরিবেশে `docker compose up` ব্যবহার করুন। `docker compose version` চালিয়ে আপনার সংস্করণ যাচাই করুন — 2.x হলেই আধুনিক।

১৭.৭ Docker কমান্ড ও লাইন-বাই-লাইন ব্যাখ্যা

নিচের কমান্ডগুলো TechNova প্রজেক্ট ডিরেক্টরি (`~/technova`) থেকে চালানো ধরা হয়েছে, যেখানে `compose.yaml` আছে। প্রতিটি কমান্ডের পর তার প্রতিটি অংশের ব্যাখ্যা দেওয়া হলো:

```
$ docker compose version
Docker Compose version v2.27.0
$ docker compose config
# ফাইলটি যাচাই করে, চলক পূরণ করে, চূড়ান্ত কনফিগারেশন দেখায় — ভুল ধরার সেরা উপায়
$ docker compose up -d
$ docker compose ps
$ docker compose logs -f api
$ docker compose down
```

লাইন-বাই-লাইন ব্যাখ্যা: (১) `docker compose version` — ইনস্টল করা Compose প্লাগইনের সংস্করণ দেখায়; 2.x নিশ্চিত করুন, কারণ v1-এর সিনট্যাক্স ও আচরণে পার্থক্য আছে। (২) `docker compose config` — ফাইলটি না চালিয়ে শুধু যাচাই করে; YAML ভুল, অজানা চলক বা ভুল সার্ভিস রেফারেন্স থাকলে এখানেই ভুল দেখায়। কোনো কিছু চালুর আগে এটি চালানো পেশাদার অভ্যাস। (৩) `docker compose up -d` — `up` মানে নেটওয়ার্ক ও ভলিউম তৈরি করে সব সার্ভিস চালু করো; `-d` (detached) মানে টার্মিনাল আটকে না রেখে পেছনে (Background) চালাও। `-d` ছাড়া চালালে লগ টার্মিনালে ভাসতে থাকে এবং `Ctrl+C` দিলে সব বন্ধ হয়ে যায় — ডেভেলপমেন্টে লগ দেখার জন্য ভালো, সার্ভারে নয়। (৪) `docker compose ps` — শুধু এই প্রজেক্টের কনটেইনারগুলোর অবস্থা (State), পোর্ট ও স্বাস্থ্য দেখায়; `docker ps` হোস্টের সব কনটেইনার দেখায়। (৫) `docker compose logs -f api` — `logs` মানে লগ দেখাও; `-f` (follow) মানে নতুন লগ আসতে থাকলে দেখাতে থাকো; শেষের `api` শুধু সেই সার্ভিসের লগ দেখায়। (৬) `docker compose down` — কনটেইনার ও প্রজেক্টের নেটওয়ার্ক থামিয়ে মুছে দেয়, কিন্তু Named Volume-এর ডেটা রেখে দেয়।

```
$ docker compose up -d --build
$ docker compose up -d --scale api=3
$ docker compose --profile tools up -d
$ docker compose -f compose.yaml -f compose.prod.yaml up -d
$ docker compose exec api sh
$ docker compose down -v --rmi local
```

ব্যাখ্যা (দ্বিতীয় সেট): `--build` চালুর আগে Image আবার Build করে (কোড বদলালে দরকার)। `--scale api=3` একই সার্ভিসের তিনটি কনটেইনার চালায় — তবে তখন সার্ভিসে নির্দিষ্ট `container_name` বা নির্দিষ্ট হোস্ট পোর্ট থাকা যাবে না (নাম ও পোর্টের সংঘাত হবে)। `--profile tools` শুধু সেই প্রোফাইলের ঐচ্ছিক সার্ভিসগুলোও চালু করে। `-f` দিয়ে একাধিক ফাইল দিলে পরের ফাইল আগেরটিকে ওভাররাইড করে (মার্জ হয়, পুরো প্রতিস্থাপন নয় — ১৭.৮ দেখুন)। `exec api sh` চলমান `api` কনটেইনারের ভেতরে একটি শেল খোলে — ডিবাগিংয়ের জন্য, নতুন কনটেইনার তৈরি করে না। `down -v` ভলিউমও মুছে দেয় (ডেটা হারাবে — সতর্ক থাকুন!) এবং `--rmi local` স্থানীয়ভাবে Build করা Image-ও মুছে দেয়।

নমুনা আউটপুট (চিত্রণমূলক):

```
$ docker compose ps
NAME                IMAGE               COMMAND                  SERVICE   CREATED
STATUS                    PORTS
technova-api-1      technova-api:1.2.0  "node server.js"         api       2 minutes ago    Up
2 minutes (healthy)  3000/tcp
technova-mongo-1    mongo:7             "docker-entrypoint.s…"   mongo     2 minutes ago    Up
2 minutes (healthy)  27017/tcp
technova-nginx-1    nginx:1.27          "/docker-entrypoint.…"   nginx     2 minutes ago    Up
2 minutes            0.0.0.0:80->80/tcp
```

```
technova-web-1      technova-web:1.2.0  "node server.js"         web       2 minutes ago    Up
2 minutes            3000/tcp
```

লক্ষ্য করুন: `STATUS`-এ `(healthy)` লেখা আছে কেবল যেসব সার্ভিসে `healthcheck` আছে সেগুলোতে; বাকিগুলো শুধু `Up` দেখায় — "চলছে" মানেই "সুস্থ" নয় (অধ্যায় ২০ ও ২২-এ বিস্তারিত)। কনটেইনারের নামের গঠন `প্রজেক্ট-সার্ভিস-সংখ্যা` — এখানে `technova-api-1`।

১৭.৮ সম্পূর্ণ কনফিগারেশন উদাহরণ

নিচে TechNova-র সম্পূর্ণ `compose.yaml` দেওয়া হলো — এটি অধ্যায় ১৮-এর ভিত্তি। প্রতিটি লাইনের উদ্দেশ্য পরের অনুচ্ছেদে ব্যাখ্যা করা হয়েছে। মনে রাখবেন: `version:` কী নেই — ইচ্ছাকৃতভাবে, কারণ এটি অপ্রচলিত।

```
name: technova

services:
  mongo:
    image: mongo:7
    restart: unless-stopped
    environment:
      MONGO_INITDB_ROOT_USERNAME: ${MONGO_USER:-technova_admin}
      MONGO_INITDB_ROOT_PASSWORD: ${MONGO_PASSWORD:?MONGO_PASSWORD চলকটি .env ফাইলে দিতে হবে}
      MONGO_INITDB_DATABASE: technova
    volumes:
      - mongo-data:/data/db
    networks:
      - backend
    healthcheck:
      test: ["CMD", "mongosh", "--eval", "db.adminCommand('ping')"]
      interval: 10s
      timeout: 5s
      retries: 5
      start_period: 30s

  api:
    build:
      context: ./api
      dockerfile: Dockerfile
    image: technova-api:1.2.0
    restart: unless-stopped
    depends_on:
      mongo:
        condition: service_healthy
    environment:
      NODE_ENV: production
      PORT: "3000"
      MONGODB_URI: mongodb://${MONGO_USER:-technova_admin}:${MONGO_PASSWORD}@mongo:27017/
technova?authSource=admin
    networks:
      - backend
    healthcheck:
      test: ["CMD", "node", "-e", "fetch('http://localhost:3000/
health').then(r=>{process.exit(r.ok?0:1)}).catch(()=>process.exit(1))"]
      interval: 15s
      timeout: 5s
      retries: 3
      start_period: 20s
```

```
  web:
    build:
      context: ./web
      dockerfile: Dockerfile
    image: technova-web:1.2.0
    restart: unless-stopped
    depends_on:
      - api
    networks:
      - frontend
      - backend
    environment:
      NODE_ENV: production

  nginx:
    image: nginx:1.27
    restart: unless-stopped
    depends_on:
      - web
      - api
    ports:
      - "80:80"
      - "443:443"
    volumes:
      - ./nginx/nginx.conf:/etc/nginx/conf.d/default.conf:ro
      - ./nginx/certs:/etc/nginx/certs:ro
    networks:
      - frontend

networks:
  frontend:
  backend:

volumes:
  mongo-data:
```

ফাইলটি পড়ার পদ্ধতি: `mongo` সার্ভিসটি অফিসিয়াল `mongo:7` Image থেকে চলে; ডেটাবেসের ব্যবহারকারীর নাম ও পাসওয়ার্ড `.env` থেকে আসে (`MONGO_PASSWORD` না থাকলে Compose ভুল দেখিয়ে থামবে — `:?` চিহ্নের কাজ); ডেটা `mongo-data` Named Volume-এ থাকে; `healthcheck` প্রতি ১০ সেকেন্ডে `mongosh` দিয়ে `ping` করে দেখে ডেটাবেস সত্যিই সাড়া দিচ্ছে কি না; `start_period: 30s` মানে প্রথম ৩০ সেকেন্ড ব্যর্থতা গোনা হবে না (প্রথমবার চালু হতে সময় লাগে)। `api` সার্ভিসটি `./api/Dockerfile` থেকে Build হয়ে `technova-api:1.2.0` নাম পায়; `depends_on`-এর `condition: service_healthy` মানে MongoDB সুস্থ না হওয়া পর্যন্ত API চালুই হবে না; `MONGODB_URI`-তে হোস্ট হিসেবে `mongo` — সার্ভিসের নাম — ব্যবহৃত হয়েছে, IP নয়। `web` দুটি নেটওয়ার্কেই আছে (ফ্রন্টএন্ডের সঙ্গে Nginx-এর এবং ব্যাকএন্ডের সঙ্গে API-এর কথা বলতে পারে), `nginx` শুধু `frontend` নেটওয়ার্কে এবং একমাত্র সার্ভিস যার পোর্ট হোস্টে প্রকাশ করা হয়েছে। `:ro` মানে কনফিগারেশন ভলিউম শুধু-পড়ার — কনটেইনার ভেতর থেকে Nginx কনফিগ বদলাতে পারবে না।

Dev/Prod ওভাররাইড: ডেভেলপমেন্টে একই বেস ফাইলের সঙ্গে `compose.dev.yaml` যোগ করা হয়, যেখানে `api` সার্ভিসে Bind Mount (`./api/src:/app/src`) ও `command: npm run dev` ওভাররাইড থাকে — ফলে কোড বদলালে কনটেইনার আবার Build করতে হয় না। চালুর কমান্ড: `docker compose -f compose.yaml -f compose.dev.yaml up -d`। প্রোডাকশনে উল্টো: `compose.prod.yaml` রিসোর্স সীমা, লগ রোটেশন ও `read_only: true` যোগ করে।

Compose ফাইল মার্জ করার নিয়ম: ম্যাপ (যেমন `environment`) একত্র হয়, তালিকা (যেমন `ports`) প্রতিস্থাপিত হয় না — সতর্কভাবে পরীক্ষা করতে `docker compose -f ... -f ... config` চালিয়ে চূড়ান্ত ফল দেখে নিন।

```
# .env — Compose ফাইলের চলক পূরণের জন্য (Git-এ কমিট করবেন না!)
MONGO_USER=technova_admin
MONGO_PASSWORD=CHANGE-ME-STRONG-PASSWORD
IMAGE_TAG=1.2.0
# compose.dev.yaml — ডেভেলপমেন্ট ওভাররাইড (সংক্ষিপ্ত)
services:
  api:
    volumes:
      - ./api/src:/app/src
    command: npm run dev
    environment:
      NODE_ENV: development
  mongo:
    ports:
      - "27017:27017"   # ডেভেলপমেন্টে ডেটাবেস সরাসরি দেখার জন্য; প্রোডাকশনে নয়
```

১৭.৯ বাস্তব প্রোডাকশন পরিস্থিতি

REAL WORLD — TechNova: TechNova-র নতুন ডেভেলপার রিয়া প্রথম দিনে প্রজেক্টটি চালু করতে গিয়ে দুই ঘণ্টা নষ্ট করলেন: তিনি README-তে লেখা পাঁচটি আলাদা `docker run` কমান্ড একে একে চালালেন, তৃতীয় কমান্ডে একটি পোর্ট ভুল লিখলেন, আর MongoDB-র ভলিউমের নাম ভুলে গিয়ে ডেটা হারালেন। দল সিদ্ধান্ত নিল — আর কোনো হাতে লেখা কমান্ড নয়। এখন `compose.yaml` একটিই সত্যের উৎস (Single Source of Truth): নতুন কেউ `git clone` করে `.env` ফাইলের নমুনা (`.env.example`) কপি করে `docker compose up -d --build` চালালেই পুরো স্ট্যাক ওঠে। ডেটাবেসের ডেটা Named Volume-এ, নেটওয়ার্কের নাম স্বয়ংক্রিয়, আর পোর্টের সংঘাত আগেই ধরা পড়ে। Onboarding সময় দুই ঘণ্টা থেকে পনেরো মিনিটে নেমে এলো — এটাই Compose-এর বাস্তব মূল্য।

১৭.১০ ধাপে ধাপে ব্যবহারিক বাস্তবায়ন

উদাহরণ: আপনার মেশিনে TechNova স্ট্যাকটি চালু করা

1. প্রজেক্ট ডিরেক্টরিতে `compose.yaml` ও `.env.example` আছে কি না যাচাই করুন; `.env.example` কপি করে `.env` বানান এবং `CHANGE-ME-...` মানগুলো বদলে নিন।
2. `docker compose config` চালিয়ে ফাইলের সিনট্যাক্স ও চলক পূরণ যাচাই করুন — ভুল থাকলে এখানেই থামুন ও ঠিক করুন।
3. `docker compose up -d --build` চালিয়ে পুরো স্ট্যাক চালু করুন; প্রথমবার Image Build হতে কয়েক মিনিট লাগতে পারে।
4. `docker compose ps` দিয়ে দেখুন সব সার্ভিস `Up` আছে কি না এবং `mongo` ও `api` `(healthy)` দেখাচ্ছে কি না।
5. `docker compose logs -f api` দিয়ে API-র লগ দেখুন — "MongoDB connected" ধরনের বার্তা আসছে কি না নিশ্চিত করুন।
6. ব্রাউজারে `http://localhost` খুলে দেখুন Nginx ফ্রন্টএন্ড দিচ্ছে কি না; `curl http://localhost/api/health` দিয়ে API পরীক্ষা করুন।

7. কাজ শেষে `docker compose down` চালান (ডেটা রাখতে) — ডেটা মুছতে চাইলে তবেই `down -v`।

১৭.১১ সাধারণ ভুল ও ভুল ধারণা

- **ভুল:** ফাইলের শুরুতে `version: "3.8"` লেখা — অপ্রচলিত; সতর্কবার্তা আসবে। নতুন ফাইলে লিখবেন না।
- **ভুল:** পোর্ট ম্যাপিং উদ্ধৃতি ছাড়া লেখা (`80:80`) — YAML ভুল ব্যাখ্যা করতে পারে; সবসময় `"80:80"` লিখুন।
- **ভুল:** `depends_on` মানেই অ্যাপ প্রস্তুত — ভুল; এটি শুধু কনটেইনার শুরুর ক্রম। প্রস্তুতির জন্য `healthcheck` + `service_healthy` + অ্যাপের Retry লাগে।
- **ভুল:** `docker compose down -v` অভ্যাসবশত চালানো — Named Volume-সহ সব ডেটা মুছে যাবে। প্রোডাকশনে কখনো ভাবনা ছাড়া নয়।
- **ভুল:** `latest` ট্যাগ ব্যবহার — কোন সংস্করণ চলছে বোঝা যায় না; সবসময় সংস্করণ ট্যাগ দিন (`mongo:7`, `technova-api:1.2.0`)।
- **ভুল ধারণা:** "Compose মানেই অর্কেস্ট্রেশন" — না। Compose একটি হোস্টে কনটেইনার সাজায়; একাধিক হোস্ট, স্বয়ংক্রিয় স্কেলিং, সার্ভিস ডিসকভারি, রোলিং আপডেটের পূর্ণ ব্যবস্থা এর কাজ নয় — সেটি Kubernetes-এর মতো অর্কেস্ট্রেটরের কাজ (অধ্যায় ২৪)।
- **ভুল ধারণা:** "`expose` দিলেই হোস্ট থেকে পোর্টে যাওয়া যায়" — না; `expose` শুধু ডকুমেন্টেশন ও আন্তঃকনটেইনার দৃশ্যমানতা, হোস্টে প্রকাশ করতে `ports` লাগে।

১৭.১২ নিরাপত্তা বিবেচনা

> **WARNING:** ডেভেলপমেন্টের সুবিধার জন্য MongoDB-র পোর্ট (`27017:27017`) হোস্টে প্রকাশ করা প্রোডাকশনে ভয়ংকর ভুল — ইন্টারনেটে খোলা ডেটাবেস পোর্ট মানে যে কেউ পাসওয়ার্ড অনুমান করে ডেটা পড়তে বা মুছতে পারে। প্রোডাকশনে কেবল Nginx-এর 80/443 প্রকাশ করুন; ডেটাবেস শুধু অভ্যন্তরীণ নেটওয়ার্কে থাকুক। আর `.env` ফাইল কখনো Git-এ কমিট করবেন না — `.gitignore`-এ `.env` রাখুন এবং শুধু `.env.example` (আসল মান ছাড়া) কমিট করুন।

Compose ফাইলে `environment`-এ সরাসরি আসল পাসওয়ার্ড লেখা আরেকটি সাধারণ ভুল — ফাইলটি Git-এ গেলে Secret-ও চলে যায়। পাসওয়ার্ড সবসময় `.env` বা বাইরের Secret উৎস থেকে আসুক। ডকার সকেট (`/var/run/docker.sock`) কখনো কনটেইনারে মাউন্ট করবেন না — সকেটে পৌঁছানো মানেই কার্যত হোস্টের root ক্ষমতা (অধ্যায় ২১-এ বিস্তারিত)। নেটওয়ার্ক আলাদা করা (`frontend` বনাম `backend`) একটি সহজ কিন্তু কার্যকর সুরক্ষা: আক্রমণকারী ফ্রন্টএন্ডে ঢুকলেও ডেটাবেস নেটওয়ার্কে সরাসরি পৌঁছাতে পারবে না।

১৭.১৩ কর্মক্ষমতা ও রিসোর্স বিবেচনা

Compose নিজে প্রায় কোনো রিসোর্স খায় না — খরচ হয় কনটেইনারগুলোর। তবে দুটি বিষয় মনে রাখুন: (১) প্রতিটি সার্ভিসের Image প্রথমবার টানতে (Pull) সময় ও ব্যান্ডউইথ লাগে — প্রোডাকশন সার্ভারে আগে থেকে Image টেনে রাখলে Deploy দ্রুত হয়। (২) `--build` প্রতিবার চালালে Build Cache ব্যবহার হয়; Cache নষ্ট হলে (যেমন Dockerfile-এর শুরুর লাইন বদলালে) পুরো Build আবার হয়। এই অধ্যায়ে রিসোর্স সীমা (`deploy.resources`) নিয়ে বিস্তারিত আলোচনা করা হয়নি — সেটি অধ্যায় ২২-এর বিষয়; তবে জেনে রাখুন, Compose-এ `deploy.resources.limits` একক হোস্টে (Swarm ছাড়া) সবসময় কার্যকর হয় না — CPU/মেমোরি সীমা দেওয়ার নির্ভরযোগ্য উপায় অধ্যায় ২২-এ শেখানো হবে।

১৭.১৪ সমস্যা সমাধান পদ্ধতি

TROUBLESHOOTING: `docker compose up` ব্যর্থ হলে ধাপে ধাপে যান: (১) `docker compose config` — YAML বা চলকের ভুল কি না; (২) `docker compose logs <service>` — কোন সার্ভিস কী বলছে; (৩) `docker compose ps` — কোনটি `Exited` বা `Restarting`; (৪) পোর্ট সংঘাত হলে `ss -tlnp | grep :80` দিয়ে দেখুন হোস্টে 80 পোর্ট কে ধরে আছে। সবচেয়ে সাধারণ ভুল বার্তা: `port is already allocated` (হোস্ট পোর্ট ব্যবহৃত) ও `network ... not found` (পুরনো নেটওয়ার্কের রেফারেন্স)।

সমস্যা	সম্ভাব্য কারণ	সমাধান
`yaml: line X: mapping values are not allowed`	ইন্ডেন্টেশন ভুল বা কোলনের পর স্পেস নেই	স্পেস দিয়ে ইন্ডেন্টেশন ঠিক করুন; ট্যাব ব্যবহার করবেন না
`port is already allocated`	হোস্টে একই পোর্টে অন্য কিছু চলছে	অন্য পোর্ট ব্যবহার করুন বা আগের প্রসেস থামান
`service "api" depends on undefined service "mongo"`	সার্ভিসের নামের বানান ভুল	`services`-এর নামের সঙ্গে মিলিয়ে নিন
API চালু হয়েই MongoDB সংযোগে ব্যর্থ	`depends_on` শুধু ক্রম দেয়, প্রস্তুতি নয়	`healthcheck` + `service_healthy` + Retry যোগ করুন (অধ্যায় ২০)
`up` করলেও পুরনো কোড চলছে	`--build` ছাড়া চালানো হয়েছে	`docker compose up -d --build` চালান

১৭.১৫ সেরা অনুশীলন ও অ্যান্টি-প্যাটার্ন

সেরা অনুশীলন: একটি প্রজেক্টে একটি মূল `compose.yaml` রাখুন এবং পরিবেশভেদে ছোট ওভাররাইড ফাইল ব্যবহার করুন; সব Image-এ সংস্করণ ট্যাগ দিন; চালুর আগে `docker compose config` দিয়ে যাচাই করুন; `.env.example` কমিট করুন, `.env` নয়; সার্ভিসের নাম ছোট ও অর্থবোধক রাখুন (`api`, `web`, `mongo`); ডেটাবেসের ডেটা সবসময় Named Volume-এ রাখুন। **অ্যান্টি-প্যাটার্ন:** দীর্ঘ `docker run` কমান্ডের তালিকা README-তে রাখা; `container_name` দিয়ে সব সার্ভিস বেঁধে দেওয়া (Scale করা যায় না); `latest` ট্যাগ; `version:` কী লেখা; প্রোডাকশনে ডেভেলপমেন্ট ওভাররাইড ফাইল চালানো; একটি বিশাল Compose ফাইলে সব পরিবেশের সবকিছু গুঁজে দেওয়া।

১৭.১৬ অধ্যায়ের সারাংশ

Docker Compose একটি মাল্টি-কনটেইনার ব্যবস্থাকে একটি ঘোষণামূলক YAML ফাইলে প্রকাশ করে: Service হলো চলমান অংশের সংজ্ঞা, Container তার চলমান দৃষ্টান্ত, Project হলো ফাইলটির পরিচয় যার নামে নেটওয়ার্ক ও ভলিউমের নাম তৈরি হয়। `services`, `networks` ও `volumes` — এই তিনটি শীর্ষ কী দিয়েই বেশিরভাগ কাজ হয়; আধুনিক ফাইলে `version:` কী লেখা হয় না। Service-এর ভেতরের DNS নামে সার্ভিসগুলো একে অপরকে ডাকে, Named Volume ডেটা স্থায়ী করে, `depends_on` শুরুর ক্রম দেয় (প্রস্তুতি নয়), আর একাধিক ফাইল ও Profiles দিয়ে পরিবেশভেদে কনফিগারেশন সাজানো যায়। Compose একটি হোস্টের সমন্বয়কারী — পূর্ণাঙ্গ অর্কেস্ট্রেটর নয়।

১৭.১৭ মূল প্রযুক্তিগত পরিভাষা

Docker Compose: একাধিক কনটেইনার-সার্ভিসকে একটি YAML ফাইলে ঘোষণা করে পরিচালনার টুল (v2: `docker compose`)।

Service: Compose ফাইলে একটি চলমান অংশের সংজ্ঞা — কোন Image, পোর্ট, ভলিউম, নেটওয়ার্ক ও পরিবেশ চলক নিয়ে চলবে।

Container: একটি Service-এর চলমান দৃষ্টান্ত (Instance)।

Project: একটি Compose ফাইল (ও তার ওভাররাইড) নিয়ে গঠিত পরিচয়; এর নামে নেটওয়ার্ক/ভলিউম/কনটেইনারের নামের উপসর্গ বসে।

Declarative: "কী চাই" ঘোষণা করার ধরন — "কীভাবে করবে" ধাপে ধাপে না বলে।

YAML: মানুষের পড়ার মতো ডেটা ভাষা; ইন্ডেন্টেশন দিয়ে গঠন বোঝায়, ট্যাব নিষিদ্ধ।

Named Volume: Docker-এর পরিচালিত স্থায়ী স্টোরেজ; কনটেইনার মুছলেও ডেটা থাকে।

Bind Mount: হোস্টের নির্দিষ্ট পথকে কনটেইনারে বাঁধা; ডেভেলপমেন্টে উপযোগী, প্রোডাকশনে ঝুঁকিপূর্ণ।

Service Discovery: সার্ভিসের নামে (DNS) এক কনটেইনার থেকে আরেক কনটেইনারকে খুঁজে পাওয়ার ব্যবস্থা।

Profile: ঐচ্ছিক সার্ভিসের দল, যা শুধু `--profile` দিলে চালু হয়।

Override File: বেস Compose ফাইলের ওপর মার্জ হওয়া অতিরিক্ত ফাইল (যেমন dev/prod)।

Interpolation: Compose ফাইলে `${VAR}` লিখে `.env` বা শেল থেকে মান বসানো।

১৭.১৮ জ্ঞান যাচাই — প্রশ্ন

1. Service ও Container-এর পার্থক্য নিজের ভাষায় ব্যাখ্যা করুন। একই Service থেকে তিনটি Container চালালে কী হবে?

2. আধুনিক Compose ফাইলে `version:` কী কেন লেখা হয় না?
3. `ports` ও `expose`-এর পার্থক্য কী? কখন কোনটি ব্যবহার করবেন?
4. Named Volume ও Bind Mount-এর পার্থক্য কী? ডেটাবেসের ডেটার জন্য কোনটি উপযুক্ত এবং কেন?
5. `depends_on` কেন অ্যাপের প্রস্তুতি নিশ্চিত করে না? প্রস্তুতি নিশ্চিত করতে কী কী লাগে?
6. `docker compose down` ও `docker compose down -v`-এর পার্থক্য কী? দ্বিতীয়টি কখন বিপজ্জনক?
7. একই Compose ফাইল দিয়ে dev ও prod পরিবেশ আলাদা করতে ওভাররাইড ফাইল কীভাবে ব্যবহার করবেন?
8. Compose কেন পূর্ণাঙ্গ অর্কেস্ট্রেটর নয় — তিনটি সীমা লিখুন।

১৭.১৯ ব্যবহারিক অনুশীলন

1. **প্রথম স্ট্যাক:** এই অধ্যায়ের `compose.yaml` লিখে `docker compose config` দিয়ে যাচাই করুন, তারপর `up -d` করে `ps` দিয়ে অবস্থা দেখুন।
2. **ওভাররাইড:** একটি `compose.dev.yaml` লিখুন যেখানে MongoDB-র পোর্ট হোস্টে প্রকাশ পায়; `config` কমান্ডে মার্জ করা ফল দেখে নিশ্চিত করুন প্রোডাকশন ফাইলে পোর্টটি নেই।
3. **Profile:** `mongo-express` নামে একটি ঐচ্ছিক সার্ভিস `tools` প্রোফাইলে যোগ করুন; সাধারণ `up`-এ সেটি চালু হয় না তা যাচাই করুন, তারপর `--profile tools up` দিয়ে চালান।
4. **ভুল ধরা:** ইচ্ছাকৃতভাবে একটি YAML ইন্ডেন্টেশন ভুল ও একটি ভুল সার্ভিস নাম লিখে `config`-এর ভুল বার্তা পড়ে ঠিক করুন।

অধ্যায় 18 • PART IV — Docker Compose and Multi-Container Systems

Chapter 18 — Building Multi-Container MERN Applications

১৮.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: (১) MERN স্ট্যাকের চারটি অংশের প্রতিটির জন্য উপযুক্ত Dockerfile লিখতে; (২) Multi-Stage Build দিয়ে ছোট ও নিরাপদ প্রোডাকশন Image তৈরি করতে; (৩) ফ্রন্টএন্ডের Build-সময়ের (Build-time) ও রানটাইমের (Runtime) পরিবেশ চলকের পার্থক্য ব্যাখ্যা ও প্রয়োগ করতে; (৪) ব্যাকএন্ডে শুধু Production নির্ভরতা (`npm ci --omit=dev`) ইনস্টল করতে; (৫) কনটেইনারগুলোর মধ্যে অভ্যন্তরীণ যোগাযোগ ও নেটওয়ার্ক সীমানা নকশা করতে; (৬) ডেটাবেসের স্থায়ীত্ব (Persistence), ব্যাকআপ ও পুনরুদ্ধার ব্যবস্থা করতে; (৭) Graceful Shutdown ও লগ ব্যবস্থা বাস্তবায়ন করতে; (৮) Dev ও Prod কনফিগারেশনের পার্থক্য স্পষ্টভাবে সাজাতে।

১৮.২ কেন এই বিষয়টি DevOps-এ গুরুত্বপূর্ণ

অধ্যায় ১৭-এ আপনি Compose-এর কাঠামো শিখেছেন; এবার সেই কাঠামোর ভেতরের প্রতিটি বাক্স — প্রতিটি Image — কীভাবে বানাতে হয় তা শিখবেন। একটি খারাপ Dockerfile পুরো ব্যবস্থাকে ধীর, ভারী ও অনিরাপদ করে দেয়: Development নির্ভরতা-সহ ১.৫ গিগাবাইটের Image, ভেতরে সোর্স কোড ও গোপন মান, root হিসেবে চলা প্রসেস, আর থামার সময় ডেটা হারানো — এগুলো সবই Dockerfile-এর ভুল। উল্টো, একটি ভালো Multi-Stage Build একই অ্যাপকে ২০০ মেগাবাইটের নিচে নামিয়ে আনে, শুধু চালানোর জন্য যা দরকার তাই রাখে, আর প্রতিটি Build থেকে একই Image তৈরি করে। MERN অ্যাপের ক্ষেত্রে অতিরিক্ত জটিলতা আছে: ফ্রন্টএন্ড Build হয় একবার (Build-time), চলে আরেকভাবে (Static ফাইল হিসেবে); ব্যাকএন্ড রানটাইমে ডেটাবেসের সঙ্গে কথা বলে; ডেটাবেসের ডেটা কনটেইনারের জীবনচক্রের বাইরে বাঁচতে হয়। এই অধ্যায়ে এই তিনটি ভিন্ন জীবনচক্রকে একসঙ্গে বেঁধে একটি সম্পূর্ণ, চালানো-যায় এমন ব্যবস্থা বানানো হবে।

১৮.৩ পূর্বশর্ত জ্ঞান

অধ্যায় ১৭-এর Compose জ্ঞান (Service, Network, Volume, `depends_on`), Dockerfile-এর মৌলিক ধারণা (FROM, COPY, RUN, CMD), Node.js-এ `npm ci` ও `package.json`-এর `dependencies` বনাম `devDependencies`-এর পার্থক্য, Next.js-এ `npm run build` ও `npm start`-এর পার্থক্য, MongoDB-র সংযোগ স্ট্রিং (URI) ও Nginx-এর রিভার্স প্রক্সির প্রাথমিক ধারণা। Express-এ `/health` এন্ডপয়েন্ট ও SIGTERM সিগন্যালের ধারণা থাকলে ভালো হয় — না থাকলে এই অধ্যায়েই ব্যাখ্যা করা হবে।

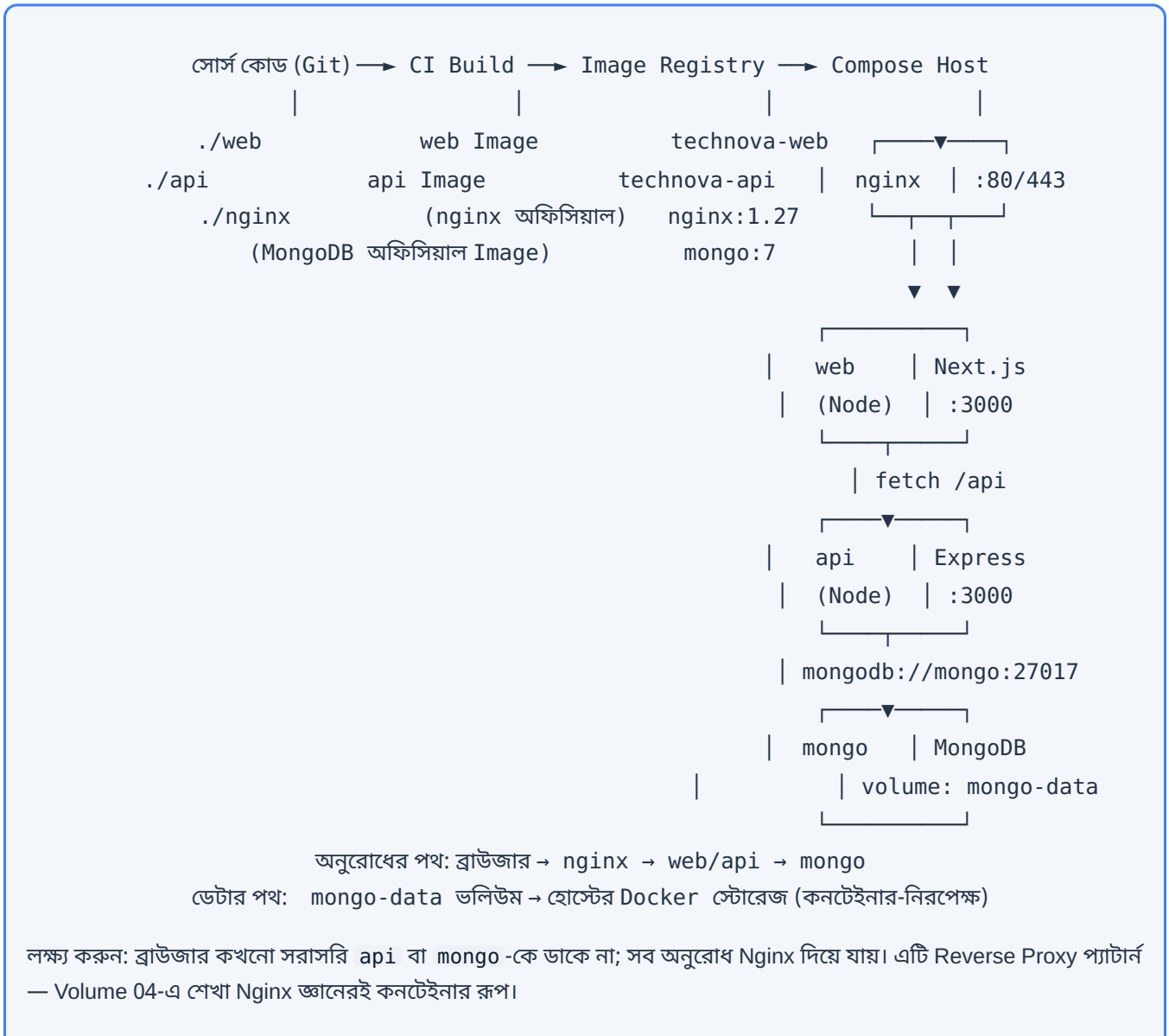
১৮.৪ প্রথম নীতি থেকে ব্যাখ্যা

একটি কনটেইনার Image হলো একটি অ্যাপ চালানোর জন্য যা যা দরকার তার একটি স্তরে স্তরে সাজানো প্যাকেজ (Layered Package)। প্রতিটি Dockerfile নির্দেশনা (Instruction) একটি নতুন স্তর (Layer) যোগ করে; স্তরগুলো Cache হয় — বদল না হলে আবার তৈরি হয় না। প্রথম নীতি: *Build* করার জন্য যা দরকার আর চালানোর জন্য যা দরকার — দুটি আলাদা জিনিস। ফ্রন্টএন্ড Build করতে Node.js, npm প্যাকেজ ও সোর্স কোড লাগে; চালাতে লাগে শুধু তৈরি হওয়া Static ফাইল ও সেগুলো পরিবেশন করার একটি ছোট সার্ভার। ব্যাকএন্ড Build করতে TypeScript কম্পাইলার বা টেস্ট টুল লাগতে পারে; চালাতে লাগে শুধু কম্পাইল করা JavaScript ও Production নির্ভরতা। Multi-Stage Build এই পার্থক্যটিকে কাজে লাগায়: প্রথম Stage-এ (Builder) সব টুল দিয়ে Build করা হয়, দ্বিতীয় Stage-এ (Runtime) শুধু ফলাফলটি কপি করে

আনা হয়। Builder Stage-এর টুল, সোর্স কোড ও অস্থায়ী ফাইল চূড়ান্ত Image-এ থাকে না — Image ছোট হয়, আক্রমণের পৃষ্ঠ (Attack Surface) কমে, আর গোপন মান ভুল করে Image-এ ঢুকে যাওয়ার ঝুঁকি কমে। দ্বিতীয় নীতি: কনটেইনার অস্থায়ী, ডেটা স্থায়ী। কনটেইনার মুছে আবার তৈরি করা যায় — এটাই স্বাভাবিক। তাই ডেটাবেসের ডেটা, আপলোড করা ফাইল ও লগ কখনো কনটেইনারের ভেতরের লেখার-যোগ্য স্তরে রাখা যাবে না; সেগুলো Volume-এ রাখতে হবে। তৃতীয় নীতি: কনটেইনারগুলো একে অপরকে নামে ডাকে, *IP*-তে নয় — কারণ কনটেইনারের IP প্রতিবার বদলায়, নাম বদলায় না।

> **CONCEPT:** Layer Cache — Dockerfile-এর প্রতিটি লাইন একটি স্তর। Docker Build করার সময় কোনো লাইনের ইনপুট (যেমন `package.json`) না বদলালে সেই স্তরটি আগের Build থেকে নিয়ে নেয়। তাই `package.json` আগে কপি করে `npm ci` চালানো, তারপর সোর্স কোড কপি করা — এই ক্রমটি গুরুত্বপূর্ণ: সোর্স কোড বদলালে শুধু শেষের স্তরগুলো আবার তৈরি হয়, নির্ভরতা আবার ডাউনলোড হয় না। উল্টো ক্রমে লিখলে প্রতিটি ছোট কোড-পরিবর্তনে পুরো `npm ci` আবার চলবে — Build ধীর হয়ে যাবে।

১৮.৫ আর্কিটেকচার / ধারণাগত চিত্র



অনুরোধের পথ: ব্রাউজার → `nginx` → `web`/`api` → `mongo`

ডেটার পথ: `mongo-data` ভলিউম → হোস্টের `Docker` স্টোরেজ (কনটেইনার-নিরপেক্ষ)

লক্ষ্য করুন: ব্রাউজার কখনো সরাসরি `api` বা `mongo`-কে ডাকে না; সব অনুরোধ Nginx দিয়ে যায়। এটি Reverse Proxy প্যাটার্ন — Volume 04-এ শেখা Nginx জ্ঞানেরই কনটেইনার রূপ।

১৮.৬ বিস্তারিত প্রযুক্তিগত ব্যাখ্যা

ফ্রন্টএন্ড: Next.js-এর Multi-Stage Build

Next.js অ্যাপের Build দুটি পর্যায়ে হয়: `npm run build` সোর্স থেকে `.next/` ডিরেক্টরি তৈরি করে (Static ফাইল ও সার্ভার-সাইড কোড), আর `npm start` সেই তৈরি ফল চালায়। Builder Stage-এ পূর্ণ Node.js Image (`node:20`) ব্যবহার করা হয়, Runtime Stage-এ ছোট `node:20-alpine` বা `node:20-slim` — Alpine ছোট কিন্তু কিছু Native মডিউলে সমস্যা করতে পারে; সন্দেহ থাকলে `slim` নিরাপদ পছন্দ। Next.js-এর একটি গুরুত্বপূর্ণ বৈশিষ্ট্য: `next.config.js`-এ `output: 'standalone'` দিলে Build-এর ফলে একটি স্বয়ংসম্পূর্ণ `.next/standalone` ডিরেক্টরি তৈরি হয়, যেখানে শুধু চালানোর জন্য দরকারি ফাইলগুলো থাকে — Runtime Image আরও ছোট হয়। এই অধ্যায়ের উদাহরণে `standalone` ব্যবহার করা হয়েছে।

Build-time বনাম Runtime পরিবেশ চলক — সবচেয়ে গুলিয়ে ফেলা পার্থক্য

Next.js-এ `NEXT_PUBLIC_*` দিয়ে শুরু হওয়া চলকগুলো Build-এর সময় JavaScript বান্ডলে স্থায়ীভাবে ঢুকে যায় — পরে বদলানো যায় না। তাই API-র ঠিকানা (যেমন `NEXT_PUBLIC_API_URL`) Build-এর সময় জানতে হয়, অথবা ফ্রন্টএন্ডকে রানটাইমে API ডাকার জন্য Relative পথ (`/api/...`) ব্যবহার করতে হয়, যা Nginx প্রক্সি করে দেয় — দ্বিতীয় পদ্ধতিই এই বইয়ে সুপারিশ করা হয়, কারণ এতে একই Image যেকোনো পরিবেশে (dev/staging/prod) চলে। অন্যদিকে সার্ভার-সাইড চলক (যেমন ডেটাবেস URI) কখনো বান্ডলে যায় না — সেগুলো রানটাইমে কনটেইনারের পরিবেশ থেকে পড়া হয়। নিয়মটি এক লাইনে: যা ব্রাউজারে যাবে তা *Build-time*, যা সার্ভারে থাকবে তা *Runtime*। `NEXT_PUBLIC_*`-এ কখনো গোপন মান দেবেন না — সেটি ব্রাউজারে দৃশ্যমান হয়ে যাবে।

ব্যাকএন্ড: শুধু Production নির্ভরতা

`npm ci` (Clean Install) `package-lock.json` অনুযায়ী হুবহু একই সংস্করণ ইনস্টল করে — `npm install`-এর মতো lock ফাইল বদলায় না, তাই Build পুনরাবৃত্তিযোগ্য (Reproducible) হয়। প্রোডাকশন Image-এ `npm ci --omit=dev` ব্যবহার করুন: `devDependencies` (টেস্ট ফ্রেমওয়ার্ক, TypeScript কম্পাইলার, Linter) ইনস্টল হবে না — Image ছোট হয়, আর টেস্ট টুলের দুর্বলতা (Vulnerability) প্রোডাকশন Image-এ ঢোকে না। ব্যাকএন্ডের `Dockerfile`-এ TypeScript থাকলে Builder Stage-এ `npm ci` (সব নির্ভরতা-সহ) করে `npm run build` চালিয়ে কম্পাইল করা JavaScript Runtime Stage-এ কপি করা হয় — ফ্রন্টএন্ডের মতোই Multi-Stage প্যাটার্ন।

অভ্যন্তরীণ যোগাযোগ ও নেটওয়ার্ক সীমানা

অধ্যায় ১৭-এর দুটি নেটওয়ার্ক (`frontend`, `backend`) এখানে কার্যকর হয়: `nginx` শুধু `frontend`-এ, `mongo` শুধু `backend`-এ, আর `web` / `api` দুটিতেই (সেতু হিসেবে)। ফলে `mongo`-কে ডাকতে পারে শুধু `api` — `web` চাইলেও সরাসরি ডেটাবেসে যেতে পারে না (সে `backend` নেটওয়ার্কে থাকলেও অ্যাপের কোডে ডেটাবেস সংযোগ লেখা নেই; নেটওয়ার্ক সীমানা ও অ্যাপের নকশা — দুটো মিলে সুরক্ষা দেয়)। API-র বেস URL ফ্রন্টএন্ডে Relative পথ হিসেবে থাকে (`/api/products`), Nginx সেটিকে `api:3000`-এ প্রক্সি করে। এই পদ্ধতির সুবিধা: ব্রাউজারকে API-র আসল ঠিকানা জানতে হয় না, CORS সমস্যা কমে, আর API-র পোর্ট হোস্টে প্রকাশ করতে হয় না।

ডেটাবেসের স্থায়ীত্ব, ব্যাকআপ ও পুনরুদ্ধার

MongoDB-র ডেটা থাকে `/data/db` পথে; `mongo-data` Named Volume সেটিকে ধরে রাখে। কনটেইনার মুছে গেলেও, Image বদলালেও, হোস্ট রিবুট হলেও ডেটা থাকে — শুধু `docker compose down -v` বা ভলিউম মুছলে যায়। ব্যাকআপের সঠিক পদ্ধতি: চলমান ডেটাবেসের বিরুদ্ধে `mongodump` চালানো — একটি অস্থায়ী কনটেইনার থেকে, একই নেটওয়ার্কে: `docker compose run --rm mongo mongodump --uri="mongodb://..." --archive=/backup/`

`dump.archive` — অথবা হোস্ট থেকে `docker compose exec mongo mongodump ...`। কখনো চলমান ডেটাবেসের ভলিউমের ফাইল সরাসরি কপি করে ব্যাকআপ নেবেন না — মাঝপথে লেখা হওয়া ফাইল দূষিত (Corrupt) ব্যাকআপ দেবে। পুনরুদ্ধার (Restore) `mongorestore` দিয়ে, Staging-এ পরীক্ষা করে, তারপর প্রোডাকশনে।

Graceful Shutdown — থামার সময়ও ভদ্রতা

কনটেইনার থামানোর সময় Docker প্রথমে SIGTERM সিগন্যাল পাঠায়, ১০ সেকেন্ড (ডিফল্ট) অপেক্ষা করে, তারপর SIGKILL দিয়ে জোর করে থামায়। Node.js অ্যাপকে SIGTERM ধরতে হয়: চলমান অনুরোধগুলো শেষ করতে দেওয়া, ডেটাবেস সংযোগ বন্ধ করা, তারপর প্রসেস থেকে বেরোনো। Express অ্যাপে এটি কয়েক লাইনের কোড: `process.on('SIGTERM', ...)`-এ সার্ভার বন্ধ করা (`server.close()`) ও MongoDB সংযোগ বন্ধ করা। `stop_grace_period` (যেমন `30s`) দিয়ে অপেক্ষার সময় বাড়ানো যায় — দীর্ঘ অনুরোধ থাকলে দরকার। শুধু SIGKILL-এ থামলে মাঝপথের লেখা (Write) অসম্পূর্ণ থেকে যেতে পারে — ডেটাবেসের জন্য সেটি ঝুঁকিপূর্ণ।

লগ: কোথায় লেখা উচিত

কনটেইনারের নিয়ম: লগ লিখতে হয় STDOUT/STDERR-এ, ফাইলে নয়। কারণ Docker Engine কনটেইনারের STDOUT/STDERR সংগ্রহ করে (`docker compose logs` দিয়ে দেখা যায়), আর ফাইলে লেখা লগ কনটেইনারের ভেতরে আটকে থাকে — কনটেইনার মুছলে লগও যায়, ডিস্কও ভরে যায়। Node.js অ্যাপে `console.log` বা একটি লগ লাইব্রেরি (যেমন Pino) STDOUT-এ JSON লগ লিখবে; ফাইলে লেখার কনফিগারেশন প্রোডাকশনে বন্ধ রাখুন। লগ রোটেশন ও কেন্দ্রীয় সংগ্রহ অধ্যায় ২২-এ।

Dev বনাম Prod কনফিগারেশন

বিষয়	Development	Production
সোর্স কোড	Bind Mount — বদলালেই প্রতিফলিত হয়	Image-এর ভেতরে — অপরিবর্তনীয়
API চালানো	`npm run dev` (Nodemon — অটো রিলোড)	`node server.js` (কম্পাইল করা কোড)
নির্ভরতা	সব (dev-সহ)	শুধু Production (`--omit=dev`)
ডেটাবেস পোর্ট	হোস্টে প্রকাশ (ডিবাগিংয়ের জন্য)	শুধু অভ্যন্তরীণ নেটওয়ার্ক
পরিবেশ চলক	`NODE_ENV=development`	`NODE_ENV=production`
লগ স্তর	`debug`	`info` / `warn`
Restart নীতি	`no` বা `on-failure`	`unless-stopped`

১৮.৭ Docker কমান্ড ও লাইন-বাই-লাইন ব্যাখ্যা

```
$ docker build -t technova-api:1.2.0 ./api
$ docker compose build --no-cache api
$ docker compose up -d --build
$ docker images technova-*
$ docker compose run --rm api npm test
$ docker compose exec mongo mongosh -u technova_admin -p
$ docker compose exec mongo mongodump --uri="mongodb://technova_admin:CHANGE-ME-STRONG-
```

```
PASSWORD@mongo:27017/technova?authSource=admin" --archive=/tmp/dump.archive
$ docker compose cp mongo:/tmp/dump.archive ./backups/dump-2026-11-09.archive
```

লাইন-বাই-লাইন ব্যাখ্যা: (১) `docker build -t technova-api:1.2.0 ./api` — `build` মানে Image বানাও; `-t` (tag) মানে নাম ও সংস্করণ দাও; শেষের `./api` হলো Build Context — Dockerfile ও সোর্স কোডের ডিরেক্টরি। (২) `docker compose build --no-cache api` — শুধু `api` সার্ভিসের Image Cache ছাড়া আবার Build করো (Cache-সংক্রান্ত সন্দেহ থাকলে)। (৩) `up -d --build` — Build করে চালু করো। (৪) `docker images technova-*` — আমাদের Image-গুলোর আকার দেখো; Multi-Stage Build সফল হলে আকার ছোট দেখাবে। (৫) `docker compose run --rm api npm test` — `run` মানে সার্ভিসের সংজ্ঞা থেকে একটি একবারের কনটেইনার চালাও; `--rm` মানে শেষে মুছে দাও; এখানে API কনটেইনারে টেস্ট চালানো হচ্ছে — প্রোডাকশন কনটেইনারকে না ছুঁয়ে। (৬) `exec mongo mongosh -u ... -p` — চলমান MongoDB কনটেইনারের ভেতরে `mongosh` শেল খোলা; `-p` পাসওয়ার্ড জিজ্ঞাসা করবে (কমান্ড-লাইনে পাসওয়ার্ড লেখা এড়ানোর জন্য)। (৭) `mongodump --uri=... --archive=...` — ডেটাবেসের ব্যাকআপ একটি আর্কাইভ ফাইলে নেওয়া; URI-তে ব্যবহারকারীর নাম ও পাসওয়ার্ড আছে — এই কমান্ডটি শেলের ইতিহাসে (History) থেকে যেতে পারে, তাই প্রোডাকশনে পাসওয়ার্ড পরিবেশ চলক থেকে নেওয়া ভালো (অধ্যায় ১৯)। (৮) `docker compose cp mongo:/tmp/dump.archive ./backups/...` — কনটেইনার থেকে ব্যাকআপ ফাইলটি হোস্টে কপি করা।

নমুনা আউটপুট (চিত্রণমূলক):

```
$ docker images technova-*
REPOSITORY       TAG      IMAGE ID       CREATED        SIZE
technova-web     1.2.0    a1b2c3d4e5f6   2 hours ago    189MB
technova-api     1.2.0    f6e5d4c3b2a1   2 hours ago    142MB
$ docker compose exec api node -e "console.log(process.env.NODE_ENV)"
production
```

দুটি Image-ই ২০০ মেগাবাইটের নিচে — Multi-Stage Build কাজ করছে তার প্রমাণ। Single-Stage Build হলে প্রতিটি Image সহজেই ১ গিগাবাইট ছাড়িয়ে যেত।

১৮.৮ সম্পূর্ণ কনফিগারেশন উদাহরণ

ফ্রন্টএন্ড Dockerfile (`./web/Dockerfile`) — Multi-Stage:

```
# Stage 1 — Builder: পূর্ণ Node.js দিয়ে Build করা হয়
FROM node:20 AS builder
WORKDIR /app
# প্রথমে শুধু package ফাইল কপি — Layer Cache-এর সুবিধা নিতে
COPY package.json package-lock.json ./
RUN npm ci
COPY . .
# standalone আউটপুটের জন্য next.config.js-এ output: 'standalone' থাকতে হবে
RUN npm run build

# Stage 2 — Runtime: শুধু চালানোর জন্য যা দরকার
FROM node:20-slim AS runtime
WORKDIR /app
ENV NODE_ENV=production
# non-root ব্যবহারকারী (অধ্যায় ২১-এর বিস্তারিত এখানে প্রয়োগ করা হয়েছে)
RUN groupadd -r appuser && useradd -r -g appuser appuser
COPY --from=builder --chown=appuser:appuser /app/.next/standalone ./
COPY --from=builder --chown=appuser:appuser /app/.next/static ./.next/static
COPY --from=builder --chown=appuser:appuser /app/public ./public
```

```
USER appuser
EXPOSE 3000
CMD ["node", "server.js"]
```

ব্যাকএন্ড Dockerfile (`./api/Dockerfile`) — Multi-Stage, TypeScript ধরে নিয়ে:

```
FROM node:20 AS builder
WORKDIR /app
COPY package.json package-lock.json ./
RUN npm ci
COPY . .
RUN npm run build

FROM node:20-slim AS runtime
WORKDIR /app
ENV NODE_ENV=production
COPY package.json package-lock.json ./
# শুধু Production নির্ভরতা — devDependencies বাদ
RUN npm ci --omit=dev && npm cache clean --force
COPY --from=builder /app/dist ./dist
RUN groupadd -r appuser && useradd -r -g appuser appuser
USER appuser
EXPOSE 3000
# Graceful Shutdown-এর জন্য SIGTERM ধরার কোড server.js-এ আছে (১৮.৬ দেখুন)
CMD ["node", "dist/server.js"]
```

Nginx কনফিগারেশন (`./nginx/nginx.conf`) — সংক্ষিপ্ত, প্রাসঙ্গিক অংশ:

```
upstream web_upstream {
    server web:3000;
}
upstream api_upstream {
    server api:3000;
}
server {
    listen 80;
    server_name app.technova.example;
    # /api/... অনুরোধ ব্যাকএন্ডে পাঠানো হয়
    location /api/ {
        proxy_pass http://api_upstream;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
    # বাকি সব অনুরোধ ফ্রন্টএন্ডে
    location / {
        proxy_pass http://web_upstream;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

`.dockerignore` — প্রতিটি অ্যাপ ডিরেক্টরিতে রাখুন (Build Context ছোট রাখতে):

```
node_modules
.next
dist
.git
.env
*.log
coverage
```

`.dockerignore` না থাকলে হোস্টের `node_modules` (লক্ষ লক্ষ ফাইল) Build Context-এ ঢুকে Build ধীর করে দেয়, আর ভুল করে `.env`-এর আসল Secret Image-এ ঢুকে যেতে পারে। `.gitignore`-এর মতোই গুরুত্বপূর্ণ — প্রতিটি Dockerfile-এর পাশে এটি রাখা অভ্যাস করুন।

Graceful Shutdown কোড (`./api/src/server.js` থেকে সংক্ষিপ্ত):

```
const server = app.listen(PORT, () => console.log(`API listening on ${PORT}`));
process.on('SIGTERM', async () => {
  console.log('SIGTERM received — shutting down gracefully');
  server.close(async () => {          // নতুন অনুরোধ নেওয়া বন্ধ, চলমান অনুরোধ শেষ হতে দাও
    await mongoose.connection.close(); // ডেটাবেস সংযোগ বন্ধ
    process.exit(0);
  });
  setTimeout(() => process.exit(1), 25000); // 25 সেকেন্ডে শেষ না হলে জোর করে বেরোও
});
```

১৮.৯ বাস্তব প্রোডাকশন পরিস্থিতি

REAL WORLD — TechNova: TechNova-র প্রথম Dockerfile ছিল Single-Stage: `FROM node:20`, তারপর পুরো সোর্স, সব dev নির্ভরতা, টেস্ট ফাইল — সব এক Image-এ। ফল: প্রতিটি Image ১.৪ গিগাবাইট, Registry-তে Push করতে ১২ মিনিট, Deploy-এ আরও ১০ মিনিট। একদিন একটি ভুলে `.env` ফাইল Image-এ ঢুকে গেল — সৌভাগ্যবশত সেটি ধরা পড়ল Staging-এ, প্রোডাকশনে নয়। Multi-Stage Build-এ যাওয়ার পর Image ১৪২ মেগাবাইটে নেমে এলো, Push/Deploy মিলিয়ে ৩ মিনিটের নিচে, আর `.dockerignore` + Builder/Runtime বিভাজনের কারণে `.env` আর কখনো Image-এ ঢোকে না। শিক্ষা: Dockerfile শুধু "কীভাবে চালাব" নয় — এটি Image-এর আকার, গতি ও নিরাপত্তার নকশা।

১৮.১০ ধাপে ধাপে ব্যবহারিক বাস্তবায়ন

উদাহরণ: MERN স্ট্যাকটি শূন্য থেকে কনটেইনারাইজ করা

1. `./api` ও `./web` ডিরেক্টরিতে `.dockerignore` ফাইল তৈরি করুন (`node_modules`, `.env`, `.git` লিখে)।
2. ওপরের Multi-Stage Dockerfile দুটি লিখুন; Next.js-এ `output: 'standalone'` যোগ করুন।
3. অধ্যায় ১৭-এর `compose.yaml` ব্যবহার করে `docker compose up -d --build` চালান।
4. `docker images` দিয়ে Image-এর আকার দেখুন — ৩০০ মেগাবাইটের বেশি হলে কোন Stage-এ কী ঢুকেছে পরীক্ষা করুন (`docker history technova-api:1.2.0`)।
5. `curl http://localhost/api/health` ও ব্রাউজারে `http://localhost` পরীক্ষা করুন; একটি পণ্য তৈরি করে MongoDB-তে লেখা হচ্ছে কি না যাচাই করুন।

6. `docker compose exec mongo mongodump ...` দিয়ে ব্যাকআপ নিন, তারপর `docker compose down` করে আবার `up` করে দেখুন ডেটা ফিরে এসেছে কি না।
7. `docker compose stop api` দিয়ে দেখুন লগে "SIGTERM received — shutting down gracefully" বার্তা আসে কি না।

১৮.১১ সাধারণ ভুল ও ভুল ধারণা

- **ভুল:** `npm install` ব্যবহার করা — lock ফাইল বদলে Build পুনরাবৃত্তিযোগ্য থাকে না; `npm ci` ব্যবহার করুন।
- **ভুল:** সোর্স কোড আগে, `package.json` পরে কপি করা — Layer Cache নষ্ট হয়ে প্রতিবার পুরো `npm ci` আবার চলে।
- **ভুল:** `.dockerignore` না রাখা — Build ধীর হয়, Secret Image-এ ঢোকার ঝুঁকি থাকে।
- **ভুল:** `NEXT_PUBLIC_*`-এ গোপন মান বা পরিবেশ-নির্দিষ্ট API ঠিকানা দেওয়া — বান্ডলে স্থায়ী হয়ে যায়; Relative পথ ও Nginx প্রক্সি ব্যবহার করুন।
- **ভুল:** প্রোডাকশন Image-এ `devDependencies` রাখা — Image ভারী হয়, দুর্বলতা বাড়ে।
- **ভুল:** ডেটাবেসের ডেটা কনটেইনারের ভেতরে রাখা (ভলিউম ছাড়া) — কনটেইনার মুছলেই ডেটা যাবে।
- **ভুল ধারণা:** "Image ছোট হলেই নিরাপদ" — না; ছোট Image ভালো শুরু, কিন্তু non-root ব্যবহারকারী, স্ক্যানিং ও আপডেট ছাড়া নিরাপত্তা সম্পূর্ণ হয় না (অধ্যায় ২১)।
- **ভুল ধারণা:** "চলমান ডেটাবেসের ভলিউম ফাইল কপি করলেই ব্যাকআপ হয়" — না; `mongodump`-এর মতো ডেটাবেসের নিজের টুল ব্যবহার করুন।

১৮.১২ নিরাপত্তা বিবেচনা

> **WARNING:** Dockerfile-এ `ENV MONGO_PASSWORD=...` বা `RUN echo "secret" > config` লিখলে সেই Secret Image-এর Layer-এ স্থায়ীভাবে থেকে যায় — Image যার হাতে যাবে সে `docker history` দিয়ে Secret দেখতে পাবে, এমনকি পরের স্তরে মুছে ফেললেও। Secret কখনো Dockerfile-এ বা Image Layer-এ নয় — শুধু রানটাইমে পরিবেশ চলক বা Secret ফাইল থেকে আসবে (অধ্যায় ১৯)। একই কারণে Build-সময়ের Secret-এর জন্য Docker BuildKit-এর `--secret` (Build Secret) ব্যবহার করুন, `ARG`-এ Secret পাস করবেন না — `ARG`-এর মানও Image-এর ইতিহাসে দেখা যায়।

এই অধ্যায়ের Dockerfile-গুলোতে ইতিমধ্যে দুটি সুরক্ষা প্রয়োগ করা হয়েছে: non-root `appuser` (root হিসেবে চলা কনটেইনার আক্রান্ত হলে ক্ষতি বেশি — অধ্যায় ২১) এবং `.dockerignore` দিয়ে `.env` বাদ দেওয়া। আরও একটি সুরক্ষা: Base Image-এর সংস্করণ নির্দিষ্ট করা (`node:20-slim`, `mongo:7`) — `node:latest` ব্যবহার করলে কোন সংস্করণ চলছে জানা যায় না, আর হঠাৎ Major আপডেটে অ্যাপ ভাঙতে পারে।

১৮.১৩ কর্মক্ষমতা ও রিসোর্স বিবেচনা

Image-এর আকার সরাসরি Deploy-এর গতিকে প্রভাবিত করে: ছোট Image দ্রুত Pull হয়, দ্রুত চালু হয়, কম ডিস্ক নেয়। Multi-Stage Build, `--omit=dev`, `.dockerignore` ও `standalone` আউটপুট — এই চারটি মিলে TechNova-র Image ১.৪ গিগাবাইট থেকে ১৫০ মেগাবাইটের কাছাকাছি নামিয়েছে। Build-এর গতির জন্য Layer Cache ক্রম (package ফাইল আগে) ও BuildKit Cache গুরুত্বপূর্ণ। রানটাইমে Node.js অ্যাপের মেমোরি ব্যবহারের দিকে নজর রাখুন: Next.js ও Express একসঙ্গে একটি ছোট সার্ভারে চললে প্রতিটির জন্য মেমোরি সীমা (অধ্যায় ২২) দেওয়া উচিত, যাতে একটির

মেমোরি লিক অন্যটিকে না মারে। MongoDB-র জন্য WiredTiger Cache ডিফল্টভাবে উপলব্ধ RAM-এর প্রায় অর্ধেক ব্যবহার করতে পারে — ছোট সার্ভারে `wiredTigerCacheSizeGB` সীমিত করা উচিত, না হলে ডেটাবেস অন্য সার্ভিসের মেমোরি খেয়ে ফেলবে।

১৮.১৪ সমস্যা সমাধান পদ্ধতি

TROUBLESHOOTING: Build ব্যর্থ হলে প্রথমে ভুলটি কোন Stage-এ হয়েছে দেখুন — Builder Stage-এ `npm ci` ব্যর্থ হলে সাধারণত নেটওয়ার্ক বা lock ফাইলের সমস্যা; Runtime Stage-এ ফাইল না পাওয়া গেলে `COPY --from=builder`-এর পথ ভুল। `docker build --progress=plain` দিয়ে বিস্তারিত লগ দেখুন। `docker compose up`-এর পর API ৫০২ (Bad Gateway) দিলে Nginx লগ দেখুন (`docker compose logs nginx`) — সাধারণত `api` সার্ভিসের নাম ভুল বা API এখনো চালু হয়নি।

সমস্যা	সম্ভাব্য কারণ	সমাধান
Build খুব ধীর, প্রতিবার `npm ci` আবার চলে	সোর্স কোড `package.json`-এর আগে কপি করা	package ফাইল আগে কপি করুন, সোর্স পরে
`Cannot find module` রানটাইমে	`--omit=dev` দিয়ে দরকারি প্যাকেজ বাদ পড়েছে	প্যাকেজটি `dependencies`-এ সরান, `devDependencies`-এ নয়
ফ্রন্টএন্ডে ভুল API ঠিকানা (পুরনো পরিবেশের)	`NEXT_PUBLIC_*` Build-সময়ে স্থায়ী হয়েছে	Relative পথ (`/api`) ব্যবহার করুন, প্রতি পরিবেশে আবার Build করুন
কনটেইনার মুছলে ডেটা হারিয়ে যায়	ভলিউম মাউন্ট করা হয়নি	`mongo-data:/data/db` Named Volume যোগ করুন
`mongodump` ব্যাকআপ দূষিত	চলমান ডেটাবেসের ফাইল সরাসরি কপি করা হয়েছে	`mongodump` / `mongorestore` ব্যবহার করুন
থামার সময় ডেটা লেখা অসম্পূর্ণ	SIGTERM ধরা হয়নি	Graceful Shutdown কোড যোগ করুন, `stop_grace_period` বাড়ান

১৮.১৫ সেরা অনুশীলন ও অ্যান্টি-প্যাটার্ন

সেরা অনুশীলন: সবসময় Multi-Stage Build; `npm ci` ও `--omit=dev`; নির্দিষ্ট Base Image ট্যাগ; `.dockerignore` প্রতিটি অ্যাপে; Relative API পথ ও Nginx প্রক্সি; ডেটা Named Volume-এ; ব্যাকআপ `mongodump` দিয়ে; SIGTERM ধরে Graceful Shutdown; লগ STDOUT/STDERR-এ। **অ্যান্টি-প্যাটার্ন:** Single-Stage বিশাল Image; Dockerfile-এ Secret; `latest` ট্যাগ; `npm install`; ফাইলে লগ লেখা; ভলিউম ছাড়া ডেটাবেস; Build-সময়ের চলকে পরিবেশ-নির্দিষ্ট ঠিকানা বেঁধে দেওয়া।

১৮.১৬ অধ্যায়ের সারাংশ

MERN অ্যাপ কনটেইনারাইজ করার মূল নীতি তিনটি: Build ও Runtime আলাদা করো (Multi-Stage), কনটেইনারকে অস্থায়ী ও ডেটাকে স্থায়ী ভাবো (Named Volume), আর সার্ভিসগুলোকে নামে ডাকো (Service Discovery)। ফ্রন্টএন্ডের `NEXT_PUBLIC_*` চলক Build-সময়ে বান্ডলে ঢোকে — তাই API ডাকতে Relative পথ ও Nginx প্রক্সি ব্যবহার করো।

ব্যাকএন্ডে শুধু Production নির্ভরতা রাখো, ডেটাবেসের ব্যাকআপ নাও `mongodump` দিয়ে, আর থামার সময় SIGTERM ধরে Graceful Shutdown করো। Dev ও Prod কনফিগারেশন আলাদা ফাইলে রাখো — একই Image, ভিন্ন পরিবেশ।

১৮.১৭ মূল প্রযুক্তিগত পরিভাষা

Multi-Stage Build: একাধিক FROM Stage-এ Dockerfile লেখা — Builder Stage-এ Build, Runtime Stage-এ শুধু ফলাফল নিয়ে ছোট Image তৈরি।

Layer / Layer Cache: Image-এর স্তর; ইনপুট না বদলালে Docker আগের স্তর আবার ব্যবহার করে, Build দ্রুত হয়।

Build Context: `docker build`-এ দেওয়া ডিরেক্টরি — এর ভেতরের ফাইলগুলোই Build-এ ব্যবহার করা যায়।

Build-time Variable: Build-এর সময় নির্ধারিত মান (যেমন `NEXT_PUBLIC_*`) — পরে বদলানো যায় না।

Runtime Variable: কনটেইনার চালুর সময় পরিবেশ থেকে পড়া মান — প্রতি পরিবেশে বদলানো যায়।

npm ci: `package-lock.json` অনুযায়ী হুবহু একই নির্ভরতা ইনস্টল করা — পুনরাবৃত্তিযোগ্য Build-এর জন্য।

Reverse Proxy: ক্লায়েন্টের অনুরোধ গ্রহণ করে ভেতরের সার্ভিসে পাঠানো সার্ভার (এখানে Nginx)।

Graceful Shutdown: SIGTERM পেয়ে চলমান কাজ শেষ করে, সংযোগ বন্ধ করে ভদ্রভাবে থামা।

mongodump / mongorestore: MongoDB-র ব্যাকআপ নেওয়া ও ফেরানোর অফিসিয়াল টুল।

.dockerignore: Build Context থেকে বাদ দেওয়া ফাইলের তালিকা — `.gitignore`-এর মতো।

Standalone Output: Next.js-এর `output: 'standalone'` — চালানোর জন্য শুধু দরকারি ফাইল নিয়ে তৈরি আউটপুট।

১৮.১৮ জ্ঞান যাচাই — প্রশ্ন

1. Multi-Stage Build কেন Single-Stage Build-এর চেয়ে ছোট ও নিরাপদ Image দেয়? দুটি কারণ লিখুন।
2. `package.json` আগে ও সোর্স কোড পরে কপি করা কেন গুরুত্বপূর্ণ? Layer Cache-এর সঙ্গে সম্পর্ক ব্যাখ্যা করুন।
3. `NEXT_PUBLIC_API_URL` দিয়ে API-র ঠিকানা দেওয়া কেন ঝুঁকিপূর্ণ? বিকল্প পদ্ধতিটি কী?

4. `npm ci` ও `npm install`-এর পার্থক্য কী? প্রোডাকশন Build-এ কোনটি ব্যবহার করবেন?
5. ডেটাবেসের ব্যাকআপ নিতে ভলিউমের ফাইল কপি করা কেন ভুল? সঠিক পদ্ধতি কী?
6. Graceful Shutdown কী? SIGTERM না ধরলে কী ক্ষতি হতে পারে?
7. Dev ও Prod কনফিগারেশনের তিনটি পার্থক্য লিখুন।
8. `.dockerignore` না থাকলে কী কী সমস্যা হতে পারে?

১৮.১৯ ব্যবহারিক অনুশীলন

1. **Image ছোট করা:** একটি Single-Stage Dockerfile লিখে Image-এর আকার মাপুন, তারপর Multi-Stage সংস্করণ লিখে পার্থক্য লিখুন।
2. **ব্যাকআপ অনুশীলন:** কিছু ডেটা তৈরি করে `mongodump` দিয়ে ব্যাকআপ নিন, ডেটা মুছে `mongorestore` দিয়ে ফেরান।
3. **Graceful Shutdown পরীক্ষা:** API-তে SIGTERM হ্যান্ডলার যোগ করে `docker compose stop api`-এর লগে ভদ্রভাবে থামার বার্তা দেখুন।
4. **Relative API পথ:** ফ্রন্টএন্ডের API কলগুলো `/api/...` Relative পথে বদলে Nginx প্রক্সি দিয়ে পরীক্ষা করুন।

অধ্যায় 19 • PART IV — Docker Compose and Multi-Container Systems

Chapter 19 — Environment Variables, Secrets and Configuration

১৯.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: (১) Configuration, Environment Variable ও Secret — তিনটি আলাদা জিনিসের পার্থক্য ব্যাখ্যা করতে; (২) Compose-এর `.env`, `env_file`, `environment` ও Interpolation-এর কাজ আলাদা করে বলতে; (৩) 12-Factor App-এর Config নীতি প্রয়োগ করতে; (৪) Compose Secrets-এর সীমা বুঝে নিরাপদ Secret ব্যবস্থাপনার পথ বেছে নিতে; (৫) `.env.example`, `.gitignore` ও Rotation-এর অভ্যাস গড়তে; (৬) ভুল করে Secret ফাঁস হলে কী করতে হয় তা বলতে।

১৯.২ কেন এই বিষয়টি DevOps-এ গুরুত্বপূর্ণ

প্রায় প্রতিটি বড় নিরাপত্তা দুর্ঘটনার পেছনে একই গল্প: GitHub-এ কমিট করা একটি পাসওয়ার্ড, একটি চুরি যাওয়া API Key দিয়ে ডেটাবেস মুছে ফেলা, বা একটি ফাঁস হওয়া টোকেন দিয়ে ক্লাউড বিল হাজার ডলার। Configuration ব্যবস্থাপনা শুধু "সুবিধা" নয় — এটি নিরাপত্তা ও পরিচালনাযোগ্যতার ভিত্তি। একই Image তিনটি পরিবেশে (dev/staging/prod) চলে — শুধু Configuration বদলায়। Configuration ঠিকমতো আলাদা না করলে হয় প্রতিটি পরিবেশের জন্য আলাদা Image বানাতে হয় (অপরিবর্তনীয়তার নীতি ভাঙে), নয়তো প্রোডাকশনের Secret ডেভেলপারদের ল্যাপটপে ছড়িয়ে পড়ে। এই অধ্যায়ে শিখবেন কোন মান কোথায় রাখা যায়, কোনটি কখনোই কোথায় রাখা যায় না, আর Compose-এর কোন সুবিধাটি আসলে কতটুকু নিরাপদ।

১৯.৩ পূর্বশর্ত জ্ঞান

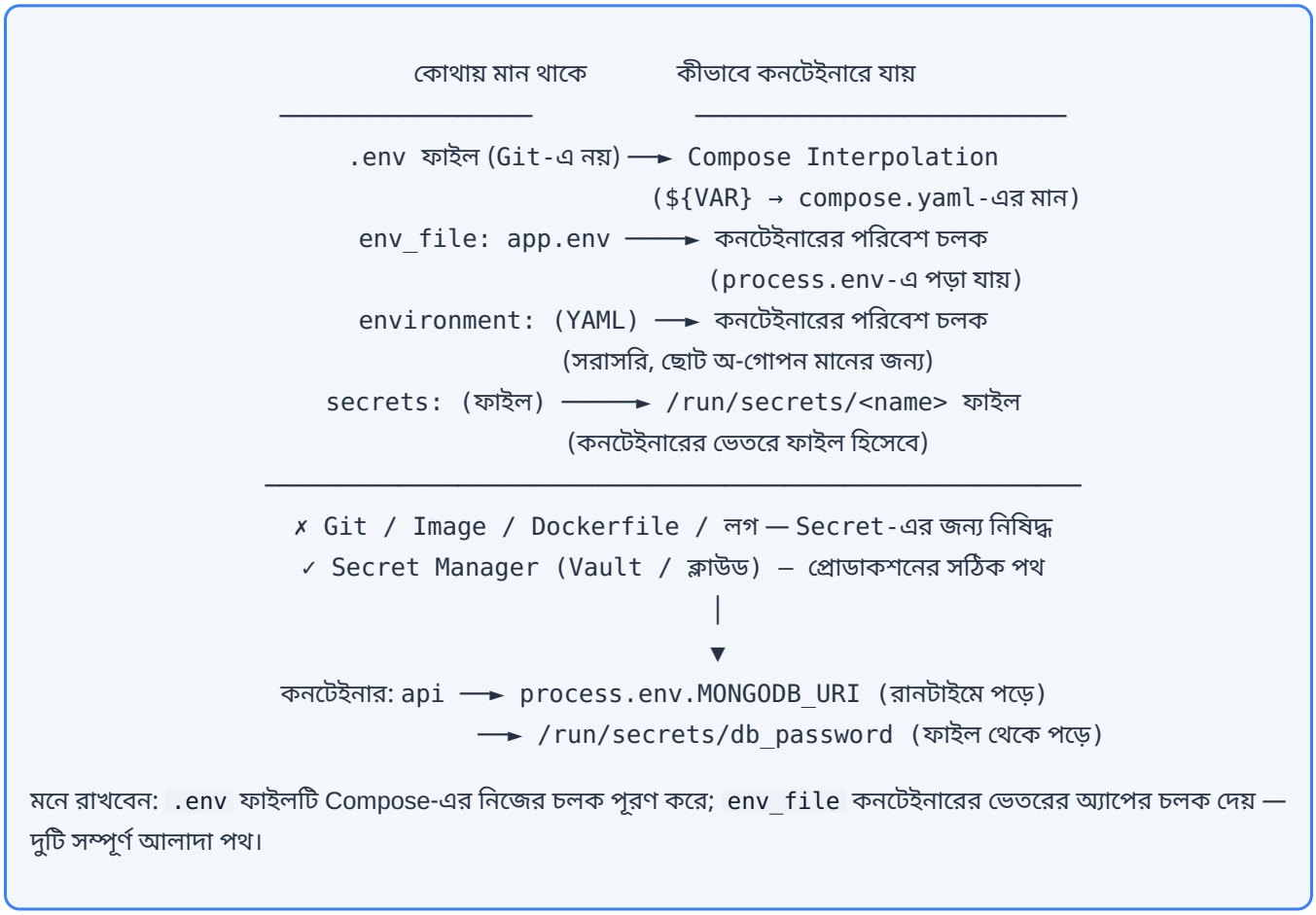
অধ্যায় ১৭-১৮-এর Compose ও Dockerfile জ্ঞান, `.env` ফাইলের প্রাথমিক ধারণা, `process.env` দিয়ে Node.js-এ চলক পড়া, Git-এ `.gitignore`-এর ব্যবহার, এবং Secret বলতে কী বোঝায় (পাসওয়ার্ড, API Key, টোকেন, প্রাইভেট Key) তার ধারণা। 12-Factor App-এর পূর্ব জ্ঞান প্রয়োজন নেই — এই অধ্যায়েই পরিচয় করিয়ে দেওয়া হবে।

১৯.৪ প্রথম নীতি থেকে ব্যাখ্যা

একটি অ্যাপের আচরণ নির্ধারণ করে তিন ধরনের বাইরের তথ্য। প্রথম, **Configuration** — পরিবেশভেদে বদলায় এমন অ-গোপন মান: পোর্ট নম্বর, লগের স্তর, API-র বেস URL, ফিচার ফ্ল্যাগ। এগুলো দেখা গেলে ক্ষতি নেই। দ্বিতীয়, **Environment Variable** — Configuration পৌঁছে দেওয়ার একটি পদ্ধতি: অপারেটিং সিস্টেমের দেওয়া নাম-মান জোড়া, যা প্রসেস শুরু হওয়ার সময় পায়। তৃতীয়, **Secret** — এমন মান যা জানা গেলে ক্ষতি হয়: ডেটাবেস পাসওয়ার্ড, API Key, JWT স্বাক্ষরের Key, TLS প্রাইভেট Key। মূল নীতিটি 12-Factor App থেকে এসেছে: *Configuration* কোড থেকে আলাদা রাখো, পরিবেশ থেকে দাও। অর্থাৎ একই কোড, একই Image — শুধু পরিবেশ চলক বদলে dev থেকে prod। Secret-এর ক্ষেত্রে আরও কঠোর নীতি: *Secret* কখনো কোডে, *Image*-এ, *Git*-এ বা লগে নয়; শুধু রানটাইমে, সীমিত সময়ের জন্য, শুধু যার দরকার তার কাছে। এই তিনটি সংজ্ঞা গুলিয়ে ফেললেই ভুল হয় — যেমন পাসওয়ার্ডকে সাধারণ Configuration ভেবে Git-এ কমিট করা, বা পোর্ট নম্বরকে Secret ভেবে অযথা লুকানো।

> **CONCEPT:** "একই Artifact, ভিন্ন Configuration" — Staging-এ পরীক্ষিত Image-টিই Production-এ চলবে, শুধু পরিবেশ চলক বদলাবে। এই নীতিটি না মানলে Staging-এ পরীক্ষা করা জিনিস আর Production-এ চালানো জিনিস আলাদা হয়ে যায় — তখন Staging-এর পরীক্ষার মূল্যই থাকে না। এ কারণেই Configuration Image-এর ভেতরে বেক (Bake) করা নিষিদ্ধ: Image বানানোর সময় পরিবেশ জানা থাকে না, জানা উচিতও নয়।

১৯.৫ আর্কিটেকচার / ধারণাগত চিত্র



মনে রাখবেন: `.env` ফাইলটি Compose-এর নিজের চলক পূরণ করে; `env_file` কনটেইনারের ভেতরের অ্যাপের চলক দেয় — দুটি সম্পূর্ণ আলাদা পথ।

১৯.৬ বিস্তারিত প্রযুক্তিগত ব্যাখ্যা

চারটি পদ্ধতির পার্থক্য — বিস্তারিত

পদ্ধতি	কী করে	কোথায় মান থাকে	কখন ব্যবহার করবেন
`.env` (Interpolation)	Compose ফাইলের `${VAR}` পূরণ করে	প্রজেক্ট ডিরেক্টরির `.env` ফাইল	Image ট্যাগ, পোর্ট, ব্যবহারকারীর নাম — Compose-এর নিজের মান
`env_file`	ফাইলের সব লাইন কনটেইনারের পরিবেশ চলক করে	`app.env` ধরনের ফাইল	অ্যাপের অনেকগুলো চলক একসঙ্গে দেওয়ার জন্য
`environment`	YAML-এ লেখা মান কনটেইনারে দেয়	Compose ফাইলে (Git-এ যায়)	শুধু অ-গোপন, ছোট মান (`NODE_ENV: production`)

`secrets`	ফাইল কনটেইনারে `/run/secrets/`-এ মাউন্ট করে	হোস্টের ফাইল	পাসওয়ার্ডের মতো গোপন মান, ফাইল হিসেবে পড়ার জন্য

অগ্রাধিকারের ক্রম (Precedence): একই চলক একাধিক জায়গায় থাকলে শেলের পরিবেশ চলক সবচেয়ে শক্তিশালী, তারপর `.env` ফাইল, তারপর Compose ফাইলের `environment`। `env_file`-এর মান `environment`-এর কাছে হারে — অর্থাৎ YAML-এ লেখা মান `env_file`-এর মানকে ওভাররাইড করে। এই ক্রমটি না জানলে "আমি `.env`-এ বদলালাম, তবু পুরনো মান চলছে" ধরনের বিভ্রান্তি হয় — তখন শেলে একই নামের চলক সেট করা আছে কি না দেখুন।

Interpolation-এর সিনট্যাক্স

`${VAR}` — মান বসাও; না থাকলে খালি। `${VAR:-default}` — না থাকলে ডিফল্ট। `${VAR-default}` — একই, তবে খালি স্ট্রিং থাকলে সেটিই ব্যবহার হয় (পার্থক্য সূক্ষ্ম, সাধারণত `:-` ব্যবহার করাই নিরাপদ)। `${VAR:?message}` — না থাকলে ভুল দেখিয়ে থামো (পাসওয়ার্ডের মতো বাধ্যতামূলক মানের জন্য)। `$${VAR}` — ডলার চিহ্নসহ আক্ষরিক `${VAR}` লিখতে চাইলে (যেমন Nginx কনফিগে)। এই সিনট্যাক্সটি Compose ফাইলের জন্য — কনটেইনারের ভেতরের অ্যাপের কোডে এর কোনো অর্থ নেই।

Compose Secrets — সত্যি কতটুকু নিরাপদ?

সততার সঙ্গে বলতে হবে: একক হোস্টে Compose-এর `secrets` মূলত একটি ফাইল মাউন্ট — Secret-টি হোস্টের একটি ফাইলে থাকে, কনটেইনারে `/run/secrets/db_password` হিসেবে দেখা যায়। এটি `environment`-এ পাসওয়ার্ড লেখার চেয়ে ভালো (কারণ `docker inspect` দিয়ে পরিবেশ চলক দেখা যায়, কিন্তু Secret ফাইল দেখা যায় না; আর লগে ভুল করে পরিবেশ চলক ছাপা হওয়ার ঝুঁকি কমে)। তবে এটি সত্যিকারের Secret Manager নয়: হোস্টের ফাইলটি যার পড়ার অধিকার আছে সে Secret পড়তে পারে, ফাইলটির ব্যাকআপ ও Rotation-এর দায়িত্ব আপনার, আর Secret-টি কনটেইনারের ভেতরে ফাইল হিসেবে থাকায় অ্যাপকে ফাইল পড়ে ব্যবহার করতে হয়। Swarm মোডে Secrets এনক্রিপ্ট করে সংরক্ষণ করা হয় — একক হোস্টের Compose-এ সেই সুরক্ষা নেই। তাই নিয়ম: ছোট প্রজেক্ট ও ডেভেলপমেন্টে Compose Secrets যথেষ্ট ভালো; গুরুত্বপূর্ণ প্রোডাকশনে HashiCorp Vault, AWS Secrets Manager, Azure Key Vault বা GCP Secret Manager ব্যবহার করুন — অধ্যায় ২৪-এ ক্লাউড প্রসঙ্গে ফিরে আসা হবে।

Secret ফাঁস হলে কী করবেন

একটি Secret একবার Git-এ, Image-এ বা লগে গেলে সেটি আর গোপন নেই — ইতিহাস মুছে ফেললেও কেউ কপি করে রেখে থাকতে পারে। সঠিক প্রতিক্রিয়া: (১) Secret-টি অবিলম্বে বাতিল (Revoke) করে নতুনটি তৈরি করুন; (২) Git ইতিহাস থেকে মুছুন (যেমন `git filter-repo`); (৩) প্রভাব যাচাই করুন — কে, কখন, কোথা থেকে ব্যবহার করেছে (ডেটাবেস ও ক্লাউডের অডিট লগ দেখুন); (৪) দলকে জানান। শুধু ফাইল মুছে কমিট করলে চলবে না — Git ইতিহাসে পুরনো কমিটে Secret থেকেই যায়।

কনফিগারেশন যাচাই (Validation) ও ডিফল্ট

অ্যাপ চালুর সময় প্রয়োজনীয় চলকগুলো যাচাই করা উচিত — না থাকলে স্পষ্ট ভুল বার্তা দিয়ে থামা উচিত, আধা-চালু অবস্থায় নয়। Node.js-এ Zod বা Joi দিয়ে পরিবেশ চলকের স্কিমা যাচাই করা যায়: `MONGODB_URI` না থাকলে "MONGODB_URI is required" বলে বেরিয়ে আসা, পরে ডেটাবেস সংযোগের সময় রহস্যজনক ভুলের চেয়ে হাজার গুণ ভালো। `docker compose config` দিয়ে Compose-স্তরের যাচাই, আর অ্যাপ-স্তরের যাচাই — দুটোই দরকার।

১৯.৭ Docker কমান্ড ও লাইন-বাই-লাইন ব্যাখ্যা

```
$ cp .env.example .env
$ docker compose config
$ docker compose exec api printenv | grep -E "NODE_ENV|PORT"
$ docker compose exec api sh -c 'cat /run/secrets/db_password'
$ docker compose exec api sh -c 'echo $MONGODB_URI'   # সতর্কতা: শুধু পরীক্ষার জন্য
$ docker compose run --rm api node -e "console.log(process.env.NODE_ENV)"
```

ব্যাখ্যা: `cp .env.example .env` — নমুনা থেকে আসল ফাইল বানানো (আসল মান বসাতে হবে)। `docker compose config` — Interpolation-এর পর চূড়ান্ত ফাইল দেখায়; এখানে Secret-এর মানও দেখা যেতে পারে — তাই এই কমান্ডের আউটপুট কখনো শেয়ার করবেন না, স্ক্রিনশটও নয়। `printenv | grep ...` — কনটেইনারের ভেতরে কোন পরিবেশ চলক সেট আছে তা যাচাই (শুধু অ-গোপন মানগুলো এভাবে দেখুন)। `cat /run/secrets/db_password` — Secret ফাইল হিসেবে মাউন্ট হয়েছে কি না যাচাই। `echo $MONGODB_URI` — শুধু ডিবাগিংয়ের উদাহরণ; এভাবে Secret টার্মিনালে ছাপা বিপজ্জনক, শেলের ইতিহাসে থেকে যায় — বাস্তবে করবেন না। `docker compose run --rm api node -e ...` — একবারের কনটেইনারে চলক পরীক্ষা।

১৯.৮ সম্পূর্ণ কনফিগারেশন উদাহরণ

```
# .env.example — Git-এ কমিট করা হয় (আসল মান নেই, শুধু কাঠামো)
MONGO_USER=technova_admin
MONGO_PASSWORD=CHANGE-ME-STRONG-PASSWORD
IMAGE_TAG=1.2.0
JWT_SECRET=CHANGE-ME-JWT-SECRET

# compose.yaml — প্রাসঙ্গিক অংশ (Secrets-সহ)
services:
  api:
    image: technova-api:${IMAGE_TAG:-1.2.0}
    env_file:
      - ./config/app.env
    environment:
      NODE_ENV: production
      PORT: "3000"
    secrets:
      - db_password
      - jwt_secret
    volumes:
      - ./config/app.env:/app/app.env:ro

  mongo:
    image: mongo:7
    environment:
      MONGO_INITDB_ROOT_USERNAME: ${MONGO_USER}
      MONGO_INITDB_ROOT_PASSWORD_FILE: /run/secrets/db_password
    secrets:
      - db_password

secrets:
  db_password:
    file: ./secrets/db_password.txt
```

```
  jwt_secret:
    file: ./secrets/jwt_secret.txt
```

ব্যাখ্যা: `MONGO_INITDB_ROOT_PASSWORD_FILE` — MongoDB-র অফিসিয়াল Image `_FILE` প্রত্যয়-সহ চলক দেখলে ফাইলের ভেতরের মান পড়ে — এটাই Docker-এর প্রচলিত Secret প্যাটার্ন (`_FILE` সাফিক্স)। `app.env` ফাইলে অ-গোপন অ্যাপ কনফিগারেশন থাকে (যেমন `LOG_LEVEL=info`, `CORS_ORIGIN=https://app.technova.example`)। `secrets/` ডিরেক্টরিটি `.gitignore`-এ রাখুন। অ্যাপের কোডে Secret পড়ার পদ্ধতি: `fs.readFileSync('/run/secrets/db_password', 'utf8').trim()` — পরিবেশ চলক না থাকলে ফাইল থেকে পড়ো, এই ফলব্যাকটি কোডে লিখুন।

```
# .gitignore — আবশ্যিক লাইনগুলো
.env
secrets/
*.pem
config/app.env
# config/app.env.example কমিট করুন, আসল app.env নয়

# config/app.env — অ-গোপন অ্যাপ কনফিগারেশন (তবু Git-এ নয়, সতর্কতার জন্য)
LOG_LEVEL=info
CORS_ORIGIN=https://app.technova.example
RATE_LIMIT_MAX=100
```

১৯.৯ বাস্তব প্রোডাকশন পরিস্থিতি

REAL WORLD — TechNova: TechNova-র একজন ইন্টার্ন Stack Overflow-এর উত্তরের জন্য একটি ভুলের স্ক্রিনশট শেয়ার করলেন — স্ক্রিনশটে `docker compose config`-এর আউটপুটে আসল MongoDB পাসওয়ার্ড দেখা যাচ্ছিল। কয়েক ঘণ্টার মধ্যে কেউ সেই পাসওয়ার্ড দিয়ে Staging ডেটাবেসে ঢুকে ডেটা মুছে ফেলল। দলের প্রতিক্রিয়া: পাসওয়ার্ড বদলানো, Git ইতিহাস পরিষ্কার করা, এবং নতুন নিয়ম — `config` কমান্ডের আউটপুট কখনো শেয়ার নয়; Secret-যুক্ত ফাইল `.gitignore`-এ; প্রতিটি পরিবেশের Secret আলাদা; আর প্রোডাকশনের Secret শুধু দুজন সিনিয়র ইঞ্জিনিয়ারের কাছে। শিক্ষা: Secret ফাঁস সাধারণত "হ্যাকিং" নয় — অসাবধান শেয়ারিং।

১৯.১০ ধাপে ধাপে ব্যবহারিক বাস্তবায়ন

1. `.env.example` লিখুন (সব চলকের নাম ও কাল্পনিক মান), `.gitignore`-এ `.env` ও `secrets/` যোগ করুন।
2. `secrets/db_password.txt` ফাইলে একটি শক্ত পাসওয়ার্ড লিখুন (ফাইলের শেষে নিউলাইন থাকলে অ্যাপে `.trim()` ব্যবহার করুন)।
3. Compose ফাইলে `secrets` ব্লক যোগ করে `MONGO_INITDB_ROOT_PASSWORD_FILE` ব্যবহার করুন।
4. `docker compose config` দিয়ে যাচাই করুন — আউটপুটটি কোথাও সংরক্ষণ বা শেয়ার করবেন না।
5. অ্যাপ চালু করে `docker compose exec api sh -c 'cat /run/secrets/db_password'` দিয়ে Secret মাউন্ট হয়েছে কি না যাচাই করুন।
6. অ্যাপে পরিবেশ চলকের যাচাই (Validation) যোগ করুন — প্রয়োজনীয় চলক না থাকলে চালুর সময় স্পষ্ট ভুল দেখাক।

১৯.১১ সাধারণ ভুল ও ভুল ধারণা

- **ভুল:** `.env` Git-এ কমিট করা — সবচেয়ে সাধারণ ও বিপজ্জনক ভুল।
- **ভুল:** `environment`-এ আসল পাসওয়ার্ড লেখা — `docker inspect` দিয়ে যে কেউ দেখতে পাবে।
- **ভুল:** `.env` ও `env_file` গুলিয়ে ফেলা — প্রথমটি Compose-এর, দ্বিতীয়টি কনটেইনারের অ্যাপের।
- **ভুল:** Secret ফাঁসের পর শুধু ফাইল মুছে দেওয়া — Secret বাতিল না করলে ফাঁস থেকেই যায়।
- **ভুল:** সব পরিবেশে একই Secret ব্যবহার — একটি ফাঁস হলে সব পরিবেশ আক্রান্ত হয়।
- **ভুল ধারণা:** "Compose Secrets মানেই এনক্রিপ্টেড Secret Manager" — না; একক হোস্টে এটি ফাইল মাউন্ট, Swarm-এ এনক্রিপ্টেড।
- **ভুল ধারণা:** "পরিবেশ চলক নিরাপদ, কারণ বাইরে থেকে দেখা যায় না" — না; `docker inspect`, লগ ও ভুল কনফিগারেশনে ফাঁস হয়।

১৯.১২ নিরাপত্তা বিবেচনা

WARNING: Secret ব্যবস্থাপনার সোনালি নিয়ম — Least Privilege: প্রতিটি Secret শুধু সেই সার্ভিসই পাবে যার সত্যিই দরকার। MongoDB-র পাসওয়ার্ড `web` সার্ভিসের দরকার নেই — তাকে দেবেন না। Secret-এর মেয়াদ (Rotation) নিয়মিত বদলান — যেমন প্রতি ৯০ দিনে ডেটাবেস পাসওয়ার্ড বদলানো, আর JWT Secret বদলালে সব ব্যবহারকারীকে আবার লগইন করতে হবে তা পরিকল্পনায় রাখুন। `docker compose config`-এর আউটপুট, `printenv`-এর ফল ও লগ — কোনোটিতেই Secret ছাপাবেন না।

১৯.১৩ কর্মক্ষমতা ও রিসোর্স বিবেচনা

Configuration ব্যবস্থাপনার সরাসরি রিসোর্স খরচ প্রায় নেই — তবে ভুল Configuration-এর পরোক্ষ খরচ বিশাল: ভুল লগ স্তর (`debug` প্রোডাকশনে) ডিস্ক ভরিয়ে দিতে পারে; ভুল Cache সেটিং ডেটাবেসকে ধীর করে দিতে পারে। তাই Configuration-কে কোডের মতো পর্যালোচনা (Review) করুন: প্রতিটি পরিবেশের `.env.example` ও ওভাররাইড ফাইলের পরিবর্তন Pull Request-এ দেখা উচিত। Secret Rotation-এর সময় ডাউনটাইম এড়াতে নতুন Secret আগে তৈরি করে, অ্যাপ আপডেট করে, তারপর পুরনোটি বাতিল করুন — একসঙ্গে বদলালে মাঝখানে সংযোগ ব্যর্থ হবে।

১৯.১৪ সমস্যা সমাধান পদ্ধতি

TROUBLESHOOTING: অ্যাপ বলছে "MONGODB_URI is not defined" — প্রথমে দেখুন চলকটি কোন পথে আসার কথা ছিল: `.env` (Interpolation), `env_file`, নাকি `environment`? `docker compose config` দিয়ে Interpolation ঠিক আছে কি না দেখুন, তারপর `docker compose exec api printenv | grep MONGODB` দিয়ে কনটেইনারের ভেতরে মান পৌঁছেছে কি না দেখুন। `.env` বদলানোর পর কনটেইনার আবার তৈরি (Recreate) করতে হয় — শুধু `restart` করলে নতুন চলক পৌঁছায় না, কারণ পরিবেশ চলক কনটেইনার তৈরির সময় নির্ধারিত হয়।

সমস্যা	সম্ভাব্য কারণ	সমাধান
`${VAR:?}` ভুল দেখিয়ে থামছে	`.env`-এ চলকটি নেই	`.env.example` দেখে মান বসান

`.env` বদলালেও মান বদলায় না	কনটেইনার Recreate করা হয়নি	`docker compose up -d` দিয়ে আবার তৈরি করুন
Secret ফাইল খালি বা পাওয়া যায় না	`secrets` ব্লকের পথ ভুল	ফাইলের পথ ও `.gitignore` যাচাই করুন
`docker inspect`-এ পাসওয়ার্ড দেখা যাচ্ছে	`environment`-এ Secret লেখা	Secret ফাইল (`_FILE` প্যাটার্ন) ব্যবহার করুন

১৯.১৫ সেরা অনুশীলন ও অ্যান্টি-প্যাটার্ন

সেরা অনুশীলন: একই Image, ভিন্ন Configuration; `.env.example` কমিট, `.env` নয়; Secret শুধু Secret ফাইল বা Secret Manager থেকে; বাধ্যতামূলক চলকে `:?`; অ্যাপ চালুর সময় চলক যাচাই; প্রতি পরিবেশে আলাদা Secret; নিয়মিত Rotation। **অ্যান্টি-প্যাটার্ন:** Secret Git/Image/লগে; `environment`-এ পাসওয়ার্ড; একই Secret সব পরিবেশে; ফাঁসের পর শুধু ফাইল মোছা; `config` আউটপুট শেয়ার করা।

১৯.১৬ অধ্যায়ের সারাংশ

Configuration অ-গোপন মান, Secret গোপন মান, আর Environment Variable হলো মান পৌঁছে দেওয়ার একটি পদ্ধতি — তিনটি গুলিয়ে ফেলবেন না। Compose-এ `.env` ফাইলের Interpolation, `env_file`, `environment` ও `secrets` — চারটি আলাদা পথ, প্রতিটির নিজস্ব কাজ ও অগ্রাধিকার। Compose Secrets একক হোস্টে ফাইল মাউন্ট — `environment`-এর চেয়ে নিরাপদ, কিন্তু সত্যিকারের Secret Manager নয়। Secret ফাঁস হলে বাতিল করাই একমাত্র প্রতিকার। একই Image সব পরিবেশে, Configuration পরিবেশ থেকে — এই নীতিটি ধরে রাখলে Deploy নিরাপদ ও পুনরাবৃত্তিযোগ্য হয়।

১৯.১৭ মূল প্রযুক্তিগত পরিভাষা

Configuration: পরিবেশভেদে বদলায় এমন অ-গোপন মান (পোর্ট, লগ স্তর, URL)।

Secret: জানা গেলে ক্ষতি হয় এমন মান — পাসওয়ার্ড, API Key, টোকেন, প্রাইভেট Key।

Environment Variable: প্রসেসকে দেওয়া নাম-মান জোড়া; Configuration পৌঁছে দেওয়ার পদ্ধতি।

12-Factor App: Configuration কোড থেকে আলাদা রাখার নীতিসহ আধুনিক অ্যাপ তৈরির ১২টি নীতি।

Interpolation: Compose ফাইলে `${VAR}` লিখে বাইরে থেকে মান বসানো।

env_file: ফাইল থেকে কনটেইনারের পরিবেশ চলক সরবরাহের Compose নির্দেশ।

Secret Rotation: নিয়মিত Secret বদলে পুরনোটি বাতিল করা।

Revoke: ফাঁস হওয়া বা পুরনো Secret বাতিল করে অকার্যকর করা।

Least Privilege: প্রত্যেককে শুধু তার দরকারি ন্যূনতম অধিকার দেওয়ার নীতি।

১৯.১৮ জ্ঞান যাচাই — প্রশ্ন

1. Configuration, Environment Variable ও Secret-এর পার্থক্য ব্যাখ্যা করুন — প্রতিটির একটি উদাহরণ দিন।
2. `.env` ও `env_file`-এর কাজের পার্থক্য কী? গুলিয়ে ফেললে কী সমস্যা হয়?
3. `${VAR:?message}` কখন ব্যবহার করবেন? একটি উদাহরণ দিন।
4. Compose Secrets কেন `environment`-এ পাসওয়ার্ড লেখার চেয়ে নিরাপদ, আর কেন এটি পূর্ণ Secret Manager নয়?
5. একটি Secret Git-এ ফাঁস হলে সঠিক প্রতিক্রিয়ার ধাপগুলো লিখুন।
6. একই Image তিনটি পরিবেশে চালাতে Configuration কীভাবে সাজাবেন?

১৯.১৯ ব্যবহারিক অনুশীলন

1. **Secret স্থানান্তর:** `environment`-এ লেখা একটি পাসওয়ার্ডকে Compose Secret ফাইলে সরিয়ে `_FILE` প্যাটার্নে ব্যবহার করুন।
2. **যাচাই:** অ্যাপে একটি Validation যোগ করুন যা প্রয়োজনীয় চলক না থাকলে স্পষ্ট ভুল দেখিয়ে থামে।
3. **ফাঁসের মহড়া:** একটি কাল্পনিক Secret "ফাঁস" হয়েছে ধরে Rotation-এর ধাপগুলো লিখে অনুশীলন করুন।

অধ্যায় 20 • PART IV — Docker Compose and Multi-Container Systems

Chapter 20 — Health Checks, Dependencies and Service Recovery

২০.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: (১) কনটেইনার "চলছে" (Running) ও "সুস্থ" (Healthy) — এই দুটি অবস্থার পার্থক্য ব্যাখ্যা করতে; (২) `healthcheck`-এর সব ক্ষেত্র (`test`, `interval`, `timeout`, `retries`, `start_period`) সঠিকভাবে লিখতে; (৩) `depends_on`-এর `service_started` বনাম `service_healthy`-এর পার্থক্য প্রয়োগ করতে; (৪) অ্যাপে Retry ও Backoff যুক্তি লিখতে; (৫) Restart নীতি দিয়ে সার্ভিস পুনরুদ্ধারের নকশা করতে; (৬) নির্ভরতার ব্যর্থতার (Cascading Failure) ঝুঁকি চিনতে ও কমাতে।

২০.২ কেন এই বিষয়টি DevOps-এ গুরুত্বপূর্ণ

সবচেয়ে বিরক্তিকর প্রোডাকশন ব্যর্থতাগুলো এভাবেই ঘটে: সার্ভার রিবুটের পর সব কনটেইনার "চলছে" দেখায়, কিন্তু সাইট কাজ করে না — কারণ API ডেটাবেসের আগেই চালু হয়ে সংযোগে ব্যর্থ হয়ে থেমে গেছে, অথবা ডেটাবেস এখনো প্রস্তুত নয়। মানুষ তখন ভাবে "Docker ভেঙে গেছে", আসলে ভেঙেছে নির্ভরতার নকশা। একটি সিস্টেমের নির্ভরযোগ্যতা নির্ধারিত হয় তার সবচেয়ে দুর্বল শুরুর ক্রম দিয়ে। Health Check হলো সিস্টেমের নিজের সম্পর্কে সত্য বলার ব্যবস্থা: "আমি প্রসেস হিসেবে বেঁচে আছি" নয়, "আমি আসল কাজটি করতে পারছি" — এই সত্য। এই অধ্যায়ে শিখবেন কীভাবে সার্ভিসগুলো একে অপরের প্রস্তুতি যাচাই করে, ব্যর্থ হলে নিজে থেকে সেরে ওঠে, আর একজনের ব্যর্থতা যাতে সবাইকে না টানে।

২০.৩ পূর্বশর্ত জ্ঞান

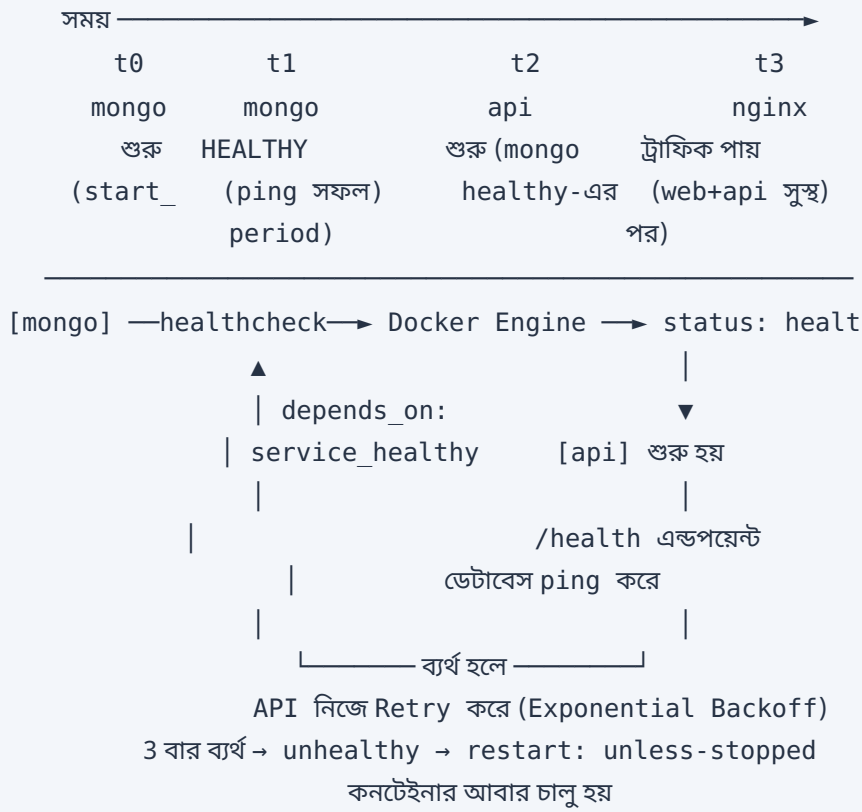
অধ্যায় ১৭-১৮-এর Compose, `depends_on` ও `healthcheck`-এর প্রাথমিক ধারণা, HTTP Status Code (বিশেষ করে 200 ও 503), Node.js-এ Promise ও `async/await`, MongoDB সংযোগের মৌলিক ধারণা, এবং `docker compose ps`-এর আউটপুট পড়া। Retry ও Exponential Backoff-এর পূর্ব জ্ঞান প্রয়োজন নেই।

২০.৪ প্রথম নীতি থেকে ব্যাখ্যা

প্রথম নীতি: একটি প্রসেস বেঁচে থাকা আর একটি সার্ভিস কাজ করা — দুটি আলাদা দাবি। Node.js প্রসেস চালু থাকতে পারে, অথচ ডেটাবেস সংযোগ বিচ্ছিন্ন থাকায় প্রতিটি অনুরোধ ব্যর্থ হচ্ছে। Docker Engine শুধু জানে প্রসেসটি বেঁচে আছে কি না (PID আছে কি না); অ্যাপটি কাজ করছে কি না তা জানে শুধু অ্যাপ নিজে। Health Check হলো অ্যাপের নিজেকে পরীক্ষা করার একটি ছোট প্রোগ্রাম — যেমন API-র `/health` এন্ডপয়েন্ট ডেটাবেসে একটি `ping` পাঠিয়ে দেখে সংযোগ আছে কি না, আর 200 (সুস্থ) বা 503 (অসুস্থ) ফেরত দেয়। দ্বিতীয় নীতি: নির্ভরতা একটি প্রতিশ্রুতি নয়, একটি সময়ের ব্যাপার। MongoDB "চালু হয়েছে" মানে এই মুহূর্তে সংযোগ নেবে — এমন নয়; প্রথমবার চালু হতে সময় লাগে, আর চলমান ডেটাবেসও মাঝে মাঝে ব্যস্ত বা পুনরায় চালু হতে পারে। তাই নির্ভরশীল অ্যাপকে সবসময় ধরে নিতে হয় — নির্ভরতা যেকোনো মুহূর্তে অনুপস্থিত হতে পারে — এবং পুনঃচেষ্টা (Retry) করতে হয়। তৃতীয় নীতি: পুনরুদ্ধার স্বয়ংক্রিয় হওয়া চাই। রাত তিনটায় ডেটাবেস পড়ে গেলে মানুষ জেগে ঠিক করবে — এমন নকশা নির্ভরযোগ্য নয়; সিস্টেম নিজে থেকে আবার চেষ্টা করবে, আর না পারলে স্পষ্টভাবে অসুস্থ বলে জানাবে — এটাই পেশাদার নকশা।

CONCEPT: Running বনাম Healthy — একটি কনটেইনারের তিনটি সম্ভাব্য অবস্থা ভাবুন: (১) `created` — তৈরি হয়েছে, চলেনি; (২) `running` — প্রসেস বেঁচে আছে; (৩) `healthy` — প্রসেস বেঁচে আছে এবং Health Check পাস করছে। `docker compose ps`-এ `Up (healthy)` দেখলে তবেই তৃতীয় অবস্থা। Health Check ছাড়া কোনো সার্ভিস কখনো "healthy" হয় না — শুধু "Up" দেখায়, যা প্রায় কোনো তথ্যই দেয় না।

২০.৫ আর্কিটেকচার / ধারণাগত চিত্র



লক্ষ্য করুন: Health Check শুধু শুরুর সময়ের নয় — চলমান অবস্থায়ও প্রতি ১৫ সেকেন্ডে চলে। ডেটাবেস পরে পড়ে গেলে API `unhealthy` হয়ে যাবে — তখনই ধরা পড়বে, ব্যবহারকারী অভিযোগ করার আগে।

২০.৬ বিস্তারিত প্রযুক্তিগত ব্যাখ্যা

Health Check-এর ক্ষেত্রগুলো

ক্ষেত্র	অর্থ	TechNova-র মান
`test`	সুস্থতা যাচাইয়ের কমান্ড; Exit Code 0 মানে সুস্থ, 1 মানে অসুস্থ	`["CMD", "mongosh", "--eval", "db.adminCommand('ping')"]`
`interval`	কতক্ষণ পরপর পরীক্ষা	10s (mongo), 15s (api)
`timeout`	একটি পরীক্ষার জন্য সর্বোচ্চ অপেক্ষা	5s

`retries`	অসুস্থ ঘোষণার আগে কতবার ব্যর্থ হতে দেবে	5 (mongo), 3 (api)
`start_period`	শুরুর এই সময়টুকু ব্যর্থতা গোনা হবে না	30s (mongo), 20s (api)
`start_interval`	শুরুর সময়ে পরীক্ষার ব্যবধান (Compose v2.20+)	5s — দ্রুত সুস্থতা ধরা পড়ে

`test`-এর তিনটি রূপ আছে: `["CMD", "..."]` (কনটেইনারের ভেতরে কমান্ড চালানো — সবচেয়ে সাধারণ), `["CMD-SHELL", "curl -f http://localhost:3000/health || exit 1"]` (শেলের মাধ্যমে — পাইপ বা `||` ব্যবহার করতে চাইলে), এবং `["NONE"]` (Health Check নিষ্ক্রিয় করা — বেস Image-এর Health Check বন্ধ করতে)। `curl -f`-এর `-f` ফ্ল্যাগটি গুরুত্বপূর্ণ: HTTP ভুল (যেমন 500) পেলে `curl` Exit Code 22 দেয়, না হলে 200 না হলেও Exit Code 0 দিতে পারে — তখন অসুস্থ সার্ভিস সুস্থ দেখাবে।

একটি ভালো /health এন্ডপয়েন্ট কেমন হয়

ভালো Health এন্ডপয়েন্ট তিনটি কাজ করে: (১) অ্যাপের নিজের অবস্থা জানায়; (২) গুরুত্বপূর্ণ নির্ভরতা (ডেটাবেস) যাচাই করে; (৩) দ্রুত সাড়া দেয় (কয়েকশো মিলিসেকেন্ডের মধ্যে)। খারাপ Health এন্ডপয়েন্ট ভারী কাজ করে (যেমন পুরো টেবিল গোনা) — তখন Health Check নিজেই সার্ভিসকে ধীর করে দেয়। দুটি স্তর আলাদা করা ভালো অভ্যাস: `/health/live` (Liveness — প্রসেস বেঁচে আছে কি না, হালকা) ও `/health/ready` (Readiness — ডেটাবেস-সহ কাজ করতে প্রস্তুত কি না)। Compose-এর Health Check সাধারণত Readiness-এর মতো কাজ করে। আরেকটি সূক্ষ্ম বিষয়: Health এন্ডপয়েন্টে সংবেদনশীল তথ্য (যেমন ডেটাবেসের ভেতরের বিবরণ) ফেরত দেবেন না — শুধু `{"status": "ok"}` বা `{"status": "degraded"}` যথেষ্ট।

depends_on-এর দুটি Condition

`condition: service_started` — নির্ভরশীল কনটেইনার শুরু হলেই পরেরটি শুরু হবে (ডিফল্ট; শুধু `depends_on: [mongo]` লিখলে এটাই হয়)। `condition: service_healthy` — নির্ভরশীল কনটেইনার Health Check পাস করলেই পরেরটি শুরু হবে। দ্বিতীয়টি ব্যবহার করতে হলে নির্ভরশীল সার্ভিসে `healthcheck` থাকতেই হবে — না থাকলে Compose ভুল দেখাবে। মনে রাখবেন: `service_healthy` শুধু শুরুর ক্রম নিয়ন্ত্রণ করে; পরে MongoDB পড়ে গেলে API-কে Compose নিজে থেকে থামায় না — সেটি অ্যাপের Retry ও Health Check-এর কাজ।

Retry ও Exponential Backoff

অ্যাপের সংযোগ কোডে পুনঃচেষ্টা লিখতে হয় — একবার ব্যর্থ হলেই হাল ছাড়া যাবে না। Exponential Backoff মানে প্রতিটি ব্যর্থ চেষ্টার পর অপেক্ষা দ্বিগুণ করা: ১ সেকেন্ড, ২ সেকেন্ড, ৪ সেকেন্ড, ৮ সেকেন্ড — এতে ব্যর্থ সার্ভিসের ওপর চাপ কমে (সবাই একসঙ্গে বারবার চেষ্টা করলে সার্ভিসটি আর উঠতেই পারে না — একে বলে Retry Storm)। Mongoose-এ সংযোগের সময় `serverSelectionTimeoutMS` ও পুনঃচেষ্টার সীমা নির্ধারণ করা যায়; আরও ভালো — সংযোগ ব্যর্থ হলে অ্যাপটি লগ লিখে নিজে থেকে আবার চেষ্টা করবে, নির্দিষ্ট সংখ্যার পর স্পষ্ট ভুল দিয়ে থামবে যাতে Restart নীতি আবার চালু করে।

Restart নীতি ও পুনরুদ্ধারের সীমা

`restart: unless-stopped` প্রোডাকশনের মান — কনটেইনার যেকোনো কারণে থামলে (ক্র্যাশ, OOM, Health Check-এর পর Docker-এর হস্তক্ষেপ) আবার চালু হবে। তবে একটি সতর্কতা: Restart নীতি অসুস্থ কনটেইনারকে নিজে থেকে আবার চালু করে না — Docker Engine Health Check-এর ফল দেখে কনটেইনার থামায় না, শুধু অবস্থা জানায়। অসুস্থ হলে পুনরায় চালু করতে বাইরের পর্যবেক্ষণ (Monitoring) বা অর্কেস্ট্রেটর লাগে — Compose-এর এই সীমাটি স্বীকার করা

জরুরি (অধ্যায় ২৩-২৪)। তাই অ্যাপের নিজের Retry + Health Check + Restart নীতি — তিনটি মিলে পুনরুদ্ধারের ব্যবস্থা হয়, কোনো একটিতে নয়।

Cascading Failure — একজনের ব্যর্থতা সবার ব্যর্থতা

MongoDB ধীর হয়ে গেলে API-র প্রতিটি অনুরোধ ডেটাবেসের উত্তরের অপেক্ষায় আটকে যায়; API-র সংযোগ পুল ভরে যায়; Nginx API-র উত্তরের অপেক্ষায় থাকে; ব্রাউজার Timeout হয়। একটিমাত্র ধীর নির্ভরতা পুরো সিস্টেমকে টেনে নামায় — একে বলে Cascading Failure। প্রতিকার: প্রতিটি সংযোগে Timeout (যেমন MongoDB-তে `serverSelectionTimeoutMS: 5000`), সংযোগ পুলের সীমা, API-তে অনুরোধের Timeout, আর Health Check-এ নির্ভরতার যাচাই যাতে অসুস্থতা তাড়াতাড়ি ধরা পড়ে। Timeout ছাড়া Retry শুধু চাপ বাড়ায় — তাই "Retry + Timeout + Backoff" — তিনটি একসঙ্গে।

২০.৭ Docker কমান্ড ও লাইন-বাই-লাইন ব্যাখ্যা

```
$ docker compose ps
$ docker inspect technova-api-1 --format='{{json .State.Health}}'
$ docker compose logs api --tail 50
$ docker compose up -d --force-recreate api
$ docker compose restart api
$ docker compose stop api && docker compose start api
```

ব্যাখ্যা: `docker compose ps` — কোন সার্ভিস `(healthy)`, কোনটি `(unhealthy)` বা `(starting)` তা দেখায়। `docker inspect ... --format='{{json .State.Health}}'` — Health Check-এর বিস্তারিত ইতিহাস (শেষ পরীক্ষাগুলোর ফল, ব্যর্থতার বার্তা) JSON আকারে দেখায় — Health Check কেন ব্যর্থ হচ্ছে তা বোঝার সেরা উপায়। `logs --tail 50` — শেষ ৫০ লাইন লগ (পুরো লগ না টেনে)। `up -d --force-recreate api` — `api` কনটেইনারটি মুছে আবার তৈরি করে (কনফিগারেশন বদলালে দরকার)। `restart api` — কনটেইনারটি থামিয়ে আবার চালু করে (তৈরি না করে)। `stop && start` — একই কাজ দুটি ধাপে; `restart`-এর বিকল্প।

নমুনা আউটপুট (চিত্রণমূলক):

```
$ docker compose ps
NAME                SERVICE   STATUS                    PORTS
technova-api-1      api       Up 5 minutes (healthy)    3000/tcp
technova-mongo-1    mongo     Up 6 minutes (healthy)    27017/tcp
$ docker inspect technova-mongo-1 --format='{{.State.Health.Status}}'
healthy
```

২০.৮ সম্পূর্ণ কনফিগারেশন উদাহরণ

```
# compose.yaml — Health Check ও নির্ভরতা (সম্পূর্ণ অংশ)
services:
  mongo:
    image: mongo:7
    restart: unless-stopped
    healthcheck:
      test: ["CMD", "mongosh", "--quiet", "--eval", "db.adminCommand('ping')"]
      interval: 10s
      timeout: 5s
      retries: 5
```

```
      start_period: 30s
      start_interval: 5s

  api:
    build: ./api
    image: technova-api:1.2.0
    restart: unless-stopped
    depends_on:
      mongo:
        condition: service_healthy
    stop_grace_period: 30s
    healthcheck:
      test: ["CMD-SHELL", "node -e \"fetch('http://localhost:3000/health/
ready').then(r=>process.exit(r.ok?0:1)).catch(()=>process.exit(1))\""]
      interval: 15s
      timeout: 5s
      retries: 3
      start_period: 20s

  web:
    build: ./web
    image: technova-web:1.2.0
    restart: unless-stopped
    depends_on:
      api:
        condition: service_healthy

  nginx:
    image: nginx:1.27
    restart: unless-stopped
    depends_on:
      web:
        condition: service_started
      api:
        condition: service_started
```

ব্যাখ্যা: `mongo`-র Health Check `mongosh` দিয়ে `ping` করে — এটি শুধু প্রসেস নয়, ডেটাবেসের সাড়া যাচাই করে। `api` শুরু হয় কেবল MongoDB সুস্থ হলে; `web` শুরু হয় API সুস্থ হলে; `nginx` শুধু `service_started` শর্তে চলে — কারণ Nginx-এর Health Check নেই, আর Nginx নিজে আপস্ট্রিম (Upstream) ব্যর্থ হলে 502 দেয়, নিজে থামে না। `stop_grace_period: 30s` API-কে থামার জন্য ৩০ সেকেন্ড দেয় (অধ্যায় ১৮-এর Graceful Shutdown কোডের সঙ্গে মিলিয়ে)।

```
// api/src/db.js — Retry ও Backoff-সহ সংযোগ (সংক্ষিপ্ত)
async function connectWithRetry(uri, maxAttempts = 10) {
  for (let attempt = 1; attempt <= maxAttempts; attempt++) {
    try {
      await mongoose.connect(uri, { serverSelectionTimeoutMS: 5000 });
      console.log('MongoDB connected');
      return;
    } catch (err) {
      const delay = Math.min(1000 * 2 ** attempt, 30000); // 2s, 4s, 8s ... সর্বোচ্চ 30s
      console.log(`MongoDB সংযোগ ব্যর্থ (চেষ্টা ${attempt}/${maxAttempts}) — ${delay}ms পর আবার`);
      await new Promise(r => setTimeout(r, delay));
    }
  }
  throw new Error('MongoDB-তে সংযোগ করা যায়নি — অ্যাপ থামছে');
```

```
}

// api/src/routes/health.js — Liveness বনাম Readiness
router.get('/health/live', (req, res) => res.json({ status: 'ok' }));
router.get('/health/ready', async (req, res) => {
  try {
    await mongoose.connection.db.admin().ping(); // ডেটাবেসে ping
    res.json({ status: 'ok' });
  } catch {
    res.status(503).json({ status: 'degraded', reason: 'database unreachable' });
  }
});
```

২০.৯ বাস্তব প্রোডাকশন পরিস্থিতি

REAL WORLD — TechNova: TechNova-র সার্ভার রিবুটের পর একদিন সকাল ৯টায় সাইট ডাউন ছিল — অথচ `docker compose ps`-এ সব `Up` দেখাচ্ছিল। তদন্তে দেখা গেল: MongoDB প্রথমবার চালু হতে ৪০ সেকেন্ড নিয়েছিল, API ৫ সেকেন্ডে চালু হয়ে সংযোগে ব্যর্থ হয়ে থেমে গিয়েছিল, আর `restart: no` থাকায় কেউ তাকে আবার চালু করেনি। Nginx চালু ছিল, তাই পোর্ট 80 সাড়া দিচ্ছিল — কিন্তু প্রতিটি অনুরোধে 502। সমাধান ছিল তিনটি একসঙ্গে: MongoDB-তে Health Check + API-তে `depends_on: service_healthy` + API-তে Retry/Backoff + `restart: unless-stopped`। এরপর থেকে সার্ভার রিবুটের পর স্ট্যাক নিজে থেকে সুস্থভাবে ওঠে — মানুষের হস্তক্ষেপ ছাড়াই।

২০.১০ ধাপে ধাপে ব্যবহারিক বাস্তবায়ন

1. API-তে `/health/live` ও `/health/ready` এন্ডপয়েন্ট যোগ করুন (ওপরের কোড অনুযায়ী)।
2. `compose.yaml`-এ MongoDB ও API-র Health Check যোগ করুন এবং `depends_on`-এ `condition: service_healthy` লিখুন।
3. `docker compose up -d` করে `docker compose ps`-এ `(healthy)` দেখা যাচ্ছে কি না যাচাই করুন।
4. ইচ্ছাকৃতভাবে MongoDB থামান (`docker compose stop mongo`) — API-র `/health/ready` 503 দিচ্ছে কি না দেখুন; তারপর MongoDB চালু করে API নিজে থেকে সেরে ওঠে কি না দেখুন।
5. `docker inspect technova-api-1 --format='{{json .State.Health}}'` দিয়ে Health Check-এর ইতিহাস পড়ুন।
6. API-র সংযোগ কোডে Retry/Backoff যোগ করে পরীক্ষা করুন: MongoDB বন্ধ থাকা অবস্থায় API চালু করলে সে অপেক্ষা করে সংযোগ করে কি না।

২০.১১ সাধারণ ভুল ও ভুল ধারণা

- **ভুল:** Health Check-এ শুধু প্রসেস আছে কি না দেখা (যেমন `ps aux | grep node`) — প্রসেস থাকলেই সার্ভিস কাজ করে না।
- **ভুল:** Health এন্ডপয়েন্টে ভারী কাজ (পুরো সংগ্রহ গোনা) — Health Check নিজেই সার্ভিসকে ধীর করে।
- **ভুল:** `start_period` না দেওয়া — শুরুর স্বাভাবিক বিলম্বকে ব্যর্থতা ভেবে কনটেইনার অসুস্থ ঘোষণা করা হয়।
- **ভুল:** `depends_on` লিখেই নিশ্চিন্ত হওয়া — পরে নির্ভরতা পড়ে গেলে Compose কিছু করে না; Retry দরকার।

- **ভুল:** Retry-তে নির্দিষ্ট ব্যবধান (প্রতিবার ১ সেকেন্ড) — Retry Storm তৈরি হয়; Exponential Backoff ব্যবহার করুন।
- **ভুল ধারণা:** "Restart নীতি অসুস্থ কনটেইনারকে আবার চালু করে" — না; Docker Health Check দেখে কনটেইনার থামায় না, শুধু অবস্থা জানায়।
- **ভুল ধারণা:** "`Up` মানেই সুস্থ" — না; `(healthy)` না দেখলে সুস্থতা অজানা।

২০.১২ নিরাপত্তা বিবেচনা

WARNING: Health এন্ডপয়েন্টে সংবেদনশীল তথ্য ফেরত দেবেন না — যেমন ডেটাবেসের সংস্করণ, ভেতরের IP, বা ভুলের বিস্তারিত Stack Trace। আক্রমণকারী `/health` থেকে সিস্টেমের মানচিত্র আঁকতে পারে। শুধু `ok` / `degraded` জানান। আর Health Check কমান্ডে পাসওয়ার্ড লিখবেন না — `mongosh`-এর Health Check-এ প্রমাণীকরণ ছাড়া `ping` ব্যবহার করা হয়েছে, এটিই সঠিক।

২০.১৩ কর্মক্ষমতা ও রিসোর্স বিবেচনা

Health Check-এর খরচ ছোট কিন্তু শূন্য নয়: প্রতি ১০–১৫ সেকেন্ডে একটি `ping` বা HTTP অনুরোধ — ডেটাবেসের ওপর নগণ্য চাপ। তবে `interval` খুব ছোট (যেমন ১ সেকেন্ড) করলে অযথা চাপ বাড়ে; খুব বড় (যেমন ৫ মিনিট) করলে ব্যর্থতা ধরা পড়তে দেরি হয়। ১০–৩০ সেকেন্ডের ব্যবধান সাধারণত সঠিক ভারসাম্য। Retry-এর ক্ষেত্রে Backoff-এর সর্বোচ্চ সীমা (`Math.min(..., 30000)`) রাখা জরুরি — না হলে ব্যর্থ সার্ভিসের জন্য অপেক্ষা মিনিটের পর মিনিট বাড়তে থাকে।

২০.১৪ সমস্যা সমাধান পদ্ধতি

TROUBLESHOOTING: সার্ভিস `(unhealthy)` দেখালে: (১) `docker inspect <container> --format='{{json .State.Health}}'` দিয়ে শেষ পরীক্ষার আউটপুট ও Exit Code দেখুন; (২) কনটেইনারের ভেতরে হাতে Health Check কমান্ডটি চালান (`docker compose exec api node -e "fetch..."`); (৩) `docker compose logs` দিয়ে অ্যাপের ভুল দেখুন — Health Check ব্যর্থতার কারণ প্রায়ই অ্যাপের লগে লেখা থাকে; (৪) `start_period` বাড়িয়ে দেখুন শুরুর বিলম্বই কারণ কি না।

সমস্যা	সম্ভাব্য কারণ	সমাধান
API চালু হয়েই থেমে যায়	MongoDB প্রস্তুত হওয়ার আগেই সংযোগের চেষ্টা	`service_healthy` + Retry/Backoff যোগ করুন
সার্ভিস সবসময় `(unhealthy)`	Health Check কমান্ড ভুল বা `curl` নেই	কনটেইনারের ভেতরে হাতে কমান্ড চালিয়ে যাচাই করুন
শুরুর পর কিছুক্ষণ `(unhealthy)`, তারপর ঠিক	`start_period` খুব ছোট	`start_period` বাড়ান
MongoDB পড়ে গেলে পুরো সাইট ডাউন	Timeout ছাড়া সংযোগ, Retry Storm	Timeout + Backoff + সংযোগ পুলের সীমা দিন

২০.১৫ সেরা অনুশীলন ও অ্যান্টি-প্যাটার্ন

সেরা অনুশীলন: প্রতিটি সার্ভিসে অর্থবহ Health Check; Liveness ও Readiness আলাদা; `service_healthy` শুরুর ক্রমে; অ্যাপে Retry + Backoff + Timeout — তিনটি একসঙ্গে; `restart: unless-stopped`; Health এন্ডপয়েন্টে সংবেদনশীল তথ্য নয়। **অ্যান্টি-প্যাটার্ন:** Health Check ছাড়া নির্ভরতা; শুধু প্রসেস দেখে সুস্থতা বিচার; `depends_on`-এর ওপর সম্পূর্ণ ভরসা; নির্দিষ্ট ব্যবধানে অসীম Retry; ভারী Health এন্ডপয়েন্ট।

২০.১৬ অধ্যায়ের সারাংশ

চলছে (Running) মানেই সুস্থ (Healthy) নয় — সুস্থতা মানে অ্যাপটি আসল কাজ করতে পারছে, যা Health Check দিয়ে যাচাই করতে হয়। `depends_on: service_healthy` শুরুর ক্রম নিয়ন্ত্রণ করে, কিন্তু চলমান অবস্থায় নির্ভরতা পড়ে গেলে অ্যাপের নিজের Retry, Backoff ও Timeout-ই ভরসা। Restart নীতি ক্র্যাশ থেকে পুনরুদ্ধার করে, কিন্তু অসুস্থ কনটেইনারকে নিজে থেকে থামায় না — তাই Health Check, Retry ও Restart — তিনটি মিলে পুনরুদ্ধারের ব্যবস্থা। একজনের ব্যর্থতা যাতে সবাইকে না টানে, তার জন্য প্রতিটি সংযোগে Timeout আবশ্যিক।

২০.১৭ মূল প্রযুক্তিগত পরিভাষা

Health Check: কনটেইনার সুস্থ কি না তা যাচাইয়ের নিয়মিত পরীক্ষা (কমান্ড বা HTTP অনুরোধ)।

Healthy / Unhealthy: Health Check পাস করা বা ব্যর্থ হওয়া অবস্থা; `Up` থেকে আলাদা।

Liveness: প্রসেস বেঁচে আছে কি না — হালকা পরীক্ষা।

Readiness: সার্ভিস কাজ করতে প্রস্তুত কি না — নির্ভরতা-সহ পরীক্ষা।

Retry: ব্যর্থ কাজ আবার চেষ্টা করা।

Exponential Backoff: প্রতিটি ব্যর্থ চেষ্টার পর অপেক্ষা দ্বিগুণ করা — Retry Storm এড়াতে।

Retry Storm: অনেক ক্লায়েন্ট একসঙ্গে বারবার চেষ্টা করে ব্যর্থ সার্ভিসকে আরও চাপে ফেলা।

Cascading Failure: এক সার্ভিসের ব্যর্থতা শৃঙ্খল ধরে অন্যগুলোকে ফেলে দেওয়া।

start_period: Health Check-এর শুরুর ছাড়ের সময় — এই সময় ব্যর্থতা গোনা হয় না।

২০.১৮ জ্ঞান যাচাই — প্রশ্ন

1. Running ও Healthy-এর পার্থক্য ব্যাখ্যা করুন। `docker compose ps`-এ কী দেখে সুস্থতা বোঝা যায়?
2. `service_started` ও `service_healthy`-এর পার্থক্য কী? দ্বিতীয়টি ব্যবহারের শর্ত কী?
3. Liveness ও Readiness এন্ডপয়েন্ট আলাদা করা কেন ভালো অভ্যাস?
4. Exponential Backoff কী? নির্দিষ্ট ব্যবধানে Retry করলে কী সমস্যা হয়?
5. Restart নীতি কেন অসুস্থ কনটেইনারকে পুনরায় চালু করে না?
6. Cascading Failure কী? Timeout কীভাবে এটি কমায়?

২০.১৯ ব্যবহারিক অনুশীলন

1. **Health Check যোগ:** আপনার API-তে `/health/live` ও `/health/ready` যোগ করে Compose-এ Health Check লিখুন।
2. **ব্যর্থতার মহড়া:** MongoDB থামিয়ে API-র আচরণ দেখুন — 503 আসে কি না, লগে কী লেখা হয়।
3. **Retry পরীক্ষা:** MongoDB বন্ধ রেখে API চালু করুন — Backoff-সহ Retry কাজ করছে কি না লগে দেখুন।

PART V • DEVOPS ENGINEERING MASTERY

Production Container Operations

কনটেইনার সিকিউরিটি, resource limit, logging ও monitoring, প্রোডাকশন ডেপ্লয়মেন্ট, আপডেট, ট্রাবলশুটিং ও সেরা অনুশীলন।

অধ্যায় 21 • PART V — Container Security, Resources and Production Operations

Chapter 21 — Container Security and Least-Privilege Design

২১.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: (১) কনটেইনারের নিরাপত্তা মডেল — Namespace, Cgroup ও Capabilities — প্রথম নীতি থেকে ব্যাখ্যা করতে; (২) Non-Root কনটেইনার, Read-Only Root Filesystem ও Capability Dropping বাস্তবায়ন করতে; (৩) Privileged কনটেইনার ও Docker Socket মাউন্টের ঝুঁকি ব্যাখ্যা করতে; (৪) Seccomp, AppArmor/SELinux-এর ভূমিকা বলতে; (৫) Image Provenance, স্ক্যানিং ও Base Image নির্বাচনের নিয়ম প্রয়োগ করতে; (৬) Supply-Chain ঝুঁকি, নেটওয়ার্ক বিচ্ছিন্নতা ও Container Escape ধারণা ব্যাখ্যা করতে; (৭) একটি Compose স্ট্যাকের নিরাপত্তা পর্যালোচনা (Review) করতে।

২১.২ কেন এই বিষয়টি DevOps-এ গুরুত্বপূর্ণ

কনটেইনার "হালকা VM" নয় — এটি একই Linux কার্নেল ভাগ করে নেওয়া বিচ্ছিন্ন প্রসেসের দল। এই ভাগাভাগিই কনটেইনারের শক্তি ও দুর্বলতা দুটোই। একটি ভুল কনফিগারেশন — root হিসেবে চলা কনটেইনার, মাউন্ট করা Docker Socket, বা অরক্ষিত হোস্ট ফোল্ডার — আক্রমণকারীকে একটি ওয়েব অ্যাপের দুর্বলতা থেকে হোস্টের সম্পূর্ণ নিয়ন্ত্রণে পৌঁছে দিতে পারে। ২০২৪–২০২৫ সালের বাস্তব ঘটনাগুলোতে দেখা গেছে: ফাঁস হওয়া Image-এর Secret, স্ক্যান না করা Base Image-এর জানা দুর্বলতা, আর ভুল করে ইন্টারনেটে খোলা Docker API — এই তিনটি পথেই সবচেয়ে বেশি আক্রমণ হয়েছে। নিরাপত্তা শেষে যোগ করার জিনিস নয়; Dockerfile-এর প্রথম লাইন (Base Image নির্বাচন) থেকেই এটি শুরু হয়। এই অধ্যায়টি এই বইয়ের নিরাপত্তা-সংক্রান্ত সবচেয়ে গভীর অধ্যায়।

২১.৩ পূর্বশর্ত জ্ঞান

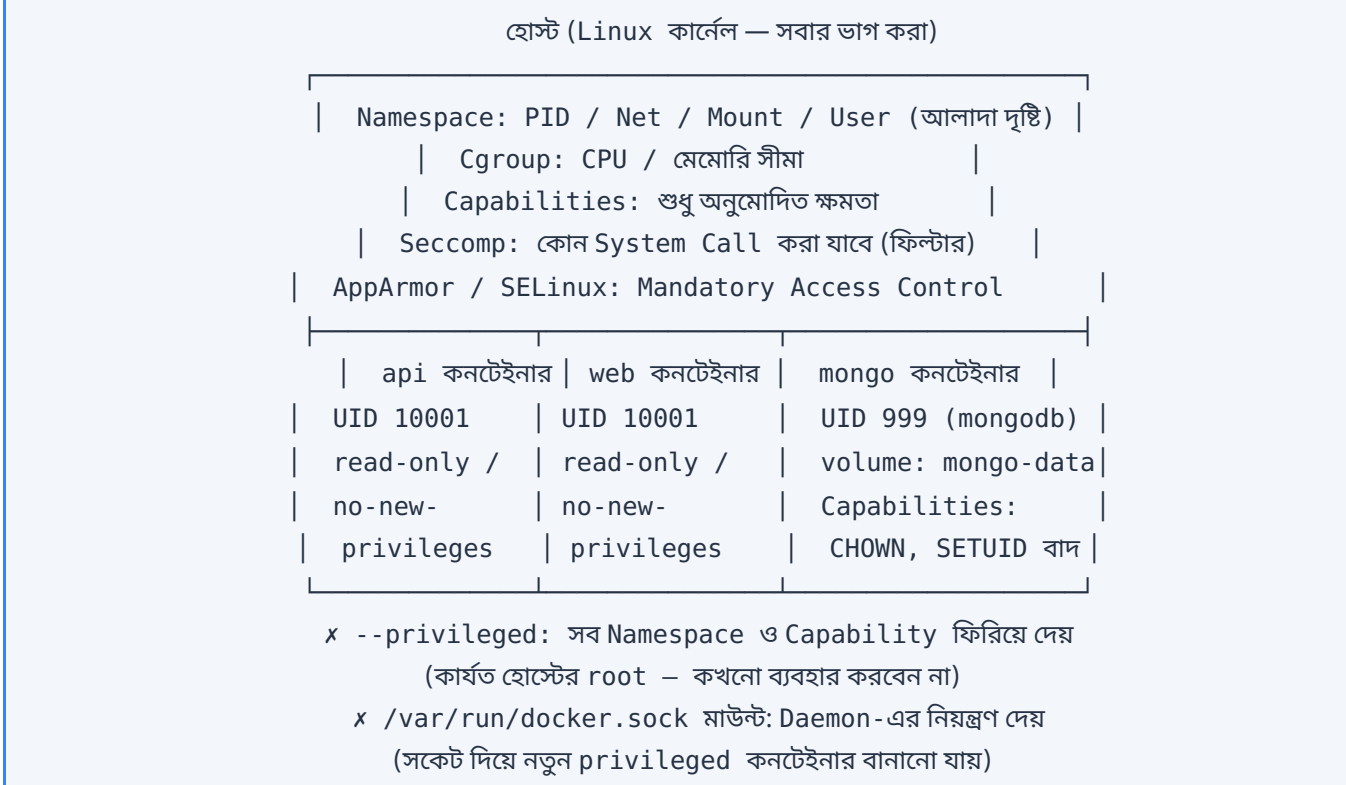
Linux ব্যবহারকারী ও অনুমতি (UID/GID, `chmod`, root বনাম সাধারণ ব্যবহারকারী), অধ্যায় ১৭–১৯-এর Compose, Dockerfile ও Secret জ্ঞান, Volume 04-এর ফায়ারওয়াল ও SSH নিরাপত্তার ধারণা, এবং HTTP ও নেটওয়ার্ক পোর্টের মৌলিক জ্ঞান। Linux কার্নেলের Namespace/Capability-এর পূর্ব জ্ঞান প্রয়োজন নেই — এখানেই ব্যাখ্যা করা হবে।

২১.৪ প্রথম নীতি থেকে ব্যাখ্যা

Linux কার্নেল কনটেইনারকে বিচ্ছিন্ন করে তিনটি ব্যবস্থায়। **Namespace** — প্রতিটি কনটেইনার নিজের আলাদা দৃষ্টিভঙ্গি পায়: নিজের প্রসেস তালিকা (PID Namespace), নিজের নেটওয়ার্ক (Network Namespace), নিজের ফাইলসিস্টেমের মূল (Mount Namespace)। একটি কনটেইনারের ভেতর থেকে হোস্টের প্রসেস দেখা যায় না — কারণ সে অন্য Namespace-এ আছে। **Cgroup** — রিসোর্সের সীমা: কত CPU, কত মেমোরি ব্যবহার করা যাবে (অধ্যায় ২২)। আর **Capabilities** — root-এর ক্ষমতাকে ছোট ছোট টুকরোয় ভাগ করা: যেমন `CAP_NET_BIND_SERVICE` (১০২৪-এর নিচের পোর্ট বাঁধা), `CAP_SYS_ADMIN` (প্রায় সব প্রশাসনিক কাজ — সবচেয়ে বিপজ্জনক)। ঐতিহ্যগত Unix-এ root মানে সব ক্ষমতা; Capabilities মানে "শুধু এইটুকু ক্ষমতা দাও, বাকিটা নয়" — এটাই Least Privilege-এর কার্নেল-স্তরের রূপ। তবে একটি সত্য সবসময় মনে রাখবেন: *Namespace* বিচ্ছিন্নতা *VM*-এর মতো শক্ত নয়। কার্নেলের একটি দুর্বলতা (Kernel Vulnerability) বা ভুল কনফিগারেশন (যেমন `--privileged`) বিচ্ছিন্নতা ভেঙে দিতে পারে — একে বলে **Container Escape**। তাই নিরাপত্তা কখনো একটি ব্যবস্থার ওপর নির্ভর করে না — স্তরে স্তরে (Defense in Depth) সাজাতে হয়।

> **CONCEPT:** Least Privilege — প্রতিটি অংশকে শুধু তার কাজের জন্য ন্যূনতম অধিকার দাও: অ্যাপ non-root ব্যবহারকারী হিসেবে চলবে, ফাইলসিস্টেম Read-Only থাকবে, নেটওয়ার্কে শুধু দরকারি পোর্ট খোলা থাকবে, আর Capabilities-এর মধ্যে শুধু যা লাগে তাই থাকবে। কোনো একটি স্তর ভেঙে গেলেও পরের স্তর আক্রমণকারীকে থামাবে — এটাই Defense in Depth।

২১.৫ আর্কিটেকচার / ধারণাগত চিত্র



ডিফল্ট Docker প্রোফাইল ইতিমধ্যে অনেক বিপজ্জনক System Call বন্ধ করে (Seccomp) ও কিছু Capability বাদ দেয়; এই অধ্যায়ে আমরা তার ওপর আরও সীমা যোগ করব — কমানোই নিরাপত্তা, বাড়ানো নয়।

২১.৬ বিস্তারিত প্রযুক্তিগত ব্যাখ্যা

Non-Root কনটেইনার — প্রথম ও সবচেয়ে গুরুত্বপূর্ণ সুরক্ষা

ডিফল্টভাবে কনটেইনারের ভেতরের প্রসেস root (UID 0) হিসেবে চলে — তবে সেটি হোস্টের root নয়, User Namespace ছাড়া কনটেইনারের root-এর UID হোস্টেও 0 হিসেবে দেখা যেতে পারে (যদি User Namespace Remapping চালু না থাকে)। অর্থাৎ কনটেইনার থেকে বেরিয়ে আসা (Escape) আক্রমণকারী হোস্টের root পেতে পারে। সমাধান: Dockerfile-এ একটি নির্দিষ্ট UID-এর ব্যবহারকারী তৈরি করে `USER appuser` (বা সংখ্যায় `USER 10001`) লেখা — সংখ্যা ব্যবহার করা ভালো, কারণ নামটি Image-এ না থাকলেও সংখ্যাটি কাজ করে। ফাইলের মালিকানা `COPY --chown=10001:10001` দিয়ে মেলাতে হয়, না হলে non-root ব্যবহারকারী নিজের ফাইল পড়তে পারবে না। MongoDB ও Nginx-এর অফিসিয়াল Image ইতিমধ্যে non-root ব্যবহারকারীতে চলে (MongoDB UID 999, Nginx-এর ক্ষেত্রে কনফিগারেশন অনুযায়ী) — নিজের Image-গুলোতেও একই অভ্যাস করুন।

Read-Only Root Filesystem ও অস্থায়ী লেখা

`read_only: true` দিলে কনটেইনারের Root Filesystem-এ লেখা যায় না — আক্রমণকারী ম্যালওয়্যার ইনস্টল করতে বা কনফিগারেশন বদলাতে পারে না। অ্যাপের যেসব জায়গায় লেখা দরকার (যেমন `/tmp`, Node.js-এর Cache) সেগুলোর জন্য `tmpfs` মাউন্ট দেওয়া হয় — মেমোরিতে থাকা অস্থায়ী ফাইলসিস্টেম, কনটেইনার থামলে মুছে যায়: `tmpfs: [/tmp]`। এই সুরক্ষাটি বিশেষভাবে কার্যকর রানটাইম আক্রমণের বিরুদ্ধে, তবে মনে রাখবেন: ডেটাবেসের মতো সার্ভিসে Root Filesystem Read-Only করা যায় না — তাদের ডেটা ডিরেক্টরি ভলিউমে থাকে, সেই ভলিউম লেখার-যোগ্য থাকতেই হয়।

Capabilities: বাদ দেওয়া ও যোগ করা

Docker ডিফল্টভাবে বিপজ্জনক Capabilities (যেমন `SYS_ADMIN`, `NET_ADMIN`) বাদ দেয়। আরও নিরাপদ হতে Compose-এ লিখুন: `cap_drop: [ALL]` — সব Capability বাদ দাও — তারপর শুধু যা দরকার তা যোগ করুন: `cap_add: [CHOWN, SETGID, SETUID]` (Node.js অ্যাপের জন্য সাধারণত এগুলোও লাগে না; `NET_BIND_SERVICE` লাগে কেবল ১০২৪-এর নিচের পোর্ট বাঁধতে — আমাদের অ্যাপ 3000 পোর্টে চলে, তাই লাগে না)। `cap_drop: [ALL]` দেওয়ার পর অ্যাপ চালিয়ে দেখুন — কোনো Capability-র অভাবে অ্যাপ ভাঙলে তবেই ন্যূনতমটি যোগ করুন। এই "সব বাদ দাও, দরকার হলে যোগ করো" পদ্ধতিই Least Privilege।

Privileged কনটেইনার ও --privileged-এর বিপদ

`privileged: true` কনটেইনারকে হোস্টের প্রায় সব ডিভাইস, সব Capability ও সব Namespace-এর অধিকার দেয় — কার্যত কনটেইনারটি হোস্টের root-এর সমান হয়ে যায়। কিছু বিশেষ কাজে (যেমন Docker-এর ভেতরে Docker চালানো — Docker-in-Docker) এটি ব্যবহার করা হয়, কিন্তু অ্যাপ্লিকেশন কনটেইনারে এর কোনো স্থান নেই। একইভাবে `host` নেটওয়ার্ক মোড (`network_mode: host`) কনটেইনারকে হোস্টের নেটওয়ার্ক সরাসরি দেয় — নেটওয়ার্ক বিচ্ছিন্নতা শেষ হয়ে যায়। দুটোই Compose ফাইলে দেখলে সঙ্গে সঙ্গে প্রশ্ন করুন: "এটি ছাড়া কি কাজটি করা যায় না?" — প্রায় সবসময় যায়।

Docker Socket — সবচেয়ে ভুল বোঝা ঝুঁকি

Docker Daemon-এর সঙ্গে কথা বলার পথ হলো Unix Socket `/var/run/docker.sock`। এই সকেটটি কনটেইনারে মাউন্ট করলে (`-v /var/run/docker.sock:/var/run/docker.sock`) কনটেইনারের ভেতর থেকে Docker API ব্যবহার করা যায় — অর্থাৎ নতুন কনটেইনার তৈরি করা যায়, যেকোনো Image চালানো যায়, হোস্টের যেকোনো ডিরেক্টরি মাউন্ট করা যায়। একটি আক্রমণকারী তখন একটি নতুন `--privileged` কনটেইনার বানিয়ে হোস্টের Root Filesystem মাউন্ট করে সম্পূর্ণ নিয়ন্ত্রণ নিতে পারে — কয়েকটি কমান্ডেই। তাই সকেটে পৌঁছানো মানেই কার্যত হোস্ট-স্তরের অধিকার। কিছু টুল (যেমন Portainer, Watchtower, CI Runner) সকেট চায় — সেগুলো চালালে সকেট শুধু-পড়ার (Read-Only) মাউন্টে দিন (তবু ঝুঁকি থাকে), আলাদা সীমিত-অধিকারের প্রক্সি (যেমন Docker Socket Proxy) ব্যবহার করুন, আর কখনোই ইন্টারনেটে-দৃশ্যমান কনটেইনারে সকেট দেবেন না। আরেকটি সম্পর্কিত ঝুঁকি: Docker API-কে TCP পোর্টে (2375) প্রমাণীকরণ ছাড়া খোলা — ইন্টারনেটে এমন খোলা Docker Daemon স্ক্যান করে আক্রমণকারীরা ক্রিপ্টো মাইনার চালায়; TLS প্রমাণীকরণ ছাড়া TCP-তে Daemon খুলবেন না।

Seccomp, AppArmor ও SELinux — কার্নেলের অতিরিক্ত পাহারা

Seccomp (Secure Computing Mode) একটি ফিল্টার: কনটেইনার কোন System Call করতে পারবে তা নির্ধারণ করে। Docker-এর ডিফল্ট Seccomp প্রোফাইল ৪০টির বেশি বিপজ্জনক System Call বন্ধ করে দেয়; নিজের প্রোফাইল লিখে আরও কঠোর করা যায়, তবে ভুল প্রোফাইলে অ্যাপ ভেঙে যেতে পারে — তাই প্রথমে ডিফল্টের ওপর ভরসা করুন, কঠোর প্রোফাইল Staging-এ পরীক্ষা করে তবেই ব্যবহার করুন। **AppArmor** (Ubuntu-তে ডিফল্ট) ও **SELinux** (RHEL-ভিত্তিক সিস্টেমে) হলো Mandatory Access Control — প্রসেস কোন ফাইল পড়তে বা লিখতে পারবে, কোন নেটওয়ার্ক ব্যবহার

করতে পারবে, তা কার্নেল-স্তরে বেঁধে দেয়, এমনকি root হলেও। Docker AppArmor প্রোফাইল (`docker-default`) স্বয়ংক্রিয়ভাবে প্রয়োগ করে; কাস্টম প্রোফাইল লিখতে পারলে আরও শক্ত হয়। Compose-এ `security_opt: [apparmor:docker-default, seccomp:default]` দিয়ে স্পষ্ট করা যায়।

Image Provenance, স্ক্যানিং ও Base Image নির্বাচন

Image Provenance মানে Image-টি কোথা থেকে এলো, কে বানিয়েছে, কী কী উপাদান আছে — তার প্রমাণ। নিয়ম: শুধু অফিসিয়াল Image (`mongo:7`, `node:20-slim`, `nginx:1.27`) বা যাচাই করা প্রকাশকের Image ব্যবহার করুন; অজানা ব্যবহারকারীর Image ব্যবহার করবেন না — তার ভেতরে ম্যালওয়্যার থাকতে পারে। **Digest Pinning** মানে ট্যাগের বদলে Image-এর SHA256 Digest দিয়ে নির্দিষ্ট করা (`mongo:7@sha256:abc123...`) — ট্যাগ বদলে যেতে পারে, Digest বদলায় না; গুরুত্বপূর্ণ প্রোডাকশনে ট্যাগ + Digest দুটোই ব্যবহার করুন। **স্ক্যানিং:** `docker scout cves technova-api:1.2.0` বা Trivy (`trivy image technova-api:1.2.0`) দিয়ে Image-এর জানা দুর্বলতা (CVE) দেখুন; CI-তে স্ক্যানিং বাধ্যতামূলক করুন — High/Critical দুর্বলতা থাকলে Build ব্যর্থ হবে। Base Image ছোট রাখুন (`slim` / `alpine`) — কম প্যাকেজ মানে কম দুর্বলতা; আর Base Image নিয়মিত আপডেট করুন (অধ্যায় ২৩)।

Supply-Chain ঝুঁকি ও নেটওয়ার্ক বিচ্ছিন্নতা

আপনার Image-এর নিরাপত্তা আপনার নির্ভরতার নিরাপত্তার ওপর নির্ভর করে: একটি আপস করা npm প্যাকেজ আপনার Build-এ ঢুকে Image-এ যেতে পারে। প্রতিকার: `package-lock.json` কমিট করা, `npm audit` চালানো, নির্ভরতা নিয়মিত আপডেট করা, আর Build-এ শুধু বিশ্বস্ত Registry ব্যবহার করা। নেটওয়ার্কে: অধ্যায় ১৭-এর `frontend` / `backend` বিভাজন বজায় রাখুন; শুধু Nginx-এর পোর্ট প্রকাশ করুন; আউটবাউন্ড নেটওয়ার্কও সীমিত করুন (প্রয়োজনে কনটেইনারকে ইন্টারনেটে যেতে বাধা দেওয়া যায় — তবে npm/DNS-এর জন্য সীমিত অনুমতি দরকার)। হোস্ট ফাইলসিস্টেম মাউন্ট (`/:/host` বা `/etc:/etc`) কখনো করবেন না — কনটেইনার থেকে হোস্টের ফাইল বদলে যাওয়া যায়, আর সেটি Escape-এর সরাসরি পথ।

২১.৭ Docker কমান্ড ও লাইন-বাই-লাইন ব্যাখ্যা

```
$ docker scout cves technova-api:1.2.0
$ trivy image technova-api:1.2.0
$ docker image inspect technova-api:1.2.0 --format='{{.Config.User}}'
$ docker compose exec api id
$ docker compose exec api sh -c 'touch /etc/testfile'  # ব্যর্থ হওয়া উচিত (read-only)
$ docker compose exec api capsh --print | head -20
```

ব্যাখ্যা: `docker scout cves` — Docker-এর নিজস্ব স্ক্যানার; Image-এর CVE তালিকা ও তীব্রতা দেখায়। `trivy image` — জনপ্রিয় ওপেন-সোর্স স্ক্যানার; CI-তে ব্যবহারের জন্য উপযোগী। `docker image inspect --format='{{.Config.User}}'` — Image-টি কোন ব্যবহারকারী হিসেবে চলবে তা দেখায় (খালি মানে root — সতর্ক হোন)। `id` — চলমান কনটেইনারে বর্তমান ব্যবহারকারীর UID/GID দেখায়; `uid=10001(appuser)` প্রত্যাশিত। `touch /etc/testfile` — Read-Only Filesystem-এ লেখার চেষ্টা; ব্যর্থ হলে সুরক্ষা কাজ করছে। `capsh --print` — কনটেইনারের কার্যকর Capabilities দেখায়; `cap_drop: [ALL]`-এর পর তালিকা প্রায় খালি থাকা উচিত।

নমুনা আউটপুট (চিত্রণমূলক):

```
$ docker compose exec api id
uid=10001(appuser) gid=10001(appuser) groups=10001(appuser)
$ docker scout cves technova-api:1.2.0
```

```
  0 Critical  •  2 High  •  5 Medium  •  12 Low
  (সংখ্যাগুলো চিত্রণমূলক — আপনার Image-এর ফল আলাদা হবে)
```

২১.৮ সম্পূর্ণ কনফিগারেশন উদাহরণ

```
# compose.yaml — নিরাপত্তা-কঠোর সংস্করণ (প্রাসঙ্গিক অংশ)
services:
  api:
    image: technova-api:1.2.0
    user: "10001:10001"
    read_only: true
    tmpfs:
      - /tmp
    cap_drop:
      - ALL
    security_opt:
      - no-new-privileges:true
      - seccomp:default
      - apparmor:docker-default
    networks:
      - backend
    # privileged: false — ডিফল্ট; কখনো true করবেন না
    # volumes-এ হোস্টের সিস্টেম পথ মাউন্ট করবেন না

  mongo:
    image: mongo:7
    user: "999:999"
    cap_drop:
      - ALL
    cap_add:
      - CHOWN
      - SETGID
      - SETUID
    security_opt:
      - no-new-privileges:true
    networks:
      - backend
    volumes:
      - mongo-data:/data/db

  nginx:
    image: nginx:1.27
    cap_drop:
      - ALL
    cap_add:
      - NET_BIND_SERVICE   # 80/443 পোর্ট বাঁধতে দরকার
    read_only: true
    tmpfs:
      - /var/cache/nginx
      - /var/run
    networks:
      - frontend
```

ব্যাখ্যা: `user: "10001:10001"` — সংখ্যায় ব্যবহারকারী নির্ধারণ (Dockerfile-এর `USER`-এর সঙ্গে মিলিয়ে)। `no-new-privileges:true` — প্রসেস চলাকালে নতুন অধিকার পাওয়া বন্ধ করে (যেমন Setuid বাইনারি দিয়ে অধিকার বাড়ানো

যায় না)। `read_only: true` + `tmpfs` — লেখা শুধু অস্থায়ী মেমোরিতে। MongoDB-র `cap_add` তিনটি Capability তার ডেটা ডিরেক্টরির মালিকানা ঠিক করতে দরকার — ন্যূনতম প্রয়োজন অনুযায়ী যোগ করা হয়েছে। Nginx-এর `NET_BIND_SERVICE` শুধু 80/443 বাঁধার জন্য — বাকি সব Capability বাদ।

২১.৯ বাস্তব প্রোডাকশন পরিস্থিতি

REAL WORLD — TechNova: TechNova-র একটি পুরনো Staging সার্ভারে ডেভেলপমেন্টের সুবিধার জন্য Docker Socket একটি মনিটরিং কনটেইনারে মাউন্ট করা ছিল। সেই কনটেইনারের একটি দুর্বলতা দিয়ে আক্রমণকারী সকেটে পৌঁছে একটি নতুন `--privileged` কনটেইনার বানিয়ে হোস্টের `/etc/shadow` পড়ে ফেলল। ঘটনার পর দল সকেট মাউন্ট সম্পূর্ণ বন্ধ করে, মনিটরিংয়ের জন্য Docker Socket Proxy (শুধু-পড়ার, সীমিত API) ব্যবহার শুরু করল, আর সব কনটেইনারে `cap_drop: [ALL]` ও non-root ব্যবহারকারী বাধ্যতামূলক করল। শিক্ষা: একটি সুবিধাজনক শর্টকাটই পুরো হোস্টের নিরাপত্তা ভেঙে দিতে পারে।

২১.১০ ধাপে ধাপে ব্যবহারিক বাস্তবায়ন

1. প্রতিটি Dockerfile-এ non-root ব্যবহারকারী যোগ করুন এবং `USER 10001` লিখুন; `docker compose exec api id` দিয়ে যাচাই করুন।
2. Compose ফাইলে `read_only: true` + `tmpfs: [/tmp]` যোগ করুন; অ্যাপ চালিয়ে দেখুন কোথায় লেখা দরকার (ভুল বার্তা থেকে বোঝা যাবে)।
3. `cap_drop: [ALL]` যোগ করে অ্যাপ পরীক্ষা করুন; ভাঙলে ন্যূনতম Capability যোগ করুন।
4. `trivy image technova-api:1.2.0` চালিয়ে CVE দেখুন; একটি High দুর্বলতা থাকলে Base Image আপডেট করে আবার স্ক্যান করুন।
5. `docker compose config | grep -i -E "privileged|docker.sock|network_mode: host"` দিয়ে বিপজ্জনক সেটিং খুঁজুন — কিছু পাওয়া গেলে সরিয়ে ফেলুন।

২১.১১ সাধারণ ভুল ও ভুল ধারণা

- **ভুল:** কনটেইনারকে "হালকা VM" ভাবা — VM-এর মতো শক্ত বিচ্ছিন্নতা এতে নেই; কার্নেল ভাগ করা হয়।
- **ভুল:** root হিসেবে চালানো — ডিফল্ট, কিন্তু সবচেয়ে বড় ঝুঁকি; সবসময় non-root ব্যবহার করুন।
- **ভুল:** Docker Socket মাউন্ট করা — সুবিধার জন্য হোস্টের নিয়ন্ত্রণ তুলে দেওয়া।
- **ভুল:** `privileged: true` দিয়ে সমস্যা "ঠিক" করা — অনুমতির সমস্যার আসল কারণ খুঁজুন, privileged দেবেন না।
- **ভুল:** স্ক্যান না করা Image প্রোডাকশনে চালানো — জানা দুর্বলতা নিয়ে চলা।
- **ভুল ধারণা:** "কনটেইনারের ভেতরের root হোস্টের root নয়, তাই নিরাপদ" — User Namespace ছাড়া কনটেইনারের root হোস্টেও root হতে পারে; Escape হলে বিপদ।
- **ভুল ধারণা:** "Read-Only Filesystem দিলেই আক্রমণ অসম্ভব" — না; এটি একটি স্তর মাত্র, মেমোরি-ভিত্তিক আক্রমণ তবু সম্ভব।

২১.১২ নিরাপত্তা বিবেচনা

> **WARNING:** এই অধ্যায়ের সব সুরক্ষা একসঙ্গে প্রয়োগ করুন — কোনো একটিতে ভরসা করবেন না। বিশেষ সতর্কতা: হোস্টের `/` বা `/etc` কখনো মাউন্ট করবেন না; Docker API TCP-তে প্রমাণীকরণ ছাড়া খুলবেন না; Image-এর Digest যাচাই করুন; আর নিয়মিত `docker scout` বা Trivy স্ক্যান CI-তে চালান। একটি Image-এ Critical CVE পাওয়া গেলে সেটি প্রোডাকশনে যাওয়া বন্ধ করুন — ব্যতিক্রমের জন্য লিখিত অনুমোদন ও সময়সীমা লাগবে।

আরও দুটি বিষয়: **User Namespace Remapping** (Daemon-স্তরের সেটিং) কনটেইনারের root-কে হোস্টে সাধারণ ব্যবহারকারী হিসেবে ম্যাপ করে — Escape-এর ক্ষতি কমায়; তবে এটি চালু করলে ভলিউমের মালিকানা জটিল হয়, তাই Staging-এ পরীক্ষা করে তবেই চালু করুন। **Image স্বাক্ষর** (Docker Content Trust / Sigstore Cosign) Image-এর উৎস যাচাই করে — Registry থেকে Image টানার সময় স্বাক্ষর যাচাই করলে মাঝপথে বদলে দেওয়া Image ধরা পড়ে।

২১.১৩ কর্মক্ষমতা ও রিসোর্স বিবেচনা

নিরাপত্তা ব্যবস্থাগুলোর রিসোর্স খরচ নগণ্য: Seccomp ও AppArmor-এর ওভারহেড প্রায় শূন্য; Read-Only Filesystem-এর কোনো খরচ নেই; ছোট Base Image উল্টো Deploy দ্রুত করে। একমাত্র সতর্কতা: `tmpfs` মেমোরি ব্যবহার করে — `/tmp`-এ বড় ফাইল লিখলে মেমোরি সীমায় (অধ্যায় ২২) ধাক্কা খেতে পারে; বড় অস্থায়ী ফাইলের জন্য ভলিউম ব্যবহার করুন। স্ক্যানিংয়ের খরচ CI-তে হয়, রানটাইমে নয় — তাই প্রোডাকশনের গতিতে এর কোনো প্রভাব নেই।

২১.১৪ সমস্যা সমাধান পদ্ধতি

> **TROUBLESHOOTING:** Non-root ব্যবহারকারীতে যাওয়ার পর অ্যাপ "Permission denied" দিলে: (১) `docker compose exec api ls -la /app` দিয়ে ফাইলের মালিকানা দেখুন; (২) Dockerfile-এ `COPY --chown=10001:10001` ব্যবহার করুন; (৩) ভলিউমের মালিকানা যাচাই করুন — Named Volume প্রথমবার তৈরি হলে Image-এর ডিরেক্টরির মালিকানা পায়, কিন্তু হোস্টের Bind Mount-এর মালিকানা হোস্টের ব্যবহারকারীর হয়। Read-Only Filesystem-এ "Read-only file system" ভুল এলে কোন পথে লেখা হচ্ছে তা লগ থেকে খুঁজে সেই পথে `tmpfs` বা ভলিউম দিন।

সমস্যা	সম্ভাব্য কারণ	সমাধান
`Permission denied` ফাইল পড়তে/লিখতে	ফাইলের মালিক root, অ্যাপ non-root	`COPY --chown` ও ভলিউমের মালিকানা ঠিক করুন
`Read-only file system`	অ্যাপ লেখার-যোগ্য পথে লিখছে	সেই পথে `tmpfs` বা ভলিউম দিন
`cap_drop: [ALL]`-এর পর অ্যাপ ভাঙে	দরকারি Capability বাদ পড়েছে	ভুল বার্তা দেখে ন্যূনতম Capability যোগ করুন
স্ক্যানে Critical CVE	পুরনো Base Image বা নির্ভরতা	Base Image ও নির্ভরতা আপডেট করুন, আবার স্ক্যান করুন

২১.১৫ সেরা অনুশীলন ও অ্যান্টি-প্যাটার্ন

সেরা অনুশীলন: Non-root ব্যবহারকারী; `read_only: true` + `tmpfs`; `cap_drop: [ALL]` + ন্যূনতম `cap_add`; `no-new-privileges:true`; অফিসিয়াল Image + Digest Pinning; CI-তে স্ক্যানিং; নেটওয়ার্ক বিভাজন; সকেট মাউন্ট নয়; Secret ফাইল থেকে। **অ্যান্টি-প্যাটার্ন:** Root হিসেবে চালানো; `privileged: true`; Docker Socket মাউন্ট; হোস্ট ফাইলসিস্টেম মাউন্ট; অজানা Image; `latest` ট্যাগ; স্ক্যান ছাড়া Deploy; TCP-তে প্রমাণীকরণ ছাড়া Docker API।

২১.১৬ অধ্যায়ের সারাংশ

কনটেইনার নিরাপত্তা মানে কার্নেলের বিচ্ছিন্নতা (Namespace, Cgroup, Capabilities) বোঝা ও তার ওপর স্তরে স্তরে সুরক্ষা যোগ করা: Non-Root ব্যবহারকারী, Read-Only Filesystem, Capability Dropping, `no-new-privileges`, Seccomp ও AppArmor। Privileged কনটেইনার ও Docker Socket মাউন্ট কার্যত হোস্টের নিয়ন্ত্রণ তুলে দেয় — কখনো ব্যবহার করবেন না। Image-এর নিরাপত্তা মানে অফিসিয়াল Base Image, Digest Pinning, নিয়মিত স্ক্যানিং ও Supply-Chain সতর্কতা। কোনো একক ব্যবস্থাই যথেষ্ট নয় — Defense in Depth-ই একমাত্র নির্ভরযোগ্য পথ।

২১.১৭ মূল প্রযুক্তিগত পরিভাষা

Namespace: কার্নেলের বিচ্ছিন্নতা — প্রতিটি কনটেইনারের আলাদা PID, নেটওয়ার্ক ও ফাইলসিস্টেম দৃষ্টি।

Cgroup: কনটেইনারের CPU/মেমোরি সীমা নির্ধারণের কার্নেল ব্যবস্থা।

Capability: Root-এর ক্ষমতার ছোট টুকরো (যেমন `NET_BIND_SERVICE`) — Least Privilege-এর ভিত্তি।

Least Privilege: শুধু ন্যূনতম প্রয়োজনীয় অধিকার দেওয়ার নীতি।

Defense in Depth: স্তরে স্তরে সুরক্ষা — একটি ভাঙলেও পরেরটি থামাবে।

Container Escape: কনটেইনারের বিচ্ছিন্নতা ভেঙে হোস্টে পৌঁছানো।

Privileged Container: হোস্টের প্রায় সব অধিকার-সহ কনটেইনার — কার্যত হোস্টের root।

Seccomp: কোন System Call করা যাবে তা ফিল্টার করার কার্নেল ব্যবস্থা।

AppArmor / SELinux: Mandatory Access Control — root-কেও ফাইল ও নেটওয়ার্কের সীমা মানতে বাধ্য করে।

CVE: জানা নিরাপত্তা দুর্বলতার সনাক্তকারী নম্বর (Common Vulnerabilities and Exposures)।

Digest Pinning: Image-এর SHA256 Digest দিয়ে নির্দিষ্ট Image বেঁধে দেওয়া।

Image Provenance: Image-এর উৎস, নির্মাতা ও উপাদানের প্রমাণ।

২১.১৮ জ্ঞান যাচাই — প্রশ্ন

1. Namespace, Cgroup ও Capabilities — তিনটির কাজ আলাদা করে ব্যাখ্যা করুন।
2. Docker Socket মাউন্ট করা কেন কার্যত হোস্টের নিয়ন্ত্রণ দেওয়ার সমান? ধাপে ধাপে ব্যাখ্যা করুন।
3. `privileged: true` কী কী অধিকার দেয়? অ্যাপ কনটেইনারে এটি কেন নিষিদ্ধ?
4. Read-Only Root Filesystem-এর সুবিধা কী? লেখার দরকার হলে কী করবেন?
5. Digest Pinning কেন ট্যাগের চেয়ে নির্ভরযোগ্য?
6. Seccomp ও AppArmor-এর পার্থক্য কী?
7. Container Escape কী? দুটি প্রতিরোধ ব্যবস্থা লিখুন।

২১.১৯ ব্যবহারিক অনুশীলন

1. **কঠোর করা:** আপনার স্ট্যাকে `cap_drop: [ALL]`, `read_only: true` ও non-root ব্যবহারকারী যোগ করে অ্যাপ পরীক্ষা করুন।
2. **স্ক্যান:** Trivy বা Docker Scout দিয়ে Image স্ক্যান করে একটি দুর্বলতা খুঁজে বের করে Base Image আপডেট করুন।
3. **সকেট মহড়া:** একটি পরীক্ষামূলক কনটেইনারে সকেট মাউন্ট করে দেখুন সেখান থেকে কী কী করা যায় — তারপর বুঝুন কেন এটি বিপজ্জনক (শুধু ল্যাবে)।

অধ্যায় 22 • PART V — Container Security, Resources and Production Operations

Chapter 22 — Resource Limits, Logging and Container Monitoring

২২.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: (১) CPU ও মেমোরি Limit বনাম Reservation-এর পার্থক্য ব্যাখ্যা করতে; (২) OOM (Out of Memory) আচরণ বোঝাতে ও OOM Killer-এর ভূমিকা বলতে; (৩) `docker stats` দিয়ে রিসোর্স ব্যবহার পড়তে; (৪) Logging Driver, লগ রোটেশন ও কেন্দ্রীয় লগের নকশা করতে; (৫) `docker events` ও Health Status দিয়ে পর্যবেক্ষণ করতে; (৬) Running বনাম Healthy পার্থক্য কাজে লাগিয়ে সতর্কতা (Alert) সাজাতে।

২২.২ কেন এই বিষয়টি DevOps-এ গুরুত্বপূর্ণ

সীমা ছাড়া একটি কনটেইনার পুরো হোস্টের মেমোরি খেয়ে ফেলতে পারে — একটি মেমোরি লিক-যুক্ত API তখন MongoDB-কেও মেরে ফেলে, আর পুরো সাইট ডাউন হয়। লগ ছাড়া আপনি জানতেই পারবেন না কেন সাইট ধীর বা ডাউন — অন্ধকারে হাতড়াতে হবে। Monitoring ছাড়া সমস্যা ধরা পড়ে ব্যবহারকারীর অভিযোগে, আপনার সতর্কতায় নয়। এই তিনটি — সীমা, লগ, পর্যবেক্ষণ — মিলে একটি সিস্টেমকে "চলছে" থেকে "পরিচালনাযোগ্য" করে তোলে।

২২.৩ পূর্বশর্ত জ্ঞান

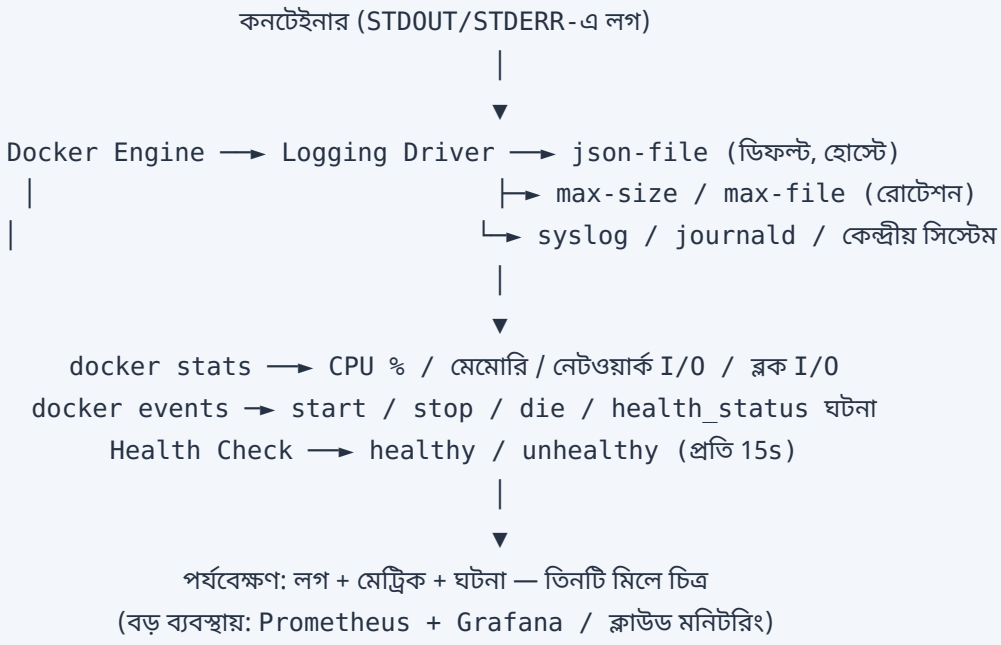
Linux-এ CPU ও মেমোরির ধারণা, অধ্যায় ২০-এর Health Check, অধ্যায় ১৮-এর STDOUT লগ নীতি, এবং `docker compose logs`-এর মৌলিক ব্যবহার। Cgroup-এর ধারণা অধ্যায় ২১-এ শেখা হয়েছে — এখানে তার প্রয়োগ দেখানো হবে।

২২.৪ প্রথম নীতি থেকে ব্যাখ্যা

প্রথম নীতি: রিসোর্স সীমিত, চাহিদা অসীম। একটি হোস্টে CPU ও মেমোরি নির্দিষ্ট; কনটেইনারগুলো তা ভাগ করে নেয়। সীমা না দিলে ভাগাভাগি হয় "যে আগে পায় সে নেয়" নিয়মে — একটি লোভী কনটেইনার বাকিদের অনাহারে রাখে। **Limit** মানে "এর বেশি নিতে পারবে না" (কঠোর সীমা); **Reservation** মানে "এতটুকু অন্তত পাবে" (নরম প্রতিশ্রুতি)। মেমোরি Limit ছাড়ালে কার্নেলের OOM Killer প্রসেস মেরে ফেলে — কনটেইনার থেমে যায়, Restart নীতি আবার চালু করে। CPU Limit ছাড়ালে প্রসেসকে থামিয়ে দেওয়া হয় না — শুধু ধীর করে দেওয়া হয় (Throttling)। দ্বিতীয় নীতি: লগ হলো সিস্টেমের স্মৃতি। কনটেইনার অস্থায়ী, কিন্তু তার লগ স্থায়ী হতে হয় — না হলে ঘটনার পর তদন্ত করা যায় না। তাই লগ STDOUT/STDERR-এ লেখা হয়, Engine সংগ্রহ করে, আর Driver তা ফাইলে, Syslog-এ বা কেন্দ্রীয় সিস্টেমে পাঠায়। তৃতীয় নীতি: পর্যবেক্ষণ মানে শুধু সংখ্যা দেখা নয় — অবস্থার পরিবর্তন ধরা। CPU ৮০% হওয়া তথ্য; Health Check `unhealthy` হওয়া একটি ঘটনা (Event) — ঘটনাতেই সতর্কতা দরকার।

> **CONCEPT:** Running বনাম Healthy — আবার মনে করুন: `Up` মানে প্রসেস বেঁচে আছে; `(healthy)` মানে Health Check পাস করছে। Monitoring-এ শুধু "চলছে কি না" দেখলে অর্ধেক সত্য দেখা হয় — একটি API ডেটাবেস সংযোগ হারিয়ে ঘণ্টার পর ঘণ্টা `Up` থাকতে পারে, অথচ কোনো অনুরোধের উত্তর দিচ্ছে না। সতর্কতা (Alert) সবসময় Health Status ও ব্যবসায়িক সূচক (যেমন 5xx ভুলের হার) দিয়ে সাজান, শুধু প্রসেসের অবস্থা দিয়ে নয়।

২২.৫ আর্কিটেকচার / ধারণাগত চিত্র



ছোট Deploy-এ `docker stats` + `docker compose logs` + Health Check যথেষ্ট; বড় ব্যবস্থায় মেট্রিক সংগ্রহকারী (Prometheus) ও ড্যাশবোর্ড (Grafana) যোগ করা হয় — অধ্যায় ২৪-এ প্রসঙ্গ।

২২.৬ বিস্তারিত প্রযুক্তিগত ব্যাখ্যা

CPU ও মেমোরি: Limit, Reservation ও OOM

সেটিং	অর্থ	উদাহরণ
`mem_limit` / `deploy.resources.limits.memory`	সর্বোচ্চ মেমোরি — ছাড়ালে OOM Kill	`512M` (api), `1G` (mongo)
`cpus` / `limits.cpus`	সর্বোচ্চ CPU — যেমন 0.5 মানে অর্ধেক কোর	`0.5` (web), `1.0` (api)
`mem_reservation`	নরম সীমা — মেমোরি চাপে এলে এই পর্যন্ত সুরক্ষিত	`256M`
`cpu_shares`	প্রতিযোগিতায় আপেক্ষিক ওজন (ডিফল্ট 1024)	সাধারণত বদলানোর দরকার নেই

একটি গুরুত্বপূর্ণ সতর্কতা: Compose-এ `deploy.resources` মূলত Swarm মোডের জন্য; একক হোস্টে `docker compose` v2 `deploy.resources.limits`-কে সমর্থন করে, তবে পুরনো সংস্করণে তা উপেক্ষিত হতো। নির্ভরযোগ্য উপায় হলো সার্ভিস-স্তরের `mem_limit` ও `cpus` ব্যবহার করা — এগুলো সরাসরি Cgroup সীমা হিসেবে কাজ করে। OOM হলে কী হয়: কার্নেল মেমোরি-খেকো প্রসেসকে মেরে ফেলে (OOM Killer), কনটেইনার Exit Code 137 নিয়ে থামে, `docker compose ps`-এ `Exited (137)` দেখায়, আর `restart: unless-stopped` থাকলে আবার চালু হয় — কিন্তু মেমোরি

লিক থাকলে চক্রটি বারবার চলবে। তাই OOM-এর প্রতিকার শুধু Restart নয় — লিক খুঁজে ঠিক করা, বা Limit বাড়ানো। Node.js-এ `--max-old-space-size` দিয়ে V8 হিপ সীমিত করা যায়, যাতে Node নিজে Limit-এর আগেই সতর্ক হয়।

Logging Driver ও রোটেশন

ডিফল্ট Driver `json-file` — প্রতিটি কনটেইনারের লগ হোস্টে JSON ফাইলে জমা হয়। সমস্যা: সীমা না দিলে ফাইল বাড়তেই থাকে, ডিস্ক ভরে যায়। সমাধান Compose-এ: `logging: {driver: json-file, options: {max-size: "10m", max-file: "3"}}` — প্রতিটি ফাইল সর্বোচ্চ ১০ মেগাবাইট, সর্বোচ্চ ৩টি ফাইল — পুরনোটি মুছে যায়। অন্য Driver: `syslog` (হোস্টের Syslog-এ), `journald` (systemd Journal-এ), `none` (লগ বন্ধ — কখনো প্রোডাকশনে নয়)। লগের ফরম্যাট: JSON Structured লগ (যেমন `{"level":"info","msg":"request","status":200}`) খোঁজা ও ফিল্টার করা সহজ — Pino-এর মতো লাইব্রেরি ব্যবহার করুন। লগে কখনো Secret, পাসওয়ার্ড বা সম্পূর্ণ অনুরোধের Body ছাপাবেন না।

docker stats, events ও পর্যবেক্ষণের অভ্যাস

`docker stats` লাইভ CPU/মেমোরি/নেটওয়ার্ক দেখায়; `docker stats --no-stream` একবারের স্ন্যাপশট দেয়। `docker events` কনটেইনারের জীবনচক্রের ঘটনা দেখায় (start, die, restart, health_status) — একটি সার্ভিস বারবার মরছে কি না তা এখানেই ধরা পড়ে। নিয়মিত পরীক্ষার তালিকা: প্রতিদিন `docker compose ps` (সব `healthy` কি না), `docker stats --no-stream` (কোনো সার্ভিস সীমার কাছাকাছি কি না), `df -h` (লগে ডিস্ক ভরেছে কি না), আর সাপ্তাহিক `docker system df` (অব্যবহৃত Image/ভলিউম কত জায়গা নিচ্ছে)।

২২.৭ Docker কমান্ড ও লাইন-বাই-লাইন ব্যাখ্যা

```
$ docker stats --no-stream
$ docker stats technova-api-1
$ docker compose logs --tail 100 -f api
$ docker compose logs --since 1h --timestamps mongo
$ docker events --filter container=technova-api-1
$ docker system df
$ docker compose exec api sh -c 'cat /sys/fs/cgroup/memory.max'
```

ব্যাখ্যা: `docker stats --no-stream` — সব কনটেইনারের বর্তমান রিসোর্স ব্যবহার একবার দেখায়। `docker stats <name>` — একটি কনটেইনারের লাইভ ব্যবহার। `logs --tail 100 -f api` — শেষ ১০০ লাইন, তারপর নতুন লগ অনুসরণ। `logs --since 1h --timestamps` — গত এক ঘণ্টার লগ, সময়-সহ। `docker events --filter container=...` — একটি কনটেইনারের ঘটনাগুলো দেখায়। `docker system df` — Image, Container, ভলিউম কত ডিস্ক নিচ্ছে। শেষ কমান্ডটি Cgroup-এর মেমোরি সীমা সরাসরি দেখায় — সীমা কার্যকর হয়েছে কি না যাচাইয়ের উপায়।

নমুনা আউটপুট (চিত্রণমূলক):

```
$ docker stats --no-stream
CONTAINER          CPU %     MEM USAGE / LIMIT     MEM %     NET I/O
technova-api-1     2.15%     148MiB / 512MiB       28.9%     1.2MB / 3.4MB
technova-mongo-1   1.02%     320MiB / 1GiB         31.2%     800kB / 1.1MB
technova-web-1     0.80%     120MiB / 512MiB       23.4%     2.1MB / 4.0MB
```

২২.৮ সম্পূর্ণ কনফিগারেশন উদাহরণ

```
# compose.yaml — রিসোর্স সীমা ও লগ রোটেশন
services:
  api:
    image: technova-api:1.2.0
    mem_limit: 512m
    cpus: 1.0
    mem_reservation: 256m
    environment:
      NODE_OPTIONS: "--max-old-space-size=384"
    logging:
      driver: json-file
      options:
        max-size: "10m"
        max-file: "3"
        tag: "technova-api"

  mongo:
    image: mongo:7
    mem_limit: 1g
    cpus: 1.0
    command: ["mongod", "--wiredTigerCacheSizeGB", "0.5"]
    logging:
      driver: json-file
      options:
        max-size: "10m"
        max-file: "3"

  web:
    image: technova-web:1.2.0
    mem_limit: 512m
    cpus: 0.5
    logging:
      driver: json-file
      options:
        max-size: "10m"
        max-file: "3"
```

ব্যাখ্যা: API-র মেমোরি Limit ৫১২ মেগাবাইট, আর Node.js-এর হিপ ৩৮৪ মেগাবাইটে সীমিত — হিপ Limit-এর আগেই থামে, OOM Killer-এর আগেই সতর্ক হয়। MongoDB-র WiredTiger Cache ০.৫ গিগাবাইটে সীমিত — না হলে ডেটাবেস উপলব্ধ RAM-এর অর্ধেক দখল করে নিত। প্রতিটি সার্ভিসের লগ ১০ মেগাবাইট × ৩ ফাইলে সীমিত — ডিস্ক ভরার ঝুঁকি নেই।

২২.৯ বাস্তব প্রোডাকশন পরিস্থিতি

REAL WORLD — TechNova: TechNova-র সাইট একদিন ধীর হয়ে গেল — পাতা খুলতে ২০ সেকেন্ড। `docker stats`-এ দেখা গেল API-র মেমোরি ৯৫% — একটি লিক। তারপর `docker compose logs api --since 1h`-এ দেখা গেল একই ভুল বারবার: একটি এন্ডপয়েন্টে ডেটাবেস সংযোগ বন্ধ না করে খোলা হচ্ছিল। লগ ও মেট্রিক একসঙ্গে না দেখলে কারণটি ধরা পড়ত না। দল এরপর থেকে প্রতিটি সার্ভিসে মেমোরি Limit ও লগ রোটেশন বাধ্যতামূলক করল, আর সাপ্তাহিক `docker stats` পর্যালোচনা রুটিনে যোগ করল।

২২.১০ ধাপে ধাপে ব্যবহারিক বাস্তবায়ন

1. প্রতিটি সার্ভিসে `mem_limit`, `cpus` ও `logging` রোটেশন যোগ করুন।
2. `docker stats --no-stream` দিয়ে বর্তমান ব্যবহার দেখে Limit-গুলো বাস্তবসম্মত কি না যাচাই করুন (Limit ব্যবহারের ২ গুণের কম হওয়া উচিত নয়)।
3. একটি সার্ভিসে ইচ্ছাকৃতভাবে ছোট Limit দিয়ে OOM পরীক্ষা করুন (ল্যাবে): Exit Code 137 ও Restart আচরণ দেখুন।
4. `docker compose logs --since 1h --timestamps api` দিয়ে লগের সময়-ক্রম পড়ার অভ্যাস করুন।

২২.১১ সাধারণ ভুল ও ভুল ধারণা

- **ভুল:** কোনো Limit না দেওয়া — একটি সার্ভিস পুরো হোস্ট দখল করে নিতে পারে।
- **ভুল:** Limit খুব কম দেওয়া — স্বাভাবিক ব্যবহারেই OOM Kill হতে থাকে।
- **ভুল:** লগ রোটেশন না দেওয়া — ডিস্ক ভরে সার্ভার থেমে যায়।
- **ভুল:** ফাইলে লগ লেখা — কনটেইনারের ভেতরে আটকে থাকে, মুছলে হারায়।
- **ভুল ধারণা:** "CPU Limit ছাড়ালে কনটেইনার থেমে যায়" — না; CPU শুধু Throttle হয়, মেমোরিতেই Kill হয়।
- **ভুল ধারণা:** "`Up` মানেই সব ঠিক আছে" — Health Status ও লগ না দেখলে অর্ধেক সত্য।

২২.১২ নিরাপত্তা বিবেচনা

WARNING: লগে Secret ছাপা একটি সাধারণ ফাঁসের পথ — অনুরোধের Header (Authorization টোকেন), পরিবেশ চলক বা ভুলের Stack Trace-এ Secret থাকতে পারে। লগ লেখার আগে সংবেদনশীল ক্ষেত্র মুছে ফেলুন (Redaction)। লগ ফাইলগুলোর অনুমতিও সীমিত রাখুন — হোস্টে `/var/lib/docker/containers/`-এর লগ শুধু root পড়তে পারে, সেটিই সঠিক। কেন্দ্রীয় লগ সিস্টেমে পাঠালে সেখানেও প্রবেশাধিকার সীমিত করুন।

২২.১৩ কর্মক্ষমতা ও রিসোর্স বিবেচনা

Limit নির্ধারণের নিয়ম: প্রথমে সীমা ছাড়া চালিয়ে স্বাভাবিক ব্যবহার মাপুন (`docker stats`), তারপর Limit দিন ব্যবহারের প্রায় দ্বিগুণ। MongoDB-র Cache সীমা (`wiredTigerCacheSizeGB`) ও Node.js হিপ সীমা (`--max-old-space-size`) অ্যাপ-স্তরের সীমা — এগুলো Cgroup Limit-এর সঙ্গে মিলিয়ে দিন, না হলে অ্যাপ নিজে জানে না কখন থামতে হবে। লগের পরিমাণও রিসোর্স: `debug` স্তর প্রোডাকশনে বন্ধ রাখুন — অতিরিক্ত লগ ডিস্ক, CPU ও কেন্দ্রীয় লগের খরচ বাড়ায়।

২২.১৪ সমস্যা সমাধান পদ্ধতি

TROUBLESHOOTING: কনটেইনার Exit Code 137 নিয়ে থেমে গেলে প্রথমে `docker stats` ও `docker compose logs` দেখুন — OOM-এর লক্ষণ: মেমোরি Limit-এর কাছাকাছি পৌঁছে হঠাৎ থামা। `dmesg | grep -i oom` (হোস্টে) OOM Killer-এর রেকর্ড দেখায়। ডিস্ক ভরে গেলে `df -h` ও `docker system df` দেখুন — পুরনো লগ বা অব্যবহৃত Image-ই সাধারণত কারণ; `docker system prune` সাবধানে ব্যবহার করুন (চলমান জিনিস মুছবে না, তবে অব্যবহৃত ভলিউম মুছতে `-v` দরকার — ডেটা হারানোর ঝুঁকি)।

সমস্যা	সম্ভাব্য কারণ	সমাধান
Exit Code 137	OOM Kill — মেমোরি Limit ছাড়িয়েছে	লিক ঠিক করুন বা Limit বাড়ান; `dmesg` যাচাই করুন
সার্ভিস অস্বাভাবিক ধীর	CPU Throttling (Limit ছুঁয়েছে)	`docker stats`-এ CPU% দেখে Limit বাড়ান
ডিস্ক পূর্ণ	লগ রোটেশন নেই বা পুরনো Image	`max-size` দিন; `docker system prune` (সাবধানে)
লগে কিছু পাওয়া যায় না	অ্যাপ ফাইলে লগ লিখছে	STDOUT/STDERR-এ লগ লিখুন

২২.১৫ সেরা অনুশীলন ও অ্যান্টি-প্যাটার্ন

সেরা অনুশীলন: প্রতিটি সার্ভিসে মেমোরি ও CPU Limit; অ্যাপ-স্তরের সীমা (হিপ, Cache) Cgroup-এর সঙ্গে মিলিয়ে; লগ রোটেশন; JSON Structured লগ; Health Status দিয়ে সতর্কতা; নিয়মিত `docker stats` পর্যালোচনা। **অ্যান্টি-প্যাটার্ন:** Limit ছাড়া চালানো; লগে Secret; `debug` লগ প্রোডাকশনে; শুধু `Up` দেখে নিশ্চিন্ত হওয়া; ডিস্ক পূর্ণ হওয়া পর্যন্ত অপেক্ষা করা।

২২.১৬ অধ্যায়ের সারাংশ

Limit মানে সর্বোচ্চ সীমা, Reservation মানে ন্যূনতম প্রতিশ্রুতি; মেমোরি ছাড়ালে OOM Killer প্রসেস মারে (Exit 137), CPU ছাড়ালে শুধু ধীর হয়। লগ STDOUT/STDERR-এ লিখে Driver দিয়ে সংগ্রহ করুন, `max-size`/`max-file` দিয়ে রোটেশন দিন। `docker stats`, `docker events` ও Health Status — তিনটি মিলে পর্যবেক্ষণের চিত্র তৈরি হয়; সতর্কতা সবসময় Health ও ব্যবসায়িক সূচকে, শুধু "চলছে" দেখে নয়।

২২.১৭ মূল প্রযুক্তিগত পরিভাষা

Limit: কনটেইনারের সর্বোচ্চ রিসোর্স — ছাড়ালে শাস্তি (OOM Kill বা Throttling)।

Reservation: কনটেইনারের ন্যূনতম রিসোর্স প্রতিশ্রুতি।

OOM (Out of Memory): মেমোরি শেষ হয়ে যাওয়া; OOM Killer প্রসেস মেরে মেমোরি ফেরত নেয়।

Throttling: CPU সীমা ছাড়ালে প্রসেসকে ধীর করে দেওয়া (থামিয়ে নয়)।

Logging Driver: Docker Engine থেকে লগ সংগ্রহ করে গন্তব্যে পাঠানোর ব্যবস্থা।

Log Rotation: লগ ফাইল আকার/সংখ্যায় সীমিত করে পুরনোটি মুছে ফেলা।

Structured Log: JSON-এর মতো কাঠামোবদ্ধ লগ — খোঁজা ও ফিল্টার করা সহজ।

docker stats: কনটেইনারের লাইভ রিসোর্স ব্যবহার দেখার কমান্ড।

docker events: কনটেইনারের জীবনচক্রের ঘটনার ধারা।

২২.১৮ জ্ঞান যাচাই — প্রশ্ন

1. Limit ও Reservation-এর পার্থক্য কী? কোনটি কঠোর?
2. মেমোরি Limit ছাড়ালে ও CPU Limit ছাড়ালে কী আলাদা ঘটে? Exit Code 137 কী বোঝায়?
3. লগ রোটেশন না দিলে কী ঝুঁকি? `max-size` ও `max-file` কী করে?
4. Running বনাম Healthy পার্থক্য Monitoring-এ কেন গুরুত্বপূর্ণ?
5. ফাইলে লগ লেখা কেন কনটেইনারে ভুল অভ্যাস?

২২.১৯ ব্যবহারিক অনুশীলন

1. **Limit নির্ধারণ:** `docker stats` দিয়ে স্বাভাবিক ব্যবহার মেপে প্রতিটি সার্ভিসে উপযুক্ত Limit দিন।
2. **OOM মহড়া:** ল্যাবে একটি ছোট Limit দিয়ে OOM Kill ও Exit 137 দেখুন, `dmesg`-এ রেকর্ড খুঁজুন।
3. **লগ অনুশীলন:** JSON Structured লগ চালু করে `logs --since 1h` দিয়ে একটি ভুলের সময়-ক্রম বের করুন।

অধ্যায় 23 • PART V — Container Security, Resources and Production Operations

Chapter 23 — Production Deployment, Updates and Troubleshooting

২৩.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: (১) Immutable Artifact, ট্যাগিং ও Digest Pinning দিয়ে নির্ভরযোগ্য Deploy নকশা করতে; (২) Registry Auth ও Image Pull-এর প্রবাহ ব্যাখ্যা করতে; (৩) Compose দিয়ে Rolling Replacement, আপডেট ও Rollback করতে; (৪) আপডেটের সময় ডেটাবেস সামঞ্জস্য (Compatibility) পরিকল্পনা করতে; (৫) ডাউনটাইম কমানোর কৌশল প্রয়োগ করতে; (৬) Deploy যাচাই (Verification) ও Runbook লিখতে; (৭) স্টোরেজ সুরক্ষা ও ব্যাকআপসহ পূর্ণ Deploy পদ্ধতি পরিচালনা করতে।

২৩.২ কেন এই বিষয়টি DevOps-এ গুরুত্বপূর্ণ

Deploy-ই সেই মুহূর্ত যখন সব প্রস্তুতি বাস্তব ব্যবহারকারীর সামনে পরীক্ষিত হয়। একটি ভুল Deploy — ভুল Image, অসামঞ্জস্য ডেটাবেস, বা যাচাই ছাড়া রিলিজ — মিনিটের মধ্যে ব্যবসার ক্ষতি করে। ভালো Deploy ব্যবস্থার বৈশিষ্ট্য তিনটি: অনুমানযোগ্য (একই ধাপ প্রতিবার), যাচাইযোগ্য (Deploy-এর পর সত্যিই কাজ করছে কি না জানা যায়), ও ফিরে-যাওয়া-যায় (ভুল হলে দ্রুত আগের সংস্করণে ফেরা যায়)। এই অধ্যায়ে TechNova-র একক-হোস্ট Compose Deploy-কে এই তিনটি গুণে সাজানো হবে — এবং সততার সঙ্গে তার সীমাও বলা হবে।

২৩.৩ পূর্বশর্ত জ্ঞান

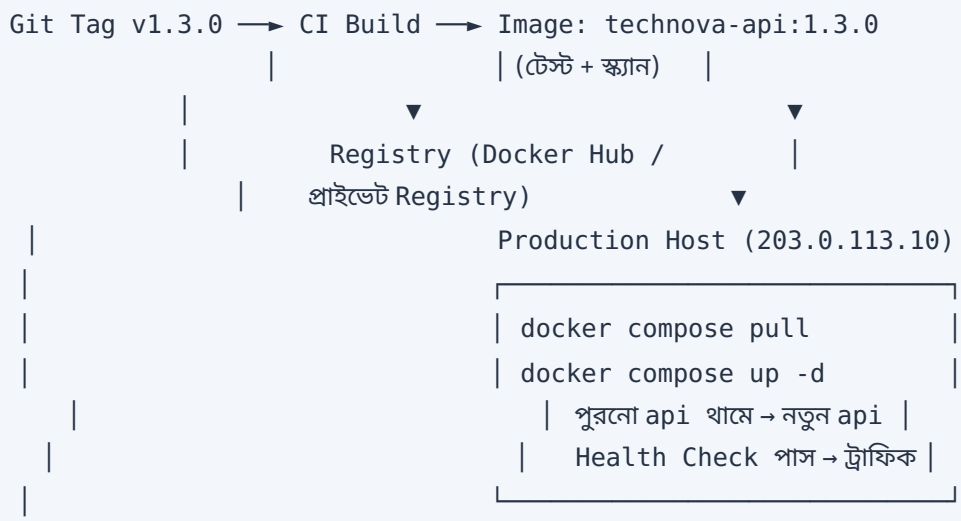
অধ্যায় ১৭–২২-এর সব জ্ঞান (বিশেষ করে Health Check, Restart, ভলিউম, Secret ও লগ), Git Tag ও Semantic Versioning, Registry-এর ধারণা (Docker Hub), MongoDB ব্যাকআপ (অধ্যায় ১৮), এবং Volume 05-এর CI/CD Pipeline-এর ধারণা। Blue-Green ও Rolling Deployment-এর পূর্ব জ্ঞান প্রয়োজন নেই — এখানেই ব্যাখ্যা করা হবে।

২৩.৪ প্রথম নীতি থেকে ব্যাখ্যা

প্রথম নীতি: *Deploy* মানে নতুন কোড চালানো নয় — নতুন *Artifact* চালানো। একটি Image একবার Build হয়ে Registry-তে গেলে তা আর বদলায় না (Immutable)। Staging-এ পরীক্ষিত Image-টিই Production-এ চলে — সার্ভারে আবার Build করা হয় না। এতে "Staging-এ চলেছিল, Production-এ চলছে না" সমস্যা দূর হয়। দ্বিতীয় নীতি: ট্যাগ একটি নাম, *Digest* একটি পরিচয়। `technova-api:1.2.0` ট্যাগটি একই নামে আবার Push করা যায় — তখন "1.2.0" আর আগের Image নয়। Digest (SHA256) কখনো বদলায় না — তাই গুরুত্বপূর্ণ Deploy-এ ট্যাগের সঙ্গে Digest-ও লিখে রাখুন, বা Registry-তে ট্যাগ ওভাররাইট নিষিদ্ধ করুন। তৃতীয় নীতি: আপডেটের সময় পুরনো ও নতুন সংস্করণ একসঙ্গে থাকতে পারে। Deploy-এর মাঝখানে কিছু ব্যবহারকারী পুরনো API-তে, কিছু নতুন API-তে যেতে পারে; ডেটাবেসকে দুই সংস্করণের সঙ্গেই কাজ করতে হবে — একে বলে Backward Compatibility। চতুর্থ নীতি: *Rollback* একটি পরিকল্পনা, আশা নয়। Rollback-এর ধাপ আগে থেকে লেখা থাকতে হবে, পরীক্ষা করা থাকতে হবে, আর এক কমান্ডে চলতে হবে — সংকটের মুহূর্তে নতুন করে ভাবার সময় থাকে না।

CONCEPT: Rolling Replacement বনাম Blue-Green — Compose-এর `up -d` দিয়ে আপডেট করলে পুরনো কনটেইনার থেমে নতুনটি ওঠে — মাঝখানে সংক্ষিপ্ত ডাউনটাইম থাকে (একটি সার্ভিস একবারে বদলায়)। Blue-Green মানে নতুন সংস্করণকে আলাদা পরিবেশে (Green) পুরো চালু করে, যাচাই করে, তারপর ট্রাফিক একবারে সরিয়ে নেওয়া — ডাউনটাইম প্রায় শূন্য, কিন্তু দ্বিগুণ রিসোর্স লাগে। একক হোস্টে Compose দিয়ে Blue-Green করা যায় (দুটি প্রজেক্ট নামে দুটি স্ট্যাক), তবে জটিল; ছোট Deploy-এ Rolling Replacement-ই সাধারণ পছন্দ — ডাউনটাইম স্বীকার করে, Health Check দিয়ে তা ছোট রাখা হয়।

২৩.৫ আর্কিটেকচার / ধারণাগত চিত্র



লক্ষ্য করুন: Deploy-এর কোনো ধাপে সার্ভারে কোড Build করা হয় না — শুধু Registry থেকে Image টানা হয়। এটাই Immutable Artifact নীতি।

২৩.৬ বিস্তারিত প্রযুক্তিগত ব্যাখ্যা

ট্যাগিং কৌশল

Semantic Versioning অনুযায়ী ট্যাগ দিন: `1.3.0` (Major.Minor.Patch)। প্রতিটি Build-এ Git Commit Hash-এর ছোট অংশও ট্যাগ করুন (`1.3.0-a1b2c3d`) — কোন কমিট থেকে Image তৈরি তা জানা যায়। `latest` কখনো Deploy-এর জন্য ব্যবহার করবেন না — "latest" এখন কোন সংস্করণ তা কেউ জানে না, Rollback-ও অসম্ভব হয়। Registry-তে পুরনো ট্যাগ মুছে ফেলার নীতি (Retention) রাখুন — যেমন শেষ ১০টি সংস্করণ রাখা, বাকি মুছে ফেলা — তবে বর্তমানে চলমান ও আগের সংস্করণটি কখনো মুছবেন না।

Registry Auth ও Image Pull

প্রাইভেট Registry-তে `docker login registry.technova.example` দিয়ে প্রমাণীকরণ করতে হয়; CI-তে Registry Credential Secret হিসেবে দেওয়া হয়, কখনো কোডে লেখা হয় না। সার্ভারে Deploy-এর আগে `docker compose pull`

দিয়ে Image আগেই টেনে রাখুন — Deploy-এর সময় Pull-এর জন্য অপেক্ষা করতে হয় না, আর নেটওয়ার্ক সমস্যা Deploy-এর আগেই ধরা পড়ে। Image Pull-এর সময় Registry-র TLS যাচাই হয়; নিজের Registry চালালে সার্টিফিকেট সঠিক রাখুন।

Compose দিয়ে আপডেট — ধাপে ধাপে

আপডেটের সঠিক ক্রম: (১) ব্যাকআপ নিন (`mongodump`); (২) নতুন Image টানুন (`docker compose pull api web`); (৩) `IMAGE_TAG=1.3.0 docker compose up -d api web` — Compose শুধু বদলে যাওয়া সার্ভিসগুলো আবার তৈরি করে, MongoDB ও Nginx ছোঁয় না; (৪) Health Check পাস হওয়া পর্যন্ত অপেক্ষা করুন (`docker compose ps`); (৫) যাচাই করুন (২৩.৬-এর Verification); (৬) সমস্যা হলে Rollback। মনে রাখবেন: Compose-এর আপডেট *Recreate* পদ্ধতিতে হয় — পুরনো কনটেইনার থামে, তারপর নতুনটি ওঠে — মাঝখানে কয়েক সেকেন্ডের ডাউনটাইম থাকে। `--no-recreate` দিলে বদলানো কনফিগারেশন প্রয়োগ হয় না; `--force-recreate` দিলে সব কনটেইনার আবার তৈরি হয়।

ডেটাবেস সামঞ্জস্য — সবচেয়ে অবহেলিত ঝুঁকি

অ্যাপের নতুন সংস্করণে ডেটাবেসের গঠন (Schema) বদলাতে পারে — যেমন একটি নতুন ক্ষেত্র (Field) যোগ হওয়া। নিয়ম: ডেটাবেসের পরিবর্তন সবসময় *Backward Compatible* হতে হবে। অর্থাৎ নতুন কোড পুরনো ডেটা পড়তে পারবে, আর পুরনো কোড (Rollback-এর ক্ষেত্রে) নতুন ডেটা পড়তে পারবে। বাস্তব নিয়ম: ক্ষেত্র যোগ করা নিরাপদ, ক্ষেত্র মুছে ফেলা বা নাম বদলানো বিপজ্জনক; মুছতে হলে দুই ধাপে করুন — প্রথম রিলিজে নতুন ক্ষেত্র যোগ করে পুরনোটি ব্যবহার বন্ধ করুন, পরের রিলিজে পুরনোটি মুছুন (Expand-Contract প্যাটার্ন)। MongoDB-তে Schema নমনীয় বলে ঝুঁকি কম, কিন্তু অ্যাপের Validation (যেমন Mongoose Schema-তে `required: true` যোগ করা) পুরনো ডকুমেন্টকে অবৈধ করে দিতে পারে — তাই Validation বদলানোর আগে পুরনো ডেটা যাচাই করুন। ডেটাবেস Migration চালানোর সময় অ্যাপ থামিয়ে Migration চালিয়ে আবার চালু করা নিরাপদ, তবে ডাউনটাইম বাড়ে — ছোট Deploy-এ এটিই গ্রহণযোগ্য।

ডাউনটাইম কমানো

একক হোস্টে ডাউনটাইম শূন্য করা যায় না, তবে কমানো যায়: (১) Image আগেই Pull করে রাখা; (২) Health Check দ্রুত পাস করা (`start_interval: 5s`); (৩) Nginx-কে সবার শেষে আপডেট করা; (৪) Deploy কম ট্রাফিকের সময়ে করা (যেমন রাত ৩টা); (৫) `stop_grace_period` ঠিক রাখা যাতে চলমান অনুরোধ শেষ হতে পারে। আরও উন্নত পদ্ধতি — Blue-Green দুটি প্রজেক্ট নামে (`-p technova-blue` ও `-p technova-green`) — Nginx-এর আপস্ট্রিম বদলে ট্রাফিক সরানো; তবে এটি Compose-এর সীমা ছাড়িয়ে যায় এবং Kubernetes-এর দিকে ইঙ্গিত করে (অধ্যায় ২৪)।

Deploy যাচাই (Verification) — চেকলিস্ট

Deploy-এর পর যাচাই না করে "হয়ে গেছে" বলা যাবে না: (১) `docker compose ps` — সব সার্ভিস `Up (healthy)`; (২) `curl http://localhost/api/health/ready` — 200; (৩) ব্রাউজারে মূল পাতা ও লগইন পরীক্ষা; (৪) একটি গুরুত্বপূর্ণ ব্যবসায়িক প্রবাহ (যেমন পণ্য যোগ করা) পরীক্ষা; (৫) `docker compose logs --since 5m api` — ভুলের ঢেউ আছে কি না; (৬) সংস্করণ যাচাই — API-র `/version` এন্ডপয়েন্টে নতুন সংস্করণ নম্বর দেখা যাচ্ছে কি না (প্রতিটি Image-এ `APP_VERSION` চলক ও Git Hash লেবেল হিসেবে রাখুন)। যাচাই ব্যর্থ হলে সঙ্গে সঙ্গে Rollback — "আরেকটু দেখি" বলে অপেক্ষা করবেন না।

স্টোরেজ সুরক্ষা

Deploy-এর সময় ভলিউম কখনো মুছবেন না: `docker compose down -v` Deploy স্ক্রিপ্টে কখনো লিখবেন না; `docker system prune -v` প্রোডাকশনে নিষিদ্ধ। ভলিউমের ব্যাকআপ Deploy-এর আগে নিন, আর ব্যাকআপটি অন্য হোস্টে বা ক্লাউড স্টোরেজে রাখুন — একই হোস্টে ব্যাকআপ রাখলে হোস্ট নষ্ট হলে ব্যাকআপও যায়।

২৩.৭ Docker কমান্ড ও লাইন-বাই-লাইন ব্যাখ্যা

```
$ docker login registry.technova.example
$ docker compose pull api web
$ IMAGE_TAG=1.3.0 docker compose up -d api web
$ docker compose ps
$ curl -s http://localhost/api/health/ready
$ curl -s http://localhost/api/version
$ IMAGE_TAG=1.2.0 docker compose up -d api web   # Rollback
$ docker image prune -a --filter "until=720h"
```

ব্যাখ্যা: `docker login` — Registry-তে প্রমাণীকরণ (Credential নিরাপদে সংরক্ষিত হয়)। `docker compose pull api web` — শুধু বদলে যাওয়া সার্ভিসের Image টানা। `IMAGE_TAG=1.3.0 docker compose up -d api web` — পরিবেশ চলক দিয়ে নতুন ট্যাগ নির্ধারণ করে শুধু `api` ও `web` আপডেট করা; MongoDB ছোঁয়া হয় না। `curl .../health/ready` ও `/version` — যাচাইয়ের দুটি কল। Rollback কমান্ডটি একই, শুধু ট্যাগ আগেরটি — এটাই Immutable Artifact-এর শক্তি। `docker image prune -a --filter "until=720h"` — ৩০ দিনের পুরনো অব্যবহৃত Image মুছে ডিস্ক পরিষ্কার করা (চলমান Image কখনো মুছে না)।

নমুনা আউটপুট (চিত্রণমূলক):

```
$ curl -s http://localhost/api/version
{"version":"1.3.0","commit":"a1b2c3d","environment":"production"}
$ docker compose ps
NAME                SERVICE   STATUS
technova-api-1      api       Up 2 minutes (healthy)
technova-web-1      web       Up 2 minutes (healthy)
technova-mongo-1    mongo     Up 3 days (healthy)
```

লক্ষ্য করুন: MongoDB ৩ দিন ধরে চলছে — আপডেটে তাকে ছোঁয়া হয়নি, ডেটা নিরাপদ।

২৩.৮ সম্পূর্ণ কনফিগারেশন উদাহরণ

```
#!/bin/bash
# deploy.sh — TechNova প্রোডাকশন Deploy স্ক্রিপ্ট (সরলীকৃত, পর্যালোচনা করে ব্যবহার করুন)
set -euo pipefail
TAG="${1:?ব্যবহার: ./deploy.sh 1.3.0}"
echo "1. ব্যাকআপ নেওয়া হচ্ছে..."
docker compose exec -T mongo mongodump --uri="mongodb://technova_admin:${MONGO_PASSWORD}
@mongo:27017/technova?authSource=admin" --archive=/tmp/dump.archive
docker compose cp mongo:/tmp/dump.archive "./backups/dump-${TAG}-$(date +%F).archive"
echo "2. নতুন Image টানা হচ্ছে..."
IMAGE_TAG="$TAG" docker compose pull api web
echo "3. আপডেট করা হচ্ছে..."
IMAGE_TAG="$TAG" docker compose up -d api web
echo "4. Health Check-এর জন্য অপেক্ষা..."
sleep 20
docker compose ps
echo "5. যাচাই..."
curl -fsS http://localhost/api/health/ready
curl -fsS http://localhost/api/version
echo "Deploy সম্পন্ন: $TAG"
```

`set -euo pipefail` মানে কোনো ধাপ ব্যর্থ হলে স্ক্রিপ্ট থেমে যাবে — অর্ধেক Deploy এগোবে না। `curl -f` মানে HTTP ভুল হলে Exit Code শূন্য নয় — যাচাই ব্যর্থ হলে স্ক্রিপ্ট থামবে। ব্যাকআপের ফাইলের নামে ট্যাগ ও তারিখ আছে — কোন ব্যাকআপ কোন সংস্করণের তা জানা যায়। এই স্ক্রিপ্টটি CI/CD Pipeline থেকে চালানো উচিত, হাতে নয় — Volume 05-এর Pipeline জ্ঞানের সঙ্গে মিলিয়ে নিন।

২৩.৯ বাস্তব প্রোডাকশন পরিস্থিতি

REAL WORLD — TechNova: TechNova একবার `latest` ট্যাগ দিয়ে Deploy করেছিল। একটি ভুল রিলিজের পর Rollback করতে গিয়ে দেখা গেল "আগের latest" আর নেই — Registry-তে `latest` এখন ভুল Image-টিই। দলকে Git থেকে পুরনো কোড আবার Build করে Deploy করতে হলো — ৪০ মিনিট ডাউনটাইম। এরপর থেকে নিয়ম: প্রতিটি রিলিজে Semantic ট্যাগ + Commit Hash; `latest` Deploy-এ নিষিদ্ধ; Rollback মানে এক কমান্ডে আগের ট্যাগ। পরের ভুল রিলিজে Rollback সময় লেগেছিল ৩ মিনিট।

২৩.১০ ধাপে ধাপে ব্যবহারিক বাস্তবায়ন

1. API-তে `/version` এন্ডপয়েন্ট যোগ করুন (সংস্করণ, Commit Hash, পরিবেশ ফেরত দেয়)।
2. ওপরের `deploy.sh` লিখে Staging-এ পরীক্ষা করুন — প্রথমে ব্যাকআপ, তারপর আপডেট, তারপর যাচাই।
3. একটি ভুল সংস্করণ Deploy করে Rollback অনুশীলন করুন — সময় মাপুন; ৫ মিনিটের বেশি লাগলে Runbook সরল করুন।
4. Registry-তে Retention নীতি লিখুন: শেষ ১০টি ট্যাগ রাখা, বাকি মুছে ফেলা।

২৩.১১ সাধারণ ভুল ও ভুল ধারণা

- **ভুল:** `latest` ট্যাগ দিয়ে Deploy — Rollback অসম্ভব হয়।
- **ভুল:** সার্ভারে কোড Build করে Deploy — Staging-এ পরীক্ষিত Artifact আর Production-এর Artifact আলাদা হয়।
- **ভুল:** ব্যাকআপ ছাড়া ডেটাবেস আপডেট — Migration ব্যর্থ হলে ফেরার পথ নেই।
- **ভুল:** `down -v` Deploy স্ক্রিপ্টে — ডেটা মুছে যাবে।
- **ভুল:** যাচাই ছাড়া Deploy শেষ ঘোষণা — Health Check ও ব্যবসায়িক প্রবাহ পরীক্ষা করুন।
- **ভুল ধারণা:** "Compose দিয়ে Zero-Downtime Deploy হয়" — না; Recreate পদ্ধতিতে সংক্ষিপ্ত ডাউনটাইম থাকে। শূন্য ডাউনটাইম চাইলে Blue-Green বা অর্কেস্ট্রেটর লাগে।

২৩.১২ নিরাপত্তা বিবেচনা

WARNING: Deploy-এর সময় Registry Credential ও `MONGO_PASSWORD` স্ক্রিপ্টে সরাসরি লিখবেন না — CI Secret বা পরিবেশ চলক থেকে নিন। Deploy স্ক্রিপ্টটি Git-এ থাকলে তার ভেতরে কোনো Secret নেই তা পর্যালোচনা করুন। নতুন Image Deploy-এর আগে স্ক্যান করুন (অধ্যায় ২১) — স্ক্যানে Critical CVE থাকলে Deploy থামান। আর Deploy করার অধিকার শুধু CI সিস্টেম ও সীমিত মানুষের হাতে রাখুন — প্রতিটি Deploy-এর লগ (কে, কখন, কোন ট্যাগ) সংরক্ষণ করুন।

২৩.১৩ কর্মক্ষমতা ও রিসোর্স বিবেচনা

Deploy-এর সময় পুরনো ও নতুন Image একসঙ্গে ডিস্কে থাকে — ডিস্কে অন্তত দুটি সংস্করণের Image-এর জায়গা রাখুন। Pull-এর সময় ব্যান্ডউইথ লাগে — কম ট্রাফিকের সময় Deploy করুন। আপডেটের সময় CPU ব্যবহার বাড়ে (নতুন কনটেইনার চালু, Health Check) — সীমা (অধ্যায় ২২) এমনভাবে দিন যাতে Deploy-এর সময় অন্য সার্ভিস অনাহারে না থাকে।

২৩.১৪ সমস্যা সমাধান পদ্ধতি

TROUBLESHOOTING: আপডেটের পর সার্ভিস `(unhealthy)` হলে: (১) `docker compose logs api --tail 100` — নতুন সংস্করণের ভুল দেখুন; (২) `docker compose ps` — কোন সার্ভিস থেমেছে; (৩) Image ট্যাগ ঠিক আছে কি না `docker images` দিয়ে দেখুন; (৪) ৫ মিনিটের মধ্যে ঠিক না হলে Rollback করুন — তদন্ত পরে, সেবা আগে। Deploy-এর পর ডেটাবেস সংযোগ ব্যর্থ হলে Migration বা Validation পরিবর্তন সন্দেহ করুন — ব্যাকআপ থেকে পুনরুদ্ধারের পরিকল্পনা Runbook-এ লেখা থাকতে হবে।

সমস্যা	সম্ভাব্য কারণ	সমাধান
নতুন সংস্করণ চালু হয় না	Image Pull ব্যর্থ বা ট্যাগ ভুল	`docker compose pull` আবার চালান; ট্যাগ যাচাই করুন
আপডেটের পর 502	API এখনো Healthy হয়নি	Health Check পাস হওয়া পর্যন্ত অপেক্ষা করুন
Rollback-এর পরও ভুল	ডেটাবেস Migration ফেরানো যায়নি	ব্যাকআপ থেকে পুনরুদ্ধার করুন (Runbook অনুযায়ী)
Deploy-এর পর ডিস্ক পূর্ণ	পুরনো Image জমে আছে	`docker image prune` (চলমান Image বাদে)

২৩.১৫ সেরা অনুশীলন ও অ্যান্টি-প্যাটার্ন

সেরা অনুশীলন: Immutable Artifact; Semantic ট্যাগ + Commit Hash; Deploy-এর আগে ব্যাকআপ; Pull আগে, Up পরে; Health Check ও `/version` দিয়ে যাচাই; এক-কমান্ড Rollback; Backward Compatible Migration; Deploy লগ সংরক্ষণ। **অ্যান্টি-প্যাটার্ন:** `latest` Deploy; সার্ভারে Build; ব্যাকআপ ছাড়া Migration; যাচাই ছাড়া রিলিজ; Rollback পরিকল্পনা ছাড়া Deploy; `down -v` স্ক্রিপ্টে।

২৩.১৬ অধ্যায়ের সারাংশ

প্রোডাকশন Deploy মানে Registry-তে থাকা Immutable Image-কে সার্ভারে টেনে চালানো — সার্ভারে Build নয়। Semantic ট্যাগ + Commit Hash দিয়ে সংস্করণ চেনা যায়, Digest দিয়ে পরিচয় নিশ্চিত করা যায়। Compose-এর আপডেট Rolling Replacement — সংক্ষিপ্ত ডাউনটাইম থাকে; Blue-Green তার বিকল্প। ডেটাবেস পরিবর্তন Backward Compatible হতে হবে, ব্যাকআপ Deploy-এর আগে নিতে হবে, আর Rollback এক কমান্ডে চলতে হবে। Deploy শেষ হয় যাচাইয়ে — Health, সংস্করণ ও ব্যবসায়িক প্রবাহ পরীক্ষা না করে রিলিজ শেষ নয়।

২৩.১৭ মূল প্রযুক্তিগত পরিভাষা

Immutable Artifact: একবার তৈরি হলে আর বদলানো যায় না এমন Image বা প্যাকেজ।

Digest: Image-এর SHA256 পরিচয় — ট্যাগের মতো বদলায় না।

Registry: Image সংরক্ষণ ও বিতরণের ভান্ডার (Docker Hub, প্রাইভেট Registry)।

Rolling Replacement: পুরনো কনটেইনার থামিয়ে নতুনটি চালু করে আপডেট করা।

Blue-Green Deployment: নতুন সংস্করণ আলাদা পরিবেশে চালু করে ট্রাফিক সরিয়ে নেওয়া।

Rollback: আগের স্থিতিশীল সংস্করণে ফিরে যাওয়া।

Backward Compatibility: নতুন সংস্করণ পুরনো ডেটা/ক্লায়েন্টের সঙ্গে কাজ করার গুণ।

Expand-Contract: প্রথমে নতুন ক্ষেত্র যোগ, পরে পুরনোটি মুছে ফেলা — নিরাপদ Schema পরিবর্তনের প্যাটার্ন।

Verification: Deploy-এর পর সিস্টেম সত্যিই কাজ করছে কি না তা যাচাই।

Runbook: ঘটনা বা Deploy-এর ধাপে ধাপে লিখিত নির্দেশিকা।

২৩.১৮ জ্ঞান যাচাই — প্রশ্ন

1. ট্যাগ ও Digest-এর পার্থক্য কী? Rollback-এর জন্য কোনটি বেশি নির্ভরযোগ্য?
2. `latest` ট্যাগ দিয়ে Deploy করলে Rollback-এ কী সমস্যা হয়?
3. Compose-এর আপডেটে ডাউনটাইম কেন থাকে? Blue-Green কীভাবে তা কমায়?
4. Backward Compatible Migration কী? ক্ষেত্র মুছে ফেলতে Expand-Contract কেন দরকার?
5. Deploy যাচাইয়ের চেকলিস্টে কী কী থাকা উচিত?
6. Deploy স্ক্রিপ্টে `down -v` থাকলে কী বিপদ?

২৩.১৯ ব্যবহারিক অনুশীলন

1. **Deploy মহড়া:** Staging-এ `deploy.sh` দিয়ে একটি নতুন সংস্করণ Deploy করে যাচাই করুন, তারপর Rollback করুন — সময় মাপুন।
2. **Runbook লেখা:** আপনার স্ট্যাকের জন্য এক পৃষ্ঠার Deploy ও Rollback Runbook লিখুন।
3. **Migration মহড়া:** একটি নতুন ক্ষেত্র যোগ করে Expand-Contract প্যাটার্নে Deploy করুন।

অধ্যায় 24 • PART V — Container Security, Resources and Production Operations

Chapter 24 — Container Operations Architecture and Best Practices

২৪.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: (১) Volume 06-এর সব ধারণা একত্র করে একটি সম্পূর্ণ Container Operations আর্কিটেকচার আঁকতে; (২) Compose-এর সীমা ব্যাখ্যা করে কখন পূর্ণাঙ্গ অর্কেস্ট্রেটর (Kubernetes) দরকার তা বলতে; (৩) AWS, Azure, GCP ও IBM Cloud-এর পরিচালিত Registry ও কনটেইনার রানটাইম বিকল্পগুলো উচ্চ-স্তরে তুলনা করতে; (৪) দৈনিক, সাপ্তাহিক ও মাসিক অপারেশন রুটিন সাজাতে; (৫) ঘটনার (Incident) প্রতিক্রিয়ার Runbook কাঠামো লিখতে; (৬) এই বইয়ের পরের ধাপ — Kubernetes অধ্যয়নের পথ — চিহ্নিত করতে।

২৪.২ কেন এই বিষয়টি DevOps-এ গুরুত্বপূর্ণ

আগের সাতটি অধ্যায়ে আপনি যন্ত্রাংশগুলো শিখেছেন — Compose, Multi-Stage Build, Secret, Health Check, নিরাপত্তা, রিসোর্স, Deploy। এবার সেগুলোকে একত্র করে একটি পরিচালনাযোগ্য ব্যবস্থা বানানোর সময়। বাস্তব অপারেশন মানে শুধু "চালু করা" নয় — প্রতিদিন দেখা, সপ্তাহে আপডেট করা, মাসে পর্যালোচনা করা, আর ঘটনা ঘটলে শান্তভাবে সাড়া দেওয়া। এই অধ্যায়ে সেই রুটিন, সিদ্ধান্তের কাঠামো ও ভবিষ্যতের পথ — তিনটি একসঙ্গে সাজানো হয়েছে। এটাই Volume 06-এর সমাপ্তি অধ্যায়।

২৪.৩ পূর্বশর্ত জ্ঞান

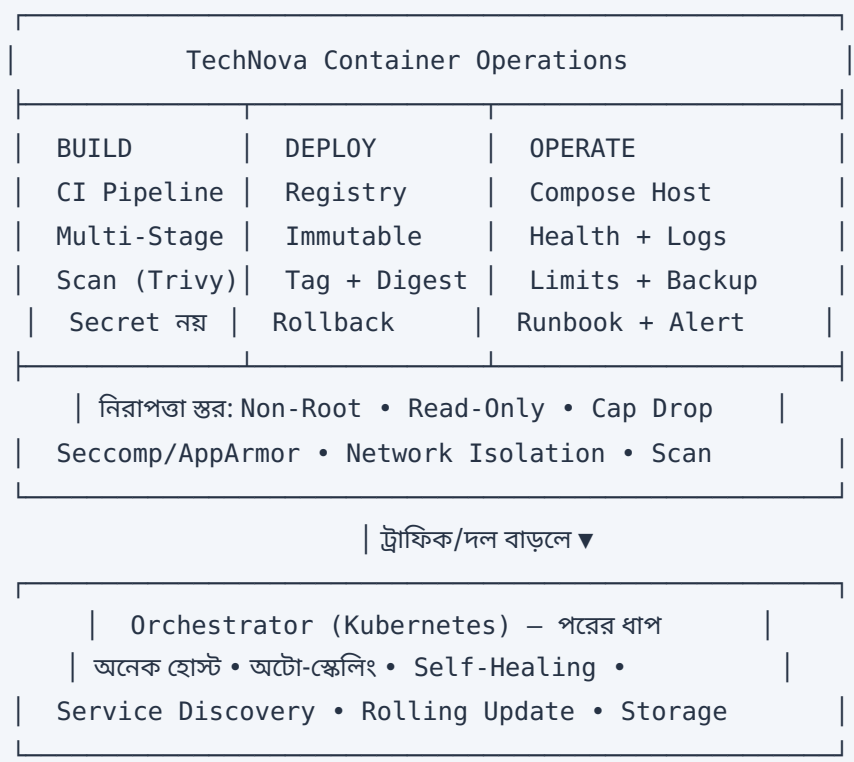
অধ্যায় ১৭–২৩-এর সব বিষয়, Volume 04-এর Linux সার্ভার অপারেশন ও Volume 05-এর CI/CD Pipeline-এর ধারণা। ক্লাউডের পূর্ব অভিজ্ঞতা প্রয়োজন নেই — তুলনাটি উচ্চ-স্তরে, ধারণাগতভাবে দেওয়া হয়েছে।

২৪.৪ প্রথম নীতি থেকে ব্যাখ্যা

প্রথম নীতি: একটি সিস্টেম ততটাই ভালো যতটা তার পরিচালনা। নিখুঁত Dockerfile লিখেও লাভ নেই, যদি কেউ লগ না দেখে, Image আপডেট না করে, আর ব্যাকআপ পরীক্ষা না করে। অপারেশন মানে তিনটি প্রশ্নের নিয়মিত উত্তর: (১) সিস্টেম কি সুস্থ? (২) সিস্টেম কি নিরাপদ? (৩) সিস্টেম কি পুনরুদ্ধারযোগ্য? দ্বিতীয় নীতি: জটিলতা ধাপে ধাপে বাড়ে। একটি হোস্টে Compose দিয়ে শুরু করা সঠিক; ট্রাফিক ও দল বাড়লে একাধিক হোস্ট, স্বয়ংক্রিয় স্কেলিং ও সার্ভিস ডিসকভারি দরকার হয় — তখন অর্কেস্ট্রেটরের সময় আসে। তৃতীয় নীতি: ক্লাউড মানে অন্যের কম্পিউটার ভাড়া করা — সুবিধা ও নির্ভরতা দুটোই। পরিচালিত (Managed) সেবা পরিচালনার ভার কমায়, কিন্তু বিক্রেতার ওপর নির্ভরতা (Vendor Lock-in) ও খরচের হিসাব নতুন করে করতে হয়। এই তিনটি নীতি মাথায় রেখে এই অধ্যায়ের তুলনা ও সিদ্ধান্তগুলো পড়ুন।

> **CONCEPT:** Compose বনাম Orchestrator — Compose একটি হোস্টে কয়েকটি কনটেইনার সাজায়; অর্কেস্ট্রেটর (যেমন Kubernetes) অনেক হোস্টে হাজারো কনটেইনার পরিচালনা করে — স্বয়ংক্রিয় স্কেলিং, নিজে থেকে পুনরুদ্ধার, সার্ভিস ডিসকভারি, রোলিং আপডেট, স্টোরেজ ব্যবস্থাপনা ও ঘোষণামূলক অবস্থা (Desired State) বজায় রাখা — সব একসঙ্গে। Compose শেখা Kubernetes শেখার প্রস্তুতি: Service, Health Check, Volume, Secret, Limit — ধারণাগুলো একই, শুধু পরিসর বড়।

২৪.৫ আর্কিটেকচার / ধারণাগত চিত্র



এই চিত্রটি Volume 06-এর সারসংক্ষেপ: বাম থেকে ডানে Build → Deploy → Operate; নিচে নিরাপত্তা; আর ভবিষ্যতের দিকে অর্কেস্ট্রেটর।

২৪.৬ বিস্তারিত প্রযুক্তিগত ব্যাখ্যা

Compose-এর সীমা — সততার সঙ্গে

ক্ষমতা	Compose (একক হোস্ট)	পূর্ণাঙ্গ অর্কেস্ট্রেটর (Kubernetes)
হোস্ট সংখ্যা	একটি	অনেক (Cluster)
স্বয়ংক্রিয় স্কেলিং	হাতে `--scale`	চাহিদা অনুযায়ী স্বয়ংক্রিয়
অসুস্থ কনটেইনার পুনরুদ্ধার	Restart নীতি (একই হোস্টে)	নতুন Pod অন্য হোস্টে চালু করে
Zero-Downtime আপডেট	সীমিত (Recreate)	Rolling Update, Blue-Green
সার্ভিস ডিসকভারি	একই হোস্টের DNS	Cluster-জুড়ে DNS ও Load Balancer
স্টোরেজ	স্থানীয় ভলিউম	নেটওয়ার্ক স্টোরেজ, Dynamic Provisioning

এই সীমাগুলো Compose-এর ব্যর্থতা নয় — এটি তার নকশার পরিসর। ছোট দল, একটি সার্ভার, কয়েকটি সার্ভিস — এখানে Compose-ই সঠিক পছন্দ: সরল, স্বচ্ছ, কম পরিচালনা। Kubernetes-এ যাওয়া মানে পরিচালনার জটিলতা অনেক বাড়ানো — শুধু তখনই যান যখন সত্যিই একাধিক হোস্ট বা স্বয়ংক্রিয় স্কেলিং দরকার।

ক্লাউড তুলনা — উচ্চ-স্তরে (পরিচালিত Registry ও কনটেইনার রানটাইম)

নিচের তুলনাটি ধারণাগত ও উচ্চ-স্তরের; দাম, কোটা বা নির্দিষ্ট বৈশিষ্ট্য এখানে দেওয়া হয়নি — সেগুলো অঞ্চল ও পরিকল্পনাভেদে বদলায়, আর অফিসিয়াল ডকুমেন্টেশনই একমাত্র নির্ভরযোগ্য উৎস। প্রতিটি ক্লাউডে দুটি ধরনের সেবা আছে: (১) **পরিচালিত Container Registry** — প্রাইভেট Image সংরক্ষণ, স্ক্যানিং ও প্রবেশাধিকার নিয়ন্ত্রণ; (২) **কনটেইনার রানটাইম/ডেপ্লয়মেন্ট বিকল্প** — একক কনটেইনার চালানো থেকে পরিচালিত Kubernetes পর্যন্ত।

ক্লাউড	পরিচালিত Registry	কনটেইনার রানটাইম / ডেপ্লয়মেন্ট বিকল্প (উচ্চ-স্তরে)	অফিসিয়াল ডকুমেন্টেশন
AWS	Amazon ECR (Elastic Container Registry)	ECS (পরিচালিত কনটেইনার সেবা), EKS (পরিচালিত Kubernetes), Fargate (সার্ভারহীন কনটেইনার), App Runner	docs.aws.amazon.com
Azure	Azure Container Registry (ACR)	Azure Container Instances (ACI), Azure Container Apps, AKS (পরিচালিত Kubernetes), App Service (কনটেইনার)	learn.microsoft.com
GCP	Artifact Registry (Container Registry-এর উত্তরসূরি)	Cloud Run (সার্ভারহীন কনটেইনার), GKE (পরিচালিত Kubernetes), Compute Engine-এ কনটেইনার	cloud.google.com
IBM Cloud	IBM Cloud Container Registry	IBM Cloud Code Engine, IBM Cloud Kubernetes Service (IKS), Red Hat OpenShift on IBM Cloud	www.ibm.com/cloud

Docker-এর নিজের ডকুমেন্টেশনও এই অধ্যায়ের ভিত্তি: Compose, Dockerfile ও নিরাপত্তার জন্য `docs.docker.com` দেখুন। **কীভাবে বেছে নেবেন:** ইতিমধ্যে যে ক্লাউডে আপনার অন্য সেবা আছে, সেখানকার Registry ও রানটাইম ব্যবহার করা সাধারণত সহজতম — একই প্রমাণীকরণ, একই নেটওয়ার্ক, একই বিল। ছোট প্রজেক্টে সার্ভারহীন কনটেইনার (Fargate, Cloud Run, Container Apps) পরিচালনার ভার সবচেয়ে কমায়; বড় ও জটিল ব্যবস্থায় পরিচালিত Kubernetes (EKS, AKS, GKE, IKS) মানসম্মত পছন্দ। অর্কেস্ট্রেশন সেবাগুলো এখানে শুধু প্রসঙ্গ হিসেবে দেওয়া হয়েছে — বিস্তারিত Kubernetes অধ্যয়ন এই সিরিজের পরের খণ্ডের বিষয়।

অপারেশন রুটিন — দৈনিক, সাপ্তাহিক, মাসিক

দৈনিক (৫ মিনিট): `docker compose ps` — সব `healthy` কি না; `docker compose logs --since 24h --timestamps api | grep -i error` — নতুন ভুল আছে কি না; ডিস্কের ব্যবহার `df -h`। **সাপ্তাহিক (৩০ মিনিট):** `docker stats --no-stream` পর্যালোচনা; `docker scout cves` বা Trivy স্ক্যান; ব্যাকআপ ফাইল আছে কি না ও তার আকার যাচাই; পুরনো Image পরিষ্কার (`docker image prune`)। **মাসিক (২ ঘণ্টা):** Base Image আপডেট ও পুনঃ-Deploy; ব্যাকআপ থেকে Staging-এ পুনরুদ্ধার পরীক্ষা (Restore Drill); Secret Rotation পর্যালোচনা; Runbook হালনাগাদ; `docker compose config` দিয়ে কনফিগারেশন পর্যালোচনা। এই রুটিনটি ক্যালেন্ডারে লিখে রাখুন — "মনে থাকলে করব" মানে কখনো করা হবে না।

ঘটনার প্রতিক্রিয়া (Incident Response) — Runbook কাঠামো

ঘটনা ঘটলে ধাপগুলো: (১) **সনাক্তকরণ** — Health Check, লগ বা ব্যবহারকারীর অভিযোগ থেকে; (২) **সেবা ফেরানো** — প্রথমে সেবা চালু করুন (Restart, Rollback, ব্যাকআপ থেকে পুনরুদ্ধার), কারণ খুঁজুন পরে; (৩) **যোগাযোগ** — দল ও ব্যবহারকারীকে জানান কী হচ্ছে; (৪) **তদন্ত** — লগ, `docker events` ও মেট্রিক দিয়ে মূল কারণ (Root Cause) খুঁজুন; (৫) **শিক্ষা** — Blameless Postmortem লিখুন: কী হয়েছিল, কেন হয়েছিল, কী বদলানো হবে। Runbook-এ প্রতিটি সাধারণ

ঘটনার (ডেটাবেস ডাউন, ডিস্ক পূর্ণ, Deploy ব্যর্থ) জন্য ধাপ, কমান্ড ও দায়িত্ব লেখা থাকে — সংকটের মুহূর্তে ভাবতে হয় না, পড়ে অনুসরণ করতে হয়।

এই বইয়ের পরের ধাপ

এই Volume শেষে আপনি পারবেন: একটি MERN অ্যাপকে Multi-Stage Dockerfile দিয়ে বানাতে, Compose দিয়ে চালাতে, নিরাপদ ও পর্যবেক্ষণযোগ্য রাখতে, আর নির্ভরযোগ্যভাবে Deploy ও Rollback করতে। পরের ধাপগুলো: (১) CI/CD Pipeline-এ এই সবকিছু স্বয়ংক্রিয় করা (Volume 05-এর সঙ্গে মিলিয়ে); (২) Kubernetes-এর মৌলিক ধারণা — Pod, Deployment, Service, Ingress, ConfigMap, Secret — শেখা (এই সিরিজের পরের খণ্ড); (৩) পর্যবেক্ষণে Prometheus + Grafana যোগ করা; (৪) Terraform দিয়ে অবকাঠামো কোড হিসেবে লেখা। প্রতিটি ধাপে এই Volume-এর ধারণাগুলোই ফিরে আসবে — শুধু পরিসর বড় হবে।

২৪.৭ Docker কমান্ড ও লাইন-বাই-লাইন ব্যাখ্যা

```
$ docker compose ps && docker stats --no-stream
$ docker compose logs --since 24h --timestamps api | grep -i error
$ docker system df && docker system prune -f
$ docker compose config --quiet && echo "Compose ফাইল বৈধ"
$ docker scout cves technova-api:1.2.0
$ docker compose exec mongo mongodump --archive=/tmp/dump.archive --uri="mongodb://
technova_admin:CHANGE-ME-STRONG-PASSWORD@mongo:27017/technova?authSource=admin"
```

ব্যাখ্যা: প্রথম লাইনটি দৈনিক পরীক্ষার এক-লাইন সংস্করণ — অবস্থা ও রিসোর্স একসঙ্গে। দ্বিতীয় লাইনটি গত ২৪ ঘণ্টার ভুল খোঁজে। `docker system df` ডিস্ক ব্যবহার দেখায়; `docker system prune -f` অব্যবহৃত কনটেইনার, নেটওয়ার্ক ও Image মুছে দেয় (ভলিউম মুছে না — `-v` ছাড়া নিরাপদ, তবু প্রোডাকশনে সাবধানে)। `docker compose config --quiet` ফাইল বৈধ কি না নীরবে যাচাই করে। শেষ দুটি স্ক্যান ও ব্যাকআপ — সাপ্তাহিক রুটিনের অংশ।

২৪.৮ সম্পূর্ণ কনফিগারেশন উদাহরণ

অপারেশন চেকলিস্ট (`operations-checklist.md` — প্রজেক্টে রাখুন):

```
# TechNova — দৈনিক / সাপ্তাহিক / মাসিক চেকলিস্ট
## দৈনিক (৫ মিনিট)
- [ ] docker compose ps — সব (healthy)?
- [ ] গত ২৪ ঘণ্টার error লগ পরীক্ষা
- [ ] df -h — ডিস্ক 80%-এর নিচে?
## সাপ্তাহিক (৩০ মিনিট)
- [ ] docker stats পর্যালোচনা — কোনো সার্ভিস সীমার কাছাকাছি?
- [ ] Image স্ক্যান (Trivy / Scout) — নতুন Critical CVE?
- [ ] ব্যাকআপ ফাইল আছে ও আকার যুক্তিসংগত?
- [ ] পুরনো Image পরিষ্কার
## মাসিক (২ ঘণ্টা)
- [ ] Base Image আপডেট ও Staging-এ Deploy
- [ ] ব্যাকআপ থেকে পুনরুদ্ধার মহড়া (Staging-এ)
- [ ] Secret Rotation পর্যালোচনা
- [ ] Runbook হালনাগাদ
## ঘটনা ঘটলে
- [ ] সেবা আগে ফেরাও (Restart / Rollback), কারণ পরে খোঁজো
- [ ] Blameless Postmortem লেখো
```

এই চেকলিস্টটি একটি ফাইল হিসেবে প্রজেক্টে রাখুন এবং প্রতিটি পর্যালোচনার তারিখ লিখুন — অপারেশন তখন ব্যক্তির স্মৃতির ওপর নির্ভর করে না, প্রক্রিয়ার ওপর নির্ভর করে।

২৪.৯ বাস্তব প্রোডাকশন পরিস্থিতি

REAL WORLD — TechNova: TechNova-র দল Volume 06-এর সবকিছু বাস্তবায়ন করার পর একটি সিদ্ধান্তের মুখে পড়ল: ট্রাফিক তিন গুণ বেড়েছে, একটি হোস্ট আর যথেষ্ট নয়। তারা Compose ছেড়ে সঙ্গে সঙ্গে Kubernetes-এ যায়নি — প্রথমে দ্বিতীয় হোস্টে একই Compose স্ট্যাক চালিয়ে Nginx-এর সামনে Load Balancer বসাল, আর ডেটাবেসকে আলাদা পরিচালিত সেবায় সরানোর পরিকল্পনা করল। শুধু যখন স্বয়ংক্রিয় স্কেলিং সত্যিই দরকার হলো, তখন তারা পরিচালিত Kubernetes (EKS-এর মতো সেবা) মূল্যায়ন শুরু করল — Compose-এর ধারণাগুলো (Health Check, Limit, Secret) Kubernetes-এর Manifest-এ প্রায় একইভাবে ফিরে এলো। শিক্ষা: প্রযুক্তি বদলানোর সিদ্ধান্ত নেওয়া হয় প্রয়োজনে, ফ্যাশনে নয়।

২৪.১০ ধাপে ধাপে ব্যবহারিক বাস্তবায়ন

1. ওপরের চেকলিস্টটি আপনার প্রজেক্টে ফাইল হিসেবে রাখুন এবং এক সপ্তাহ ধরে দৈনিক পরীক্ষা চালান।
2. একটি Incident Runbook লিখুন: "ডেটাবেস ডাউন" পরিস্থিতির ধাপ, কমান্ড ও দায়িত্ব।
3. Staging-এ ব্যাকআপ থেকে পুনরুদ্ধারের মহড়া করুন — সময় মাপুন ও লিখে রাখুন।
4. একটি ক্লাউডের (যেমন AWS বা GCP) অফিসিয়াল ডকুমেন্টেশন খুলে তার Registry ও একটি কনটেইনার রানটাইম বিকল্পের পাতা পড়ুন — দাম নয়, ধারণাটি বুঝুন।

২৪.১১ সাধারণ ভুল ও ভুল ধারণা

- **ভুল:** Deploy-এর পর আর না দেখা — অপারেশন ছাড়া সিস্টেম ধীরে ধীরে নষ্ট হয়।
- **ভুল:** ব্যাকআপ নেওয়া কিন্তু পুনরুদ্ধার পরীক্ষা না করা — ব্যাকআপ কাজ করে কি না জানা যায় না।
- **ভুল:** প্রয়োজন ছাড়া Kubernetes-এ যাওয়া — জটিলতা বাড়ে, লাভ হয় না।
- **ভুল:** ঘটনার পর দোষ খোঁজা (Blame) — মানুষ ভুল লুকাতে শেখে, সিস্টেম ঠিক হয় না।
- **ভুল ধারণা:** "Compose শিখেছি মানেই Kubernetes জানি" — না; ধারণাগুলো একই, কিন্তু অর্কেস্ট্রেটরের নিজস্ব ধারণা (Pod, Deployment, Ingress) আলাদাভাবে শিখতে হবে।
- **ভুল ধারণা:** "ক্লাউডে গেলেই নিরাপত্তা ও ব্যাকআপের দায়িত্ব শেষ" — না; ক্লাউড অবকাঠামোর দায়িত্ব নেয়, আপনার অ্যাপ ও ডেটার দায়িত্ব আপনারই থাকে (Shared Responsibility)।

২৪.১২ নিরাপত্তা বিবেচনা

WARNING: অপারেশনের নিরাপত্তা মানে নিয়মিততা: Base Image আপডেট না করলে জানা দুর্বলতা জমতে থাকে; Secret Rotation না করলে ফাঁস হওয়া Secret চিরকাল কাজ করে; ব্যাকআপ পরীক্ষা না করলে প্রয়োজনের সময় তা কাজ নাও করতে পারে। মাসিক রুটিনে এই তিনটি বাধ্যতামূলক করুন। আর ক্লাউডে গেলে Registry-র প্রবেশাধিকার (কে Image Push/Pull করতে পারে) ও ক্লাউডের IAM অনুমতি Least Privilege অনুযায়ী দিন — ক্লাউডের Root Credential কখনো CI-তে ব্যবহার করবেন না।

২৪.১৩ কর্মক্ষমতা ও রিসোর্স বিবেচনা

অপারেশনের খরচ মূলত সময়: দৈনিক ৫ মিনিট, সাপ্তাহিক ৩০ মিনিট, মাসিক ২ ঘণ্টা — একটি ছোট দলের জন্য এটাই বাস্তবসম্মত। এই সময় না দিলে খরচ পরে আসে — ঘটনার সময় ঘণ্টার পর ঘণ্টা, ডেটা হারানোর ঝুঁকি, আর ব্যবহারকারীর আস্থা হারানো। ক্লাউডে যাওয়ার খরচের হিসাব করার সময় শুধু সার্ভারের দাম নয় — পরিচালনার সময়, ব্যাকআপ, মনিটরিং ও ডেটা স্থানান্তরের খরচও ধরুন; দাম অফিসিয়াল প্রাইসিং পাতায় যাচাই করুন, এই বইয়ে কোনো দাম দেওয়া হয়নি।

২৪.১৪ সমস্যা সমাধান পদ্ধতি

TROUBLESHOOTING: "সবকিছু ঠিক আছে, তবু সাইট ধীর" — ধাপে ধাপে যান: (১) `docker stats` — কোন সার্ভিস সীমায় পৌঁছেছে; (২) `docker compose logs --since 1h` — ভুলের ধরন; (৩) `docker compose ps` — Health Status; (৪) হোস্টের `df -h` ও `free -h` — ডিস্ক ও মেমোরি; (৫) Nginx লগ — কোন এন্ডপয়েন্ট ধীর। এই ক্রমটি একটি সিদ্ধান্ত-গাছ (Decision Tree) — প্রতিটি ধাপে উত্তর "হ্যাঁ/না" অনুযায়ী পরের ধাপে যান, এলোমেলোভাবে কমান্ড চালাবেন না। প্রতিটি ঘটনার পর Runbook-এ এই গাছটি হালনাগাদ করুন।

লক্ষণ	প্রথমে দেখুন	তারপর
সাইট ডাউন	`docker compose ps`	লগ → Health → Rollback
সাইট ধীর	`docker stats`	লগ → ডেটাবেস → Nginx
ডিস্ক পূর্ণ	`df -h`, `docker system df`	লগ রোটেশন → পুরনো Image পরিষ্কার
Deploy ব্যর্থ	`docker compose logs`	ট্যাগ যাচাই → Rollback

২৪.১৫ সেরা অনুশীলন ও অ্যান্টি-প্যাটার্ন

সেরা অনুশীলন: দৈনিক/সাপ্তাহিক/মাসিক রুটিন; ব্যাকআপ + পুনরুদ্ধার মহড়া; Runbook ও Blameless Postmortem; নিয়মিত Base Image আপডেট; প্রয়োজনে অর্কেস্ট্রেটরে যাওয়া; ক্লাউড সিদ্ধান্তে অফিসিয়াল ডকুমেন্টেশন যাচাই। **অ্যান্টি-প্যাটার্ন:** Deploy-এর পর না দেখা; পরীক্ষা ছাড়া ব্যাকআপ; ফ্যাশনে Kubernetes; ঘটনায় দোষ খোঁজা; দাম বা বৈশিষ্ট্য অনুমান করে ক্লাউড বাছাই।

২৪.১৬ অধ্যায়ের সারাংশ

Container Operations মানে Build → Deploy → Operate-এর পুরো চক্রকে নিয়মিত রুটিন, Runbook ও যাচাই দিয়ে পরিচালনা করা। Compose একটি হোস্টের জন্য সঠিক টুল; তার সীমা (একক হোস্ট, হাতে স্কেলিং, সীমিত Zero-Downtime) ছাড়িয়ে গেলে পরিচালিত Kubernetes-এর দিকে যাওয়া যায় — ধারণাগুলো একই থাকে। ক্লাউডে AWS (ECR/ECS/EKS/Fargate), Azure (ACR/ACI/Container Apps/AKS), GCP (Artifact Registry/Cloud Run/GKE) ও IBM Cloud (Container Registry/Code Engine/IKS/OpenShift) — প্রতিটির Registry ও রানটাইম বিকল্প আছে; বেছে নেওয়ার আগে অফিসিয়াল ডকুমেন্টেশন (docs.docker.com, docs.aws.amazon.com, learn.microsoft.com, cloud.google.com, www.ibm.com/cloud) যাচাই করুন। এই Volume-এর যাত্রা এখানেই শেষ — পরের যাত্রা Kubernetes।

২৪.১৭ মূল প্রযুক্তিগত পরিভাষা

Container Operations: কনটেইনার ব্যবস্থার দৈনন্দিন পরিচালনা — পর্যবেক্ষণ, আপডেট, ব্যাকআপ ও ঘটনার প্রতিক্রিয়া।

Orchestrator: অনেক হোস্টে কনটেইনার পরিচালনার ব্যবস্থা (যেমন Kubernetes)।

Managed Registry: ক্লাউডের পরিচালিত Image ভান্ডার (যেমন Amazon ECR, Azure Container Registry)।

Managed Kubernetes: ক্লাউডের পরিচালিত Kubernetes সেবা (যেমন EKS, AKS, GKE, IKS)।

Serverless Container: সার্ভার পরিচালনা ছাড়া কনটেইনার চালানো (যেমন Fargate, Cloud Run)।

Runbook: ঘটনা বা অপারেশনের ধাপে ধাপে লিখিত নির্দেশিকা।

Blameless Postmortem: দোষ না খুঁজে কারণ ও প্রতিকার লেখার ঘটনা-পরবর্তী পর্যালোচনা।

Root Cause: ঘটনার মূল কারণ — লক্ষণ নয়।

Shared Responsibility: ক্লাউডে নিরাপত্তার দায়িত্ব ক্লাউড ও গ্রাহকের মধ্যে ভাগ করা থাকে।

Desired State: সিস্টেম কেমন থাকা উচিত তার ঘোষণা — অর্কেস্ট্রেটর তা বজায় রাখে।

২৪.১৮ জ্ঞান যাচাই — প্রশ্ন

1. Compose-এর তিনটি সীমা লিখুন এবং প্রতিটির জন্য কখন অর্কেস্ট্রেটর দরকার তা বলুন।
2. AWS, Azure, GCP ও IBM Cloud-এর পরিচালিত Registry-এর নাম লিখুন।
3. সার্ভারহীন কনটেইনার (যেমন Cloud Run, Fargate) কখন উপযুক্ত?
4. দৈনিক, সাপ্তাহিক ও মাসিক রুটিনে কী কী থাকে? একটি করে উদাহরণ দিন।
5. Blameless Postmortem কী? দোষ খুঁজলে কী ক্ষতি হয়?
6. ব্যাকআপ নেওয়া কিন্তু পুনরুদ্ধার পরীক্ষা না করা কেন বিপজ্জনক?

২৪.১৯ ব্যবহারিক অনুশীলন

1. **চেকলিস্ট চালানো:** এক সপ্তাহ ধরে দৈনিক চেকলিস্ট চালিয়ে একটি লগ রাখুন — কী পেলেন লিখুন।
2. **Runbook:** "ডেটাবেস ডাউন" পরিস্থিতির জন্য এক পৃষ্ঠার Runbook লিখুন।
3. **ক্লাউড অনুসন্ধান:** একটি ক্লাউডের অফিসিয়াল ডকুমেন্টেশন থেকে তার Registry ও একটি কনটেইনার রানটাইম বিকল্পের নাম ও ধারণা লিখুন।
4. **পুনরুদ্ধার মহড়া:** Staging-এ ব্যাকআপ থেকে ডেটাবেস পুনরুদ্ধার করে সময় মাপুন।